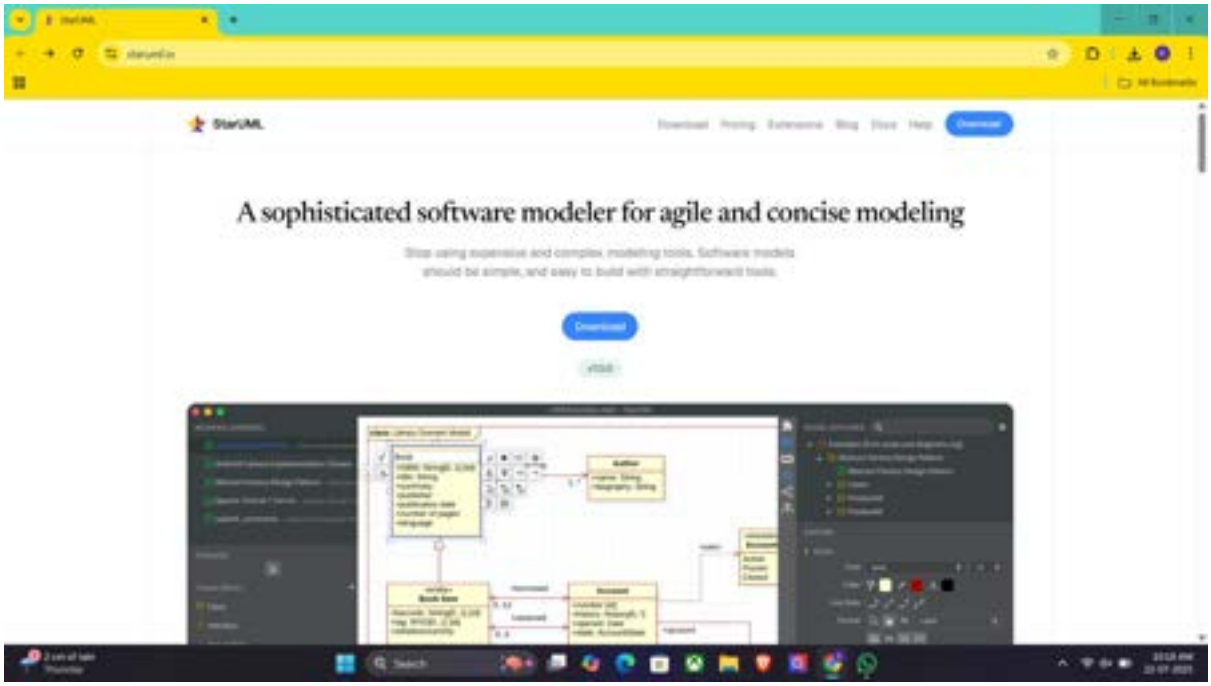
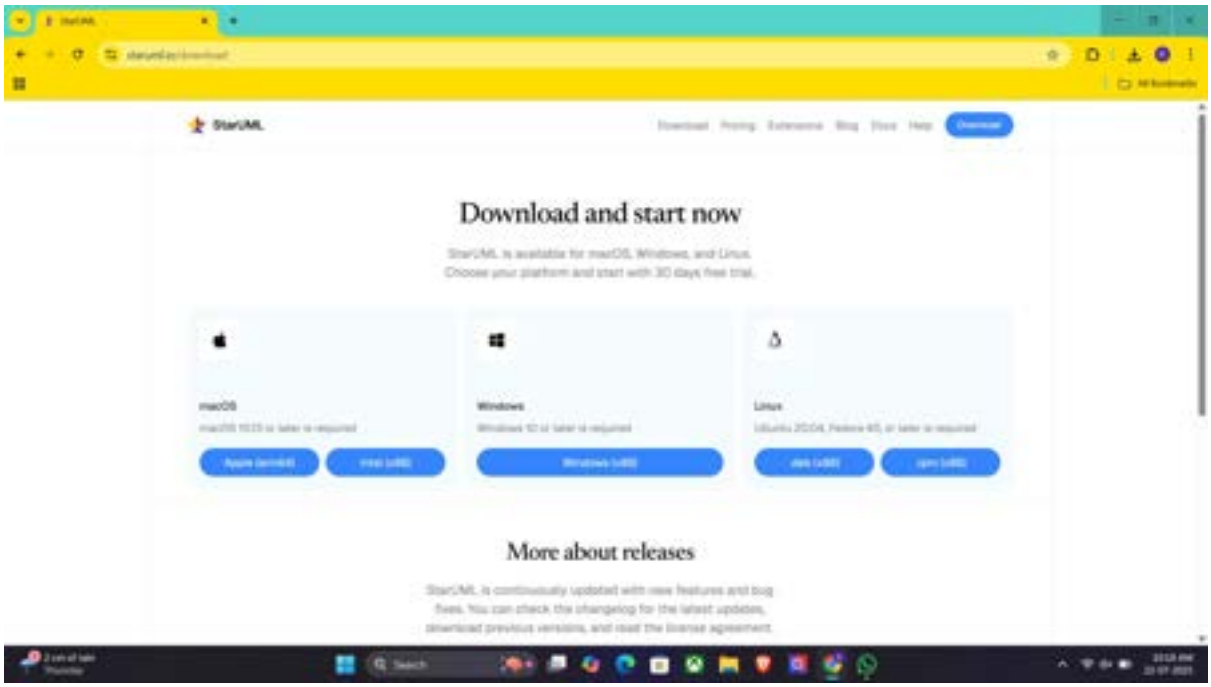
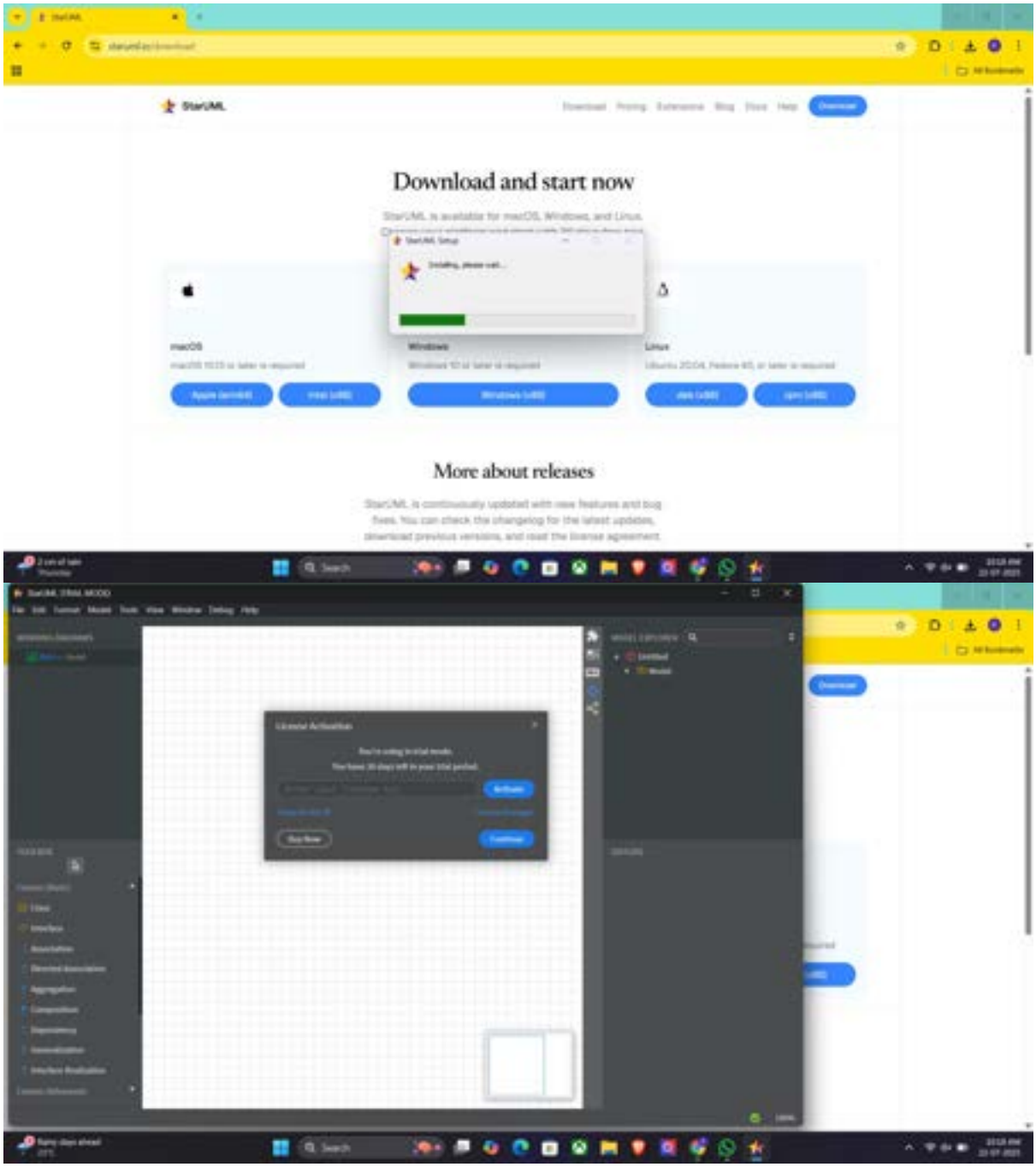
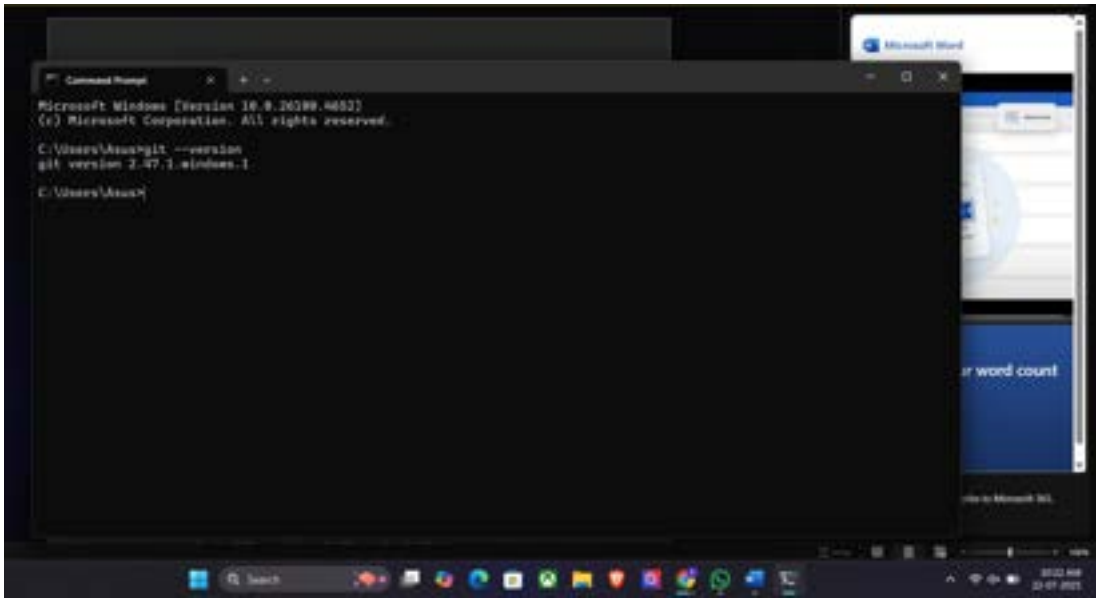
STARUML:

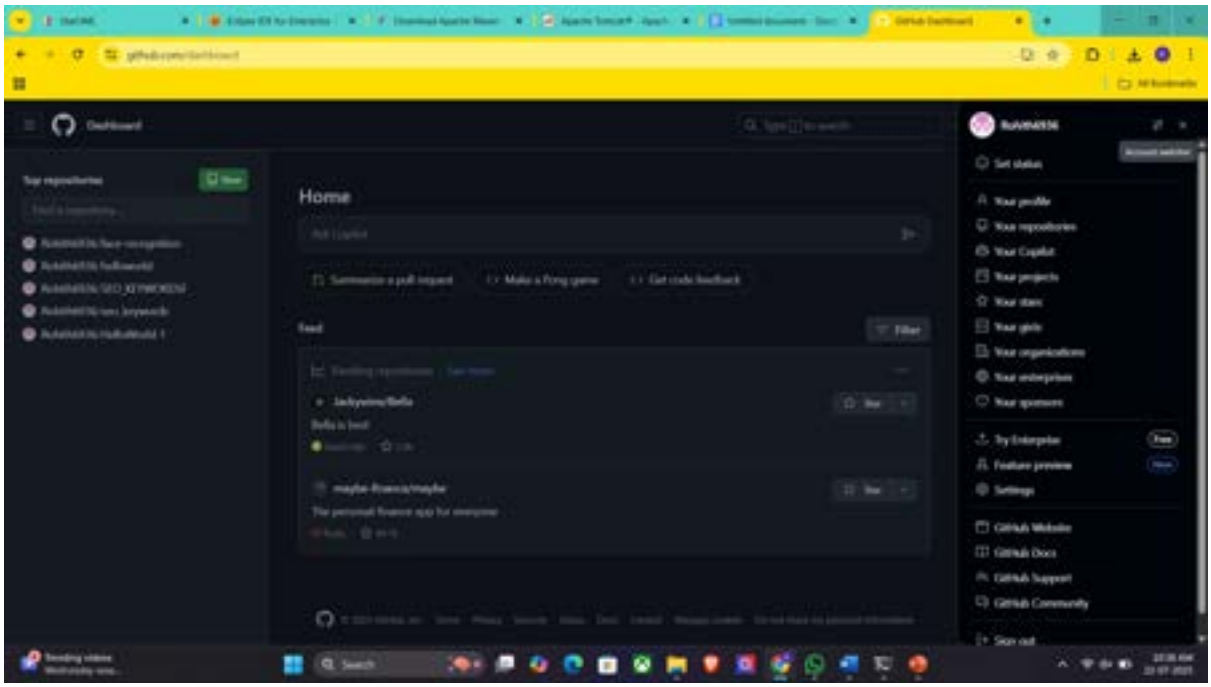

StarUML
A sophisticated software modeler for agile and concise modeling


StarUML
Download and start now
macOS
Windows
Linux
More about releases

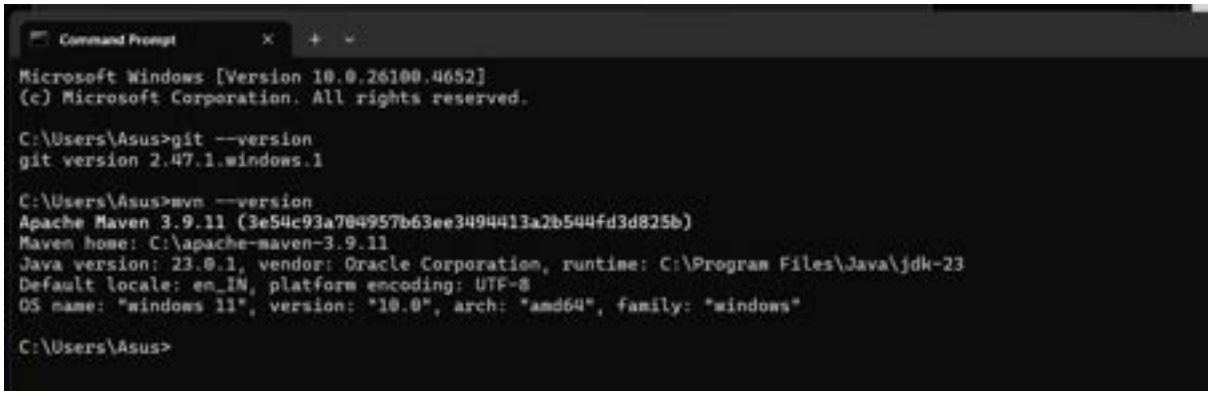

StarUML
Download and start now
More about releases

GIT:


Microsoft Windows [Version 10.0.26100.4652]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Asus>git --version
git version 2.47.1.windows.1
C:\Users\Asus>

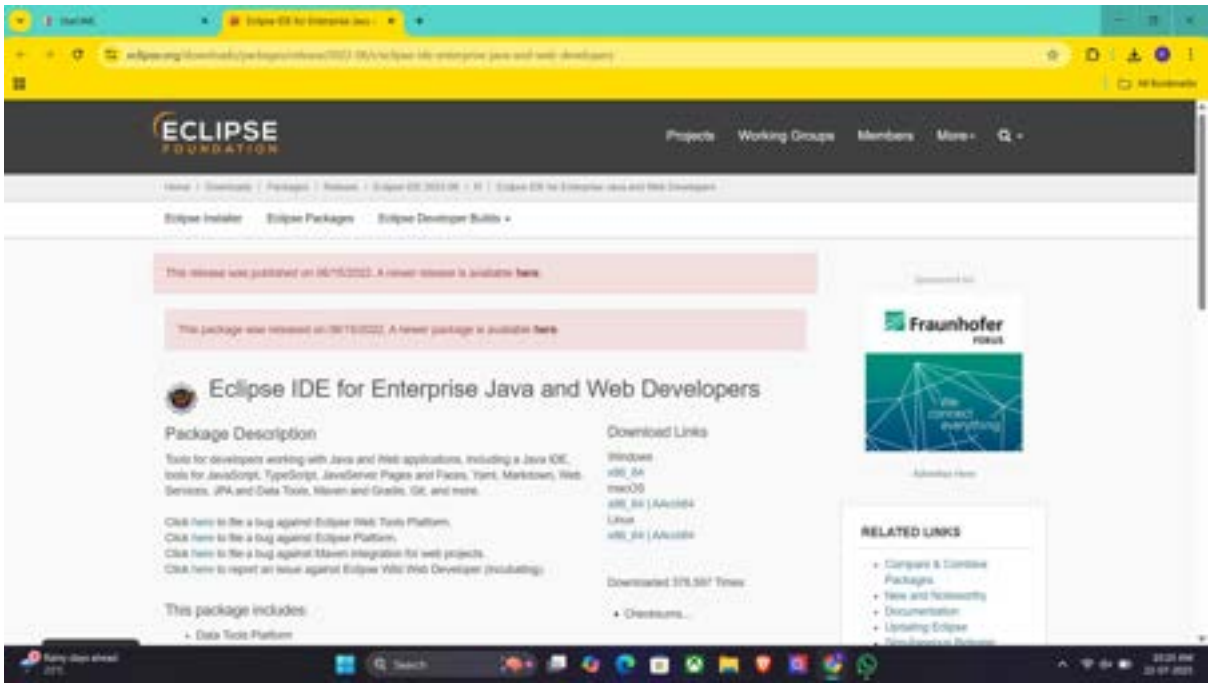

Dashboard
Home

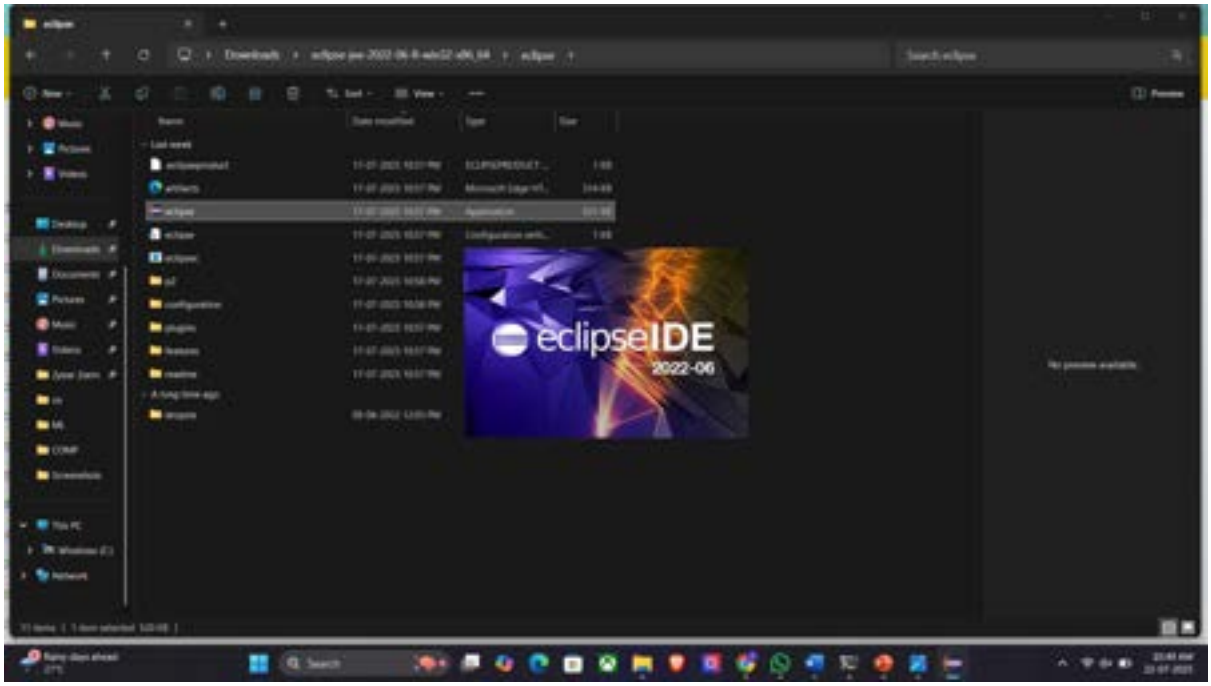
MAVEN:

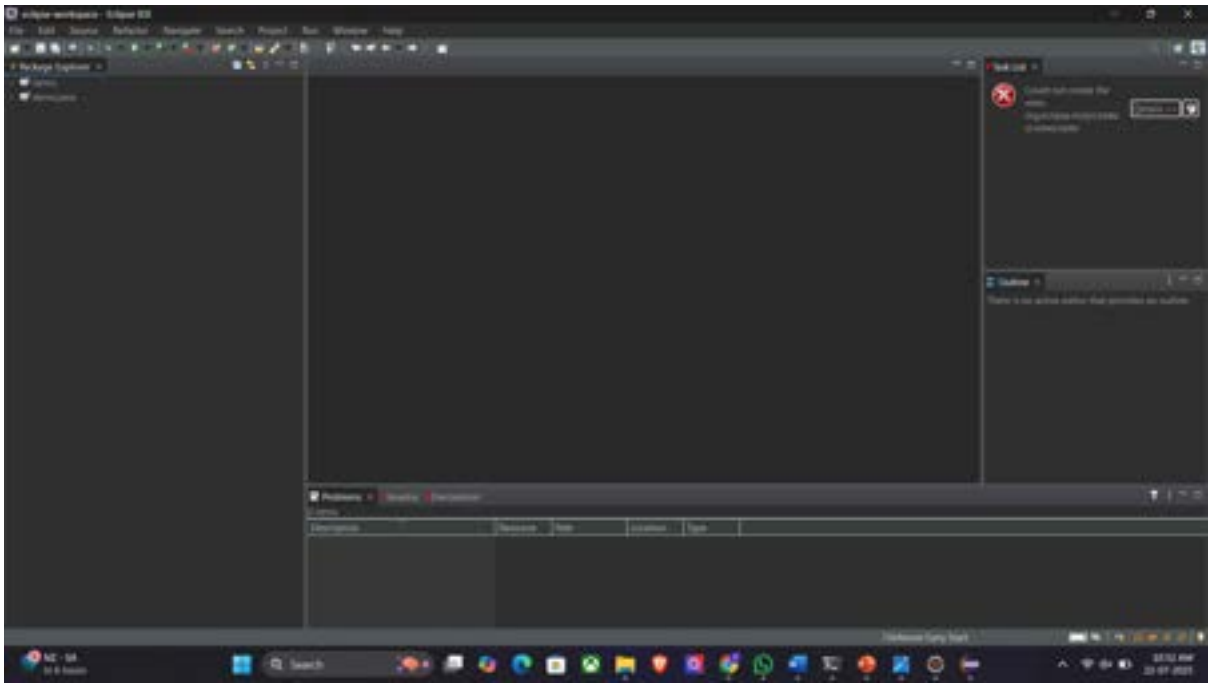


JAVA DEVELOPMENT KIT:

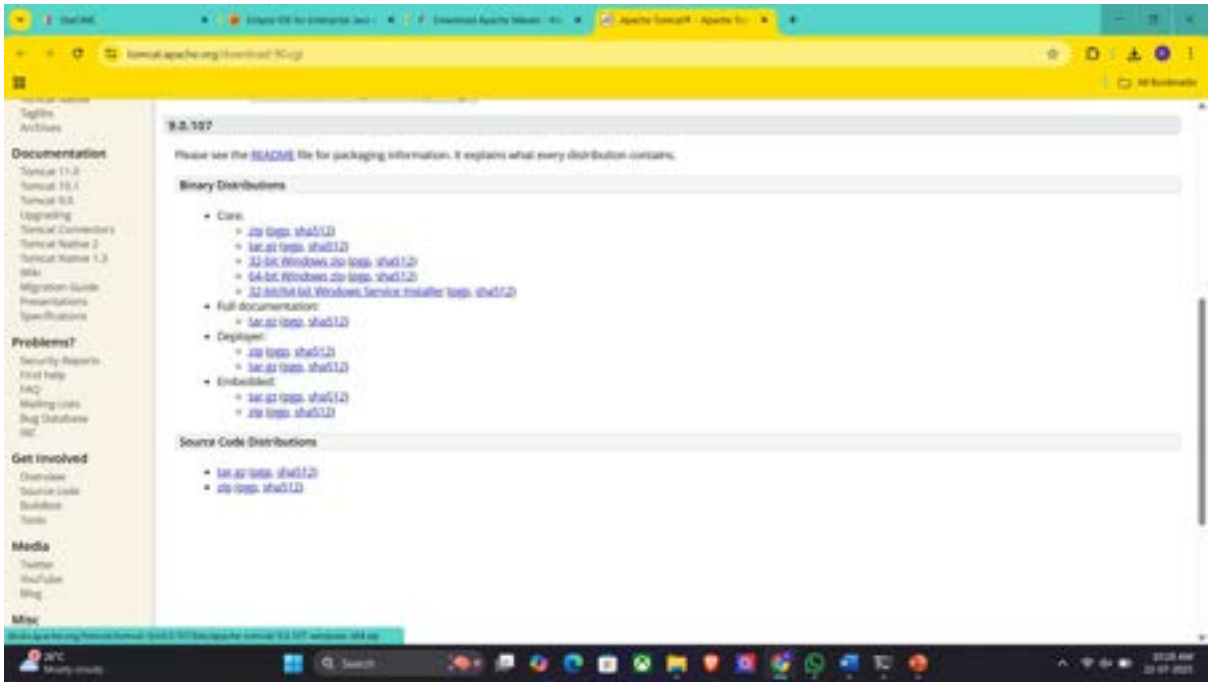
Eclipse IDE:

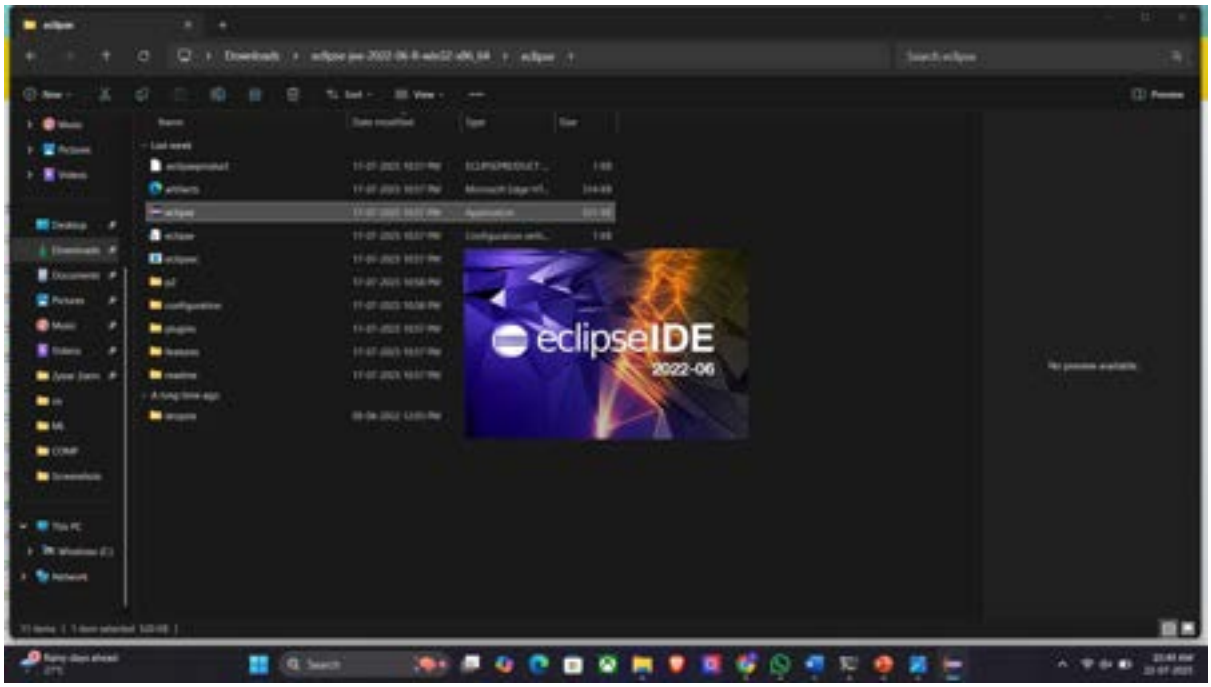


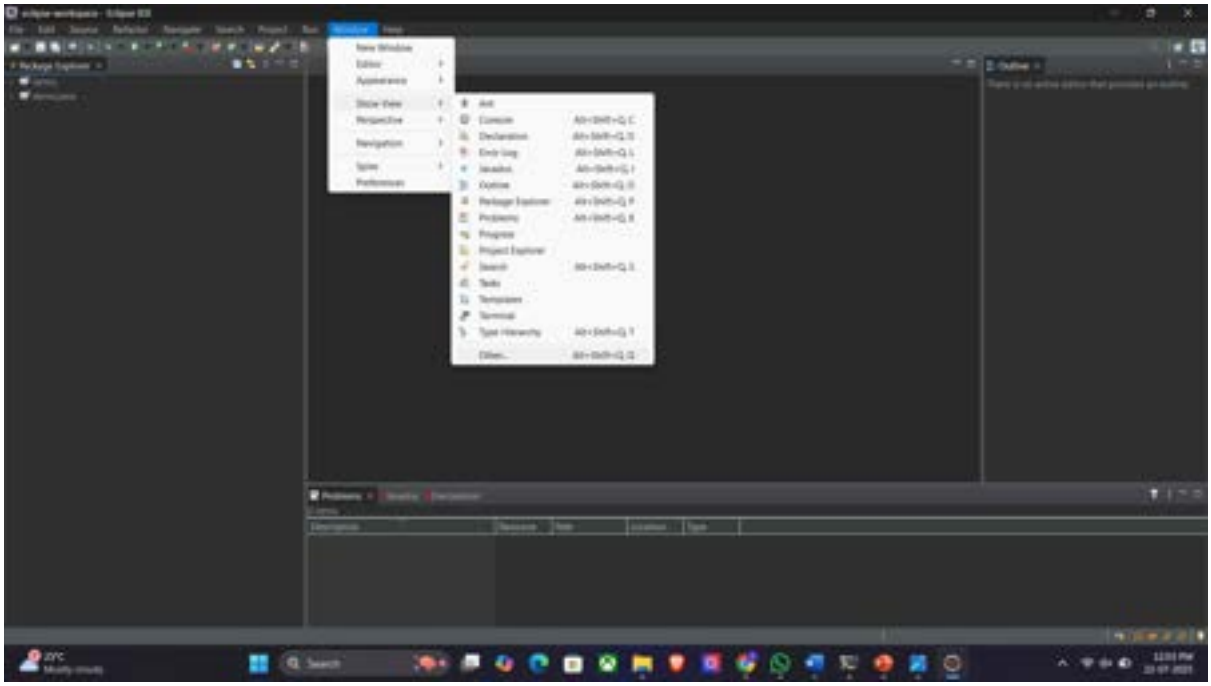


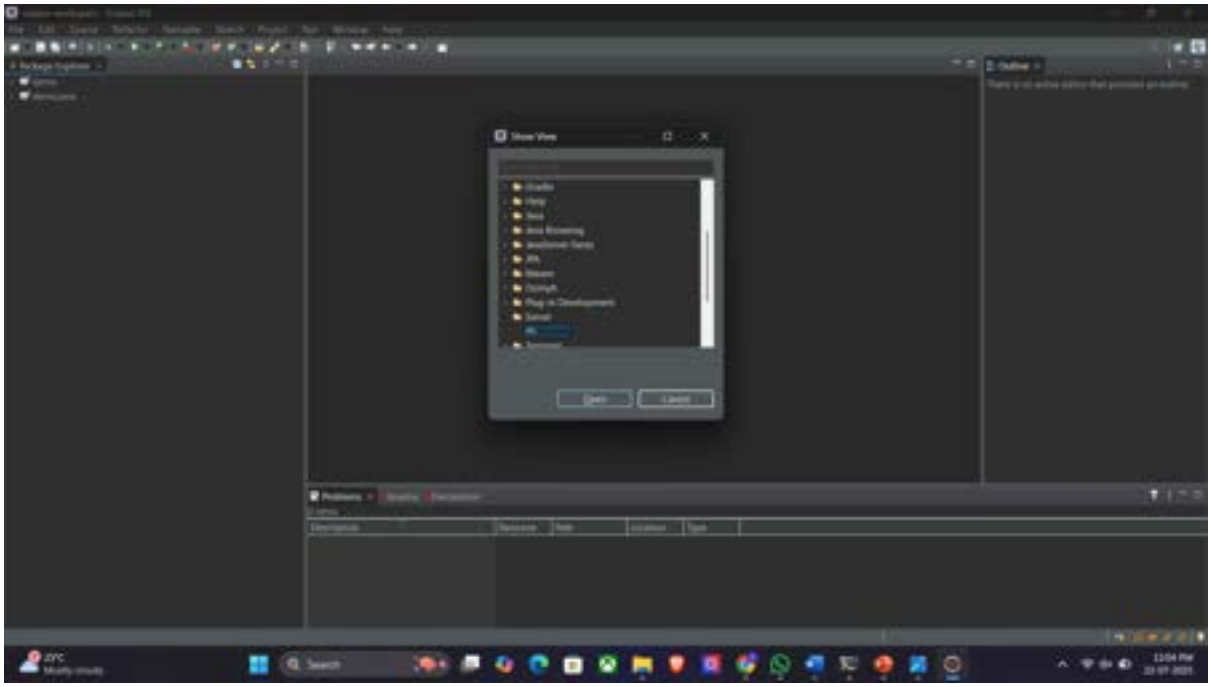


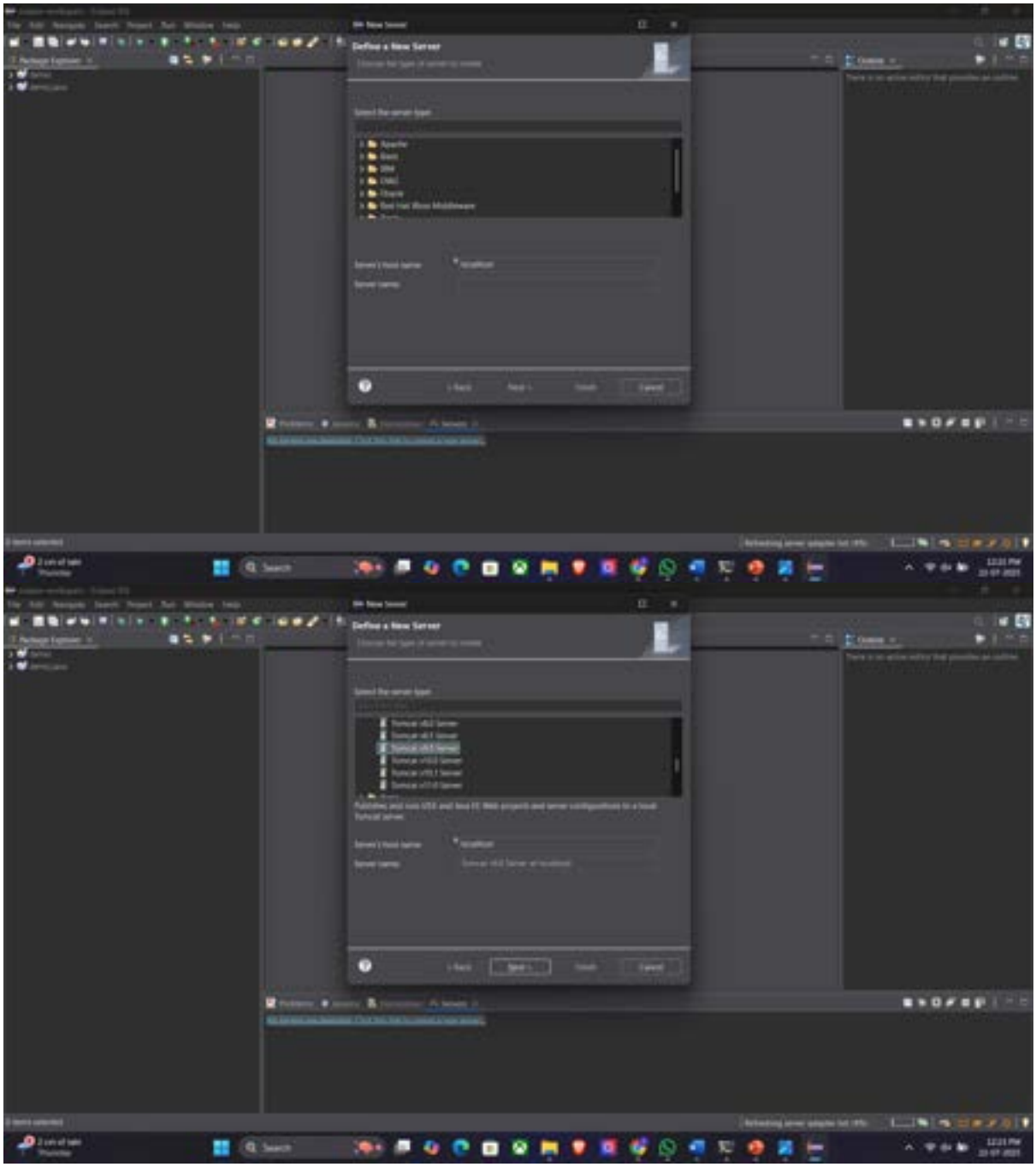
Apache Tomcat:


9.0.107
Please see the README file for packaging information. It explains what every distribution contains.
Binary Distributions
Core:
zip (pgp, sha512)
tar.gz (pgp, sha512)
32-bit Windows zip (pgp, sha512)
64-bit Windows zip (pgp, sha512)
32-bit/64-bit Windows Service Installer (pgp, sha512)
Full documentation:
tar.gz (pgp, sha512)
Deployer:
zip (pgp, sha512)
tar.gz (pgp, sha512)
Embedded:
tar.gz (pgp, sha512)
zip (pgp, sha512)
Source Code Distributions
tar.gz (pgp, sha512)
zip (pgp, sha512)
Documentation
Problems?
Get involved
Media
Misc

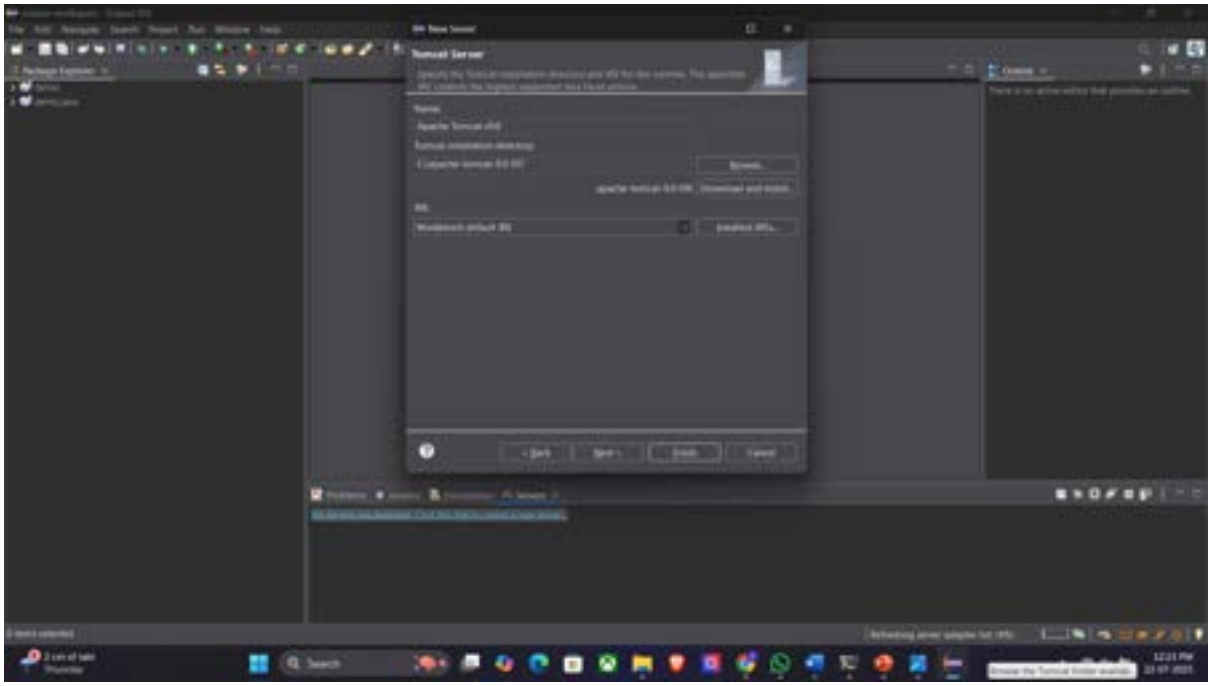

eclipseIDE
2022-06

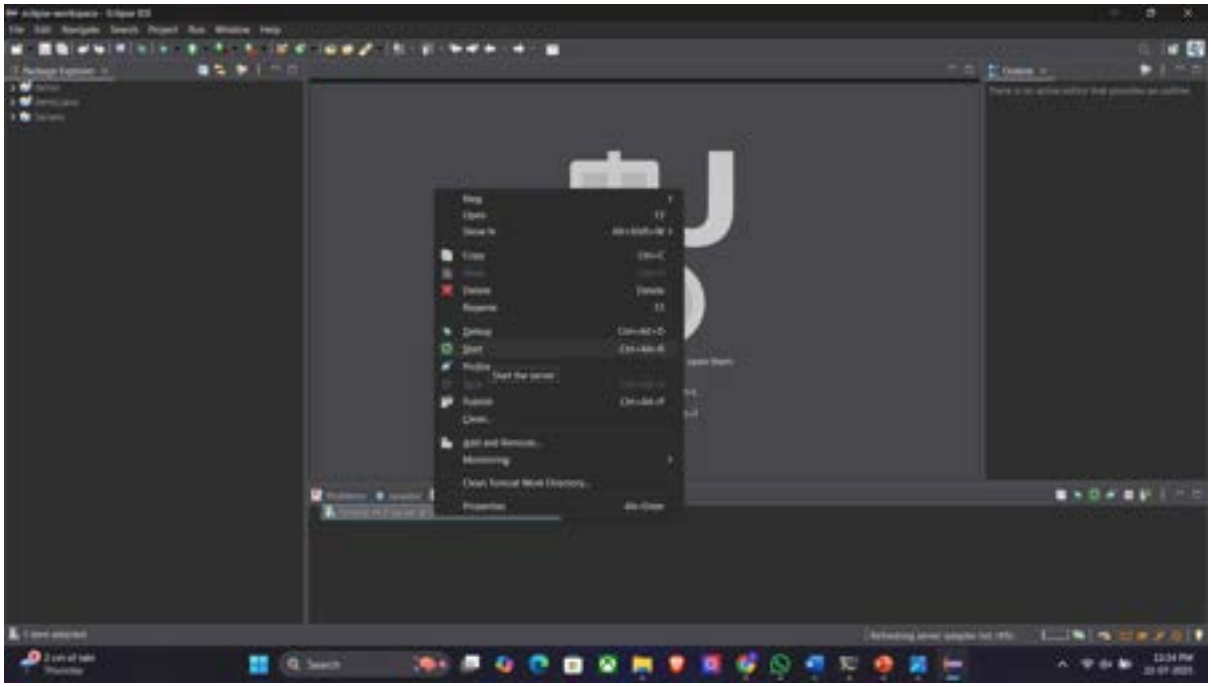


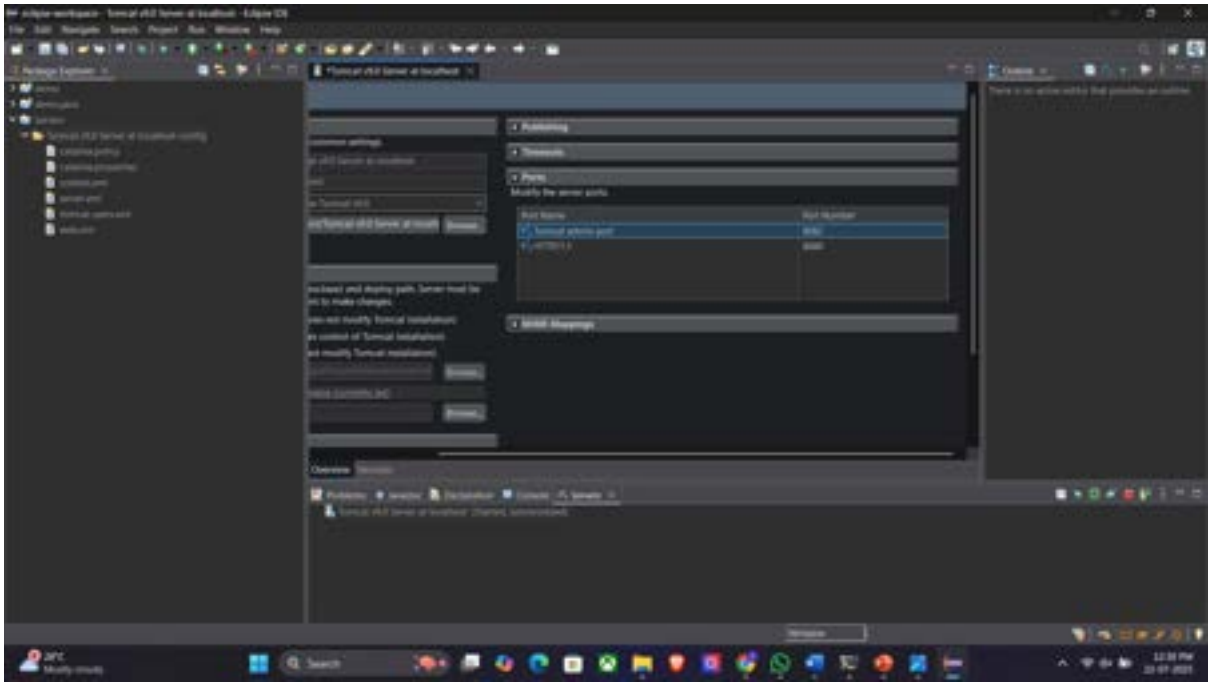


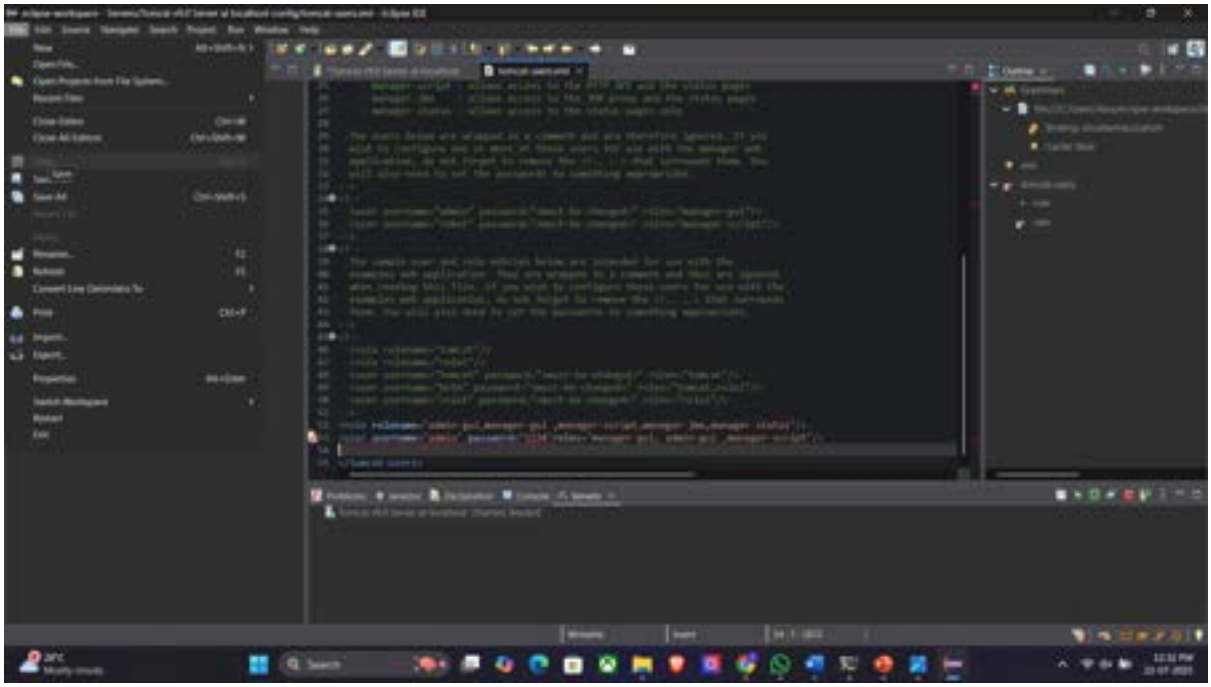

Define a New Server
Select the server type:
Apache
Basic
IBM
OW2
Oracle
Red Hat JBoss Middleware
Server's host name:
localhost
Server name:
< Back
Next >
Finish
Cancel
Problems
Javadoc
Declaration
Servers
No servers are available. Click this link to create a new server...
Outline
There is no active editor that provides an outline.
0 items selected
Refreshing server adapter list: 0%
Search
12:21 PM
22-07-2021

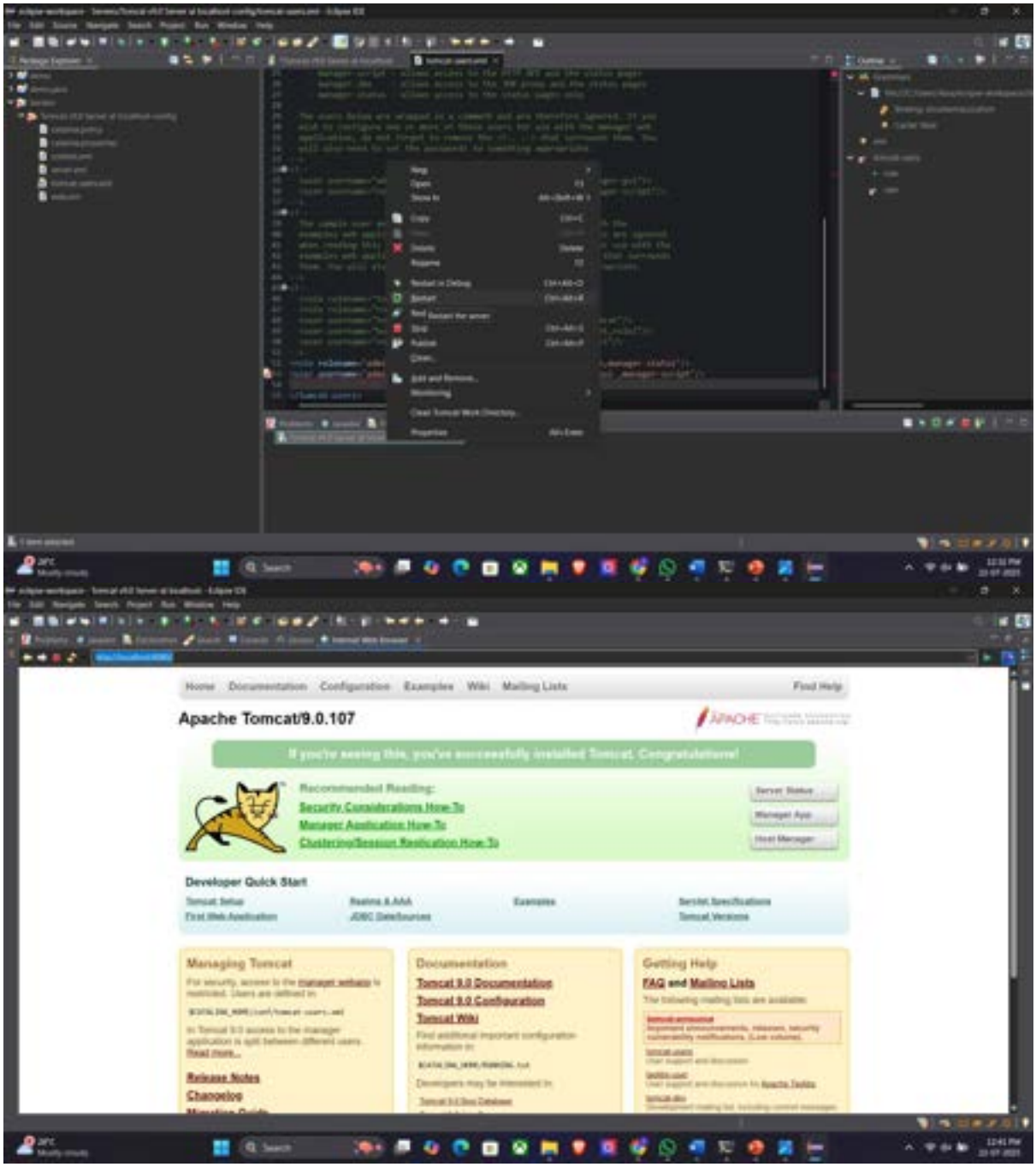
Define a New Server
Select the server type:
Tomcat v8.0 Server
Tomcat v8.5 Server
Tomcat v9.0 Server
Tomcat v10.0 Server
Tomcat v10.1 Server
Tomcat v11.0 Server
Publishes and runs J2EE and Java EE Web projects and server configurations to a local Tomcat server.
Server's host name:
localhost
Server name:
Tomcat v9.0 Server at localhost
< Back
Next >
Finish
Cancel
Problems
Javadoc
Declaration
Servers
No servers are available. Click this link to create a new server...
Outline
There is no active editor that provides an outline.
0 items selected
Refreshing server adapter list: 0%
Search
12:21 PM
22-07-2021










Home
Documentation
Configuration
Examples
Wiki
Mailing Lists
Find Help
Apache Tomcat/9.0.107
If you're seeing this, you've successfully installed Tomcat. Congratulations!
Recommended Reading:
Security Considerations How-To
Manager Application How-To
Clustering/Session Replication How-To
Server Status
Manager App
Host Manager
Developer Quick Start
Tomcat Setup
Realms & AAA
Examples
Servlet Specifications
First Web Application
JDBC DataSources
Tomcat Versions
Managing Tomcat
Documentation
Tomcat 9.0 Documentation
Tomcat 9.0 Configuration
Tomcat Wiki
Getting Help
FAQ and Mailing Lists
Release Notes
Changelog

PROJECT: WHATSAPP

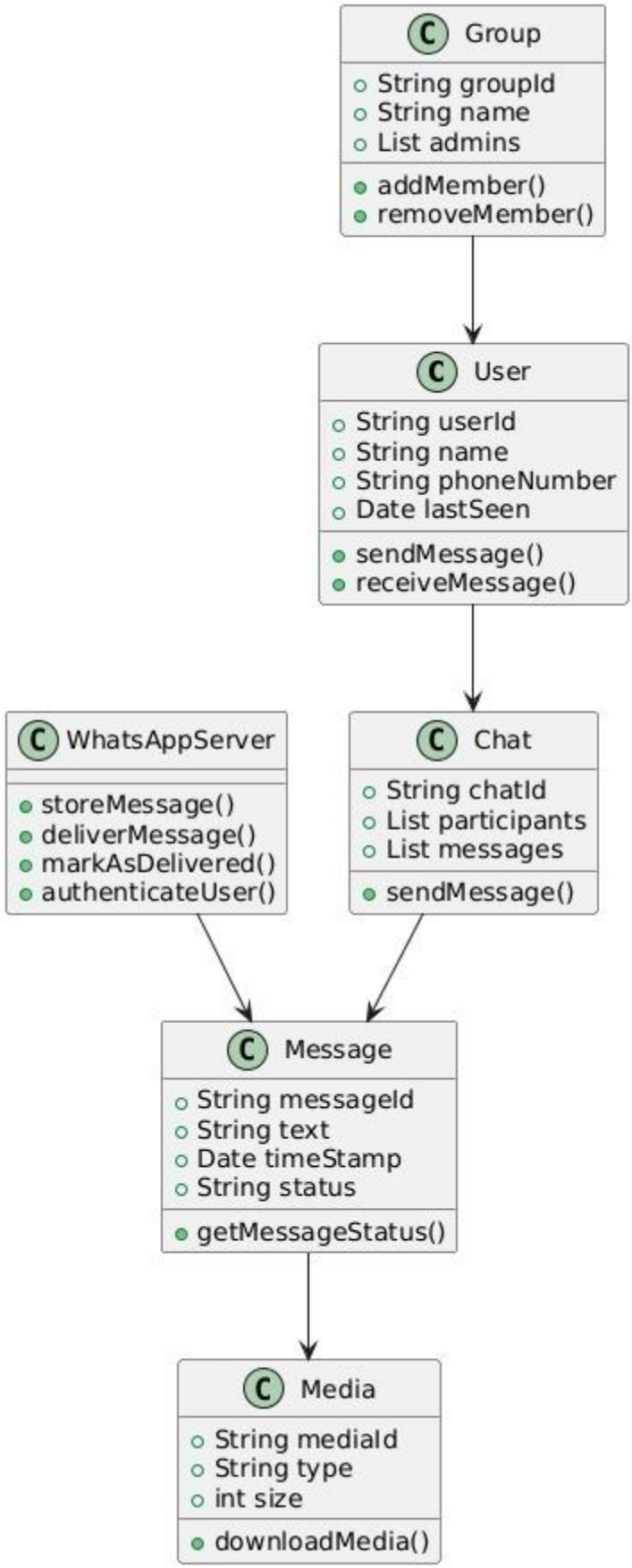
Abstract:

WhatsApp is a widely used instant messaging application that enables users to communicate in real time through text messages, images, videos, voice notes, documents, and voice/video calls. It operates on a client–server architecture where the mobile application interacts with a central WhatsApp server to authenticate users, synchronize contacts, and manage message delivery.

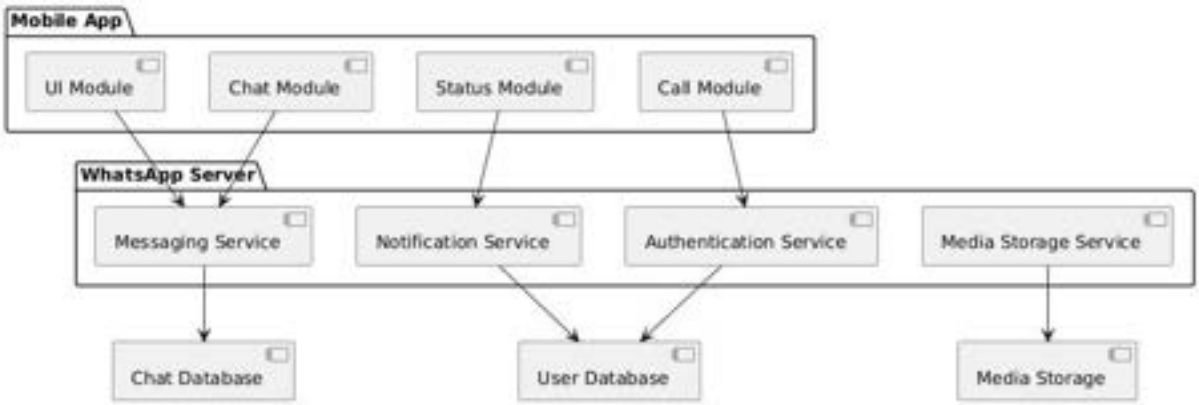
The system allows users to engage in one-to-one chats as well as participate in group conversations. Messages are transmitted using end-to-end encryption, ensuring secure communication between users. WhatsApp also supports features such as media sharing, status updates, group management, message synchronization, and push notifications, enabling seamless communication across devices.

In this project, core functionalities such as user management, chat handling, message flow, media transmission, and server interactions are analyzed. UML diagrams—including use case, class, sequence, and component diagrams—are used to model the internal structure and behavior of WhatsApp. This analysis provides a clear understanding of how a modern real-time messaging system works, focusing on reliability, scalability, and secure communication

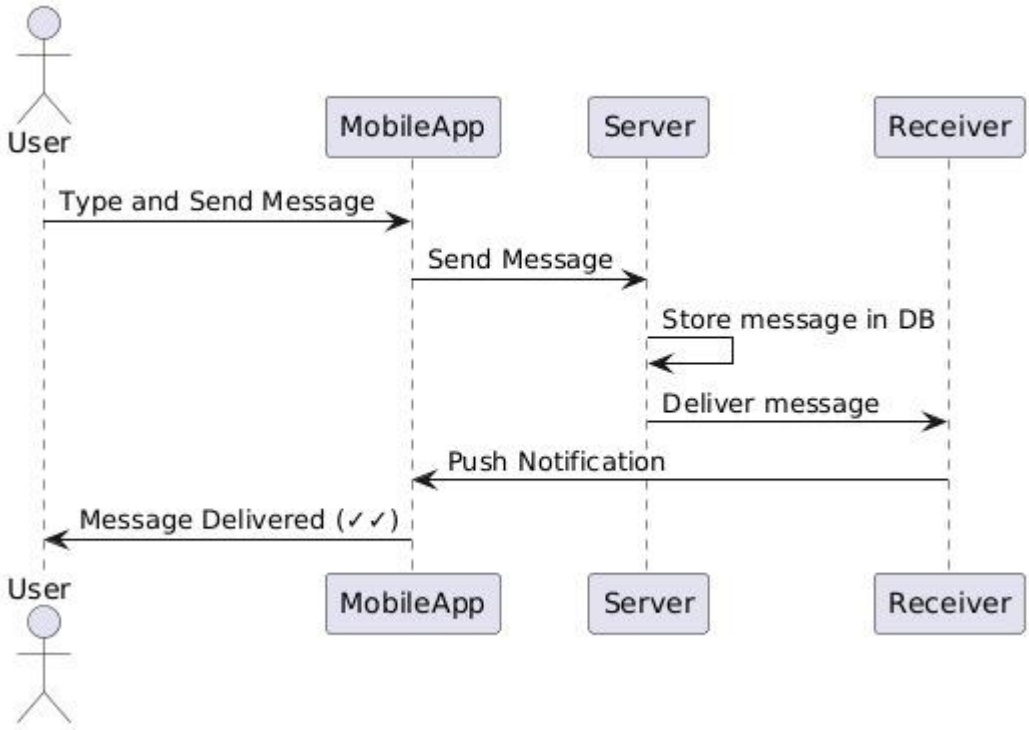
Class Diagram:



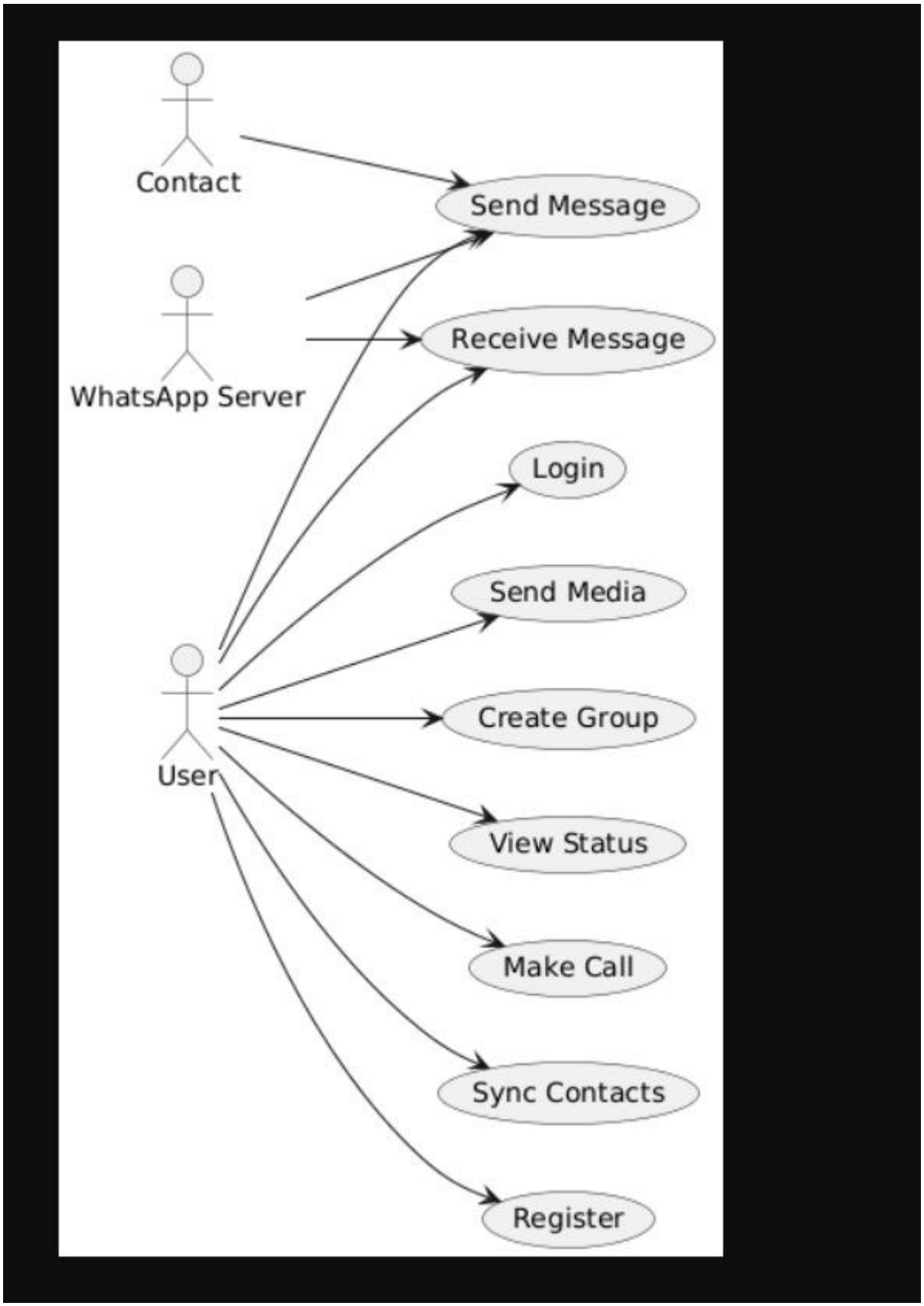
COMPONENT DIAGRAM:



SEQUENCE DIAGRAM:



USE CASE DIAGRAM:



LAB ACTION **PLAN FOR WEEK 2**

Objective:

1. Pushing multi-folder project into private repository (by student).
2. Students must explore all listed git commands on the multi-folder project in local and remote repository.
3. Students must explore all git commands on given scenario-based question.
4. Students need to make note in observation book and also upload the executed exercise on scenario-based question.

Task 1: Pushing multi-folder project into private repository(Note: student take screenshots for upload).

1. Create a Private Repository on GitHub

- Go to https://github.com
- Click **New repository**
- Give it a name
- Check **Private**
- Click **Create repository**

2. Initialize Git in Your Local Project (if not already done)

Open a terminal in the **root** of your multi-folder project:

cd /path/to/your/project

git init

3. Add the Remote Repository (in bash)

You'll see a URL on GitHub after creating the repo. It will look as below , run this command in git bash:

git remote add origin https://github.com/yourusername/your-private-repo.git

4. Add and Commit Your Files(in bash)

git add .

git commit -m "Initial commit"

5. Push to the Private Repository

If the repo is new and empty, push like this from git bash:

git push -u origin master

If you're using the main branch (GitHub default):

git push -u origin main

You may be prompted to log in (via username/password or token).

Task 2: Students must explore all listed git commands on the multi-folder project in local and remote repository.

1. **Git Commands:**

- git version : The command git version is used to check the version of git.

  git --version

- git config: Configures Git settings. Commonly used to set up user information

  git config --global user.name "Your Name"

  git config --global user.email "youremail@example.com"

- **git config --list**: Displays all the Git configurations for the current user.

2. **Repository Management**

- **git init**: Initializes a new Git repository in the current directory

  git init

- **git clone**: Creates a copy of an existing Git repository from a remote source (e.g., GitHub) to your local machine

  git clone https://github.com/username/repository.git

- **git remote –v :** To see the remote repository that is connected to your local repository

  git remote –v

- **git remote add :** To add a new remote repository to your local repository

  git remote add origin https://github.com/username/repository.git

- **Pushing Changes to Remote:** To push your local commits to a remote repository:

  git push <remote_name> <branch_name>

  git push origin main

- **Pulling Updates from Remote:** To pull the latest changes from the remote repository and merge them into your local branch

  git pull <remote_name> <branch_name>

  git pull origin main

- **git remote remove:** To remove a remote repository from your local configuration:

  git remote remove <remote_name>

  git remote remove origin

- **Renaming a Remote:** to rename an existing remote:

git remote rename <old_name> <new_name>

git remote rename origin upstream

- **Fetching Updates from Remote:** To fetch updates from the remote repository but not merge them into your local branch

  git fetch <remote_name>

- **Changing Remote URL :** To change the URL of a remote (e.g., after changing the remote repository address):

  git remote set-url <remote_name> <new_url>

  git remote set-url origin https://github.com/username/new-repository.git

- Viewing the Remote Repository's Information: to see detailed information about a remote repository

  git remote show <remote_name>

  git remote show origin

3. Staging and Committing

- **git status**: Shows the status of changes in your working directory and staging area. It tells you which files are untracked, modified, or ready to be committed.

  git status

- **git add**: Adds changes in the working directory to the staging area.

  git add filename.txt # Adds a specific file

  git add . # Adds all changes in the directory

- **git commit**: Commits the staged changes to the repository with a descriptive message. The -m option allows you to include a commit message.

  git commit -m "Commit message describing changes"

4. Branching and Merging

- **git branch**: Lists all branches or creates a new branch

  git branch # Lists all branches or

  git branch -a # Lists all branches

  git branch branch-name # Creates a new branch

  git branch -d <branch_name> # Deletes a branch

Use -D to force-delete if it hasn't been merged.

- **Listing All Remote Branches** : To you want to list all branches on the remote repository

git branch –r

- **git checkout**: Switches to a different branch

  git checkout branch-name # Switches to an existing branch

  git checkout -b new-branch # Creates and switches to a new branch

- Pruning Deleted Remotes : If a branch was deleted on the remote but still shows up locally.

  git remote prune origin

  When someone **deletes a branch on the remote**, your local Git doesn't automatically remove the corresponding remote-tracking branch (like origin/old-feature).
  git remote prune origin removes these outdated references.

- Fetching a Specific Remote Branch: To fetch a specific branch from a remote.
  git fetch <remote_name> <branch_name>

  git fetch origin feature-branch

- **Setting the Upstream Branch for Pushing** : When pushing for the first time and want to set the remote branch you are pushing to:

  git push --set-upstream <remote_name> <branch_name>

git push --set-upstream origin feature-branch

- **git merge**: Merges the specified branch into the current branch. This command integrates the changes from the feature branch into the main branch.

  git checkout main # Switch to the main branch

  git merge branch-name # Merge branch-name into main

- Rebasing a Local Branch onto a Remote Branch: If you want to rebase your local branch onto a remote branch (this can be useful to keep your history linear)

  git fetch <remote_name>
  git rebase <remote_name>/<branch_name>

Example:

git fetch origin
git rebase origin/main

5. Undoing Changes

- git reset : Removes the specified file from the staging area but leaves the working directory unchanged. git reset --hard can also reset the working directory and staging area to the last commit.

  git reset <file>:

- git revert**:** Creates a new commit that undoes the changes from a specified commit, leaving the history intact**.**

  git revert <commit>:

6. **Viewing History**

- git log**:** Shows a history of commits in the repository, including commit hashes, messages, and timestamps. Use git log --oneline for a more concise view.

  git log:

- git diff**:** Displays differences between various commits, the working directory, and the staging area. git diff without arguments shows changes not yet staged.

  git diff:

- git show**:** Shows the details of a specific commit, including the changes made and the commit message.

  git show <commit>

7. To undo the changes made to file before staging it.

- **git restore filename**

8. To correct committed with the wrong message

- **git commit --amend -m "Corrected commit message"**

9. Recovering deleted branches

- **git reflog**
- **git checkout -b feature-ui <commit_hash>**

10. To download the latest changes from the remote without merging

- **git fetch origin**

11. To remove accidentally committed sensitive file from Git history.

- **git filter-branch --force --index-filter **
- **"git rm --cached --ignore-unmatch secrets.txt" **
- **--prune-empty --tag-name-filter cat -- --all**

12. To merge changes from another branch

First switch to branch where changes are to applied and then merge another-branch

- **git checkout previous-branch**
- **git merge another-branch**

13. To resolve a merge conflict manually

You tried to merge two branches and Git reported a conflict in x.js. Then for conflict resolution do:

1. **Open x.js and resolve/remove the conflict markers (<<<<<<<, =======, >>>>>>>)**
2. After resolving:
    - ✓ **git add app.js**
    - ✓ **git commit # If Git didn't auto-create a merge commit**

14. The .gitignore

The .gitignore file is used to tell Git which files or directories to ignore in your project. It's a useful way to avoid committing unnecessary files like log files, build outputs, and IDE configurations.

a. Create a .gitignore File

In the root directory of your Git project, create a file called .gitignore

touch .gitignore

b. Add Rules to .gitignore

Inside the .gitignore file, you add patterns for the files and directories you want Git to ignore. Each pattern should be written on a new line.

Here are some common examples:

- Ignore all .log files: ***.log**
- Ignore a specific file: **secret_file.txt**
- Ignore all files in a temp/ directory: **temp/**
- Ignore all files except important_file.txt inside a folder:

  folder/*
  !folder/important_file.txt

- Ignore files with a specific extension: ***.bak**

15. To see who changed a particular line in a file

- **git blame script.py**

16. Git stash

You modified files A and B, but not committed them.

git stash # saves changes and reverts to clean state

git switch another-branch

 # do something else...

git switch main

git stash apply # brings back your changes

17. To check branch is merge

- **git branch –merged**

  If branch not merged then displays its name

18. To Delete multiple local branches at once

- **git branch –d branc1 branch2 branch3**

LAB ACTION **PLAN FOR WEEK 3**

Objective:

1. To work on collaborative coding by:
   a. Creating Organization.
   b. Coordinating with others through a shared repository
2. To resolve conflicts when collaborating on same part of code.
3. To create and apply patch
4. Students need to make note in observation book and also upload the screenshots of above executed exercise and scenario-based question.

1. Collaborative coding.

Code collaboration refers to the process of multiple people working together on the same codebase.

There are **two ways** of collaborative coding:

a. Creating Organization
b. Coordinating with others through a shared repository

a. <u>Facilitating Collaborative Work by creating Organization</u>

<u>Organization</u> - Organizations are shared accounts where businesses and open-source projects can collaborate across many projects at once. Owners and administrators can manage member access to the organization's data and projects with sophisticated security and administrative features. Steps to facilitate collaborative work by creating organization are:

Step 1: Click on New organization in Git hub

Click on Create a free organization.

S**tep 2:** Set up your organization by entering the details like name, associated email, account verification, etc.

Click on Next

Step 3: Complete the setup by

- ✓ adding members to the group.
- ✓ Now, the invitation goes to other collaborator and they must accept.
- ✓ Next click on <u>complete setup</u>
- ✓ Goto Github and click on

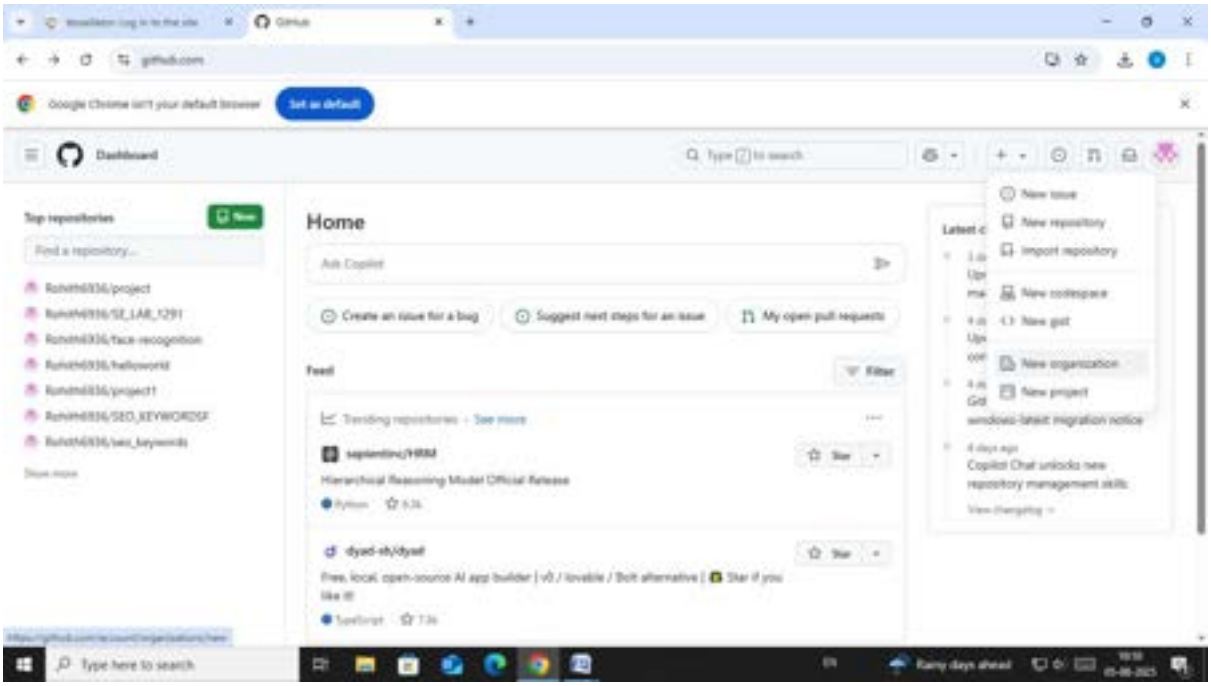
Settings → Member privileges → Base permissions → Select write → change base permission to "Write"

Next under,
Projects base permissions → Select Write → change base permission to "Write"

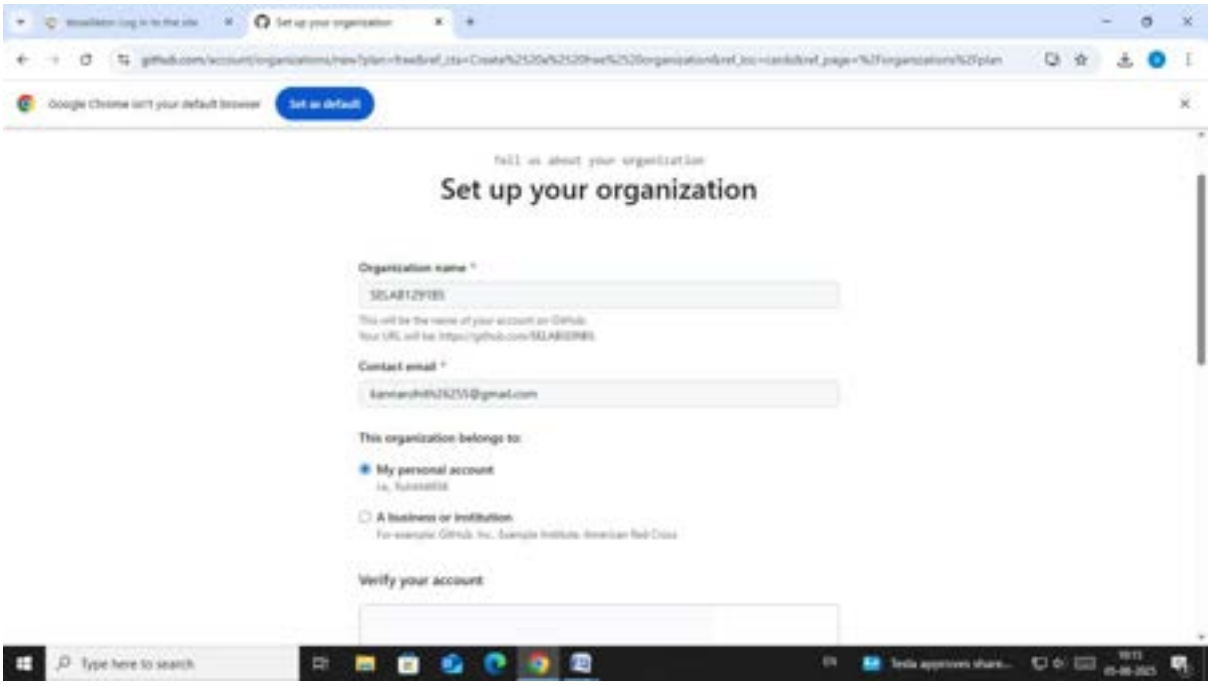
Step 4: Create the remote repository for storing the project related files in Organization. This repository is accessible to every member of the team as per the permissions given.

Note: The repository can be made private so that it is accessible only to the group members rather than being in a public domain.

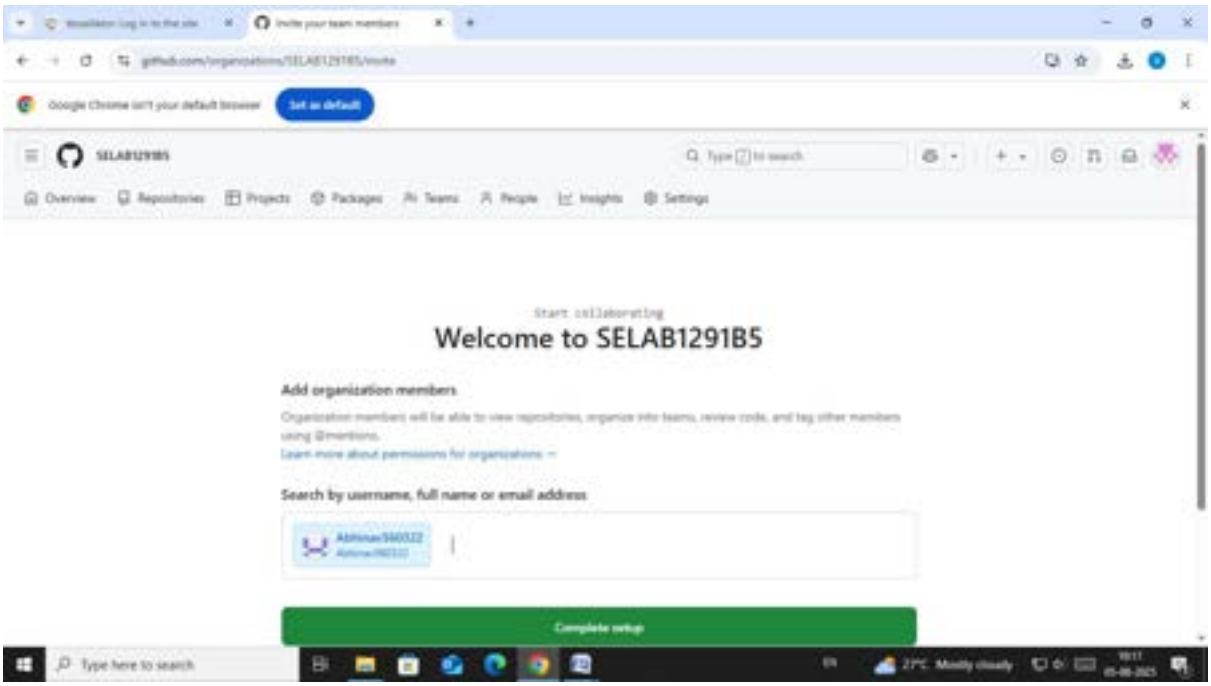
Now all the members of the team can contribute to the development of the project and the different files with all the versions and modification notices will be available in the respective repositories and is accessible to all the members



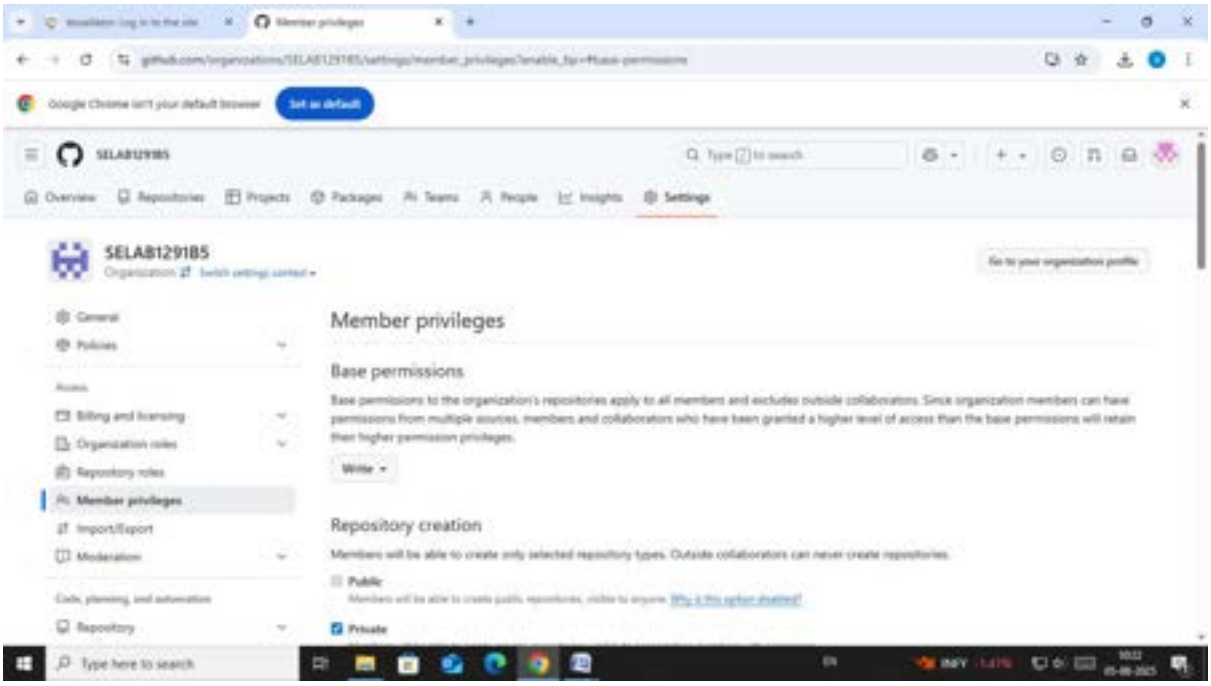
Organization details:

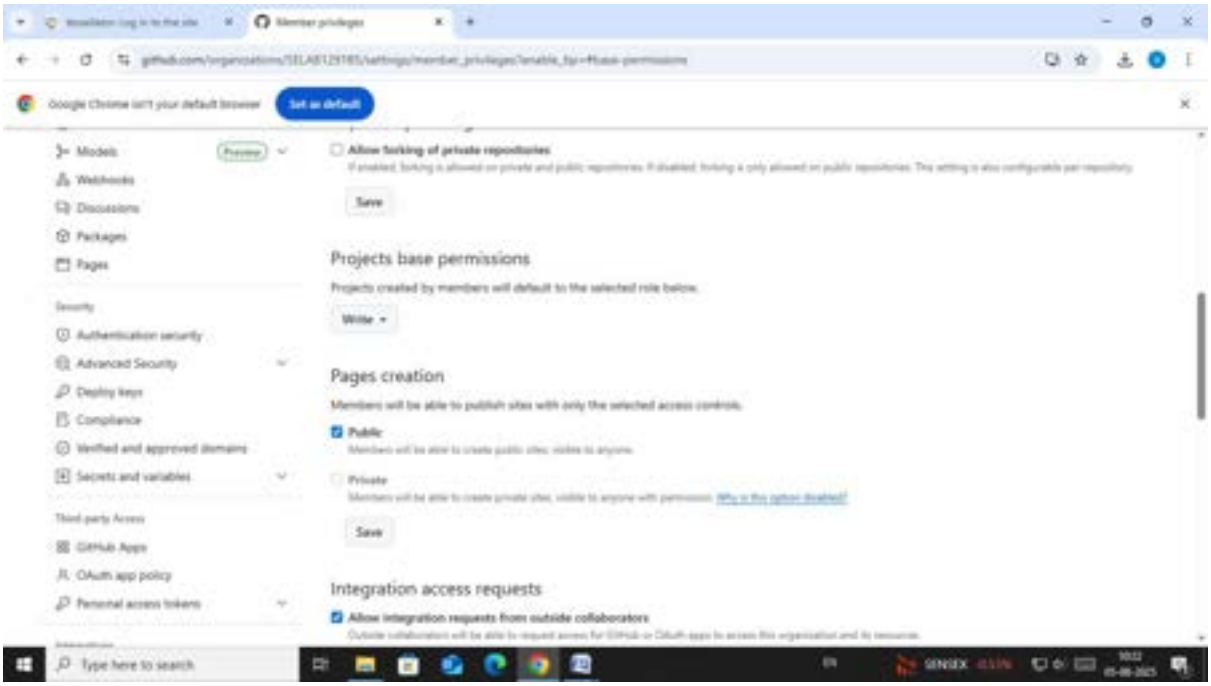


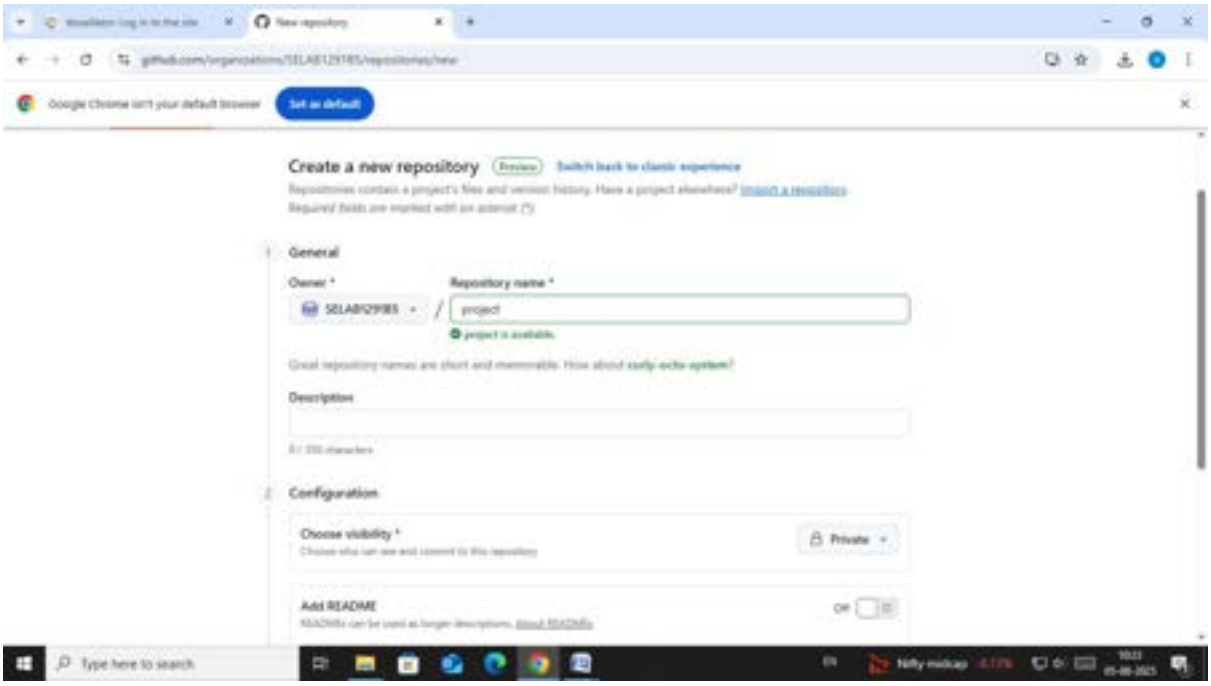
Collaborator adding:

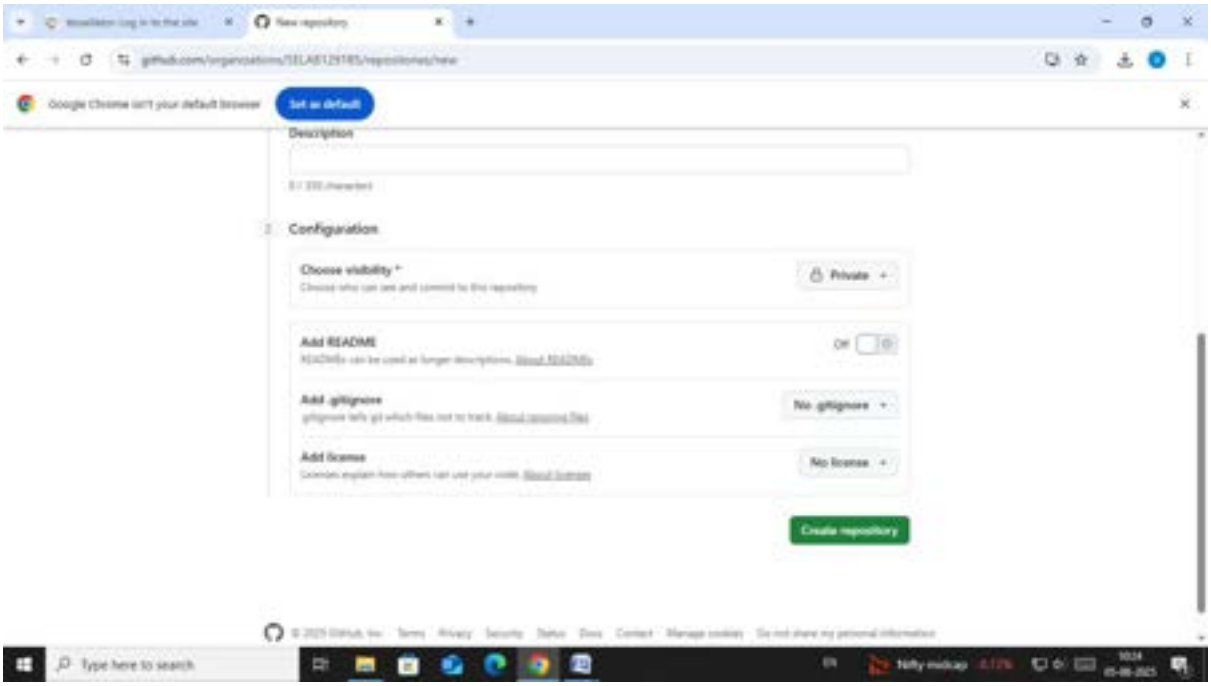


Settings:


Google Chrome isn't your default browser
Set as default
SELAB1291B5
Overview
Repositories
Projects
Packages
Teams
People
Insights
Settings
SELAB1291B5
Organization
Go to your organization profile
General
Policies
Billing and licensing
Organization roles
Repository roles
Member privileges
Import/Export
Moderation
Repository
Member privileges
Base permissions
Base permissions to the organization's repositories apply to all members and excludes outside collaborators. Since organization members can have permissions from multiple sources, members and collaborators who have been granted a higher level of access than the base permissions will retain their higher permission privileges.
Write
Repository creation
Members will be able to create only selected repository types. Outside collaborators can never create repositories.
Public
Private
Type here to search


Google Chrome isn't your default browser
Set as default
Models
Preview
Webhooks
Discussions
Packages
Pages
Authentication security
Advanced Security
Deploy keys
Compliance
Verified and approved domains
Secrets and variables
GitHub Apps
OAuth app policy
Personal access tokens
Allow forking of private repositories
Save
Projects base permissions
Projects created by members will default to the selected role below.
Write
Pages creation
Members will be able to publish sites with only the selected access controls.
Public
Private
Save
Integration access requests
Allow integration requests from outside collaborators
Type here to search





b. <u>To collaborate effectively on GitHub shared repository by sharing code and coordinating with others through a shared repository.</u>

→**Steps for Collaborator 1 are**: (collaborator-1 has repository that he wants to share with collaborator-2)

- ✓ Go to Settings > collaborators
- ✓ Now in Manage Access click on Add people
- ✓ Now you will be able to see ONE collaborator added
- ✓ After this invitation goes to collaborator 2 and they need to accept it.

→**Steps for Collaborator 2 are:**

✅ 1. Set Up Git and GitHub

In git bash
git config --global user.name "Your Name"
git config --global user.email you@example.com

Note : run `$ git remote -v`

If
```
origin  git@github.com:OrgYOG/commonREPO.git (fetch)
origin  git@github.com:OrgYOG/commonREPO.git (push)
```
then

```
$ git remote –v // Note now, not connected to any remote
```

✅ 2. Goto Git hub and copy url of shared repository by collaborator 1

- Copy url (option 1)

✅ 3. Clone the Repository

Get a local copy of the repo:
In git bash
git clone https://github.com/username/repository-name.git
cd repository-name

✅ 4. Create a Branch for Your Work

Always work on a separate branch:
In git bash
git checkout -b feature/your-feature-name

✅ 5. Make Changes and Commit

Edit files, then stage and commit your changes:
In git bash to existing file / new file
git add .
git commit -m "Add a clear and descriptive commit message"

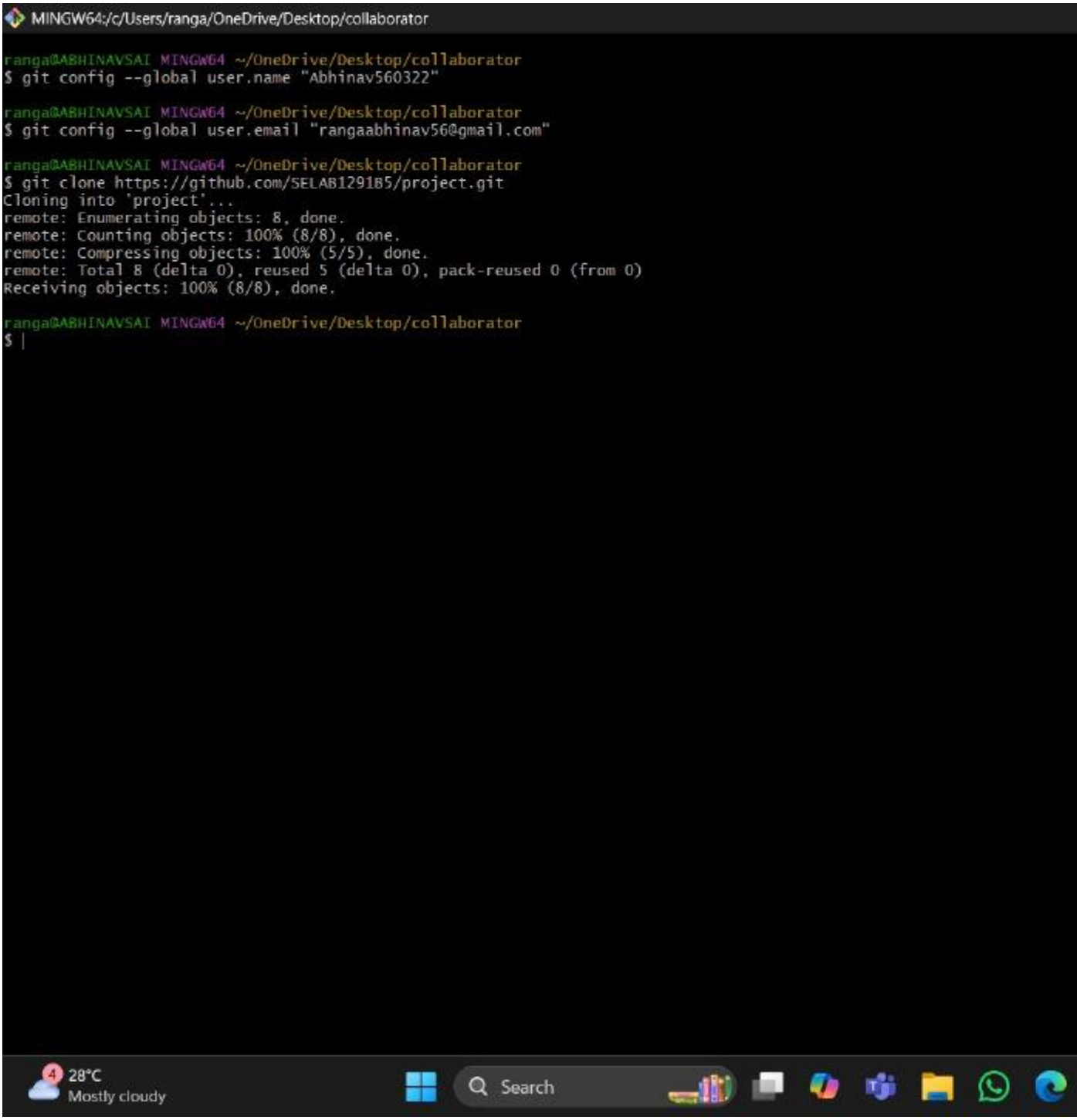
✅ 6. Push Your Branch to GitHub

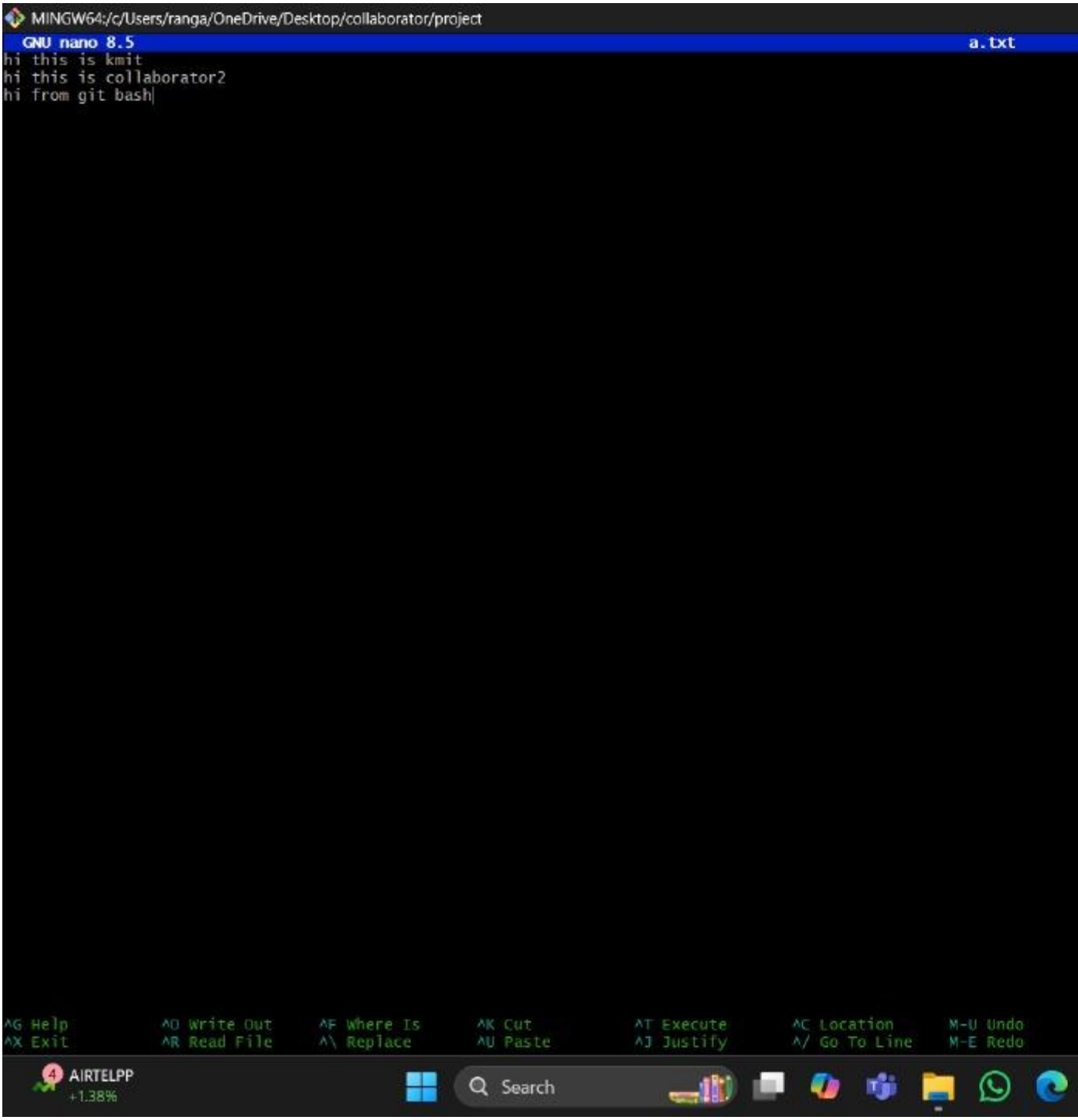
➔ In git bash
➔ git push origin feature/your-feature-name

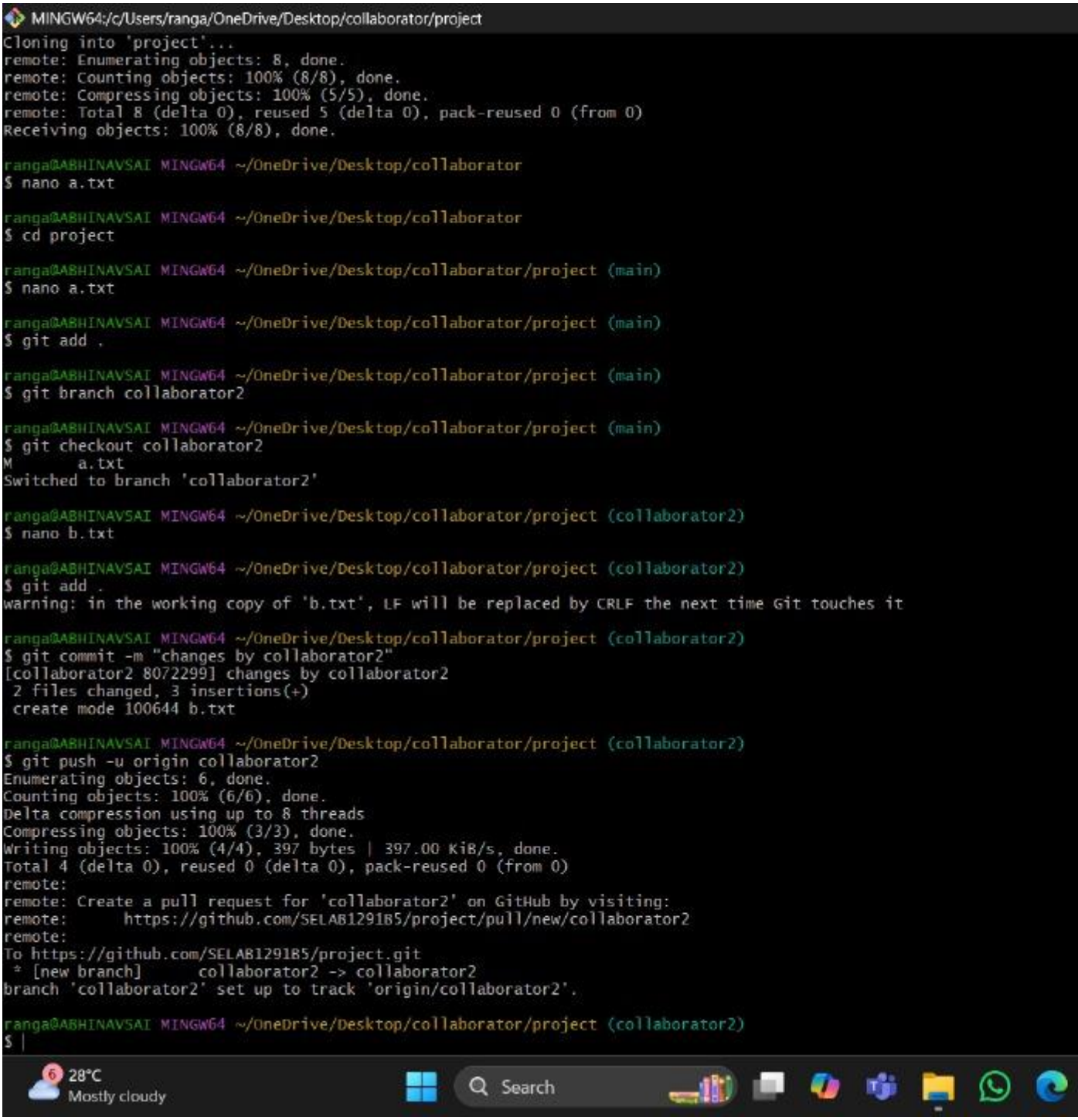
✅ 7. Keep Your Branch Updated (has to be done by owner collaborator 1)

Before making new changes:
In git bash
git checkout main
git pull origin main
git checkout feature/your-new-feature
git merge main

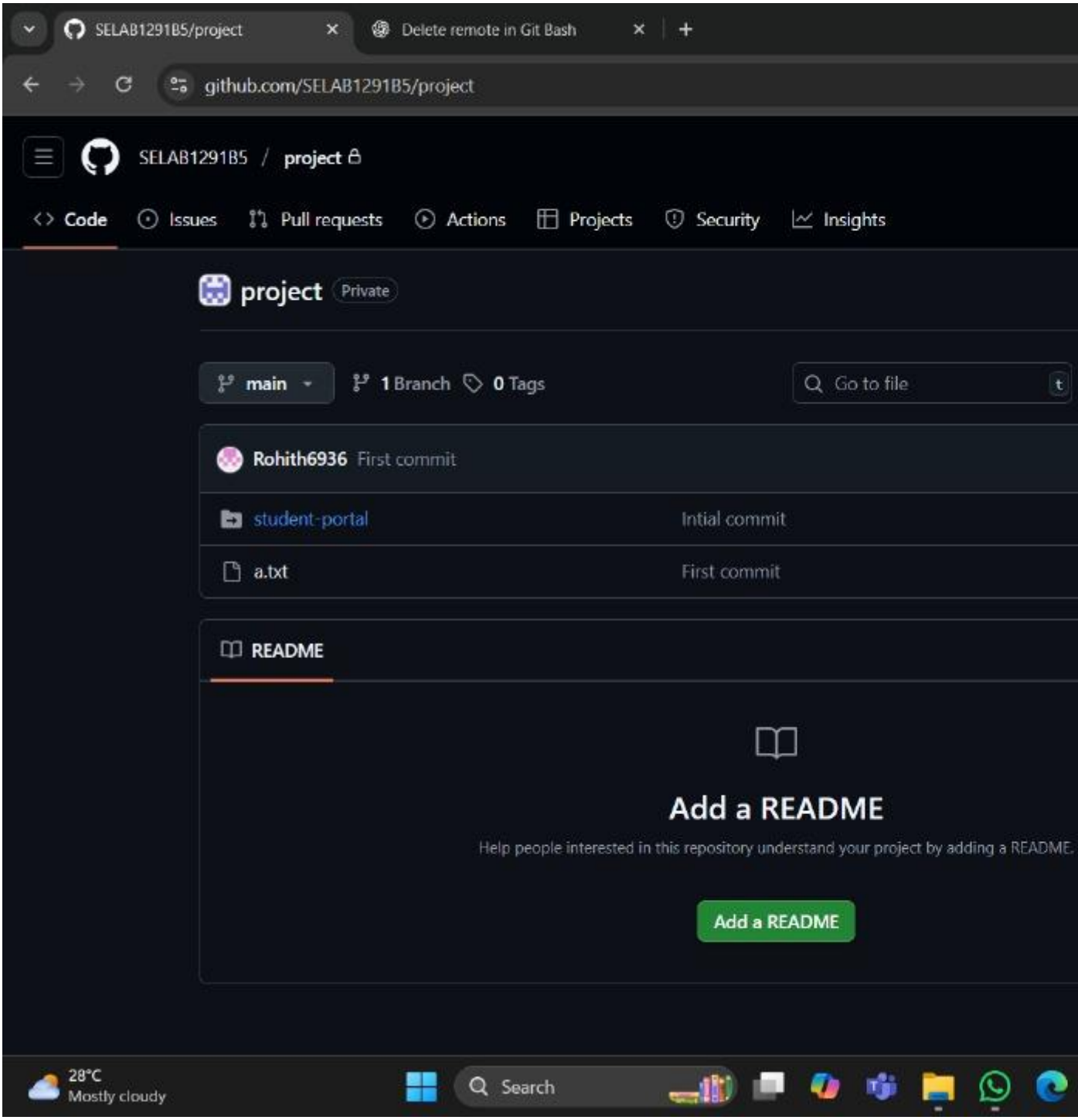
method-1:


MINGW64:/c/Users/ranga/OneDrive/Desktop/collaborator
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git config --global user.name "Abhinav560322"
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git config --global user.email "rangaabhinav56@gmail.com"
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ git clone https://github.com/SELAB1291B5/project.git
Cloning into 'project'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$
28°C
Mostly cloudy
Search

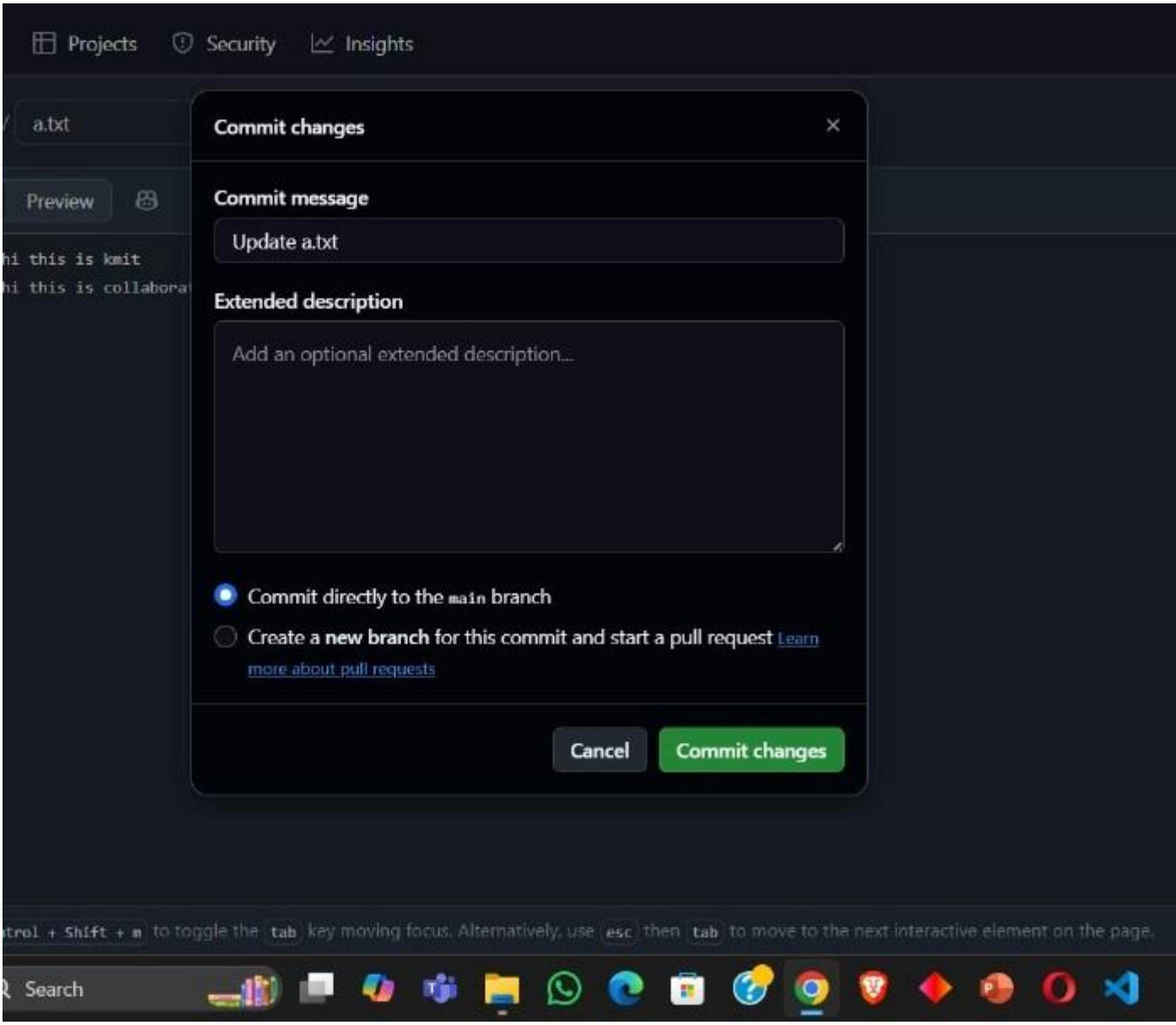

MINGW64:/c/Users/ranga/OneDrive/Desktop/collaborator/project
GNU nano 8.5 a.txt
hi this is kmit
hi this is collaborator2
hi from git bash
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo
AIRTELPP
+1.38%
Search

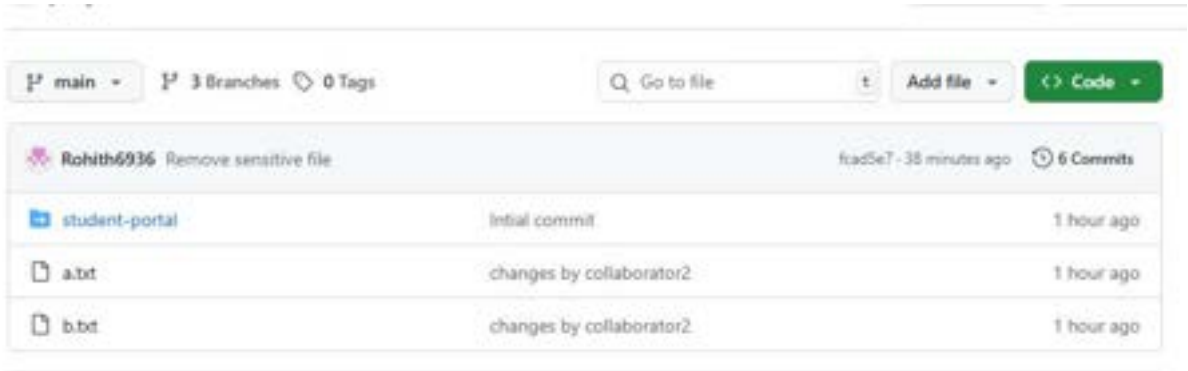

MINGW64:/c/Users/ranga/OneDrive/Desktop/collaborator/project
Cloning into 'project'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ nano a.txt
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator
$ cd project
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ nano a.txt
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git add .
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git branch collaborator2
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git checkout collaborator2
M a.txt
Switched to branch 'collaborator2'
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ nano b.txt
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git add .
warning: in the working copy of 'b.txt', LF will be replaced by CRLF the next time Git touches it
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git commit -m "changes by collaborator2"
[collaborator2 8072299] changes by collaborator2
2 files changed, 3 insertions(+)
create mode 100644 b.txt
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$ git push -u origin collaborator2
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 397 bytes | 397.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'collaborator2' on GitHub by visiting:
remote: https://github.com/SELAB1291B5/project/pull/new/collaborator2
remote:
To https://github.com/SELAB1291B5/project.git
* [new branch] collaborator2 -> collaborator2
branch 'collaborator2' set up to track 'origin/collaborator2'.
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (collaborator2)
$
28°C
Mostly cloudy
Search

Method -2:


SELAB1291B5/project
Delete remote in Git Bash
github.com/SELAB1291B5/project
SELAB1291B5 / project
Code
Issues
Pull requests
Actions
Projects
Security
Insights
project
Private
main
1 Branch
0 Tags
Go to file
Rohith6936 First commit
student-portal
Intial commit
a.txt
First commit
README
Add a README
Help people interested in this repository understand your project by adding a README.
Add a README
28°C
Mostly cloudy
Search

Update file:





Exercise on Fork

- If you're contributing to an existing project: (in public repository)
  Visit the repository page and click Fork

Note: Following steps needed if you forked from any public repository to contribute

1. Go to: `https://github.com/othername/awesome-project`
2. Click the **"Fork" button at the top-right of the page.**
3. GitHub will create a copy of that repository in your account:

https://github.com/your-username/awesome-project

4. Clone your forked repo: in bash

- ✓ **git clone https://github.com/your-username/awesome-project**
- ✓ **Make changes.**
- ✓ **Commit**
- ✓ **git push origin main**

5. Create a **pull request back to the original repo (`othername/awesome-project) to suggest your changes.**

- ✓ **Click at top Pull request**
- ✓ **Add title**
- ✓ **Create pull request**

Now you see on page message "NO conflicts with base branch"

- ✓ **Add comment any thing**
- ✓ **Click on comment**
- ✓ **Submit the pull request for review**

✅ 6. Review and Discuss Changes

- Collaborators can comment, request changes, or approve the PR(Pull Request)
- Make edits if needed and push again; they will update the PR automatically

✅ 7. Merge the Pull Request (has to be done by both collaborators)

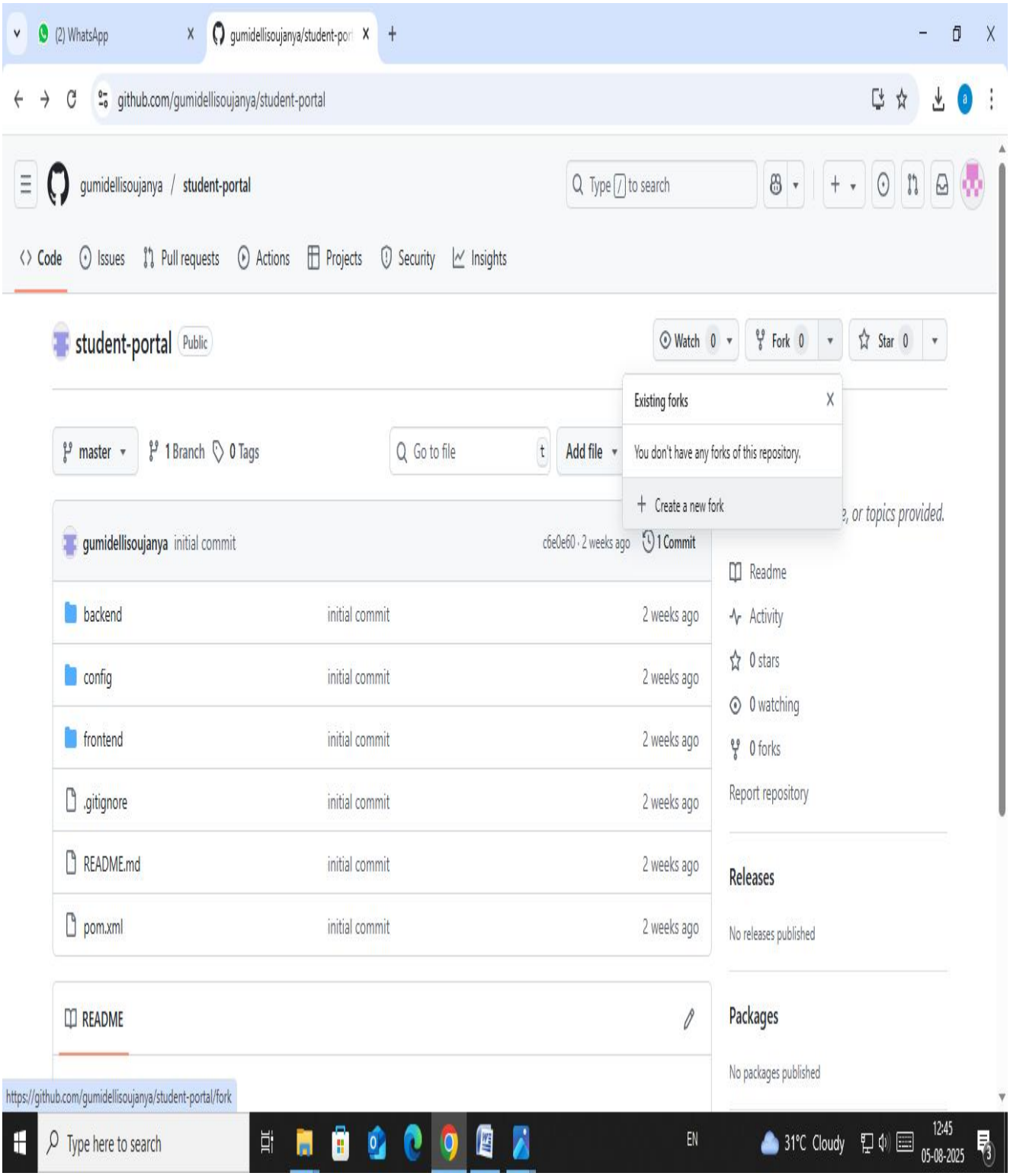
Once approved:

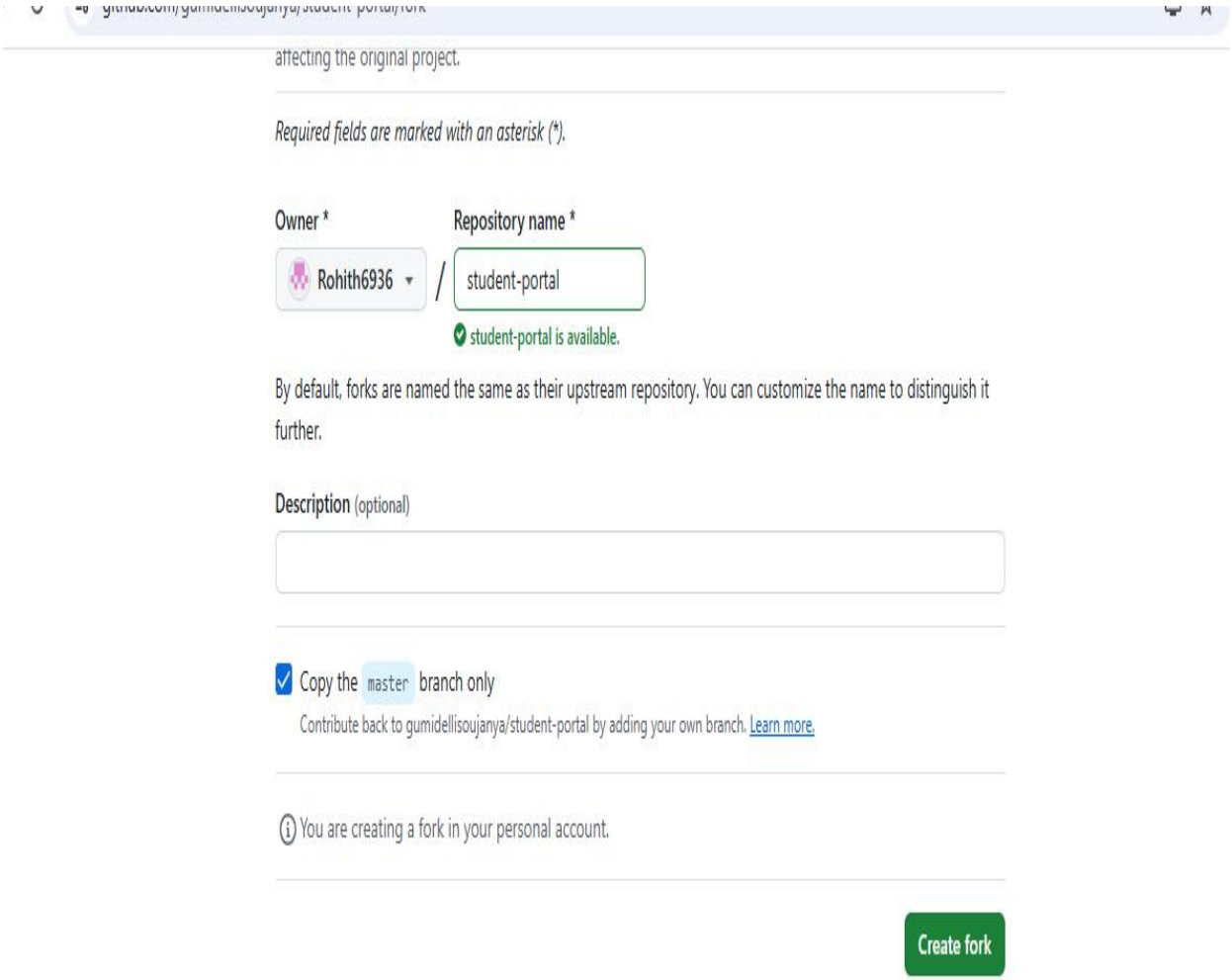
- The repo owner can click Merge pull request
- Or you can squash and merge, depending on the team's workflow

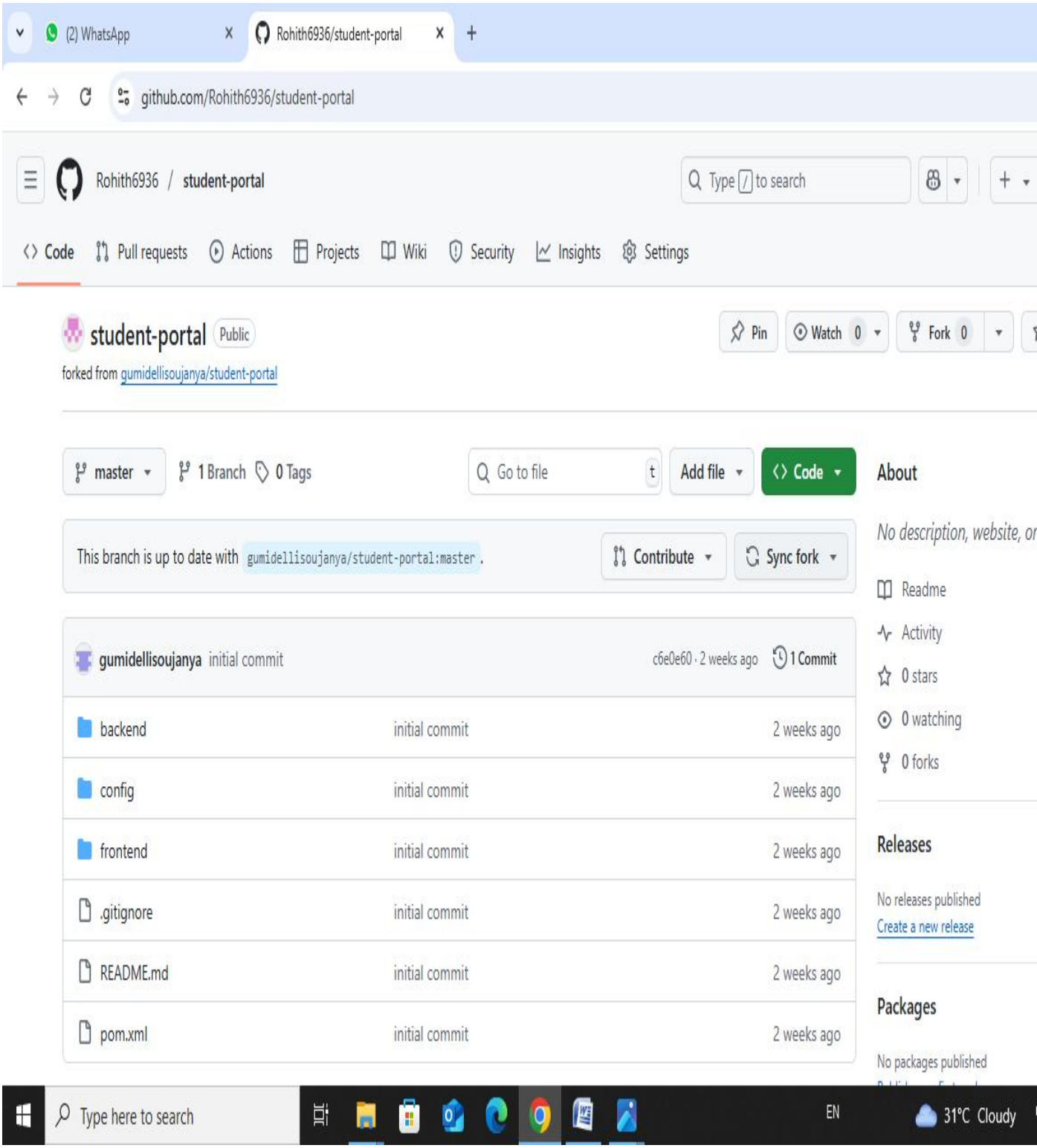
✅ 8. Keep Your Branch Updated (has to be done by owner collaborator 1)

Before making new changes:

In git bash
git checkout main
git pull origin main
git checkout feature/your-new-feature
git merge main
cd repository-name


(2) WhatsApp
gumidellisoujanya/student-por
github.com/gumidellisoujanya/student-portal
gumidellisoujanya / student-portal
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security
Insights
student-portal
Public
Watch 0
Fork 0
Star 0
Existing forks
You don't have any forks of this repository.
Create a new fork
master
1 Branch
0 Tags
Go to file
t
Add file
gumidellisoujanya initial commit
c6e0e60 · 2 weeks ago
1 Commit
backend initial commit 2 weeks ago
config initial commit 2 weeks ago
frontend initial commit 2 weeks ago
.gitignore initial commit 2 weeks ago
README.md initial commit 2 weeks ago
pom.xml initial commit 2 weeks ago
README
Readme
Activity
0 stars
0 watching
0 forks
Report repository
Releases
No releases published
Packages
No packages published
https://github.com/gumidellisoujanya/student-portal/fork
Type here to search
EN
31°C Cloudy
12:45
05-08-2025


affecting the original project.
Required fields are marked with an asterisk (*).
Owner *
Repository name *
Rohith6936
/
student-portal
student-portal is available.
By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.
Description (optional)
Copy the master branch only
Contribute back to gumidellisoujanya/student-portal by adding your own branch. Learn more.
You are creating a fork in your personal account.
Create fork



2. **<u>To resolve conflicts when collaborating on same part of code.</u>**

Resolving conflicts when collaborating with GitHub (or Git in general) is a normal part of team development. Conflicts happen when two people make changes to the same part of the code, and Git doesn't know which version to keep.

✅ Steps to Resolve a Conflict

1. See Which Files Are in Conflict (Say f1.txt)

In git bash
git status
Conflicted files will be marked like:
In git bash
both modified: (Say f1.txt)

---

3. **Open the File and Look for Conflict Markers by clicking on**

- ✓ Click on **Resolve Conflict**

Git will insert conflict markers in the file like this:

```
def greet():
<<<<<<< HEAD
   print("Hello from First collaborator ")
=======
   print("Hello from second collaborator ")
>>>>>>> feature/second collaborator branch
```

This shows the two conflicting changes:

- HEAD is your version
- The section below ======= is the other branch's version or second collaborator branch version

---

3. Edit the File to Fix the Conflict

- ✓ Choose one version or combine them:

```
def greet():
    print("Hello from First collaborator ")
    print("Hello from second collaborator ")
```

- ✓ Then **delete** all the **conflict markers** (<<<<<<<, =======, >>>>>>>).

---

4. Mark the Conflict as Resolved

Once you've edited and saved the file:

- ✓ git add f1.txt

---

5. Complete the Merge or Pull

- ✓ git commit # Only needed if you're merging

Or if you're rebasing:

- ✓ git rebase --continue

⚠ If You Want to Abort the Merge

If it gets messy and you want to cancel:
git merge --abort

Here below screen shot shows conflict raised when collaborator 1 wanted to merge changes to same line that already collaborator 2 added the line of code at that line in same file but in their own branch.

Example :

Merge Conflict – Changes made to a file by two different users leads to merge conflict as below.

Say I am collaborator 2, now

- ➔ **Accept invitation from collaborator 1**
- ➔ **Next goes to Shared repository by collaborator 1 (say abcrepo)**
- ➔ **Now open Git bash and clone the shared repo as below steps**

Step 1: first connect to this "abcrepo" to local repository by clone

- ✓ Now change directory to "abcrepo"
- ✓ Switch to branch feature-abc
- ✓ Now add line in README.md file
- ✓ Now git add, commit
- ✓ Now see status and push to remote branch feature-abc
- ✓ Now go to Github and refresh or click on "abcrepo", you can see README.md

Step 2: Now collaborator 2 adding line in README.md, save, git add, commit, push. No conflicts

Step 3: Also collaborator 1 adding line in README.md at same line collaborator 2 added, then git add, commit, push gives **merge conflict.**

NOTE IMPORTANT :

Now collaborator 2 has to give git pull command then open conflicting file to see conflict markers.

Now Edit the File to Fix the Conflict by keeping either or both collaborator changes and remove conflict markers. Add file, commit, and push.

3. To create and apply patch

A patch in Git is a text file that represents changes between two sets of files, or commits. It's essentially the output of the git diff command, packaged in a format that can be applied to another set of files.

Steps:

1. Goto github and take public project url
2. Clone into git bash
3. Now open file and edit it, add, commit

4. Run git log

5. Create patch with commit hash code as:

   git format-patch -1 commit-hash-code

   This creates 0001-commit-from-cloned-person.patch file.

Note: git format-patch -1 <commit-hash>

- -1 means "create a patch from 1 commit"
- This creates a .patch file for a single commit.
- Example:

  git format-patch -1 abc1234
- This creates a file like 0001-Your-commit-message.patch

To create patches for the **last 3 commits**: git format-patch -3

or

Create patches from multiple commits
git format-patch <base_commit>..HEAD

6. Get the path of above patch file and email or what's up to the remote owner of file.

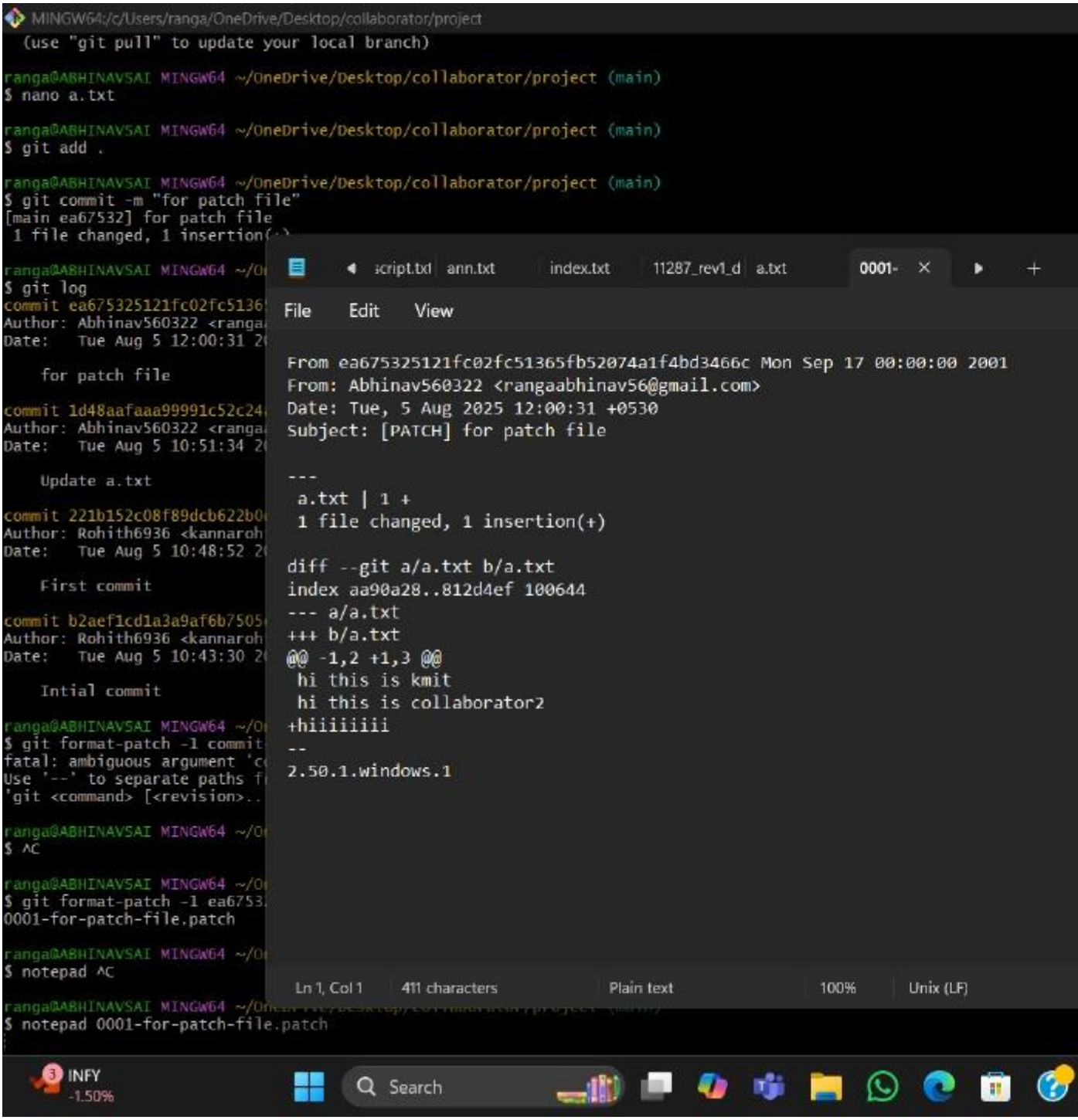
7. Once the remote owner gets the patch file 0001-commit-from-cloned-person.patch next the owner need to apply patch file in his git bash using git command below:

   Syntax:
   git apply my-changes.patch

   Or, if it's a patch made by git format-patch, use:

   git am 0001-Your-commit-message.patch

   ✓ **git apply 0001-Your-commit-message.patch**



Command:

```
ranga@ABHINAVSAI MINGW64 ~/OneDrive/Desktop/collaborator/project (main)
$ git format-patch -1 ea675325121fc02fc51365fb52074a1f4bd3466c
0001-for-patch-file.patch
```

Conclusion:

The effective use of Git in student collaborative projects promotes **teamwork, organization, and clear communication**, allowing students to manage code efficiently and track contributions accurately. By following structured Git workflows and best practices—such as regular commits, feature branching, and resolving conflicts early—students gain valuable experience in real-world development processes. This not only enhances their technical skills but also prepares them for **professional teamwork in software engineering**, where version control and collaboration are essential. Ultimately, mastering Git empowers students to deliver cleaner, more reliable code and succeed in collaborative environments.

LAB ACTION PLAN FOR WEEK 4

Objective:

To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications. src/

```
├── main/
│   ├── java/          → Application source code
│   └── resources/     → Configuration files like config.properties
└── test/
    ├── java/          → Unit test source code
    └── resources/     → Test resources
```

The compiled files and reports are generated in the **`target/`** directory after a successful build.

2. Steps to Perform Maven Build and Testing

----mvn clean install

- **`clean`:** Deletes the previous build (**`target/`** folder)
- **`install`:** Builds, tests, and installs the package to local **`.m2`** repository

After execution:

- **`target/`** folder contains compiled **`.class`** files and the final **`.jar`** or **`.war`**

- Artifact is stored in:
  `~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In **`pom.xml`**:

<dependency>

<groupId>com.google.code.gson</groupId>

<artifactId>gson</artifactId>

<version>2.10</version>

</dependency>

After running:

mvn clean install

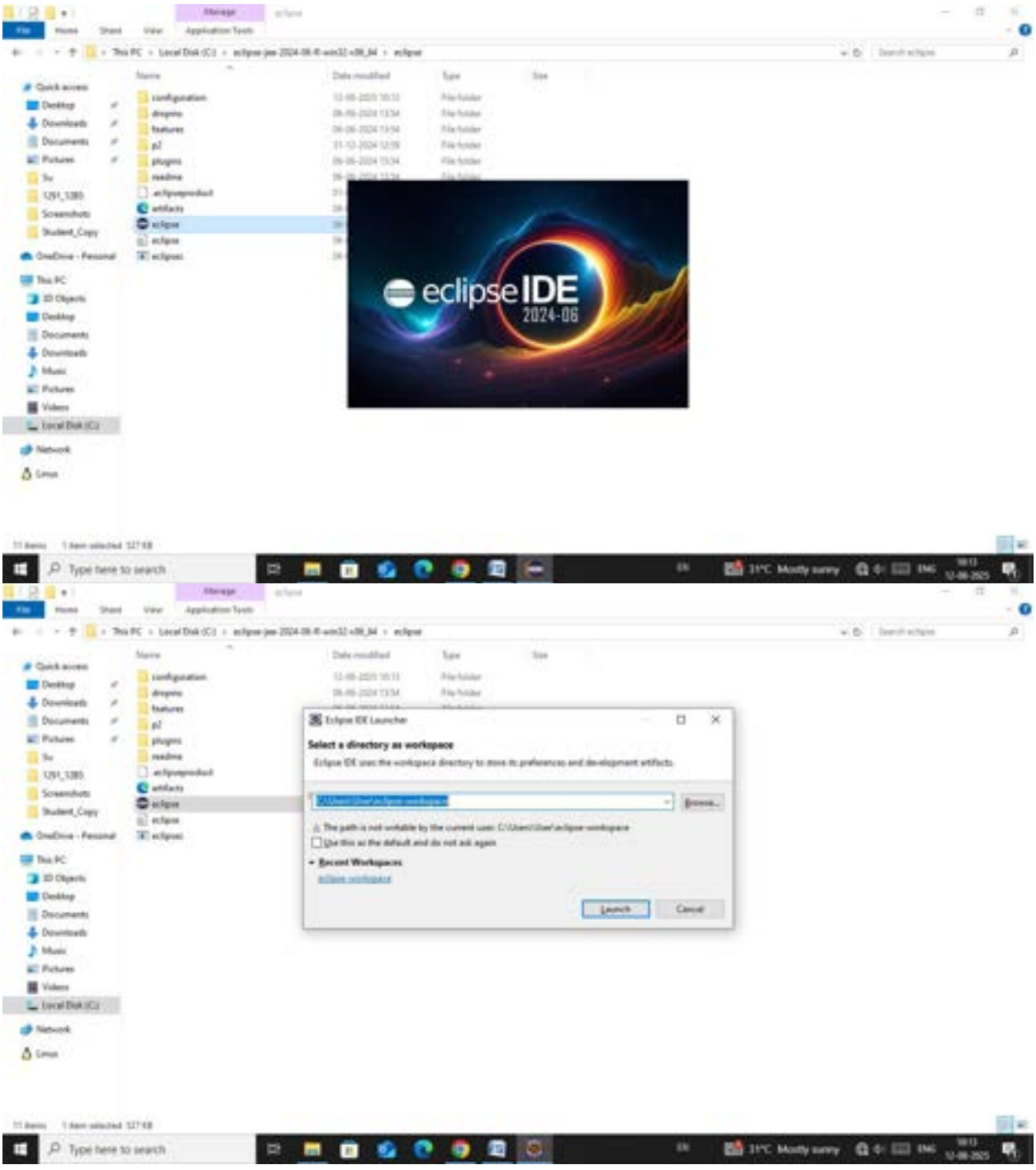
Check:

- Gson JAR is downloaded to **`.m2/repository/com/google/code/gson/gson/`**
- If version is wrong or not found → **`BUILD FAILURE`** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└── 1.1. Launch Eclipse workspace



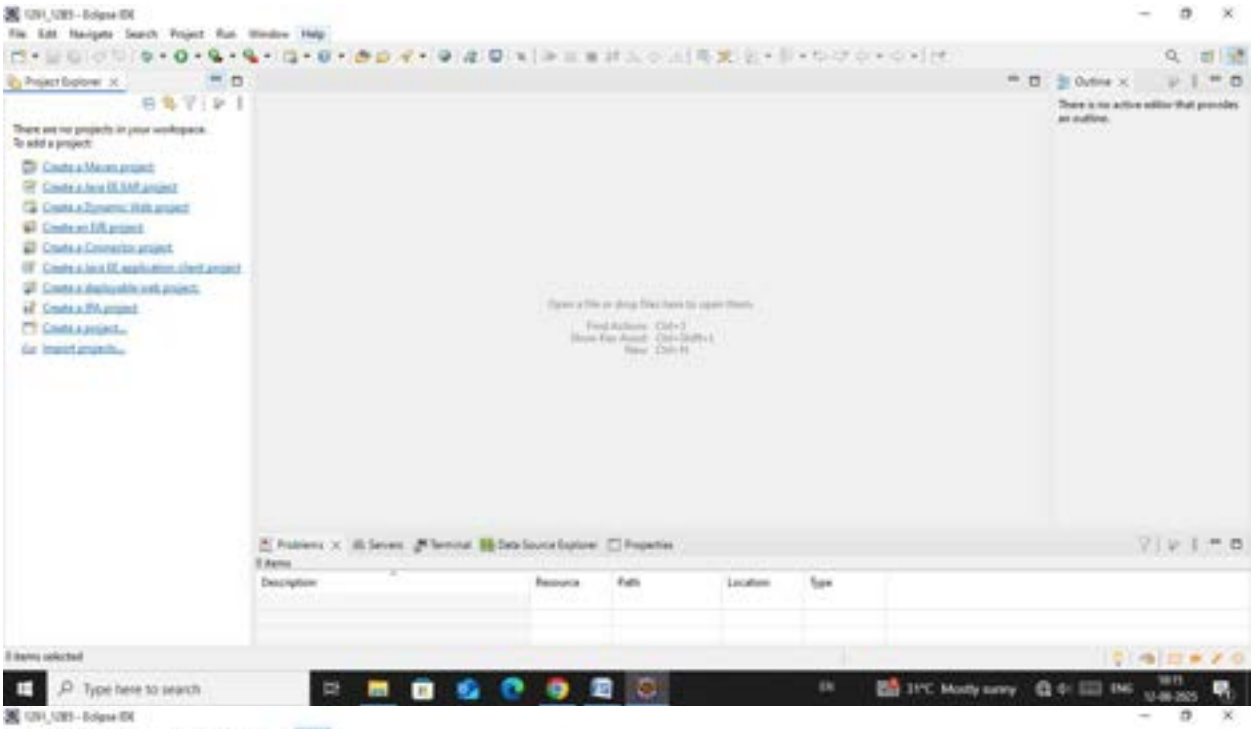
Step 2. Install Maven Plugin (if not installed)

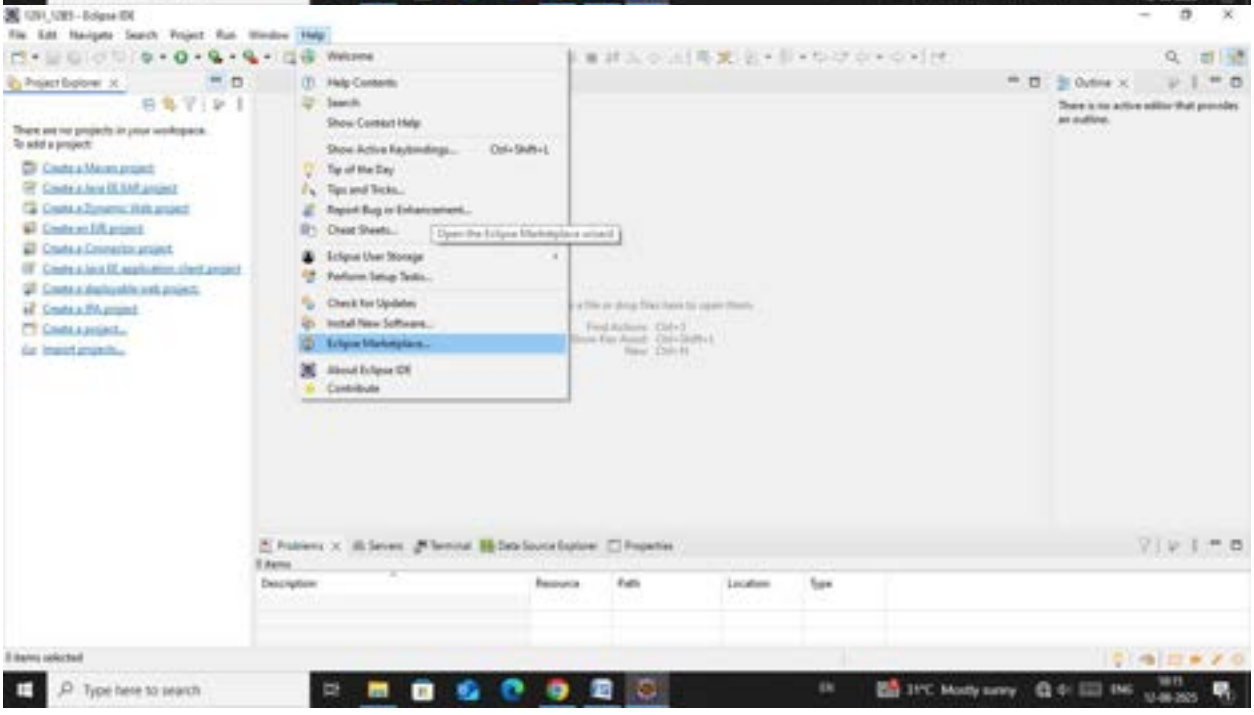
└── 2.1. Go to "Help" in the top menu

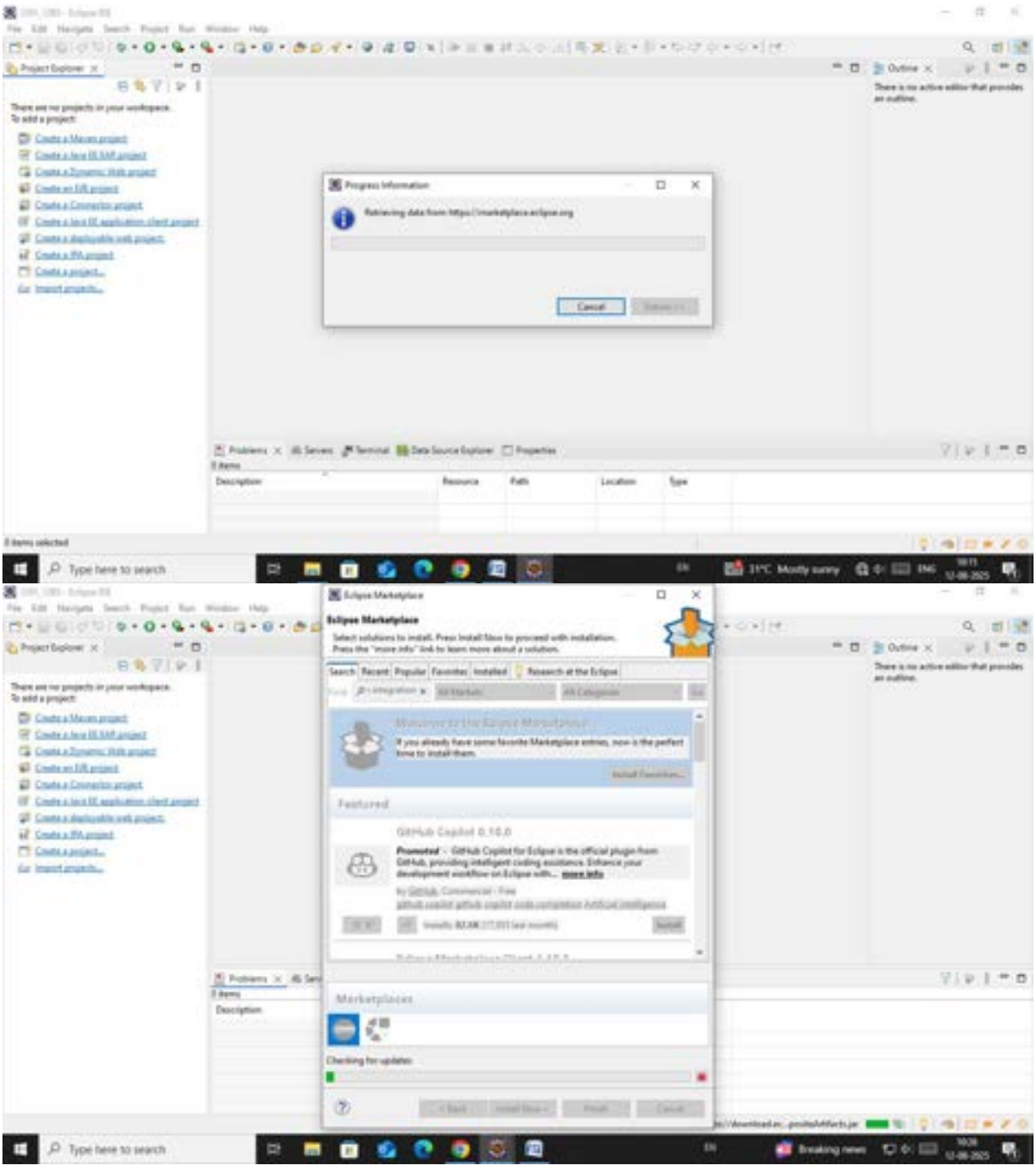
└── 2.1.1. Click "Eclipse Marketplace"

└── 2.1.2. Search for "Maven Integration for Eclipse"

└── 2.1.3. Install the plugin if not already installed







Step 3. Create a New Maven Project

└── 3.1. File -> New -> Project...

└── 3.1.1. Expand "Maven"

└── 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

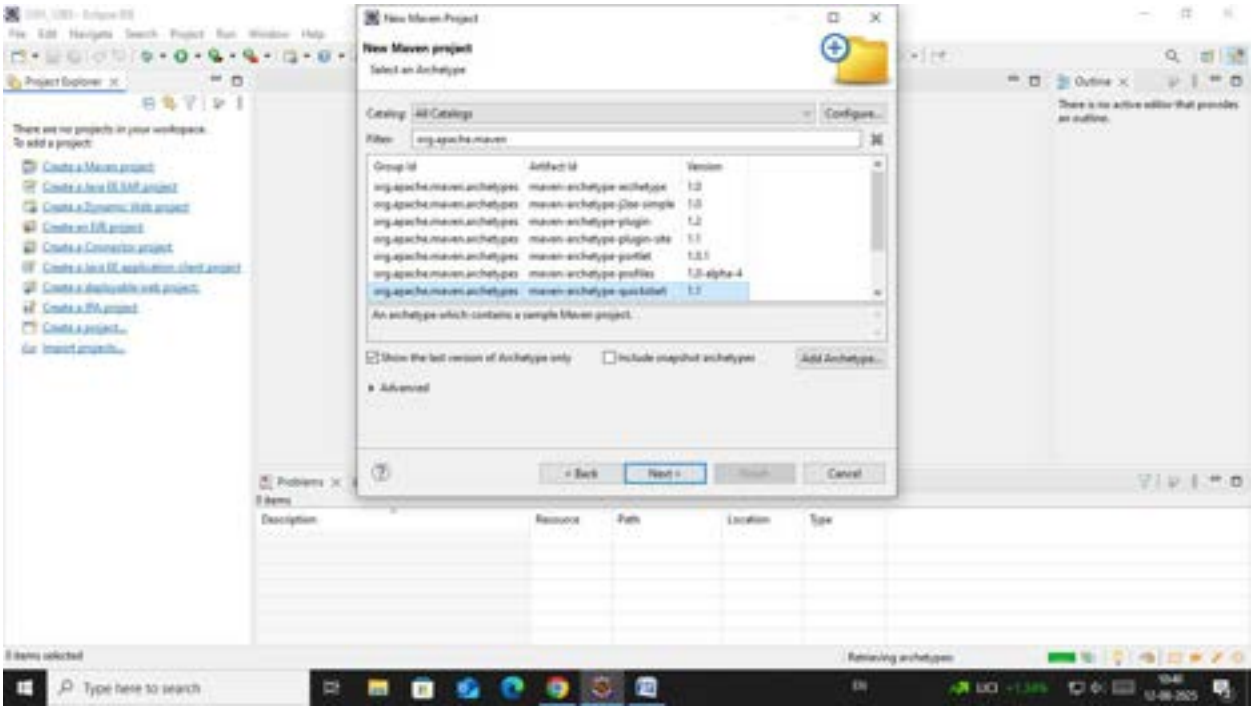
└── 4.1. Select workspace location (default or custom)

└── 4.2. Click "Next"

Step 5. Choose Maven Archetype

└── 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")

└── 5.2. Click "Next"



Step 6. Define Project Metadata

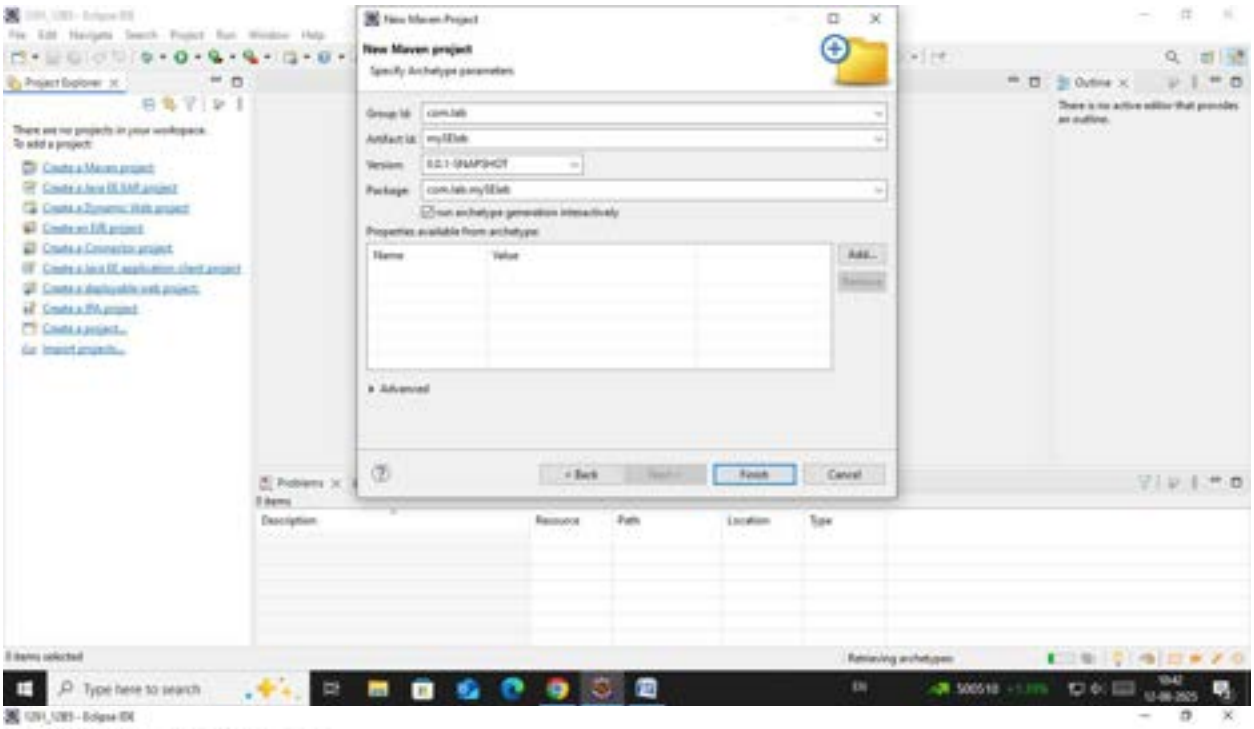
└── 6.1. Group ID: (e.g., com.example)

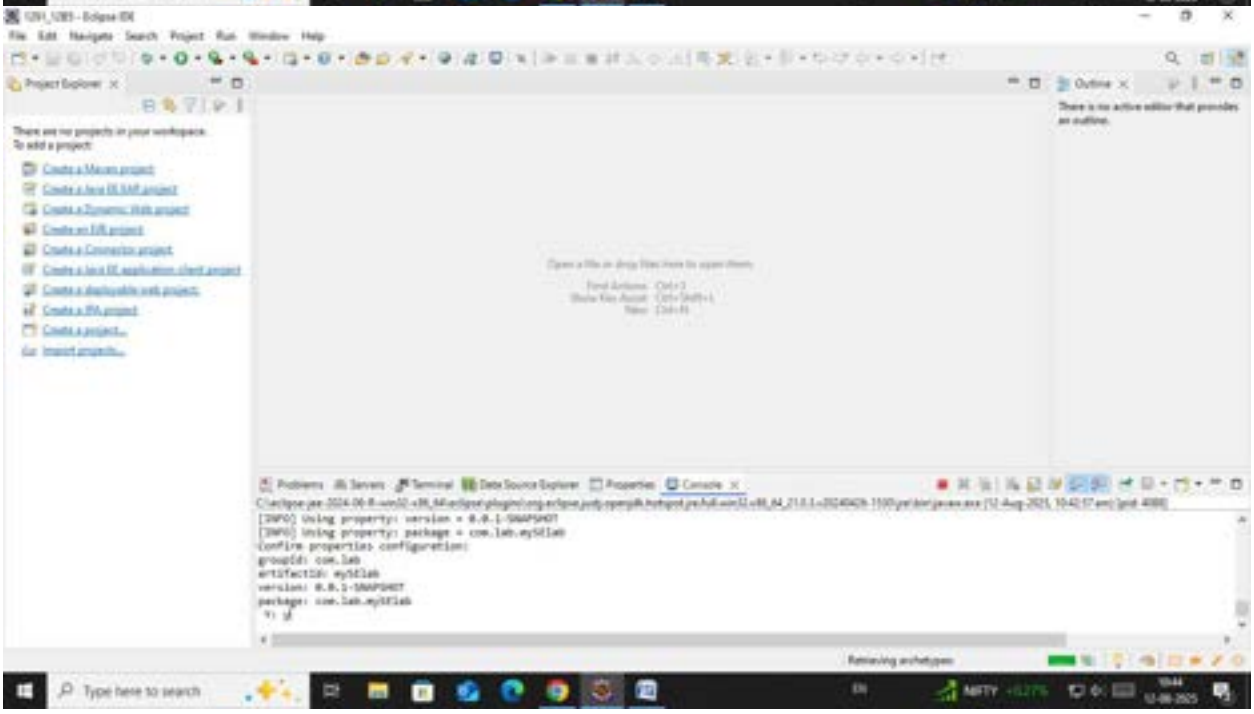
└── 6.2. Artifact ID: (e.g., my-maven-project)

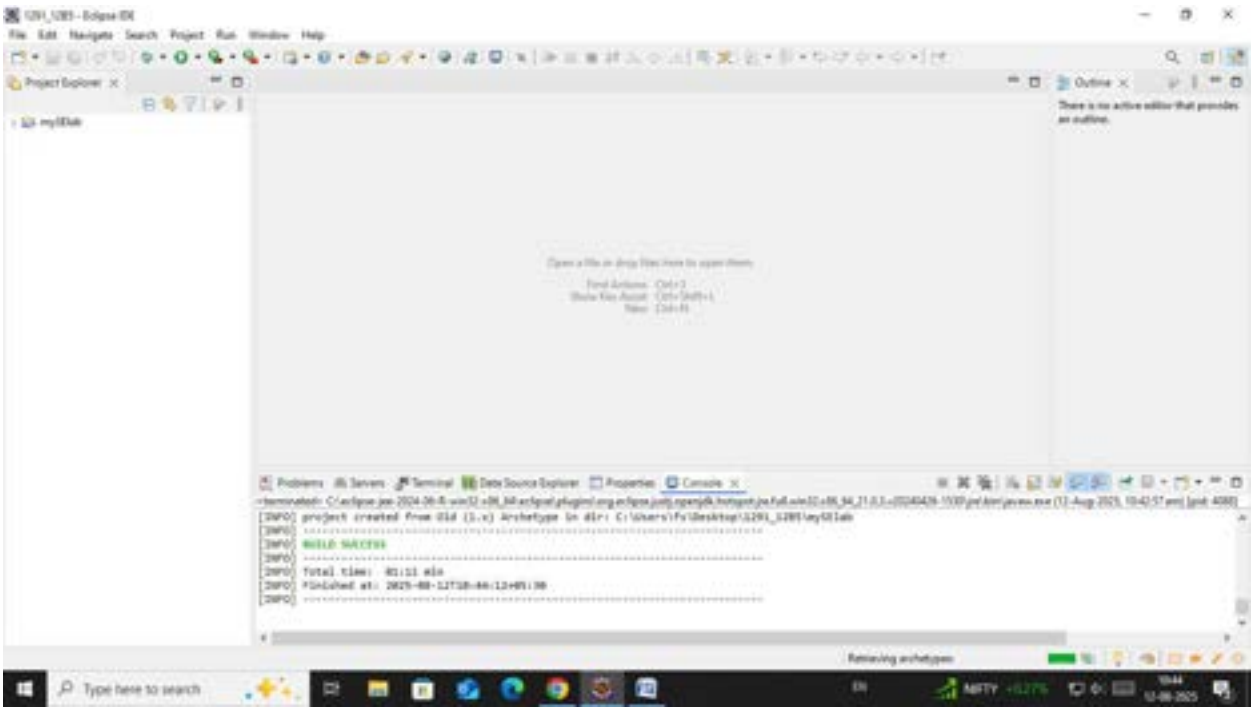
└── 6.3. Version: (default is usually fine)

└── 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.


New Maven Project
New Maven project
Specify Archetype parameters
Group Id: com.lab
Artifact Id: mySlab
Version: 0.0.1-SNAPSHOT
Package: com.lab.mySlab
Properties available from archetype:
Name
Value
Add...
Remove
Advanced
< Back
Next >
Finish
Cancel
There are no projects in your workspace.
To add a project:
Create a Maven project
Create a Java EE EAR project
Create a Dynamic Web project
Create an EJB project
Create a Connector project
Create a Java EE application client project
Create a deployable web project
Create a JPA project
Create a project...
Import projects...
Project Explorer
Outline
There is no active editor that provides an outline.
Problems
Description
Resource
Path
Location
Type
0 items selected
Retrieving archetypes
Type here to search


There are no projects in your workspace.
To add a project:
Create a Maven project
Create a Java EE EAR project
Create a Dynamic Web project
Create an EJB project
Create a Connector project
Create a Java EE application client project
Create a deployable web project
Create a JPA project
Create a project...
Import projects...
Project Explorer
Outline
There is no active editor that provides an outline.
Problems
Servers
Terminal
Data Source Explorer
Properties
Console
[INFO] Using property: version = 0.0.1-SNAPSHOT
[INFO] Using property: package = com.lab.mySlab
Confirm properties configuration:
groupId: com.lab
artifactId: mySlab
version: 0.0.1-SNAPSHOT
package: com.lab.mySlab
Y: :
Retrieving archetypes
Type here to search



Step 7. Maven Project Created

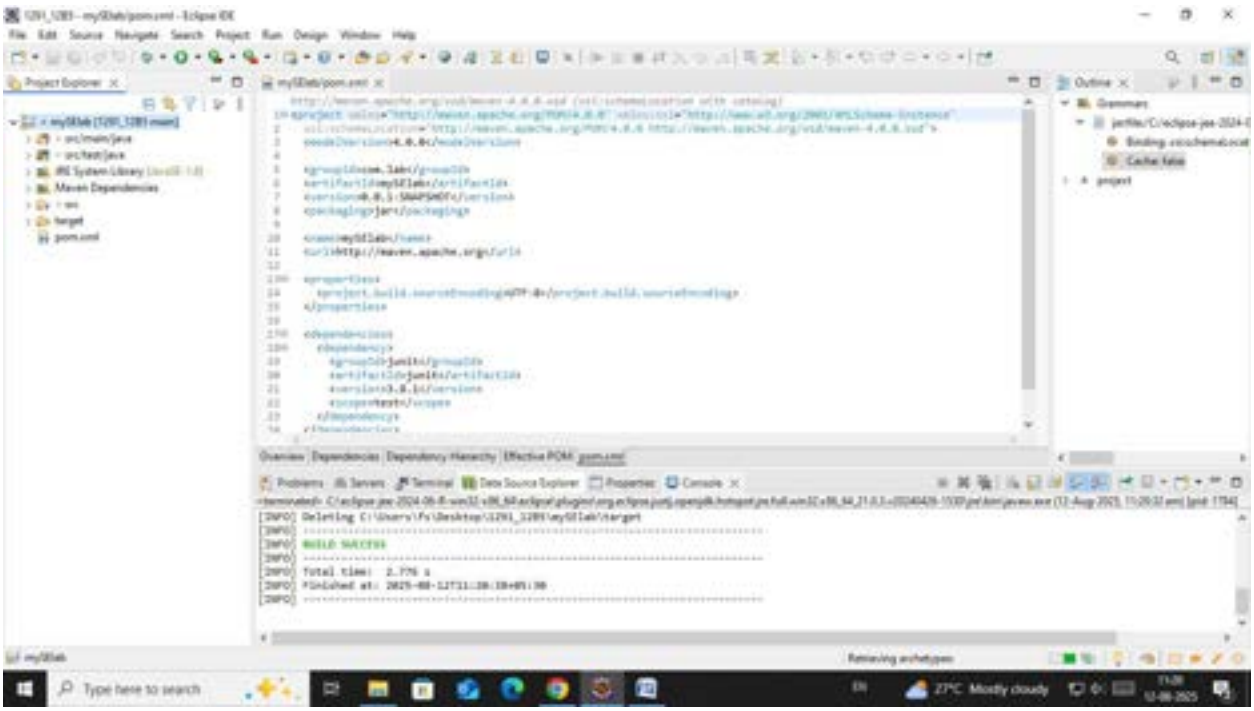
└── 7.1. Project structure is generated with a standard Maven layout

└── 7.2. Includes:

└── src/main/java (for Java source code)

└── src/test/java (for test code)

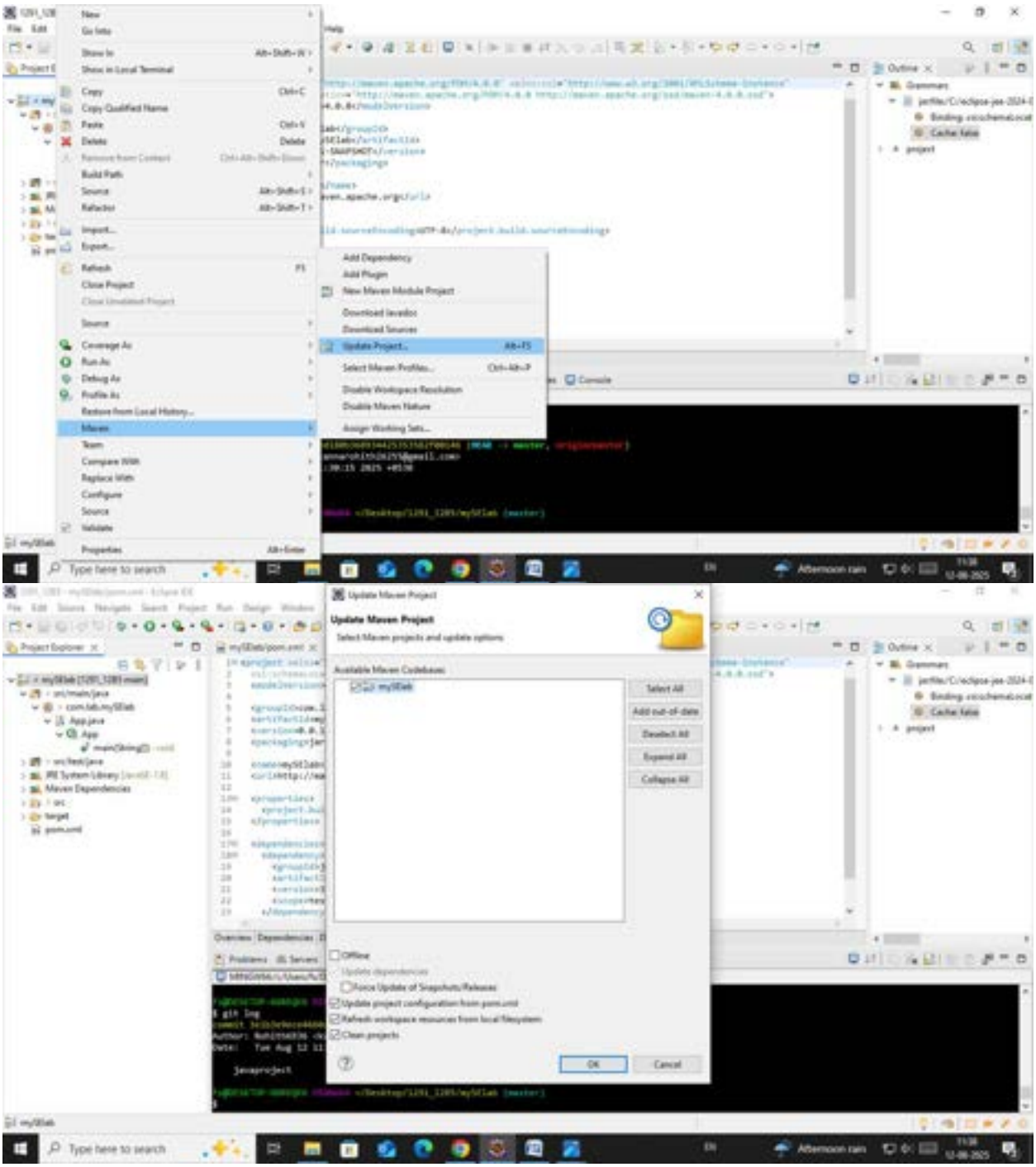
└── pom.xml (Maven configuration file)



Step 8. Update Project Settings (if needed)

└── 8.1. Right-click on the project -> Maven -> Update Project...

└── 8.2. Ensure dependencies are up to date



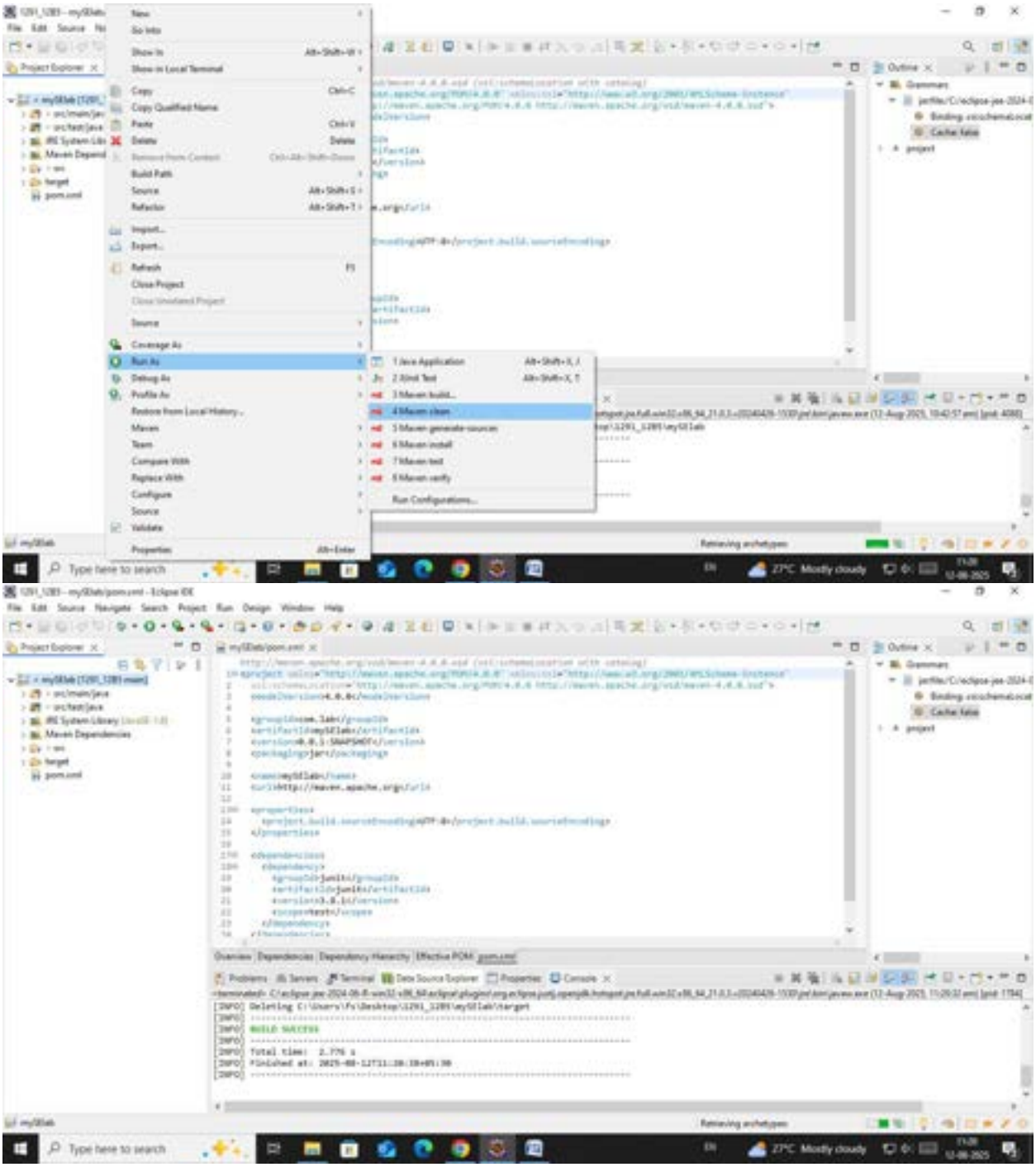
Step 9. Build and Run Maven Project

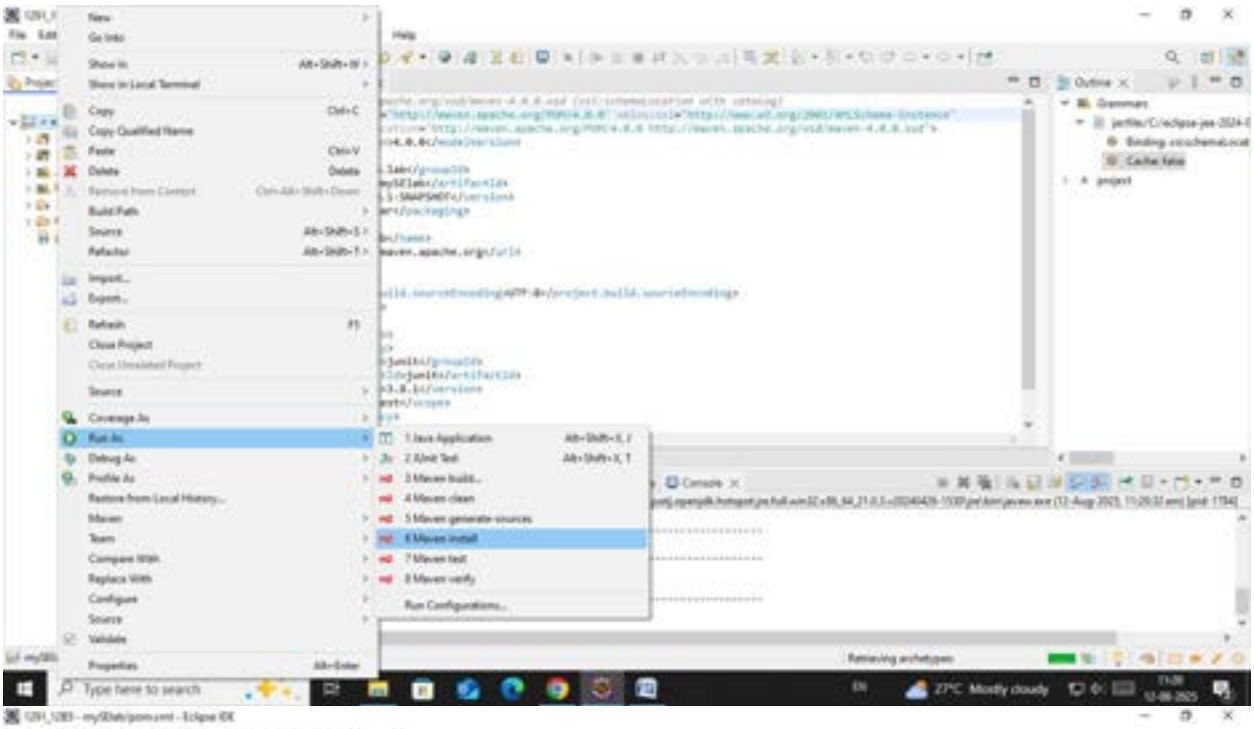
└── 9.1. Right-click on App.java -> Run As -> Maven Clean

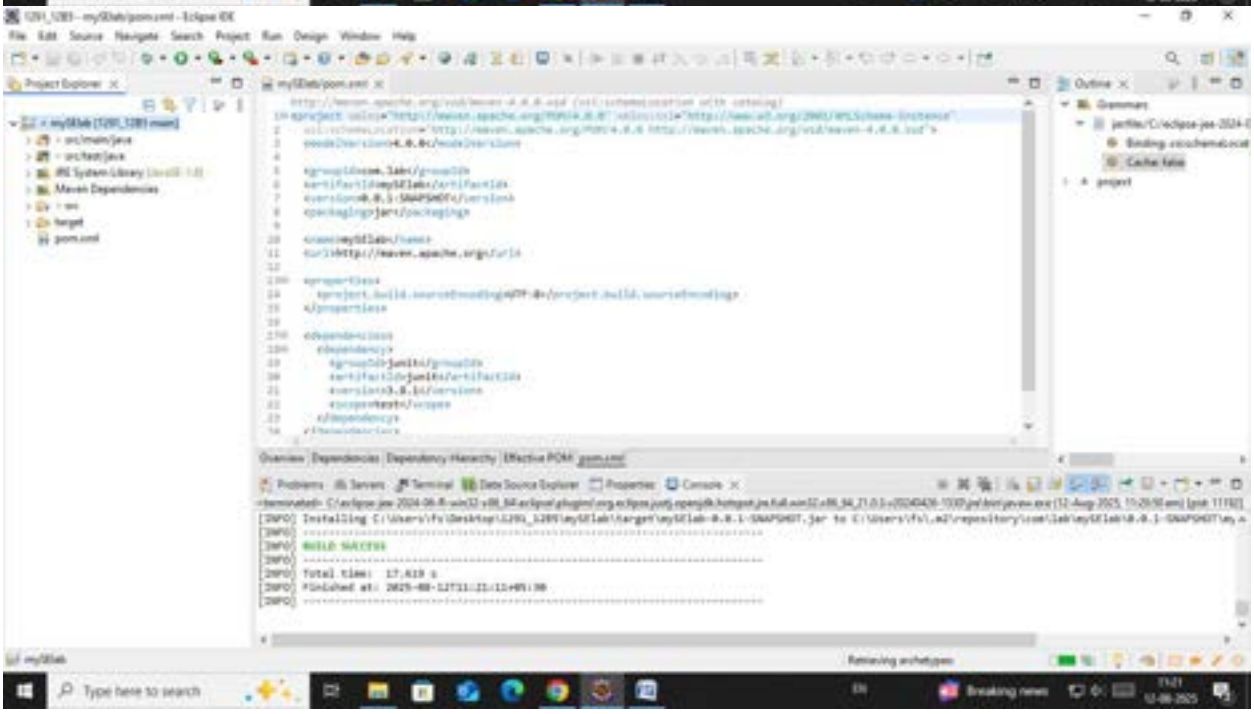
└── 9.1.1. Right-click on App.java -> Run As -> Maven Install

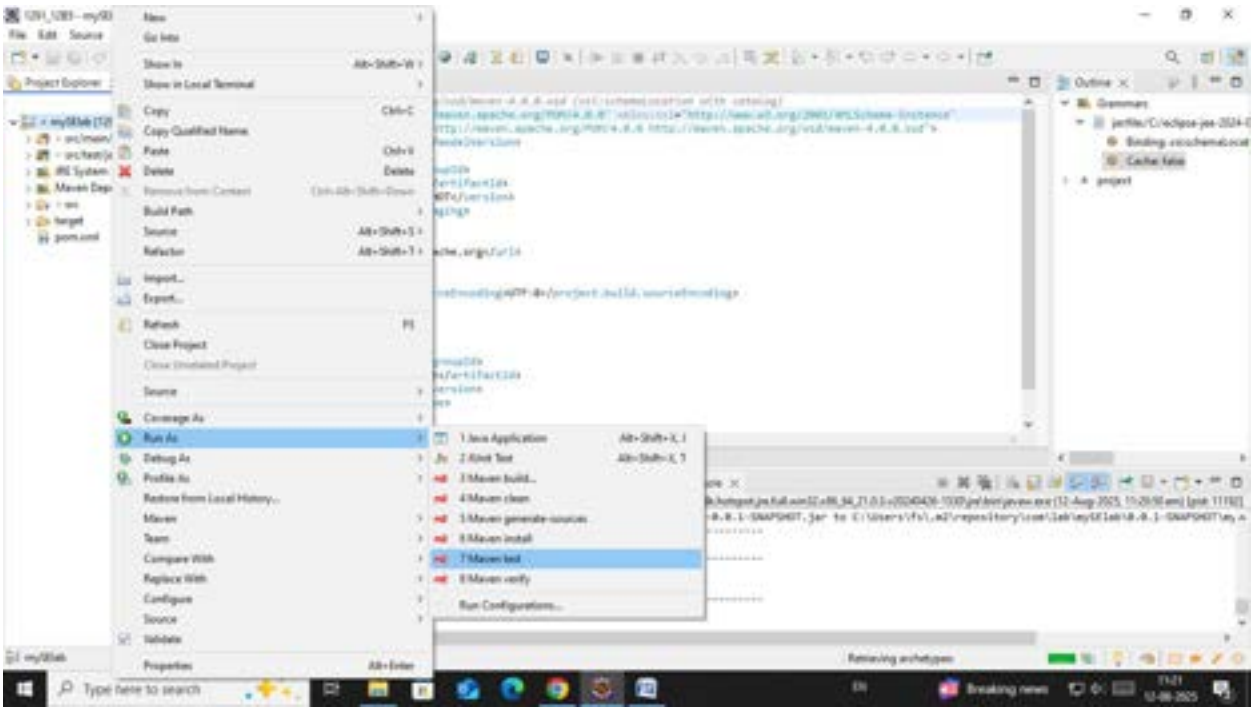
└── 9.1.2. Right-click on App.java -> Run As -> Maven Test

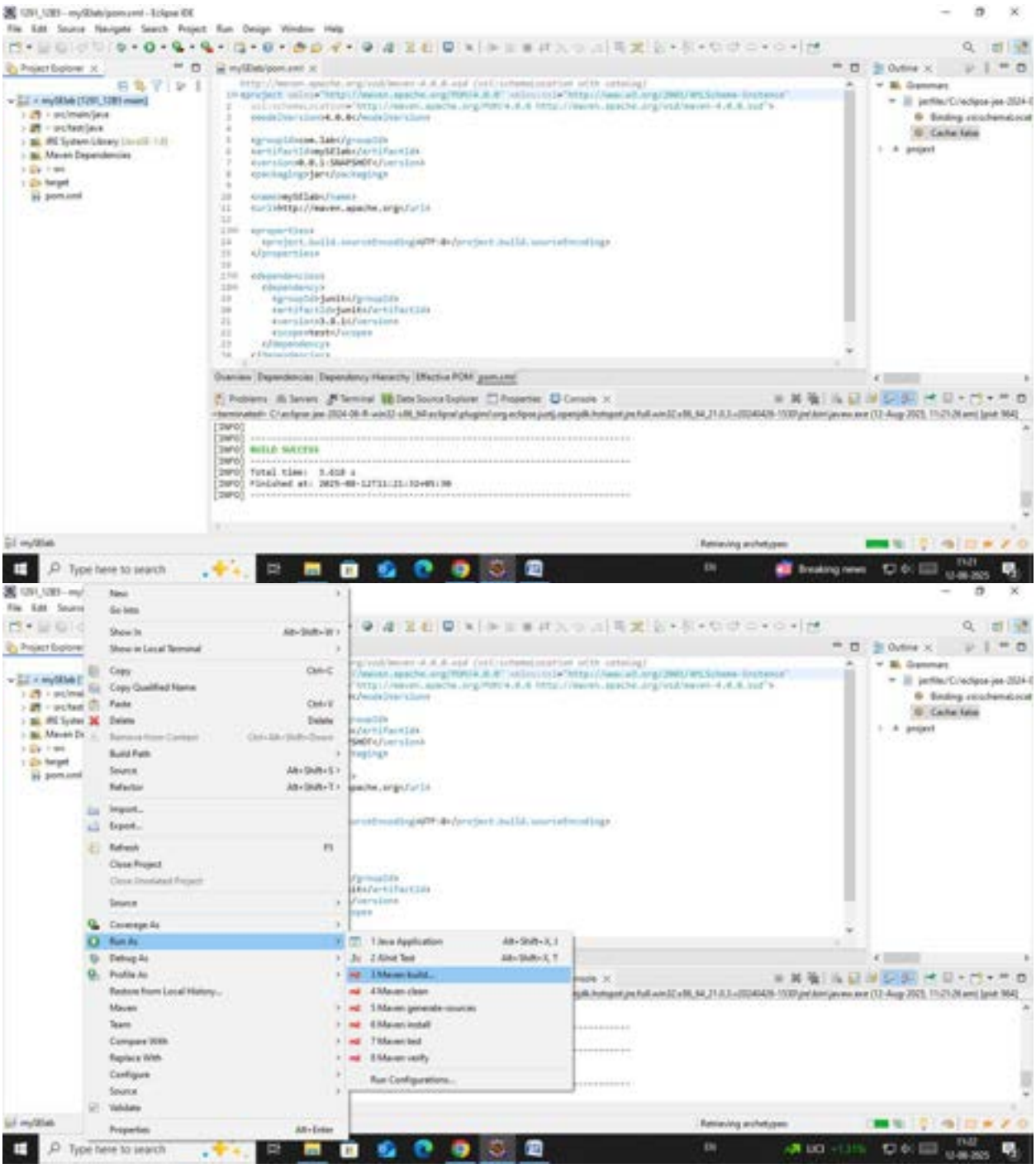
└── 9.1.3. Right-click on App.java -> Run As -> Maven Build




New
Go Into
Show In
Show in Local Terminal
Copy
Copy Qualified Name
Paste
Delete
Remove from Context
Build Path
Source
Refactor
Import...
Export...
Refresh
Close Project
Close Unrelated Projects
Source
Coverage As
Run As
Debug As
Profile As
Restore from Local History...
Maven
Team
Compare With
Replace With
Configure
Source
Validate
Properties
1 Java Application
2 JUnit Test
3 Maven build...
4 Maven clean
5 Maven generate-sources
6 Maven install
7 Maven test
8 Maven verify
Run Configurations...
Outline
Grammars
Cache Ratio
project
Console
Retrieving archetypes
Type here to search
27°C Mostly cloudy


Project Explorer
myGlab [120_128] main]
src/main/java
src/test/java
JRE System Library
Maven Dependencies
src
target
pom.xml
Outline
Grammars
Cache Ratio
project
Overview
Dependencies
Dependency Hierarchy
Effective POM
pom.xml
Problems
Servers
Terminal
Data Source Explorer
Properties
Console
[INFO] Installing C:\Users\fs\Desktop\120_128\myGlab\target\myGlab-0.0.1-SNAPSHOT.jar to C:\Users\fs\.m2\repository\com\lab\myGlab\0.0.1-SNAPSHOT\my
[INFO] BUILD SUCCESS
[INFO] Total time: 17.619 s
[INFO] Finished at: 2025-08-12T11:21:11+05:30
Retrieving archetypes
Type here to search
Breaking news

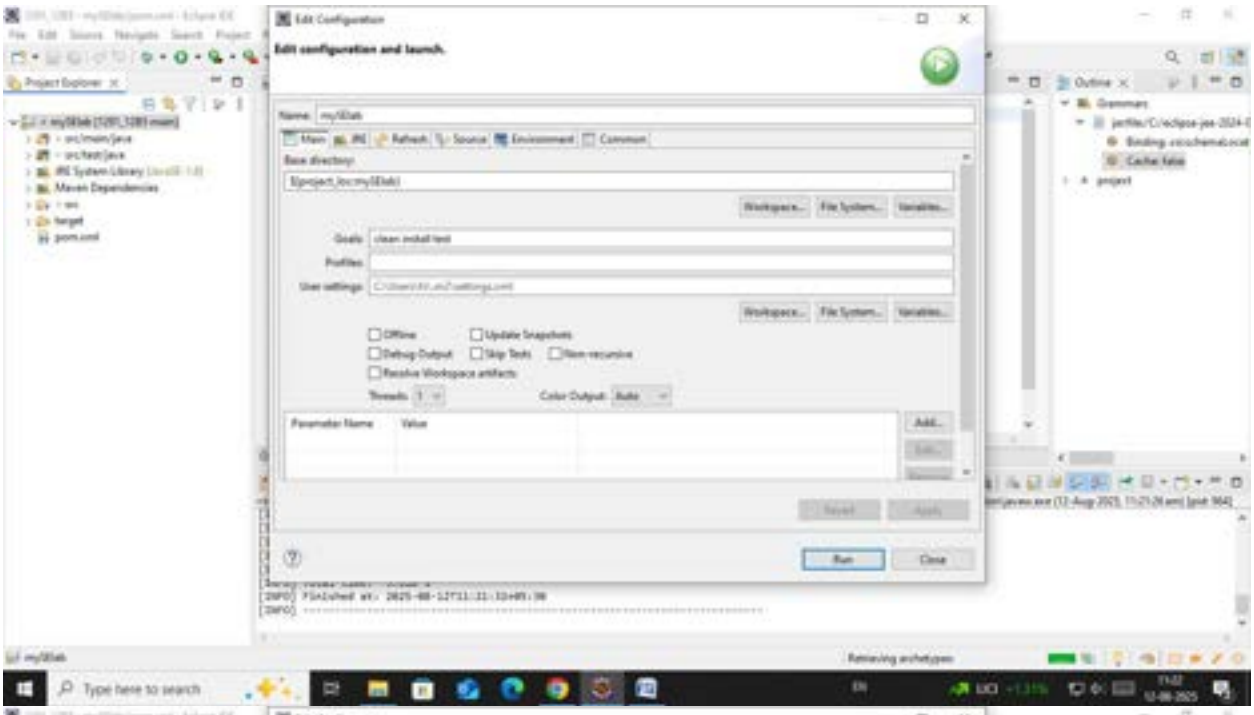

New
Go Into
Show In
Show in Local Terminal
Copy
Copy Qualified Name
Paste
Delete
Build Path
Source
Refactor
Import...
Export...
Refresh
Close Project
Coverage As
Run As
Debug As
Profile As
Restore from Local History...
Maven
Team
Compare With
Replace With
Configure
Source
Validate
Properties
1 Java Application
2 JUnit Test
3 Maven build...
4 Maven clean
5 Maven generate-sources
6 Maven install
7 Maven test
8 Maven verify
Run Configurations...
Outline
Grammars
Cache Xsds
project
Retrieving archetypes
Type here to search
Breaking news

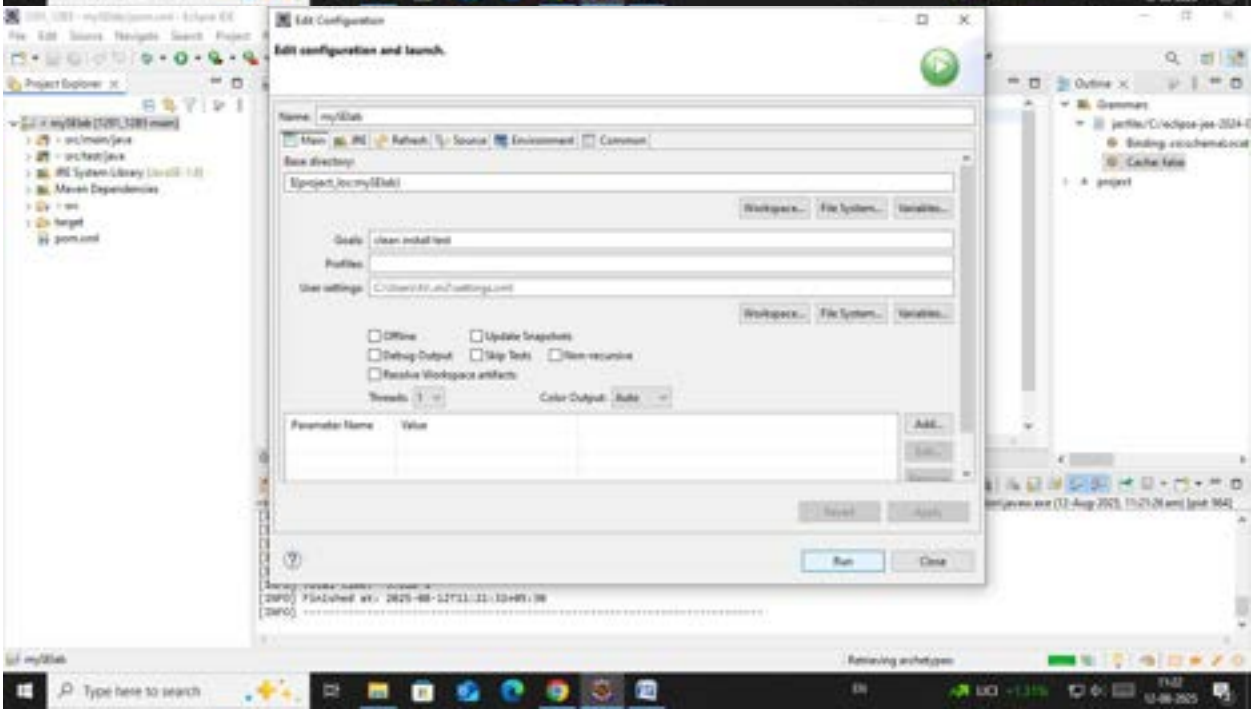


Step 10. In the Maven Build dialog:

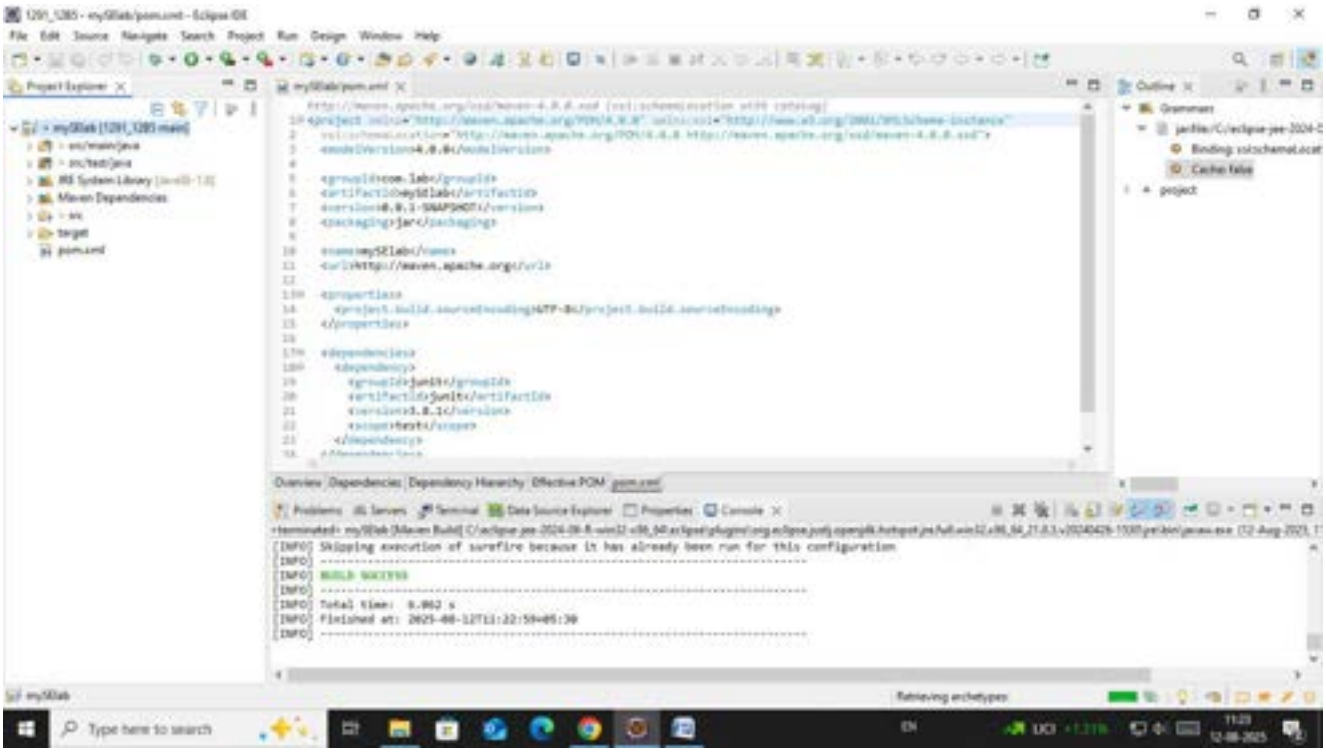
└— Enter Goals: clean install test

└— Click on Apply -> Click on Run





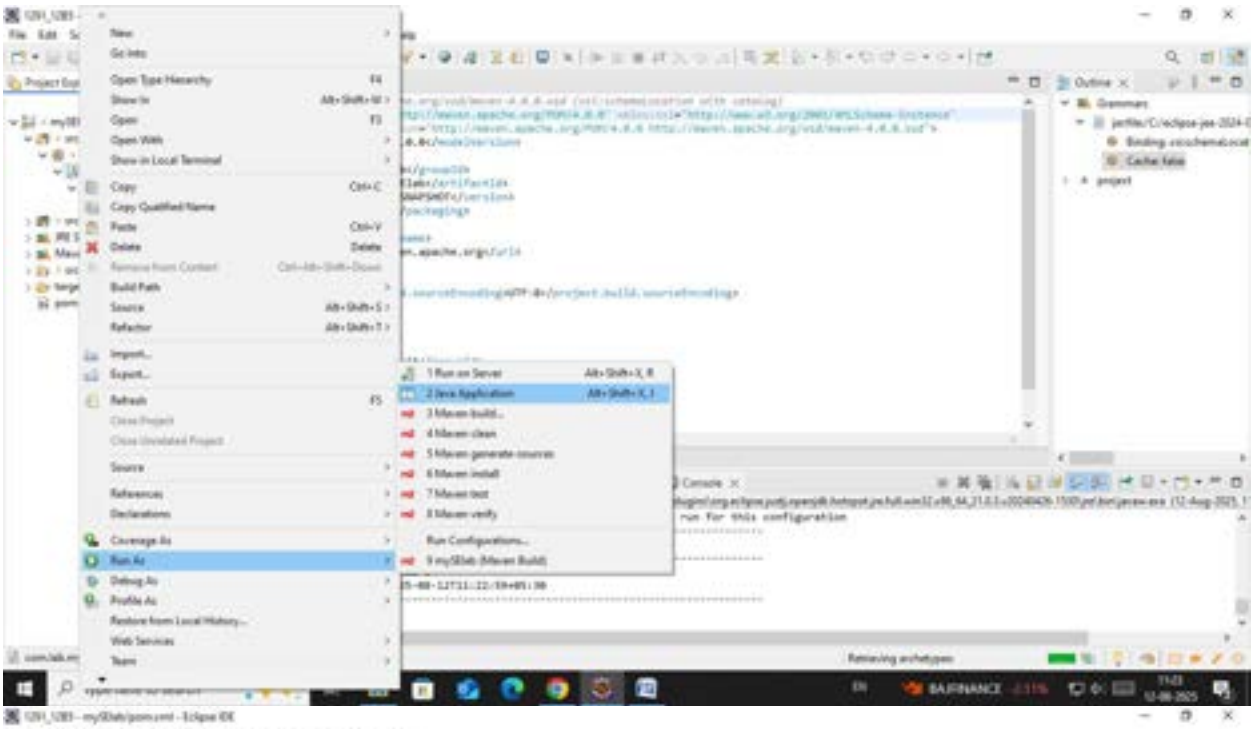
Step 11. Check console for BUILD SUCCESS message.

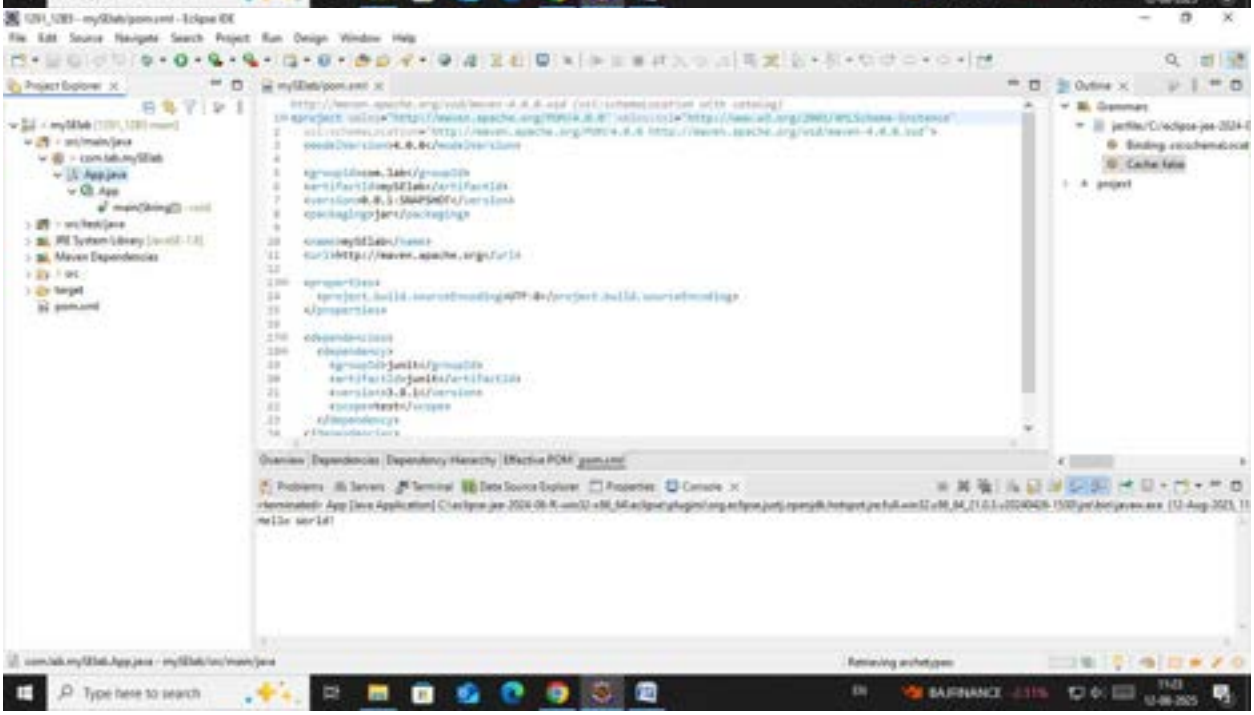


Step 12. Run the application:

└── Right-click on App.java -> Run As -> Java Application

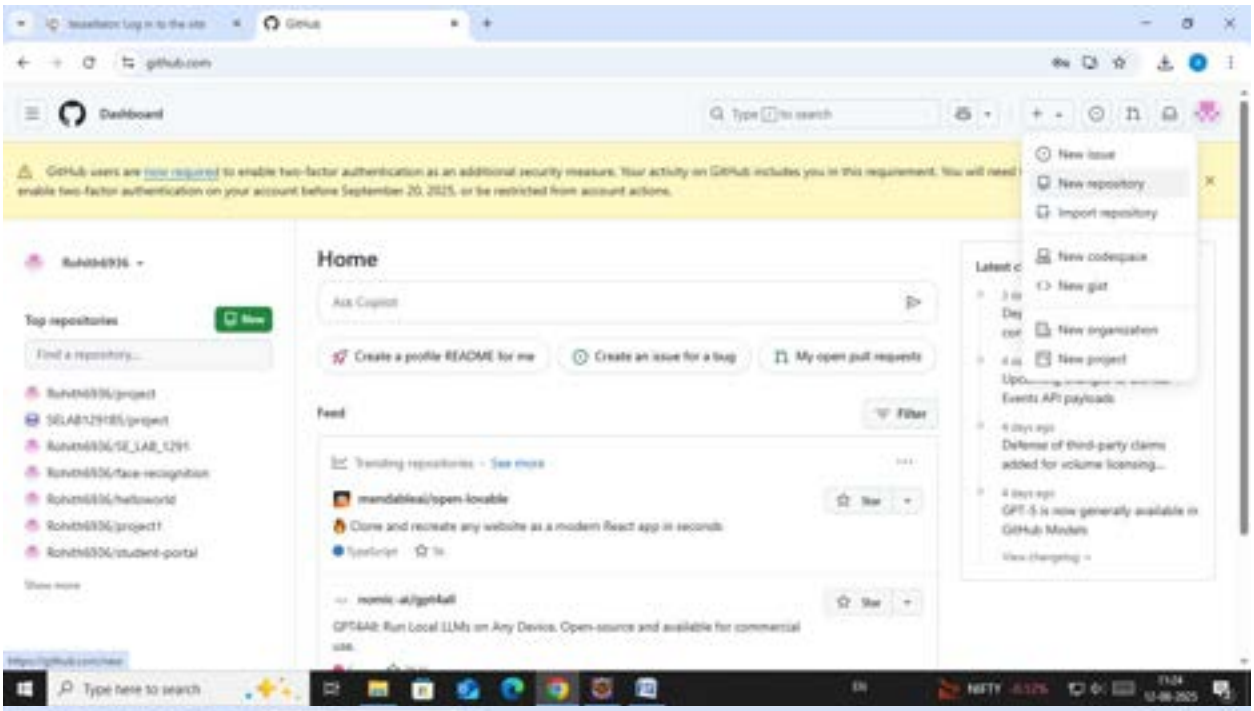
└── Output: "Hello World" displayed.

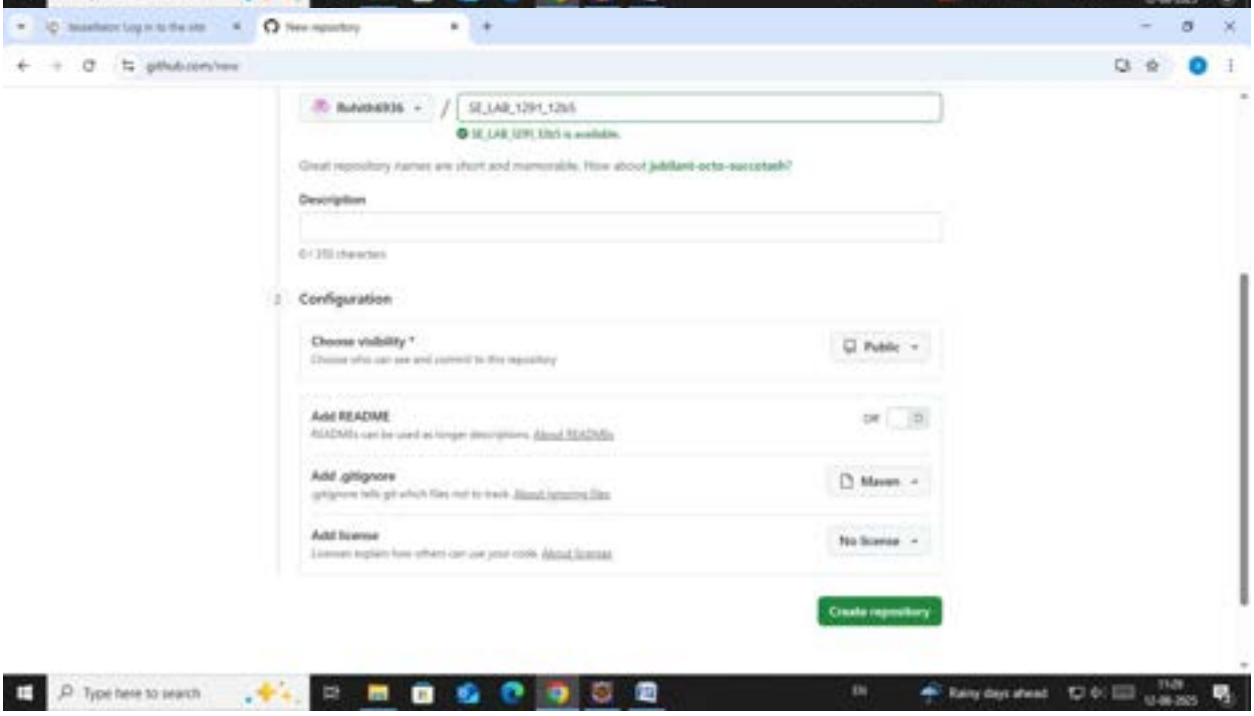


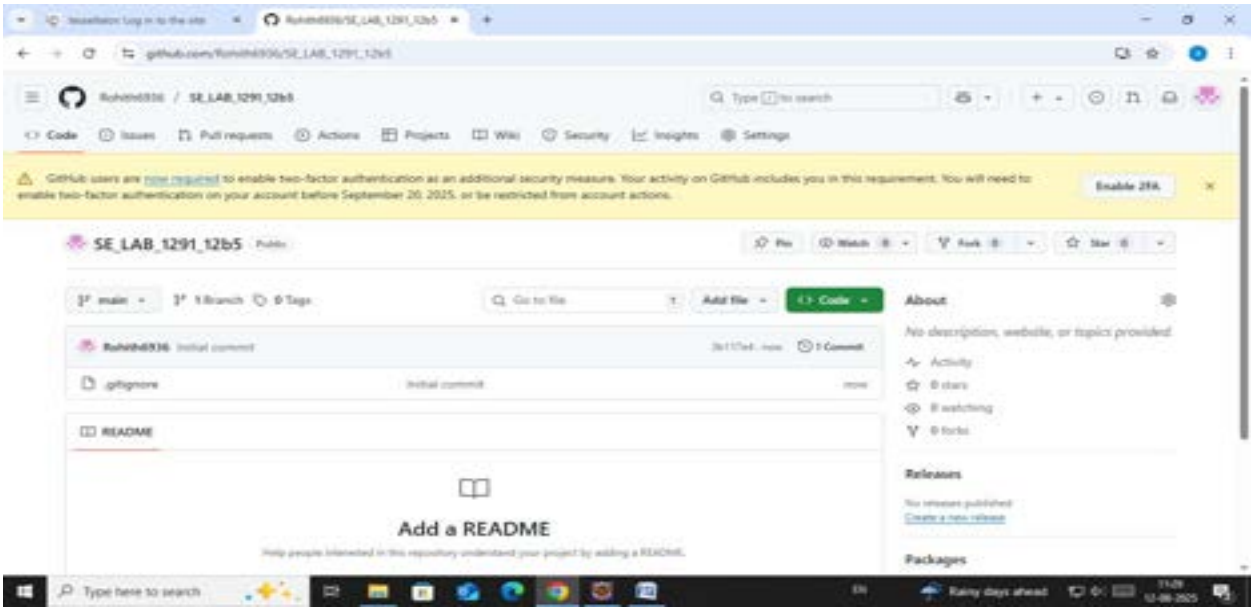


GITHUB:

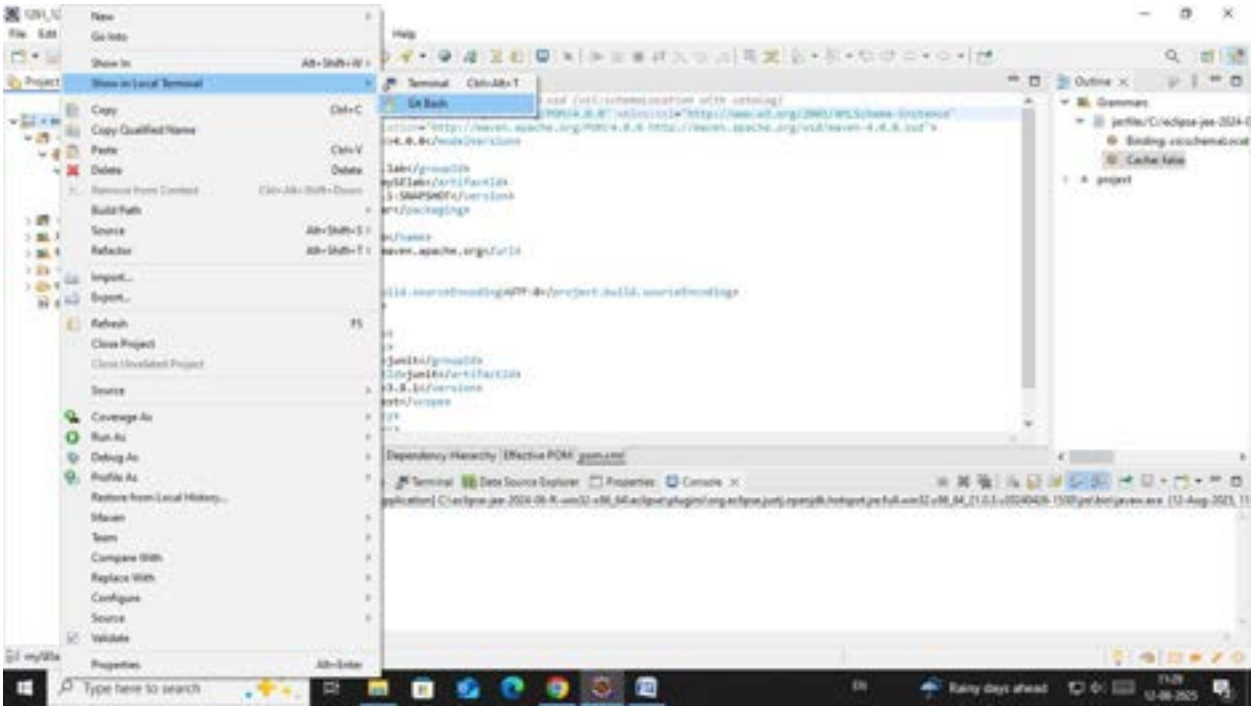
Step: create a repository


Dashboard
New issue
New repository
Import repository
New codespace
New gist
New organization
New project
Home
Create a profile README for me
Create an issue for a bug
My open pull requests
Top repositories
Feed

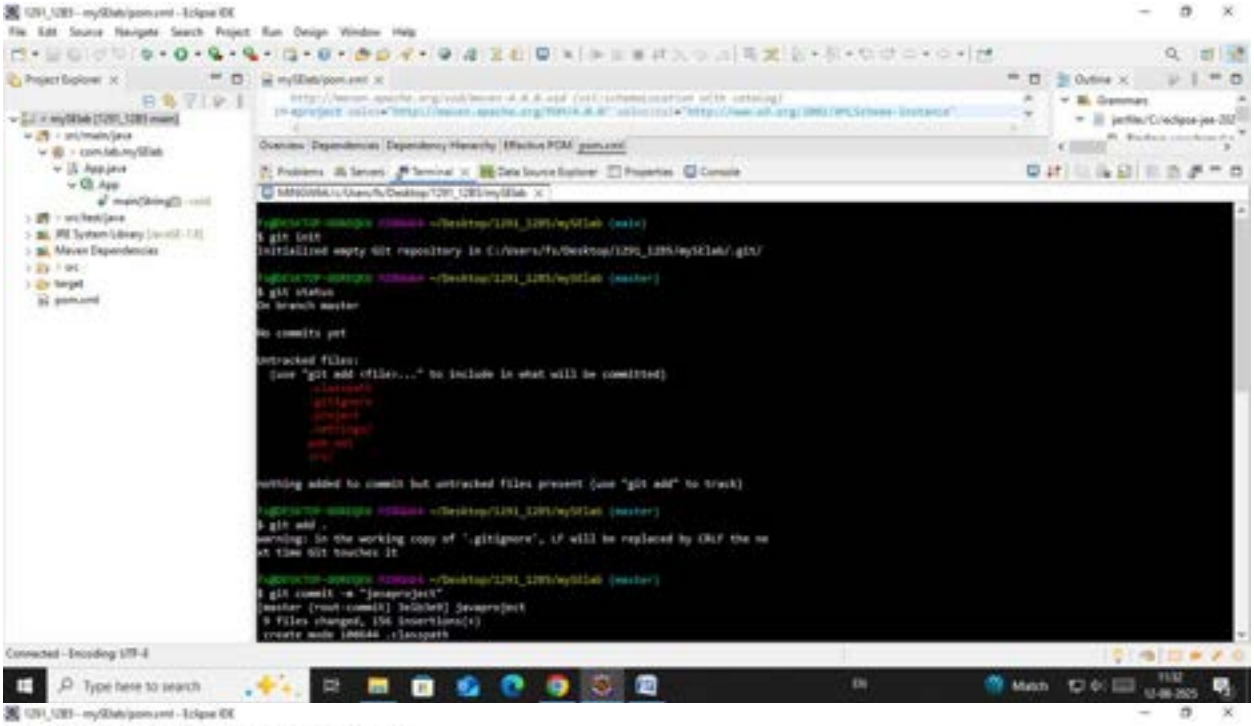

Configuration
Choose visibility *
Public
Add README
Add .gitignore
Maven
Add license
No license
Create repository

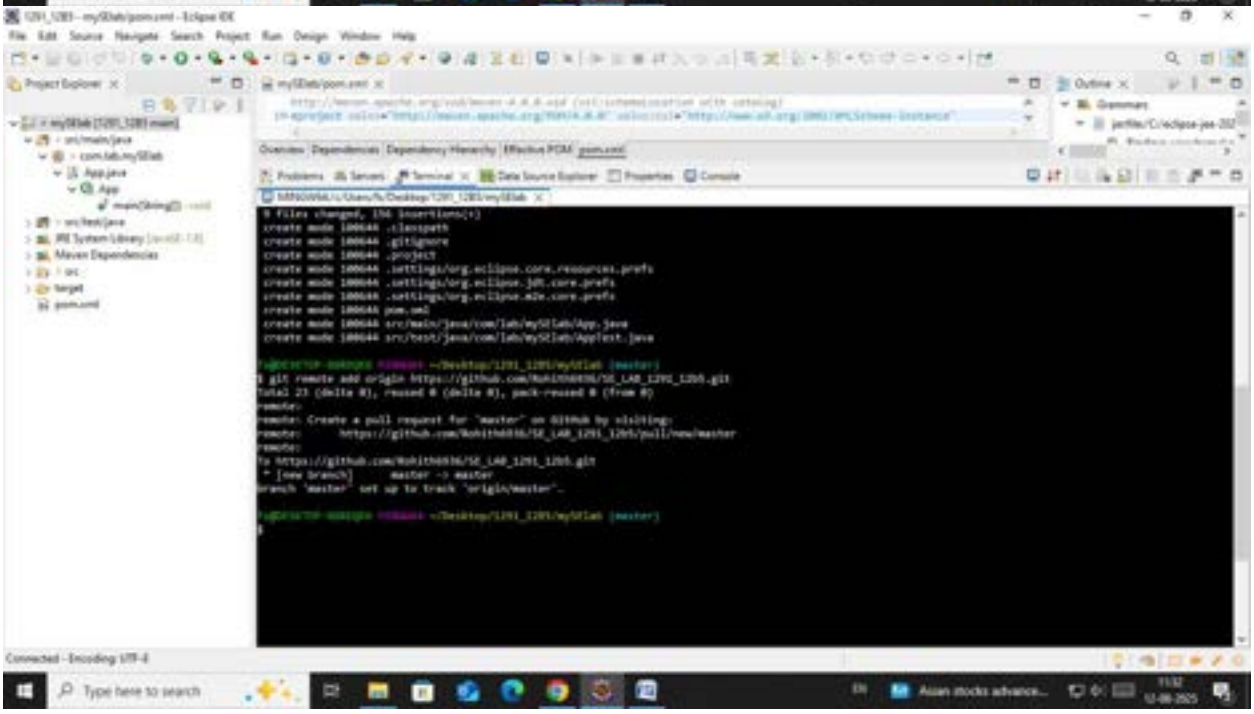


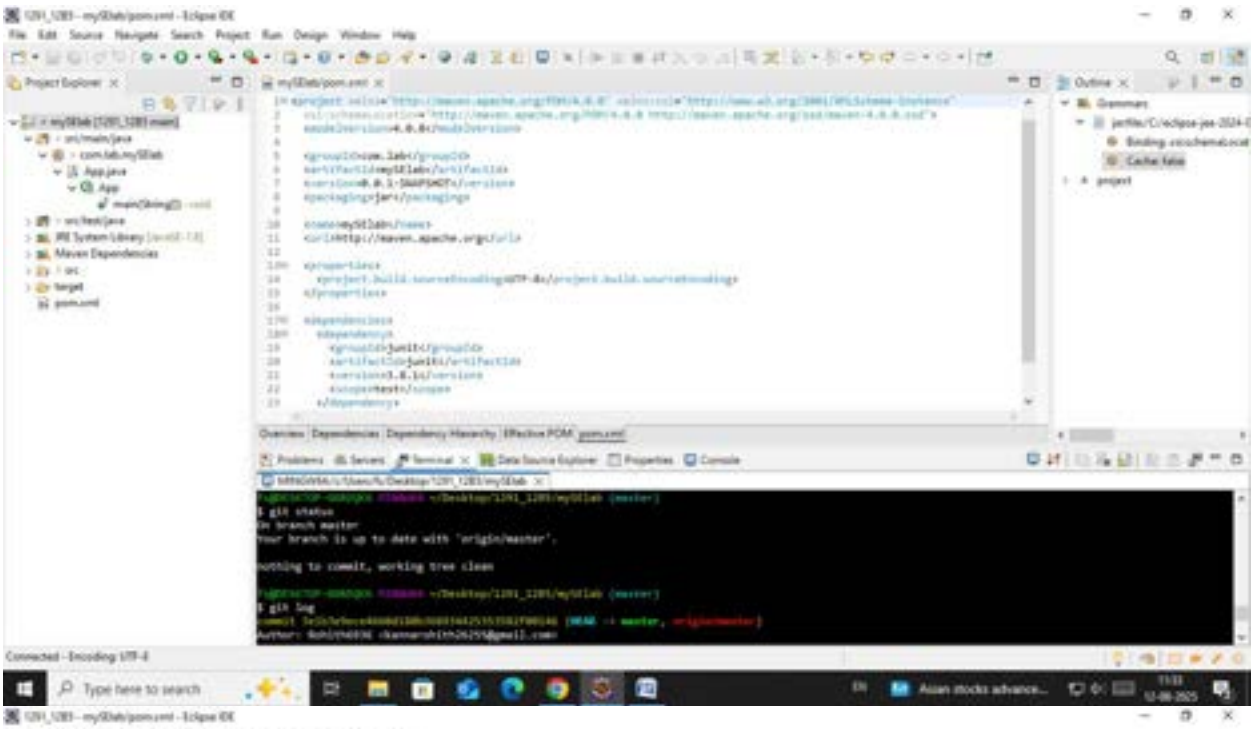
Step: open git bash in eclipse

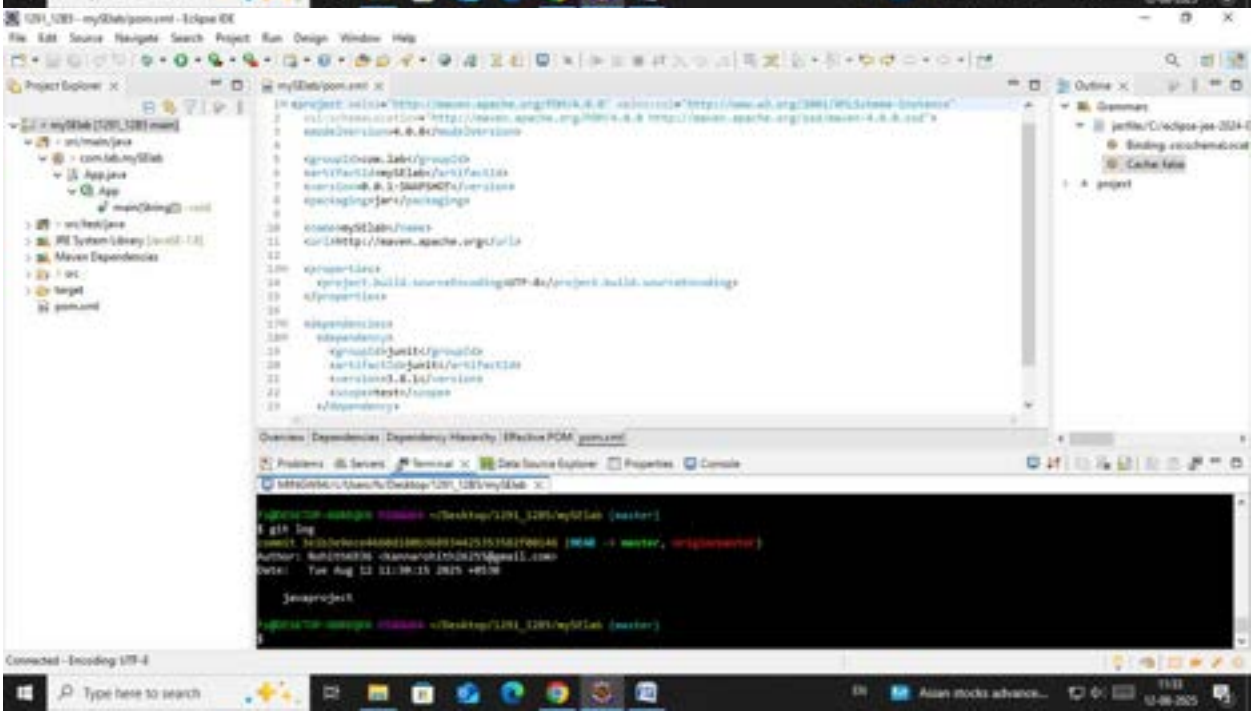


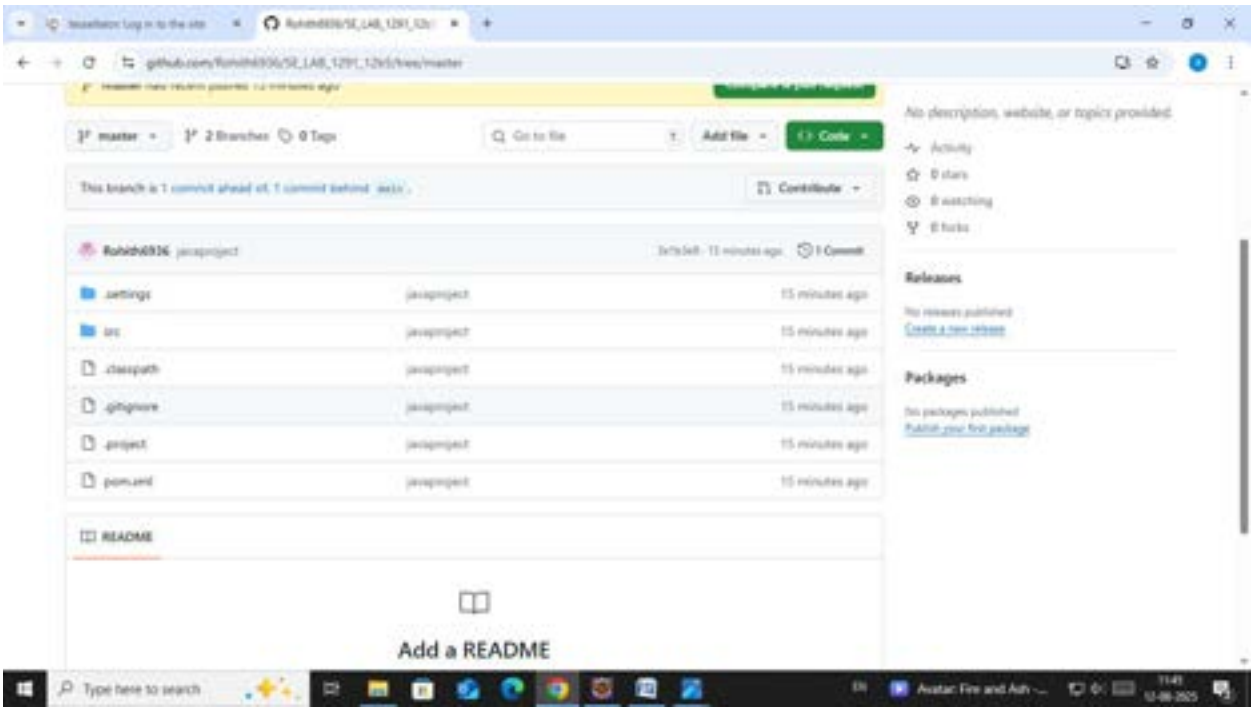
Step: push maven java project into Git hub repository









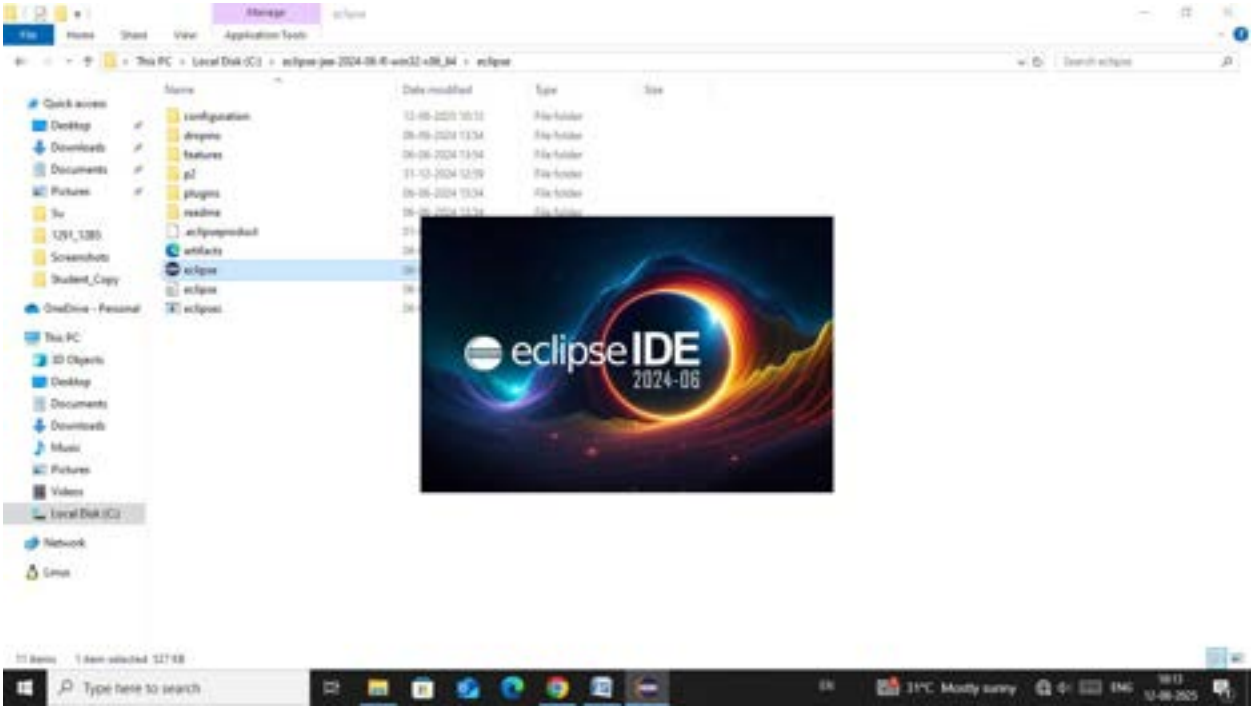


Creation of Maven web Java Project

Step 1: Open Eclipse

└── 1.1 Launch Eclipse IDE.

└── 1.2 Select or create a workspace.

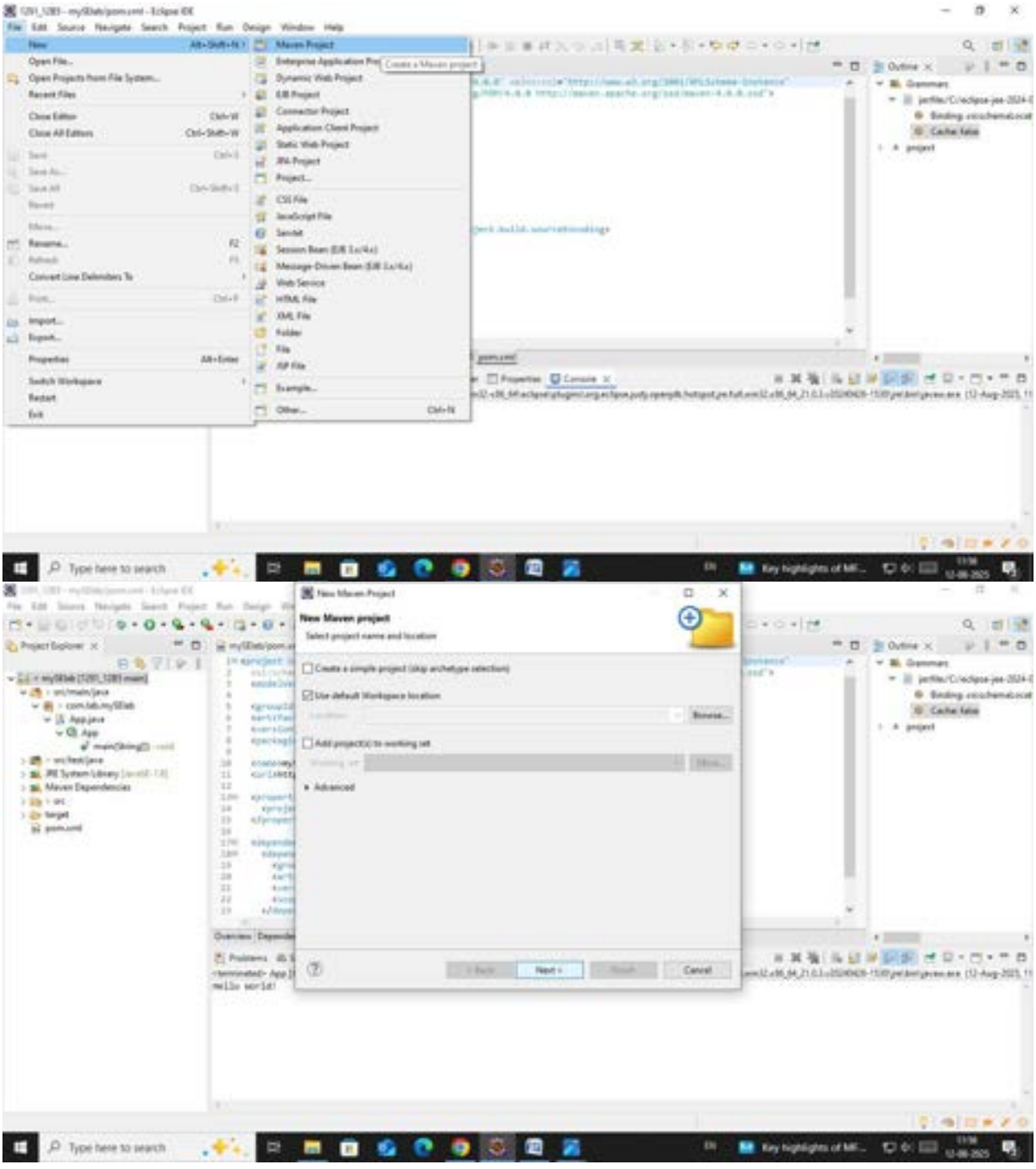


Step 2: Create a New Maven Project

└── 2.1. File -> New -> Project...

└── 2.1.1. Expand "Maven"

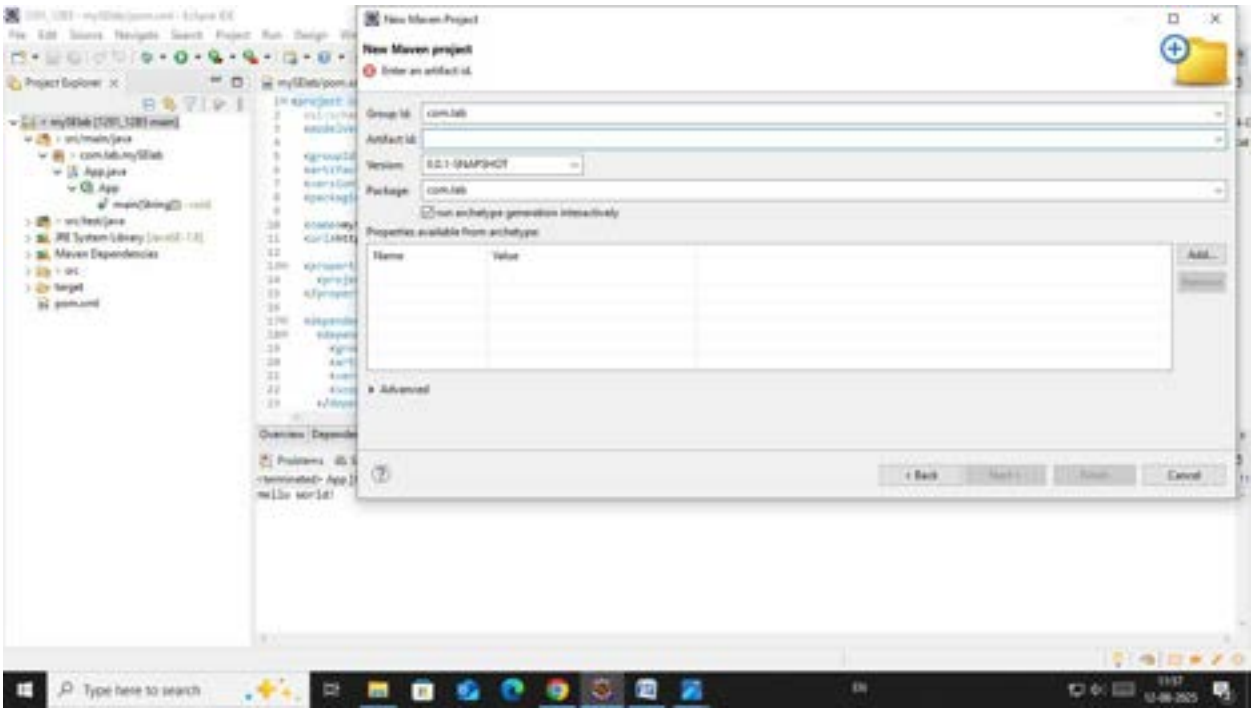
└── 2.1.2. Select "Maven Project" and click "Next"



Step 3: Choose Maven Archetype

└── 3.1. Select an archetype(e.g "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

└── 3.2. Click "Next"

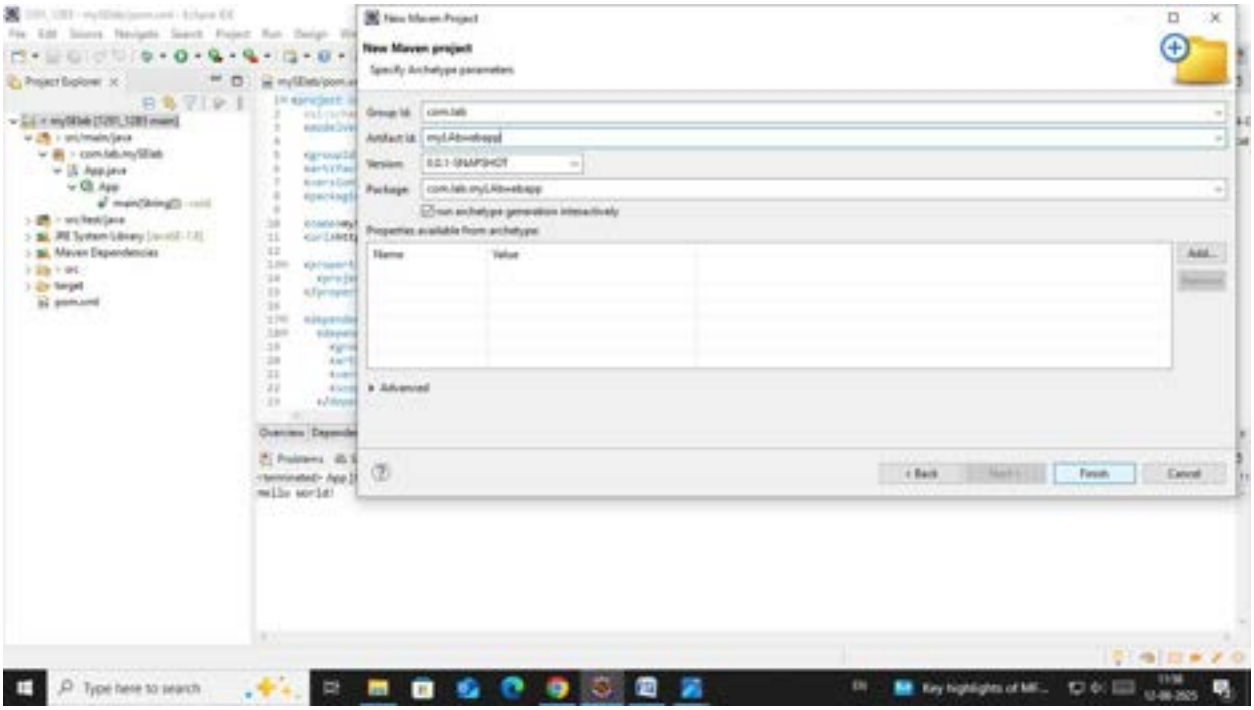


Step 4: Configure the Maven Project

└── 4.1 Group Id: Enter a group ID (e.g., com.example).

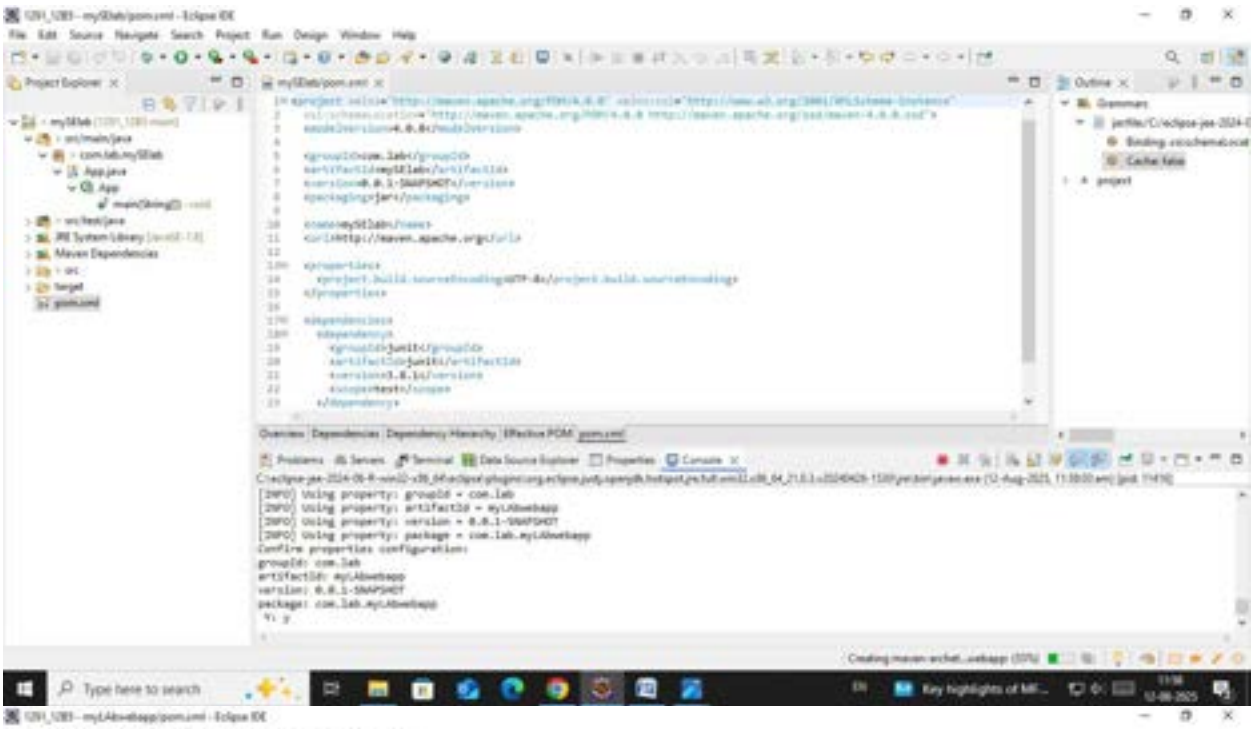
└── 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

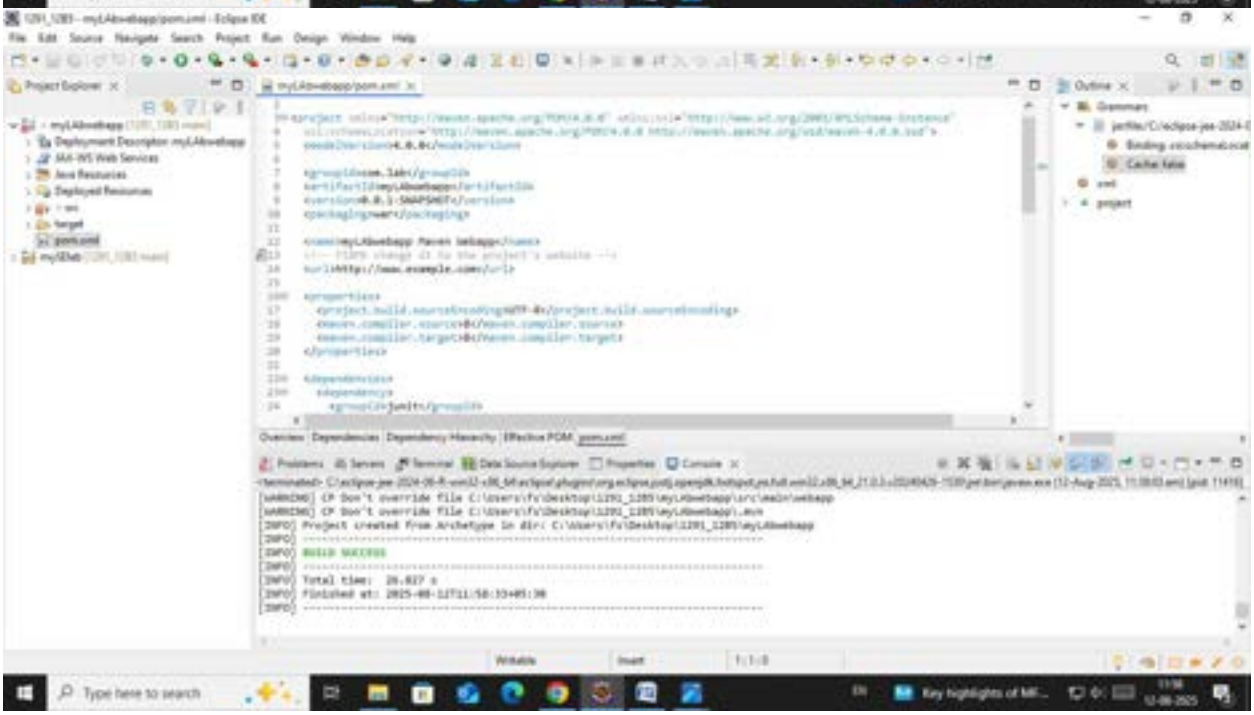
└── 4.3 Click **Finish** to create the project.



Step 5: Add Maven Dependencies

└── 5.1 Open the **pom.xml** file in the Maven project.



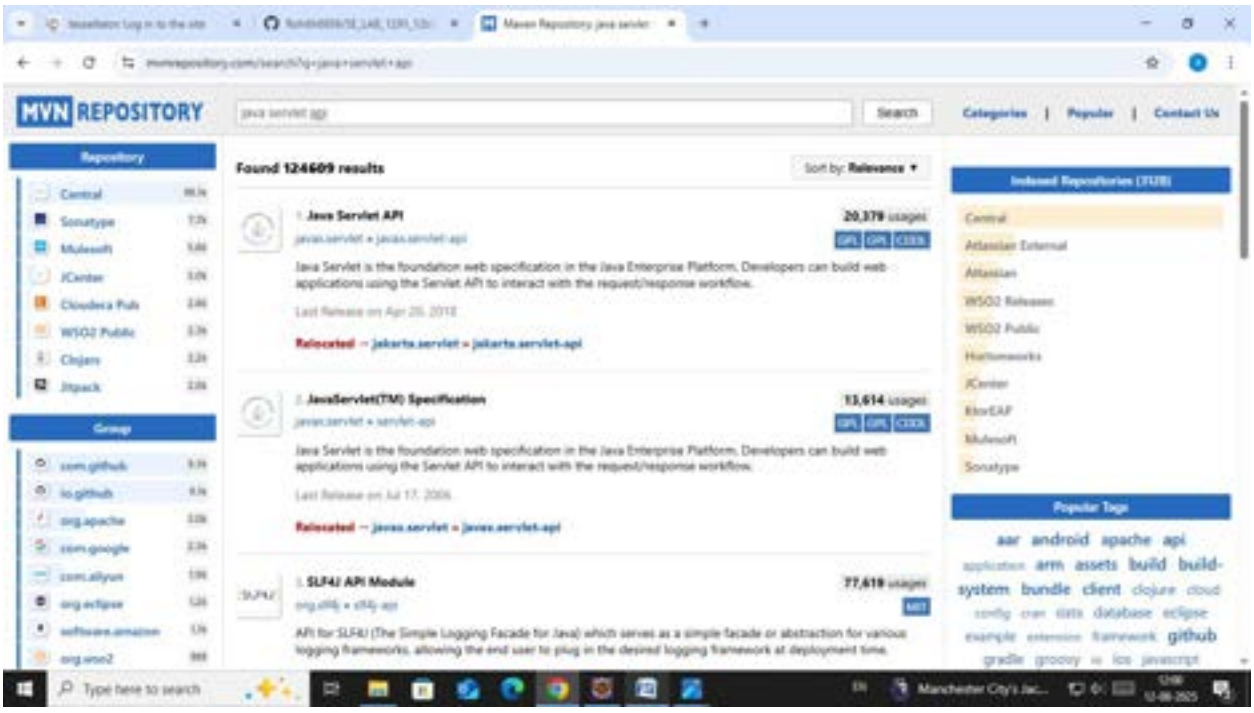


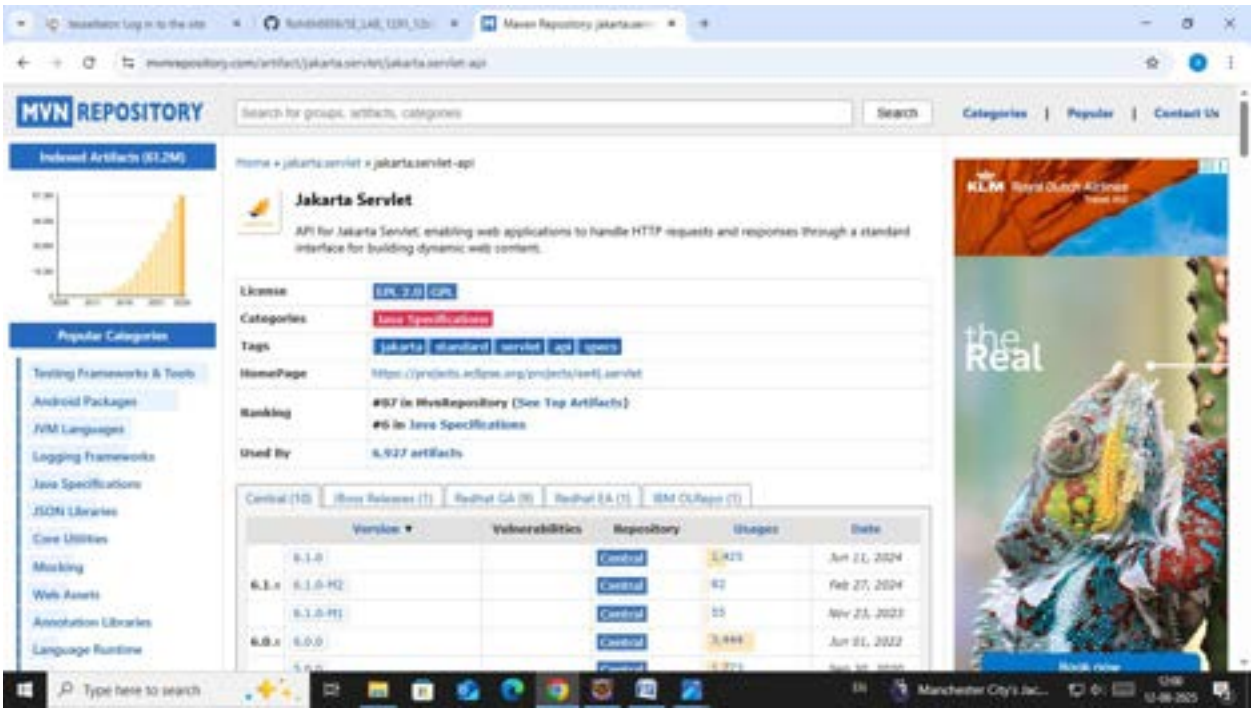
└── 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

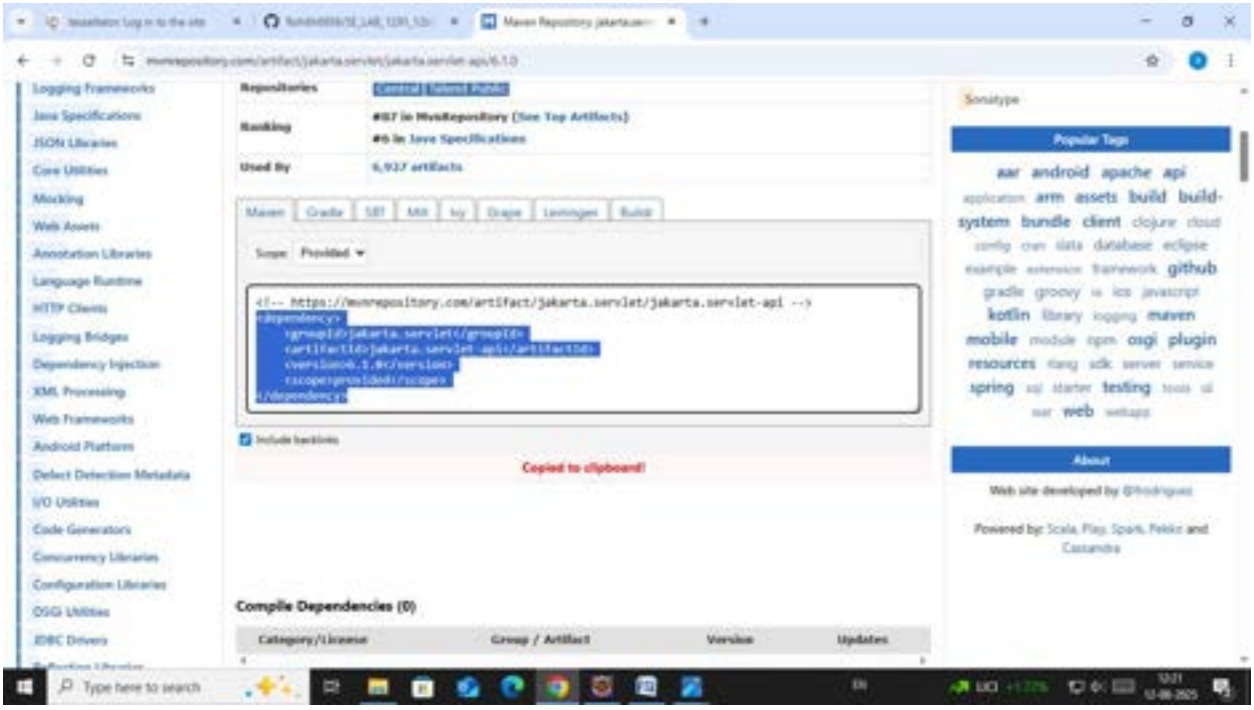


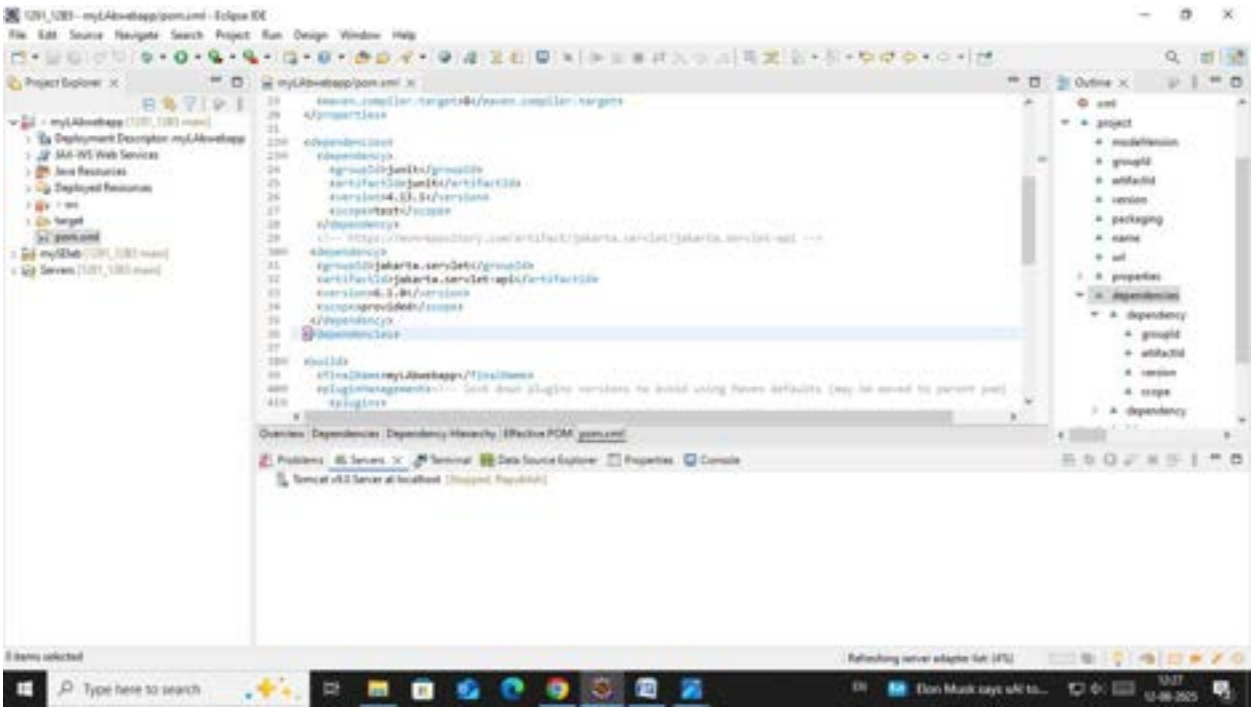


└── Example:

```xml

<dependency>

<groupId>javax.servlet</groupId>
```

```
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
</dependency>
```



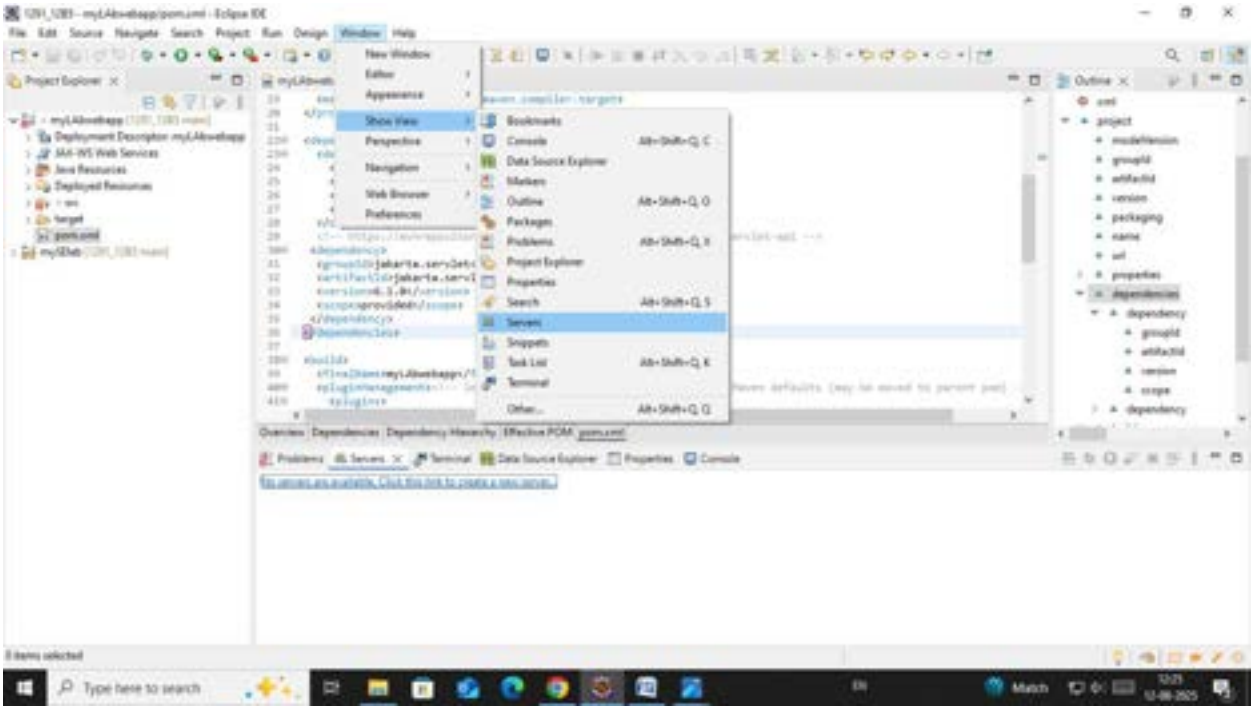


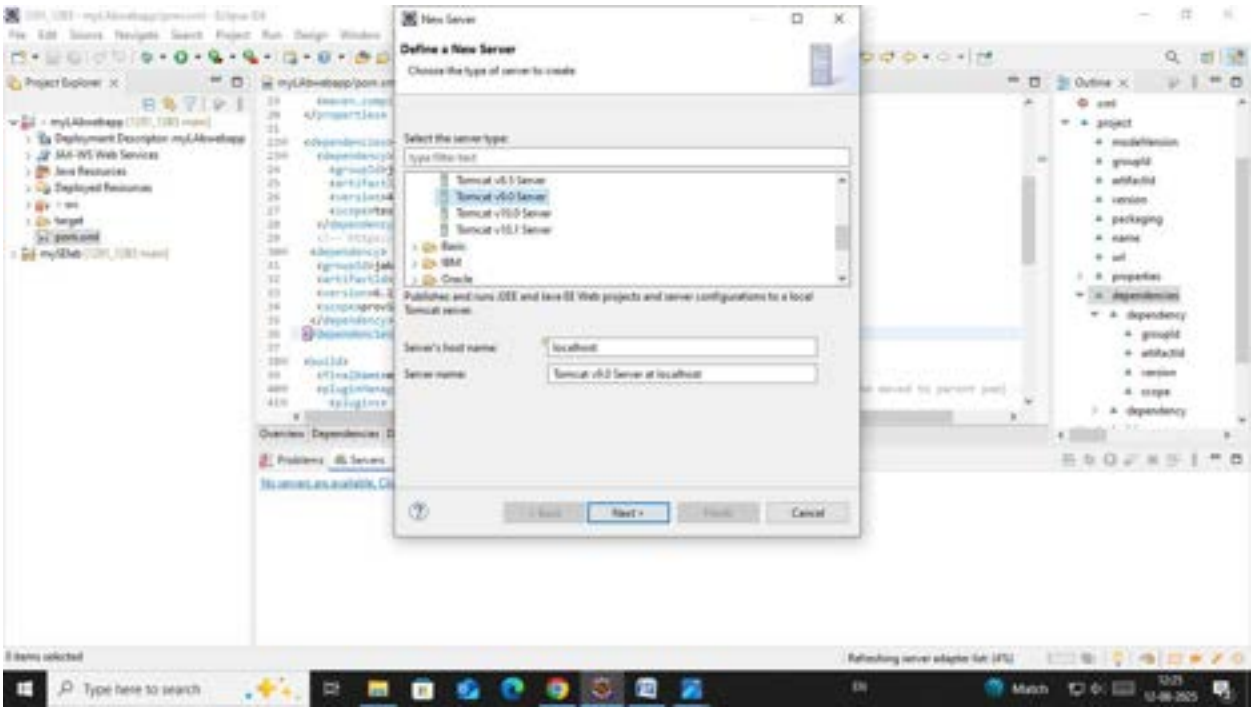
Step 6:-. Configure server:

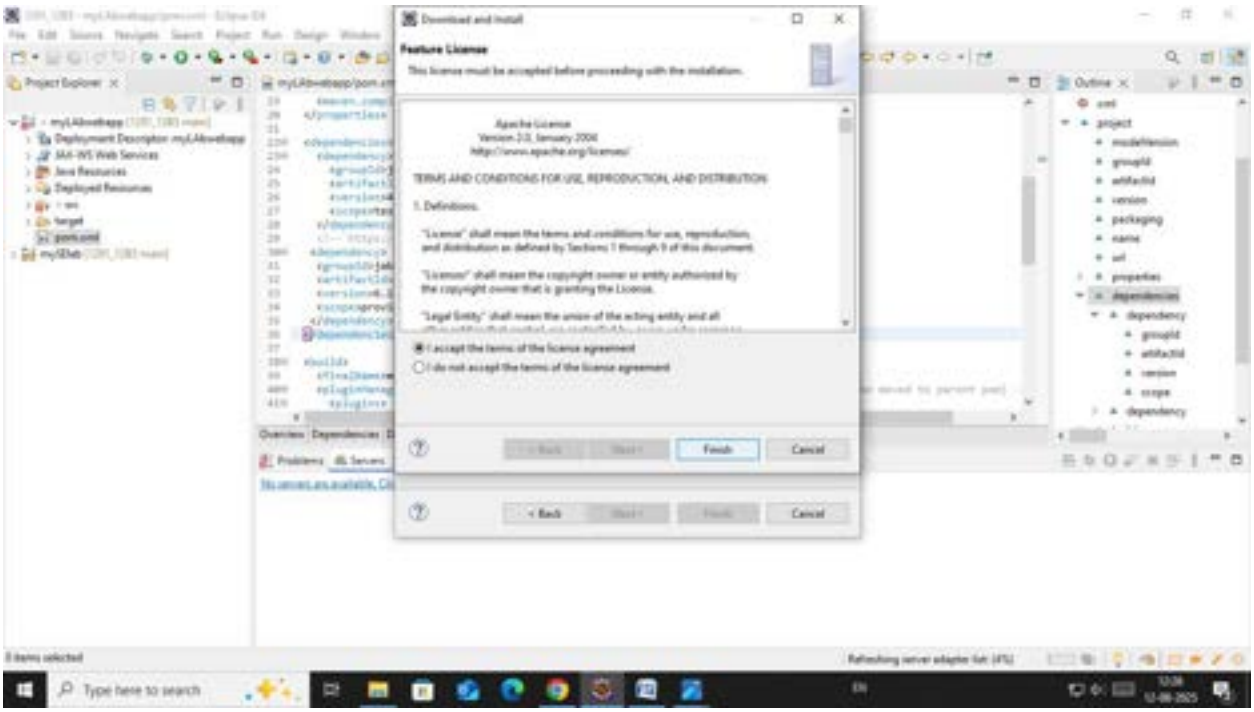
└── Window -> Show View -> Servers

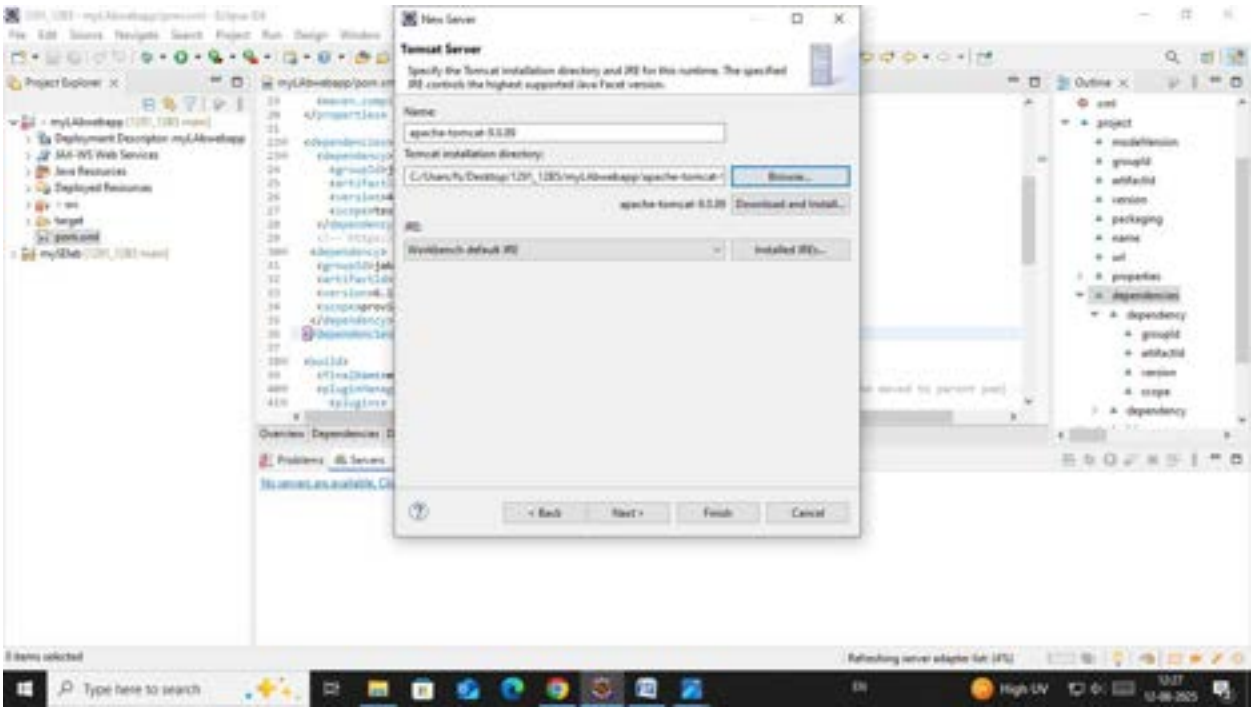
└── Add server -> Select Tomcat v9.0 server -> Click Next

└── Configure server options (e.g., ports, server location).



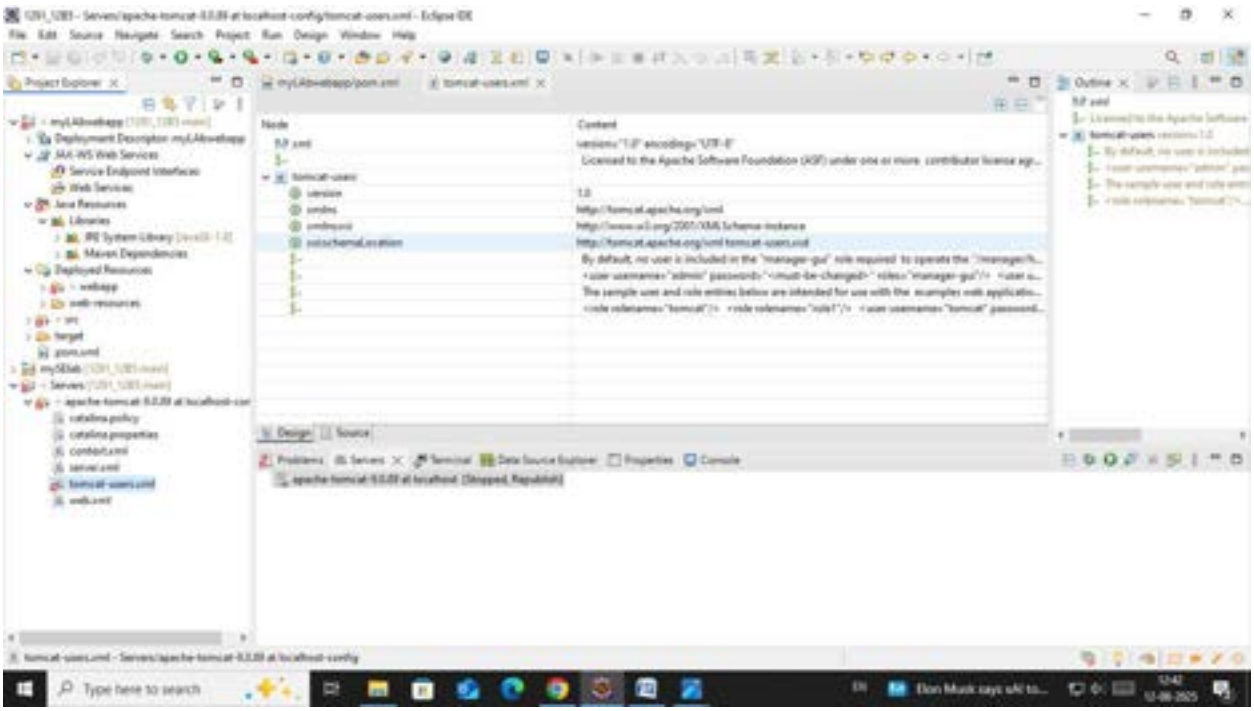

New Server
Define a New Server
Choose the type of server to create
Select the server type:
Tomcat v8.5 Server
Tomcat v9.0 Server
Tomcat v10.0 Server
Tomcat v10.1 Server
Basic
IBM
Oracle
Server's host name:
localhost
Server name:
Tomcat v9.0 Server at localhost
Next >
Cancel


Download and Install
Feature License
This license must be accepted before proceeding with the installation.
Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/
TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION
1. Definitions.
"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.
"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.
"Legal Entity" shall mean the union of the acting entity and all
I accept the terms of the license agreement
I do not accept the terms of the license agreement
Finish
Cancel
< Back
Cancel



Step 7:-. Modify 'tomcat-users.xml':

└── Add role and user details under <tomcat-users> tag.



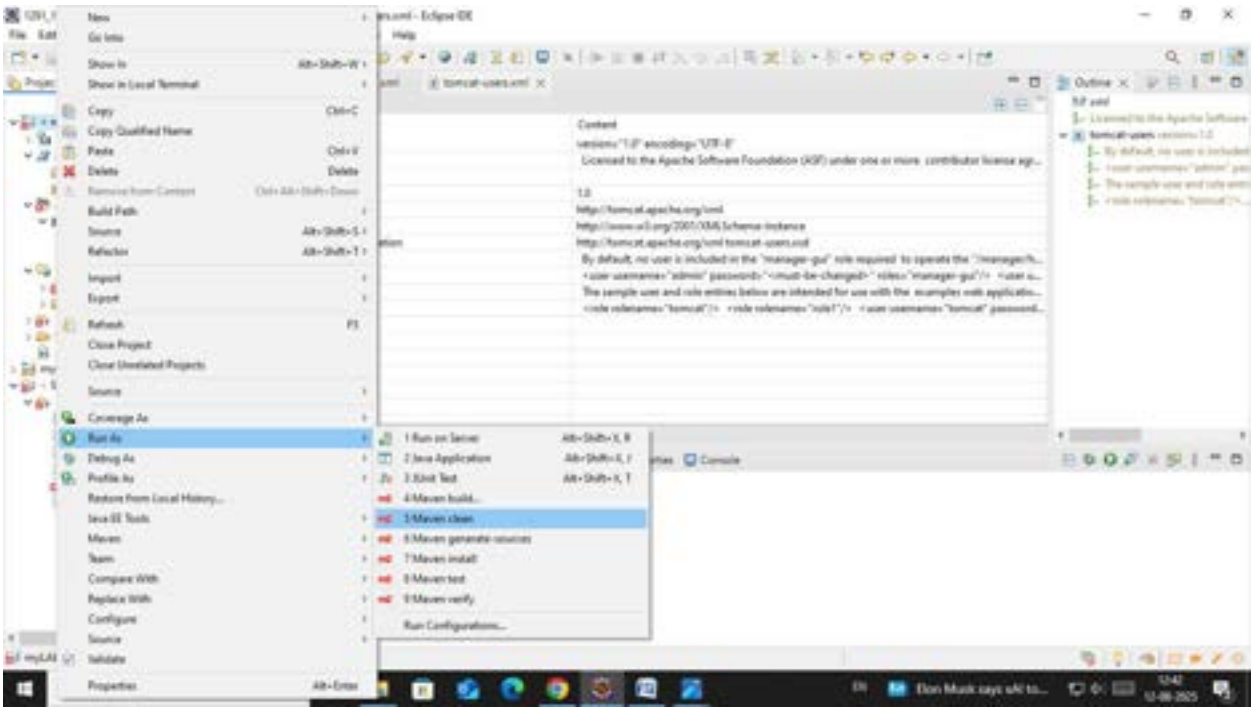
Step 8:. Build the project:

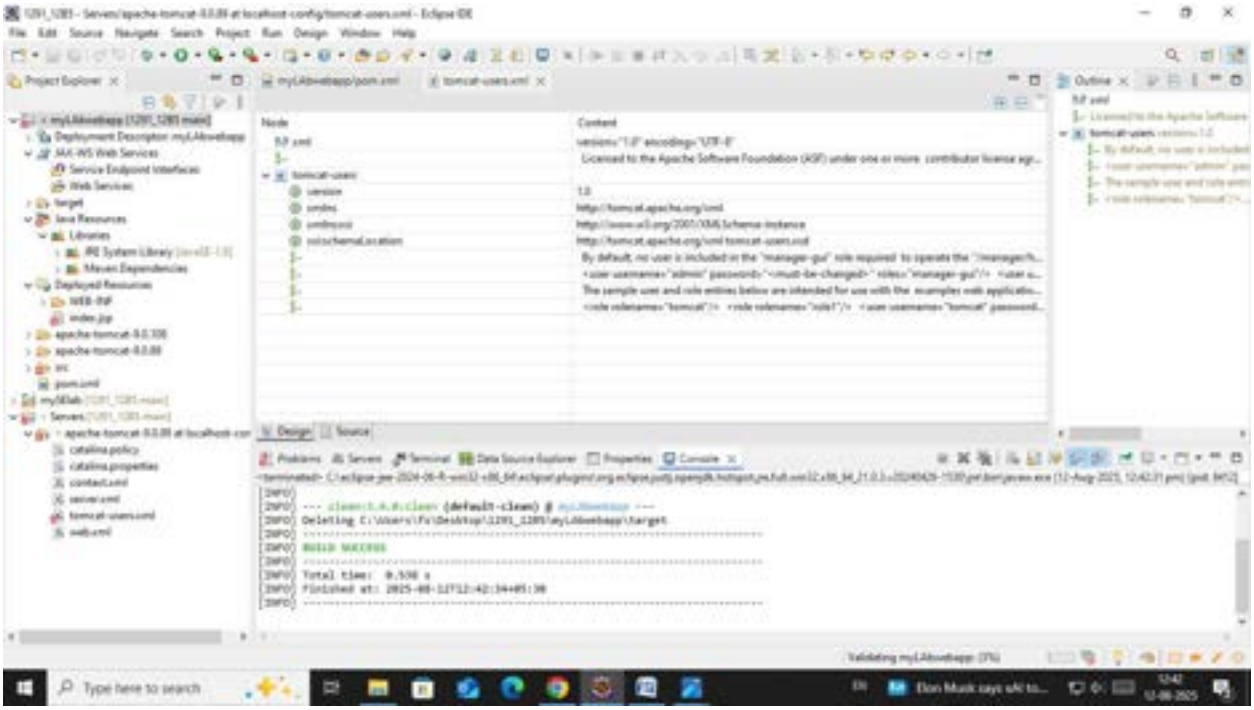
└── Right-click on index.jsp -> Run As -> Maven Clean

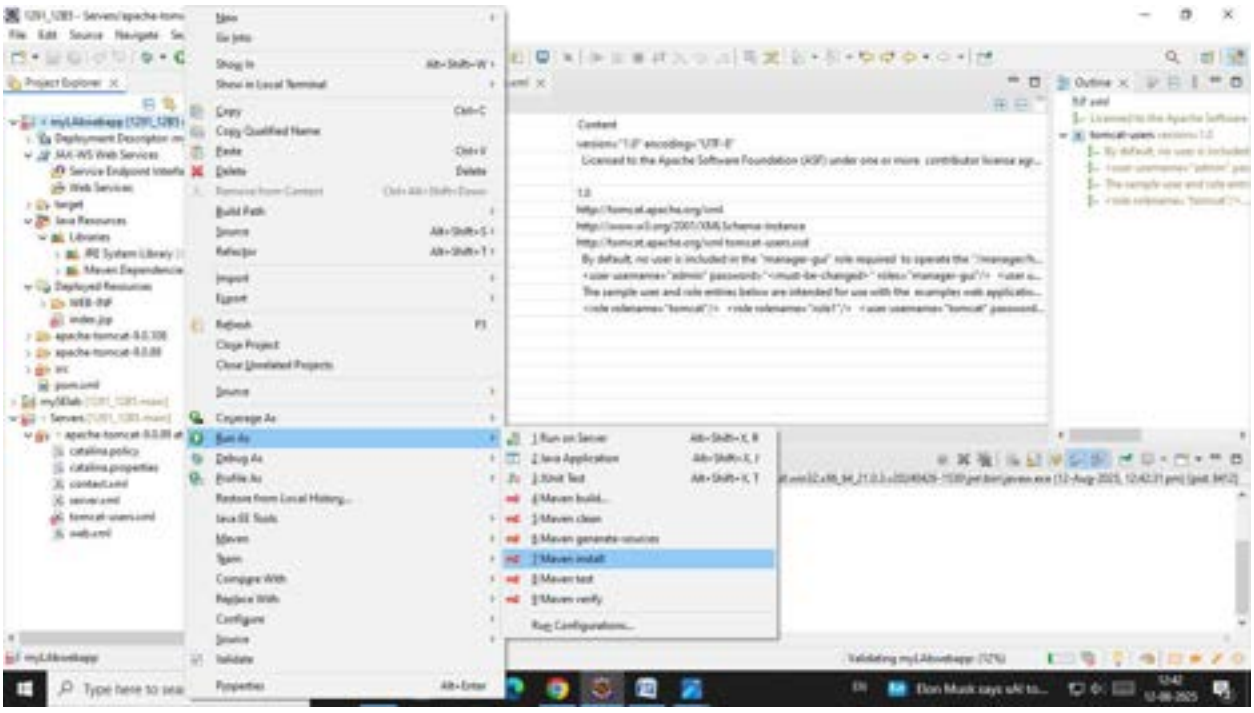
└── Right-click on index.jsp -> Run As -> Maven Install

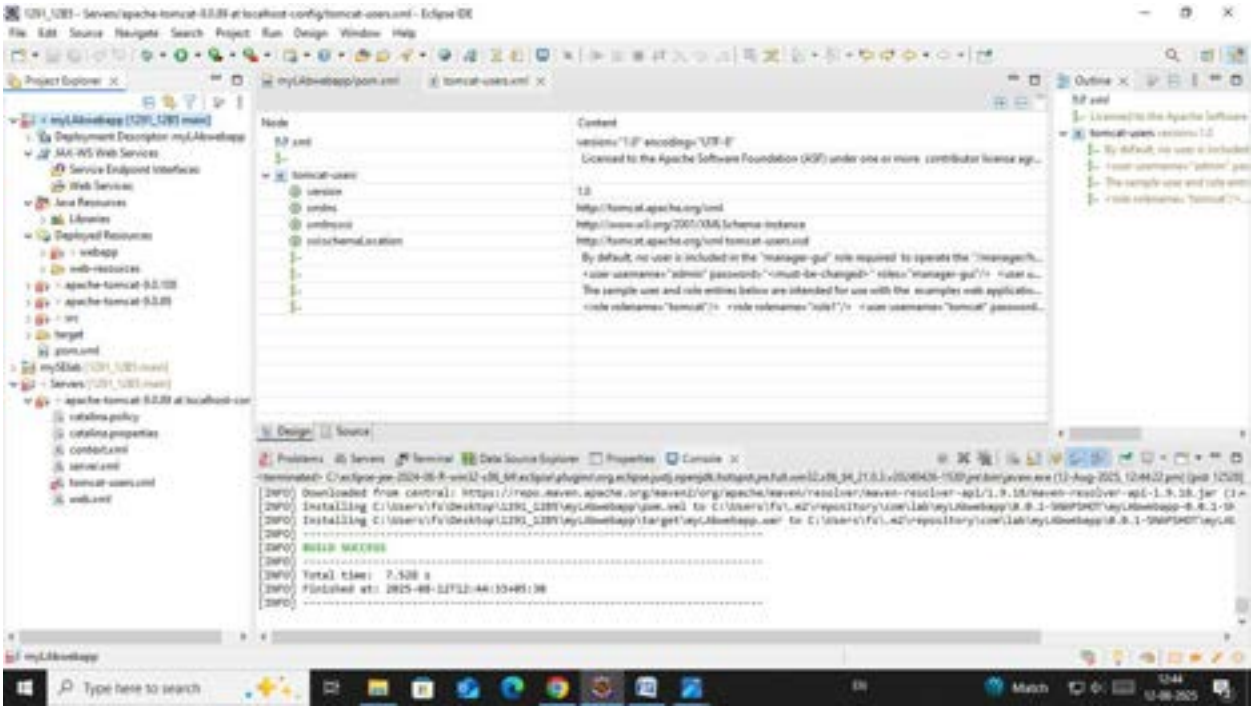
└── Right-click on index.jsp -> Run As -> Maven Test

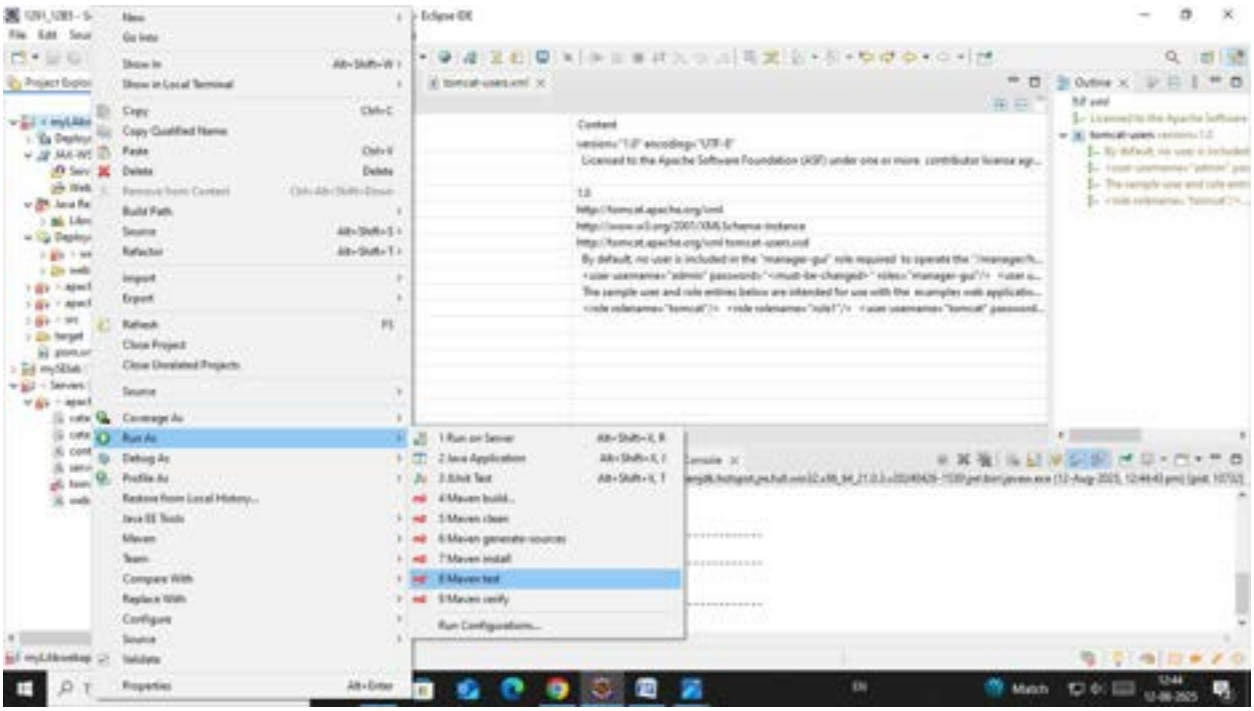
└── Right-click on index.jsp -> Run As -> Maven Build

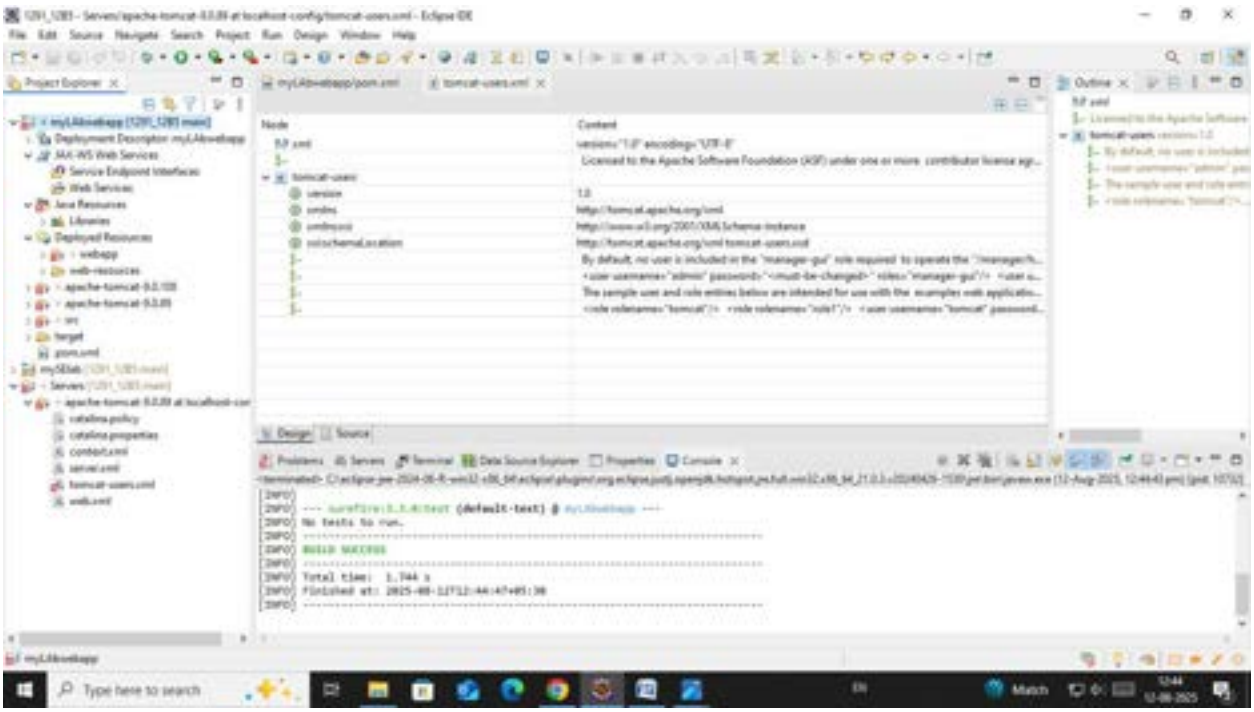

New
Go Into
Show In
Show in Local Terminal
Copy
Copy Qualified Name
Paste
Delete
Remove from Context
Build Path
Source
Refactor
Import
Export
Refresh
Close Project
Close Unrelated Projects
Source
Coverage As
Run As
Debug As
Profile As
Restore from Local History...
Java EE Tools
Maven
Team
Compare With
Replace With
Configure
Source
Validate
Properties
1 Run on Server
2 Java Application
3 JUnit Test
4 Maven build...
5 Maven clean
6 Maven generate-sources
7 Maven install
8 Maven test
9 Maven verify
Run Configurations...
Outline
Console


Project Explorer
Outline
Console
Problems
Servers
Terminal
Data Source Explorer
Properties
Design
Source
Type here to search


Project Explorer
Outline
Run As
Maven install
Maven build...
Maven clean
Maven generate-sources
Maven test
Maven verify
Run Configurations...


Project Explorer
Outline
Design
Source
Problems
Servers
Terminal
Data Source Explorer
Properties
Console
[INFO] BUILD SUCCESS
[INFO] Total time: 7.520 s
[INFO] Finished at: 2025-08-12T12:44:35+05:30

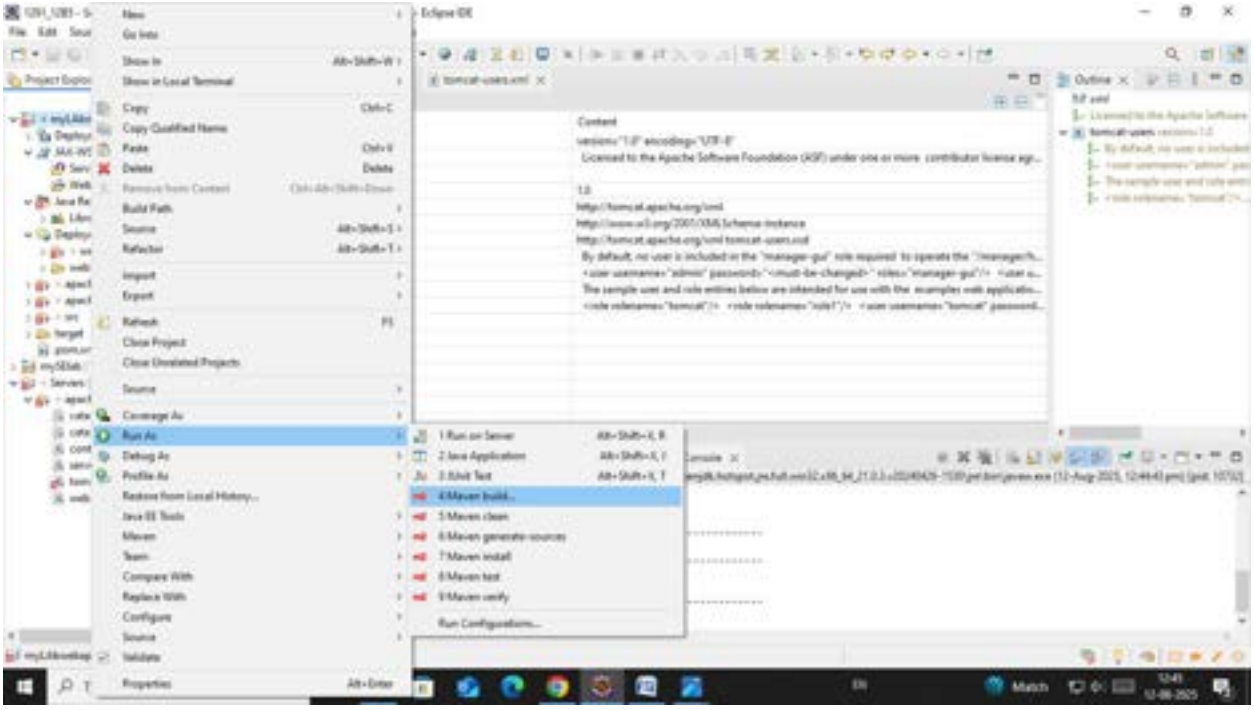


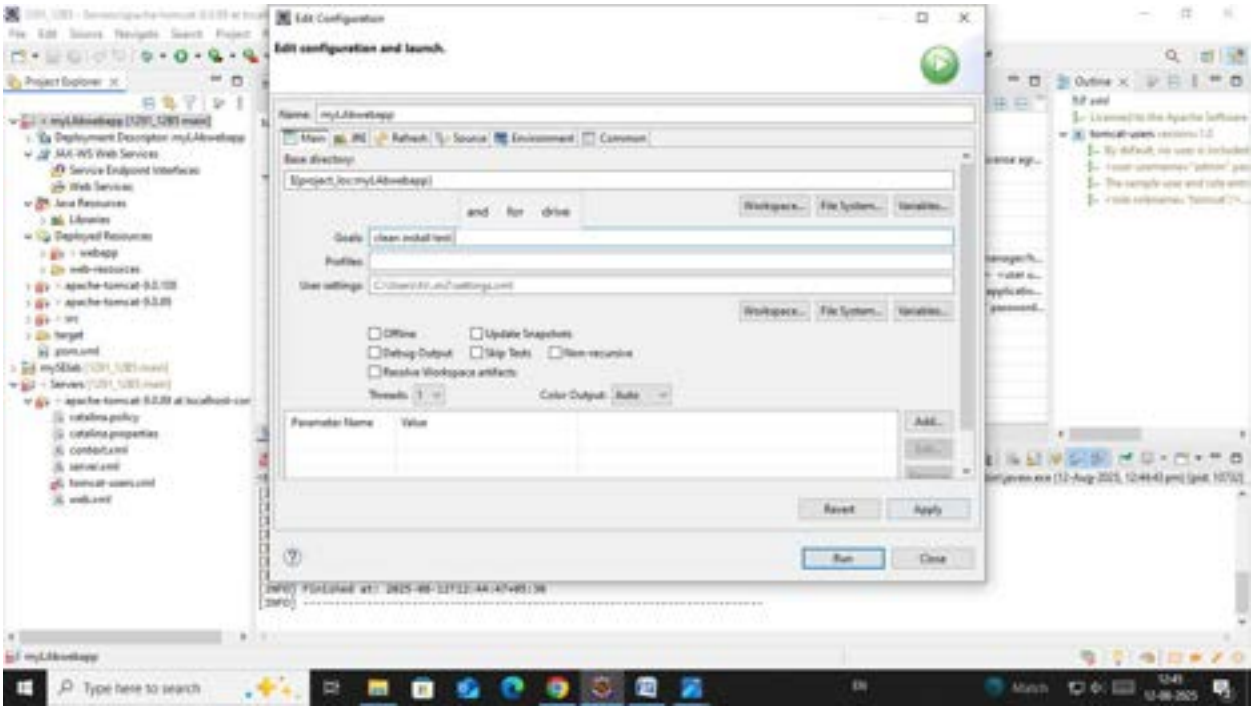


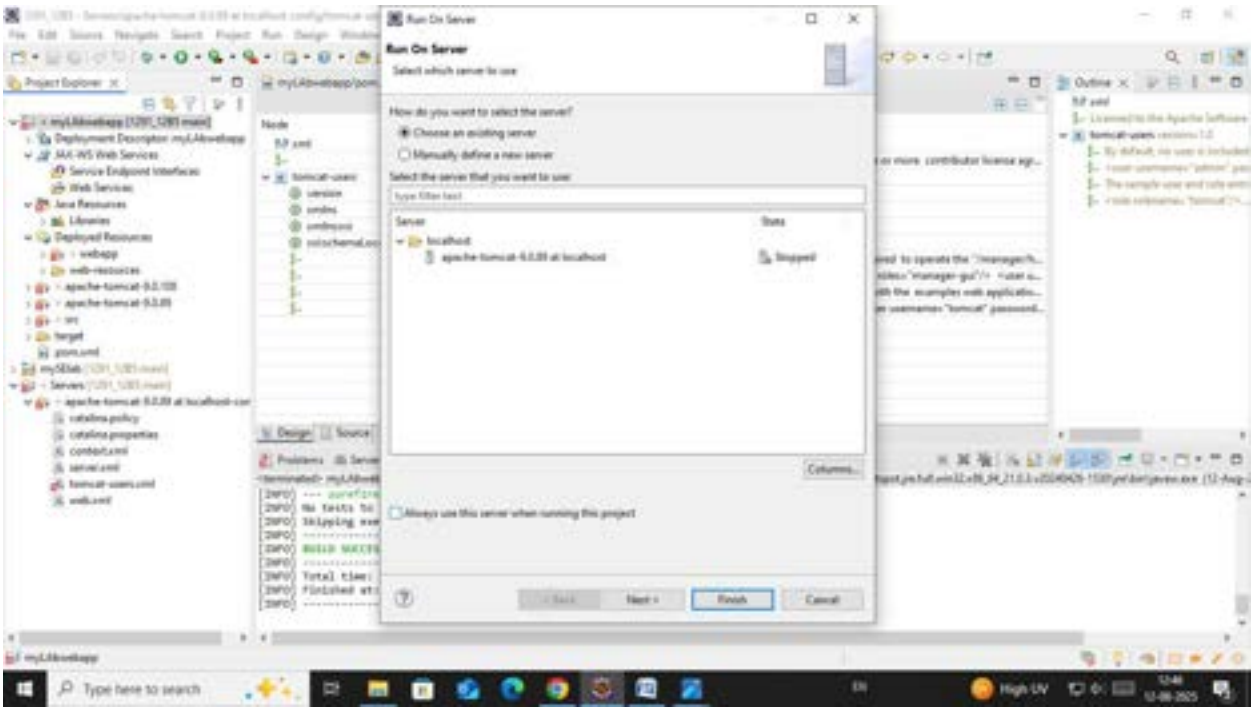
Step 9. In the Maven Build dialog:

└── Enter Goals: clean install test

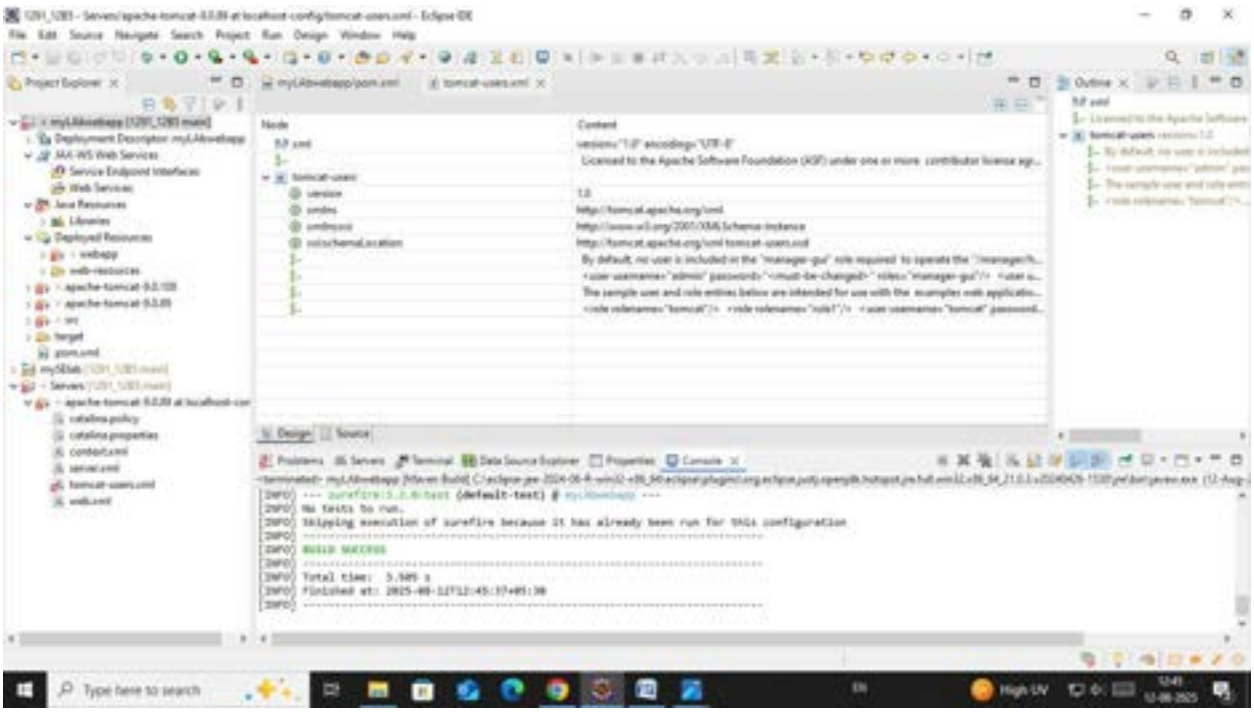
└── Click on Apply -> Click on Run







Step 10. Check console for BUILD SUCCESS message.



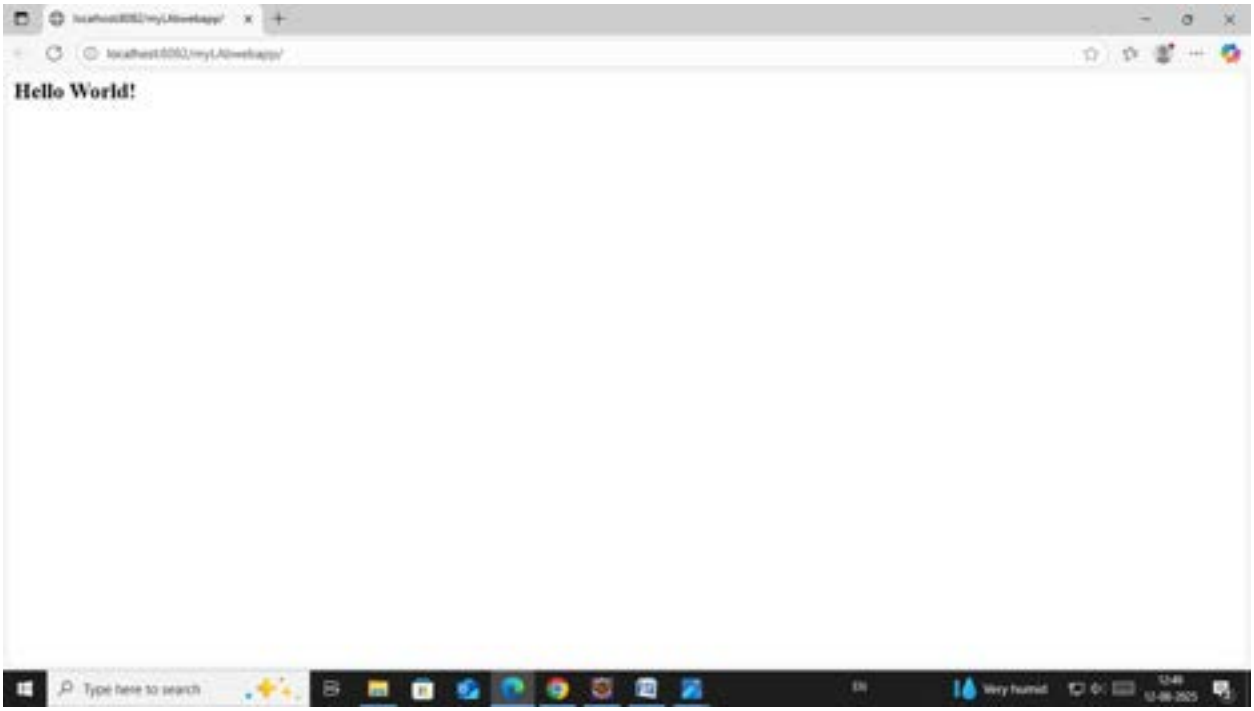
Step 11. Run the application:

└— Right-click on index.jsp -> Run As -> Run on Server

└— Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github



3. Configure Java Version via Compiler Plugin

In **`pom.xml`**:

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>17</source>

<target>17</target>

</configuration> </plugin>

Check:

- Run `mvn package`
- Inspect generated JAR: *jar tf target/myapp.jar*

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```
public class AppTest {
   @Test
   public void testSum() {
      assertEquals(5, 2 + 3);
   }
}
```

Run:

mvn test

Check:

- `target/test-classes/` → compiled test `.class` files
- `target/surefire-reports/` → `.txt` or `.xml` test results
  If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause `BUILD FAILURE`
- Maven shows precise error in console
- Fix the dependency tag → re-run `mvn clean install`

6. Adding Resource Files

Put config.properties inside:
src/main/resources/config.properties
To read:

```
InputStream input =
getClass().getClassLoader().getResourceAsStream("config.properties");
```

After build, check target/classes/ → file should exist there.

7. Multi-Module and Parent Projects

Structure:

```
parent/
├── pom.xml (packaging: pom)
├── core/
```

```
│   └── pom.xml
└── web/
    └── pom.xml
```

- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

8. Executable JARs

Add to `pom.xml`:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.Main</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Run:

```
mvn package
java -jar target/myapp.jar
```

9. Building a WAR File

Create a Maven web project with structure:

```
src/main/webapp/

  └── WEB-INF/web.xml
```

Add:

<packaging>war</packaging>

Command:

mvn package

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.

LAB ACTION PLAN FOR WEEK 5

Objectives:

Students will able to:

- Learn how to pull, run, stop, start, remove, and inspect containers and images.
- Gain the ability to create, monitor, and troubleshoot running containers.
- Configure and manage networks for container communication.
- Create and manage persistent storage for containers.
- Learn how to list, remove, and manage images efficiently.

Docker:

Docker is an open-source platform that automates the process of building, packaging, and running applications inside lightweight, portable containers, ensuring they run consistently across different environments.

Docker Images:

A read-only template that contains the application code, runtime, libraries, and dependencies. It is like a blueprint for creating containers.

Docker Containers:

A running instance of a Docker image.

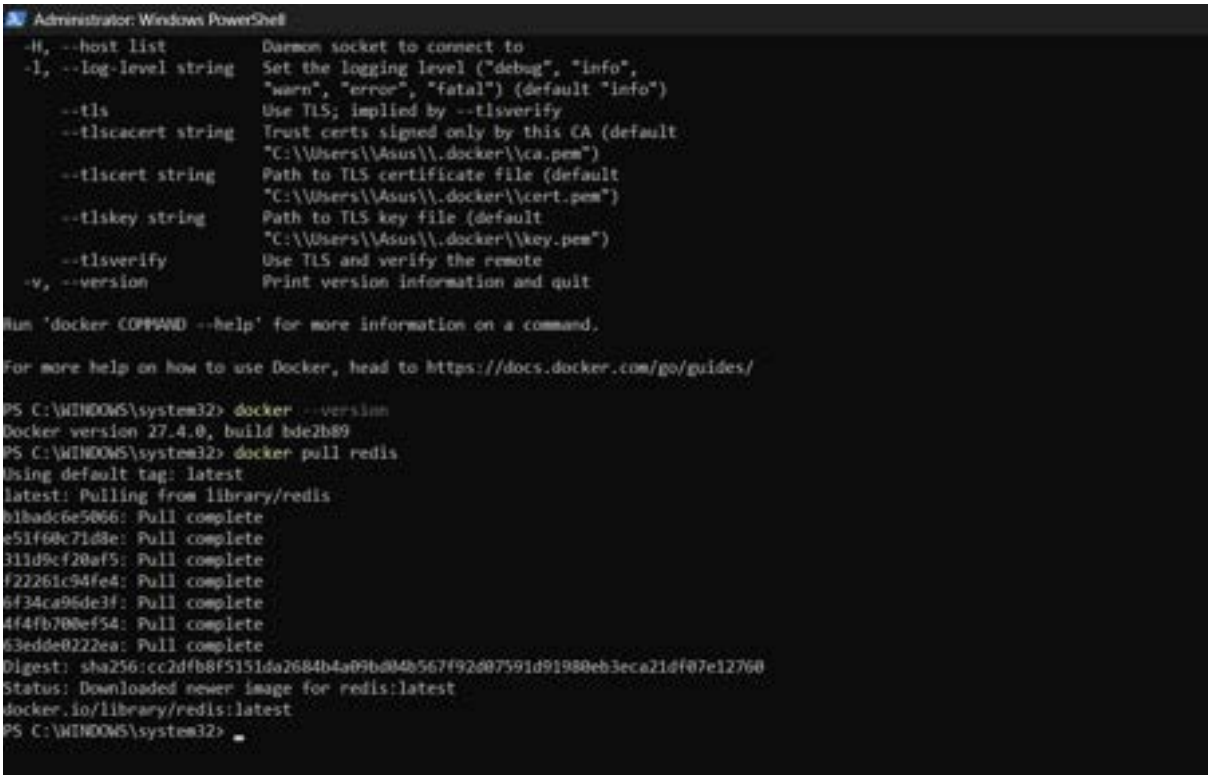
Working with Docker CLI Commands:

Docker CLI Commands with redis

Step 1: Pull the redis Image

Command:

docker pull redis



Step 2: Run a Redis Container

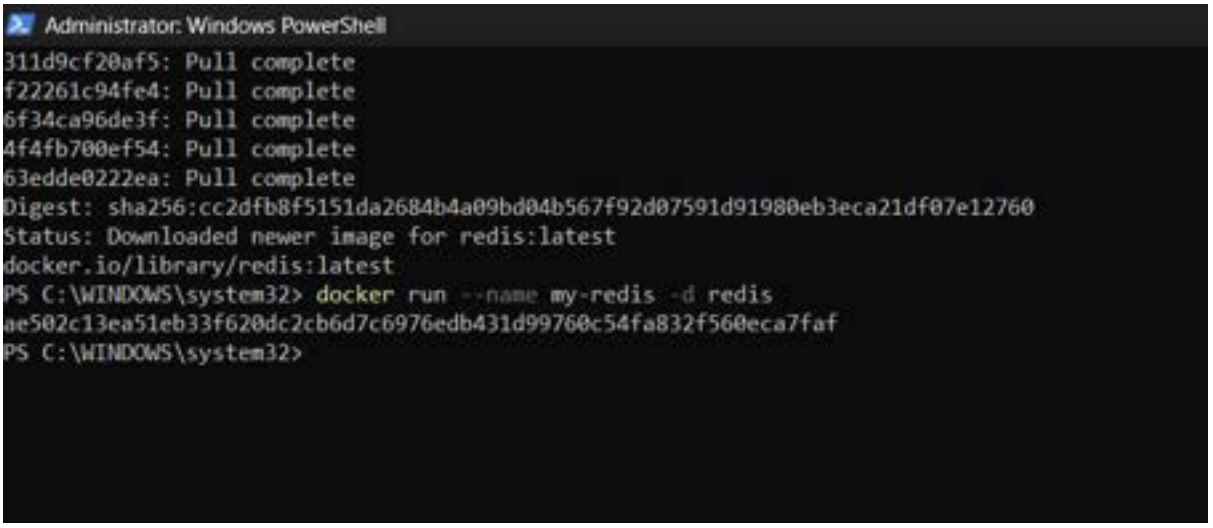
Command:

docker run --name my-redis -d redis

What It Does:

🎬 Creates and starts a container named my-redis from the redis image.

🎬 The -d flag runs the container in the background.



Step 3: Check Running Containers

Command:

docker ps

What It Does:

🎬 Lists all running containers.

Step 4: Access Redis

Command:

docker exec -it my-redis redis-cli

Opens the Redis command-line tool (redis-cli) inside the container.

Example Redis Commands:

127.0.0.1:6379> SET name "Alice"

OK

127.0.0.1:6379> GET name

"Alice"

Step 5: Stop the Redis Container

Command:

docker stop my-redis

What It Does:

🎬 Stops the Redis container but doesn't delete it.

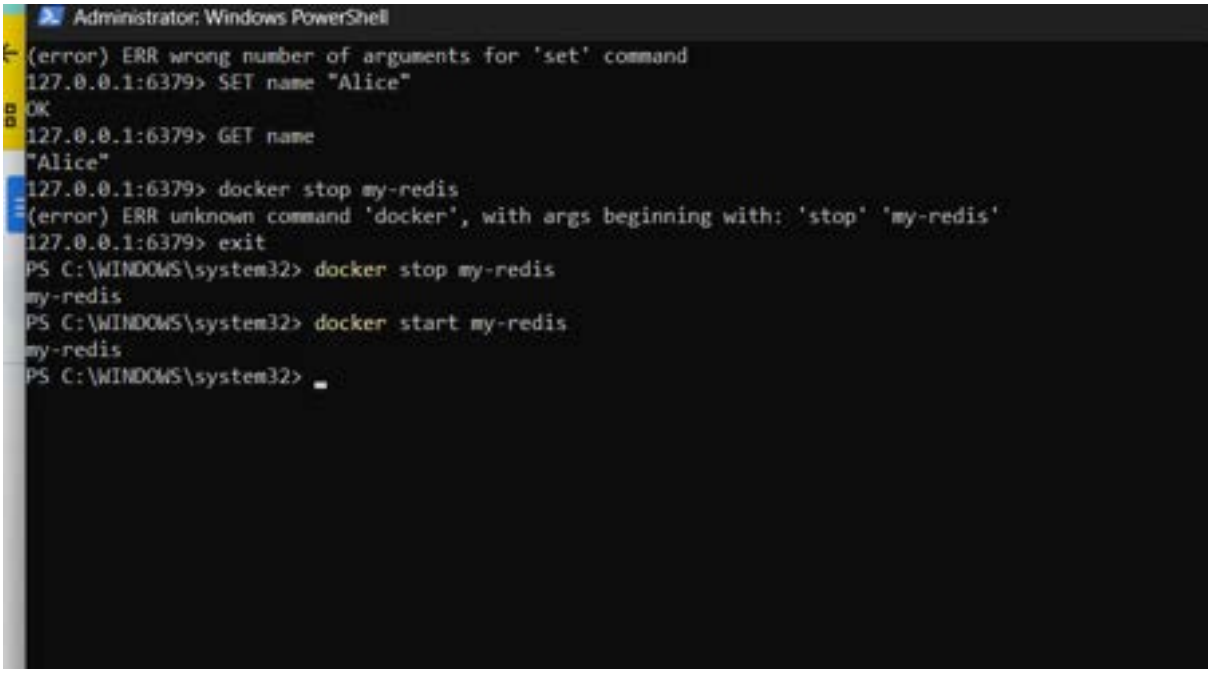
Step 6: Restart the Redis Container

Command:

docker start my-redis

What It Does:

🎬 Restarts the stopped container.



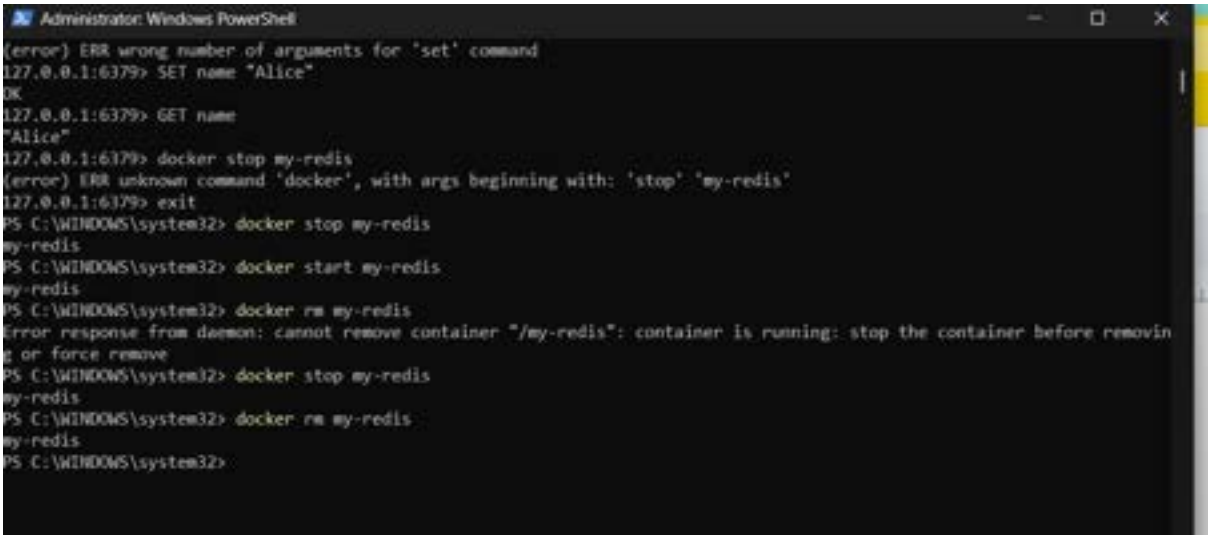
Step 7: Remove the Redis Container

Command:

docker rm my-redis

What It Does:

🎬 Deletes the container permanently.



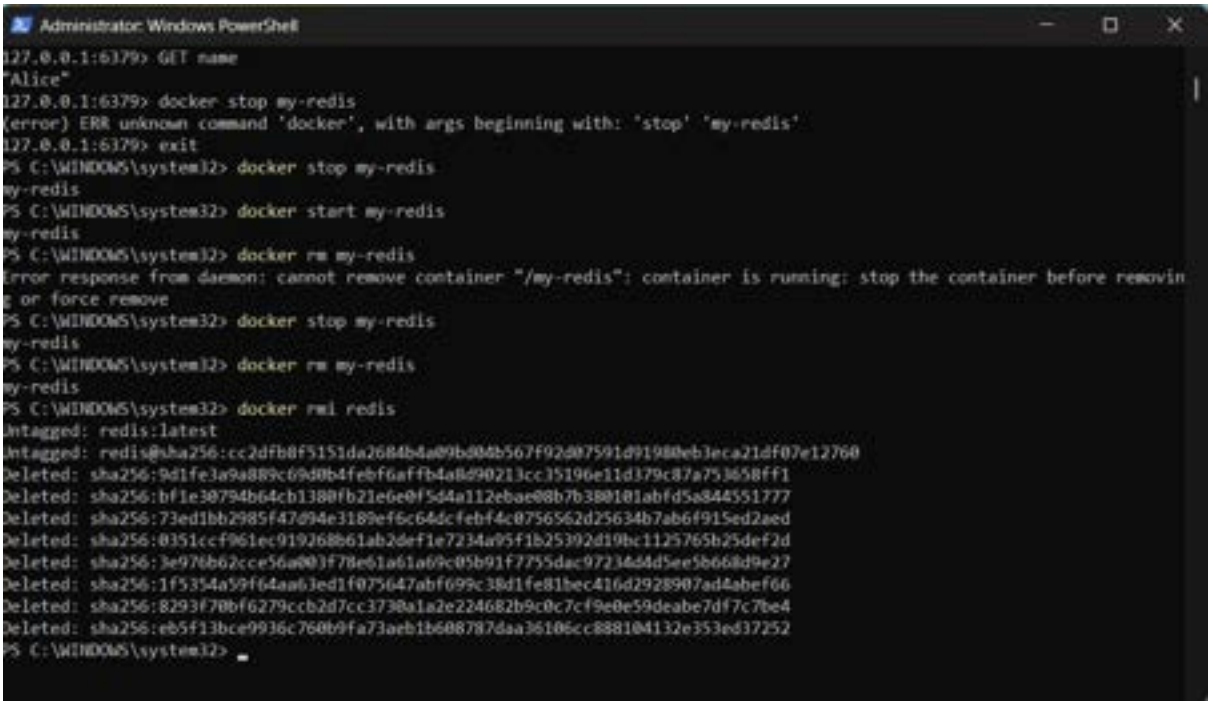
Step 8: Remove the Redis Image

Command:

docker rmi redis

What It Does:

🎬 Deletes the Redis image from your local system.



2.Working with Docker file

A Docker file is a text file with instructions to create a custom Docker image.

Step 1: Set Up Your Folder

1. Windows:

o Create a folder like C:\DockerProjects\Redis.

o Open Git Bash and navigate to the folder:

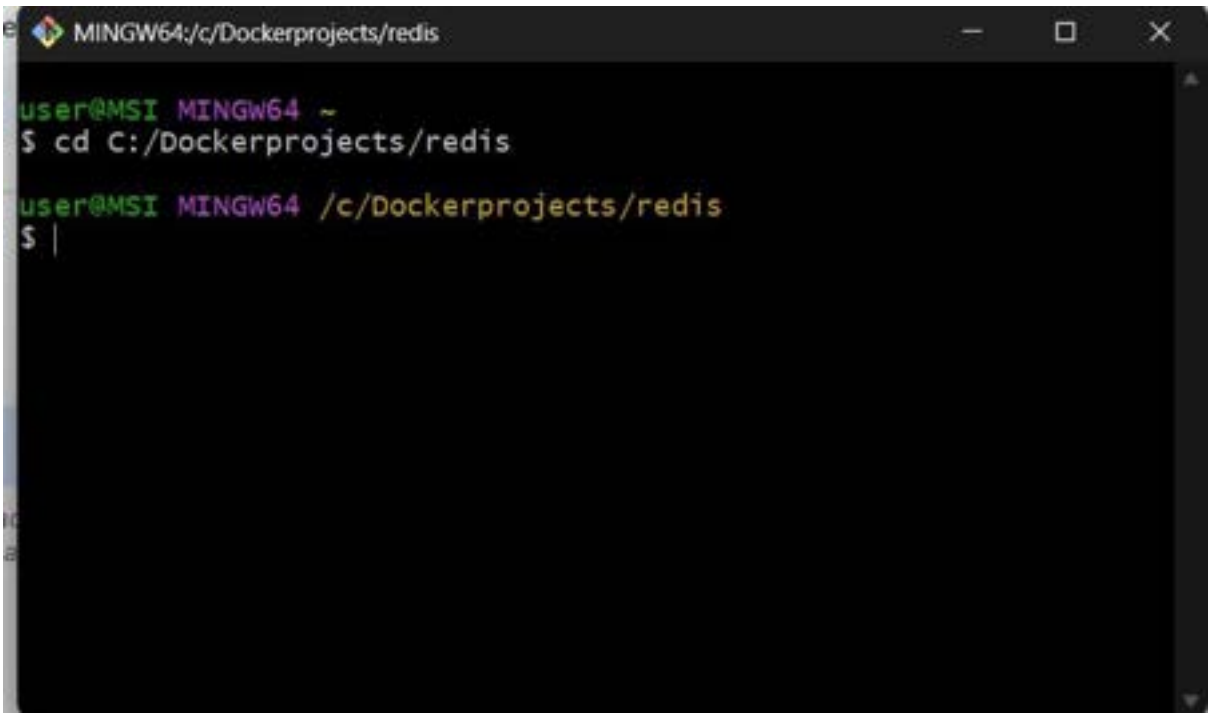
cd /c/DockerProjects/Redis

2. Mac/Linux:

o Create a folder:

mkdir ~/DockerProjects/Redis

cd ~/DockerProjects/Redis



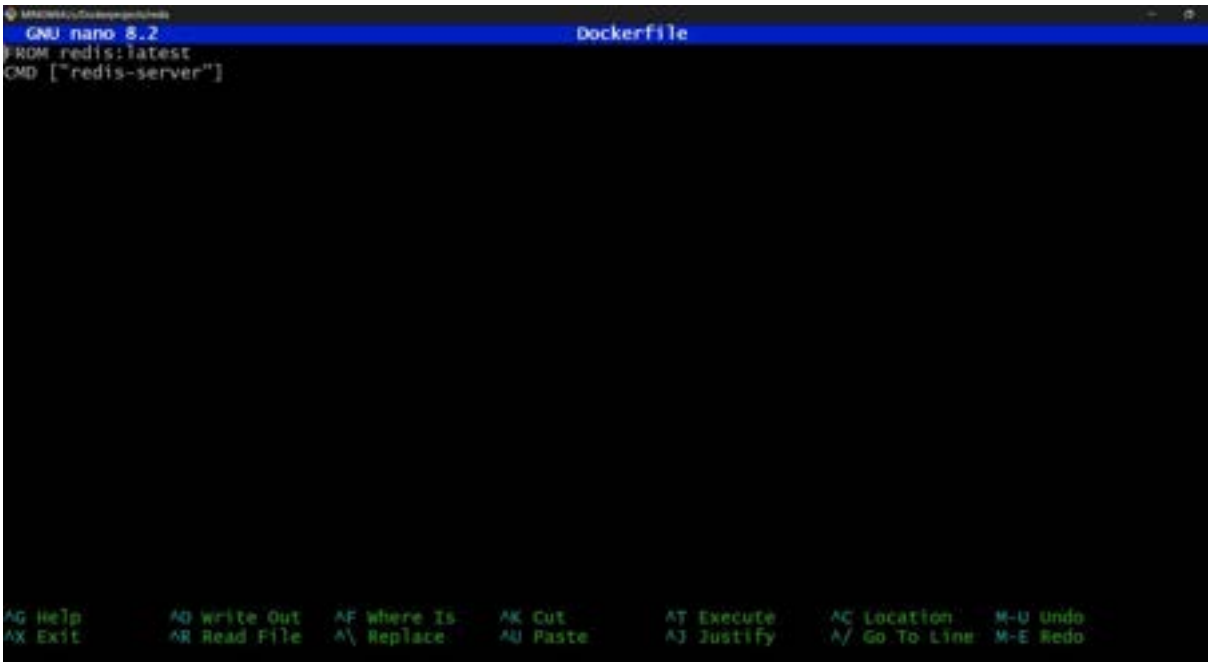
Step 2: Write the Dockerfile

1. Inside the folder, create a file named Dockerfile (no extension).

2. Add the following content:

FROM redis:latest

CMD ["redis-server"]



Docker Commands (Step-by-step):

1. docker build -t redisnew .

What it does:

🎬 This creates (builds) a Docker image using the recipe (Dockerfile) in the current folder (.).

🎬 -t redisnew: Gives the image a name/tag ("redisnew"), so you can find it easily.

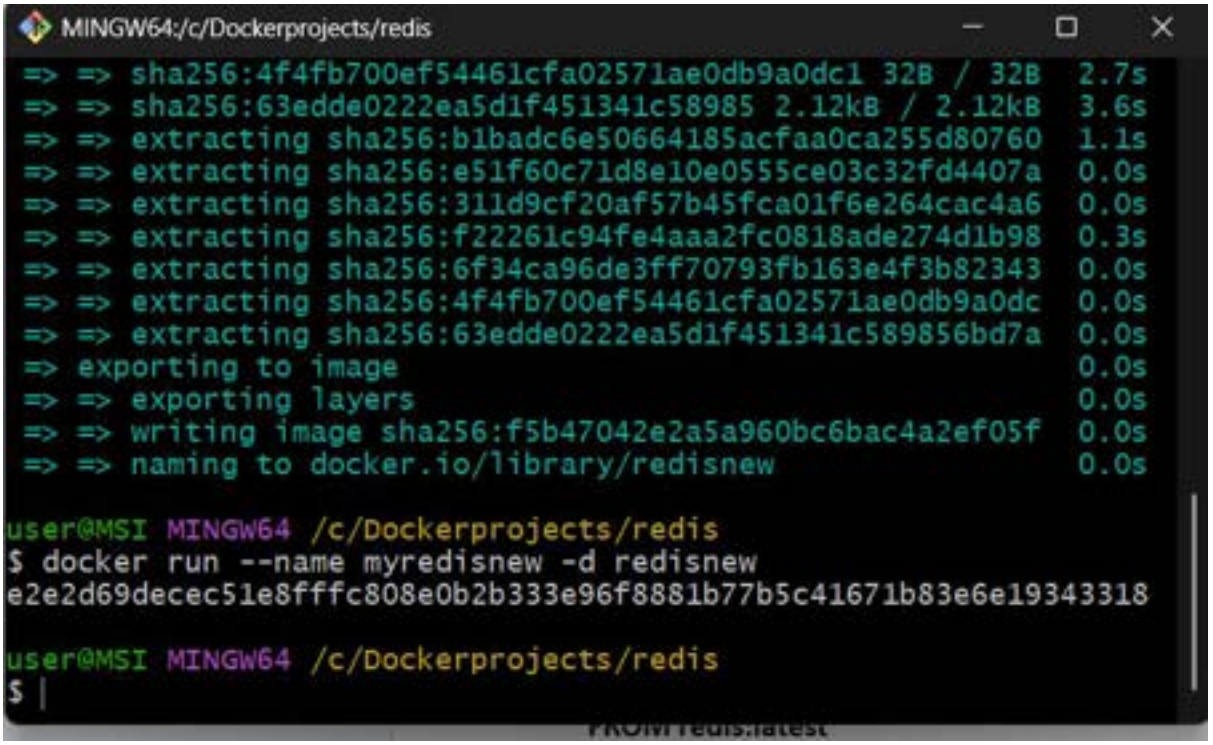
2. docker run --name myredisnew -d redisnew

What it does:

🎬 Starts a new container (mini computer) from the redisnew image.

🎬 --name myredisnew: Names the container "myredisnew" so it’s easy to identify.

🎬 -d: Runs the container in the background.



3. docker ps

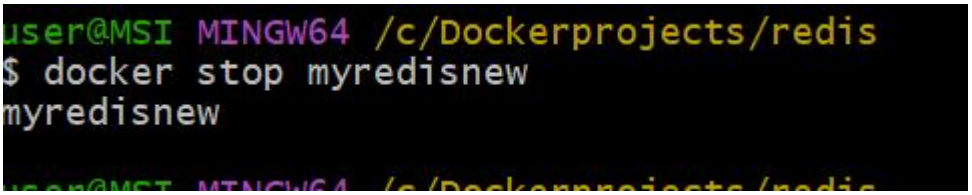
What it does:

🗔 Shows a list of containers that are running right now.

4. docker stop myredisnew

What it does:

🗔 Stops the container named "myredisnew" (like turning off a computer).



5. docker login

What it does:

🗔 Logs you into your Docker Hub account, so you can upload images.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker login
Authenticating with existing credentials...
Login Succeeded
```

6. docker ps -a

What it does:

🎬 Shows a list of all containers, including stopped ones.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
       STATUS                      PORTS     NAMES
e2e2d69decec   redisnew       "docker-entrypoint.s…"   10 minutes
 ago   Exited (0) 4 minutes ago                 myredisnew
8588a6892d40   ubuntuos       "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                 happy_nobel
b8d1ae283578   ubuntuos       "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                 confident_carson
f0e68f11eaa8   ubuntu         "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                 mystifying_rubin
f2fc9ad71114   8a5693abca66   "/bin/bash"              3 months a
go     Exited (0) 3 months ago                     intelligent_mayer
17985489d97a   8a5693abca66   "/bin/bash"              3 months a
go     Exited (0) 3 months ago                     lucid_mahavira
2ab2e7ac2d6a   d11c13fac209   "/bin/bash"              3 months a
go     Exited (255) 3 months ago                   elastic_aryabhata
f685ef32f244   58d643b91c96   "/bin/bash"              3 months a
go     Exited (255) 3 months ago                   great_tharp
4141b686aecf   58d643b91c96   "/bin/bash"              3 months a
go     Exited (1) 3 months ago                     quizzical_jang
29e8140a9f26   58d643b91c96   "/bin/bash"              4 months a
go     Exited (1) 4 months ago                     optimistic_tharp
d2e76707a21d   58d643b91c96   "/bin/bash"              4 months a
go     Exited (255) 4 months ago                   flamboyant_pare
1d5ee7758aac   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     boring_tesla
f9a8b63be0d9   ubuntu         "/bin/bash"              4 months a
go     Exited (127) 4 months ago                   quizzical_khorana
0d08d43b7fbb   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     trusting_moser
240a163e91c8   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     bold_chebyshev
6d2872e4e3aa   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     serene_chandrasekh
ar
7e46e1969cd2   9b24e6f54c27   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     sad_golick
a99350d09dea   9b24e6f54c27   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     charming_agnesi
55a367ad63bc   0de6c07bff58   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     relaxed_yalow
fecf5cd1f362   a0c3eaf8c629   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     brave_lederberg
d1d062568223   a0c3eaf8c629   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     quizzical_swanson
ab8b59225fb4   7d9df11d39f0   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                     affectionate_satos
hi
```

7. docker commit 0e993d2009a1 budarajumadhurika/redis1

What it does:

🎬 Takes a snapshot (saves changes) of the container with ID 0e993d2009a1 and creates a new image called budarajumadhurika/redis1.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker commit e2e2d69decec kannarohith369/redis
sha256:dee473af92a5449ec8dad7a308abf96e2076fc25adf0e996195e976ea7a1d607

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

8. docker images

What it does:

🎬 Lists all images saved on your system.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
kannarohith369/redis    latest    dee473af92a5   35 seconds ago   137MB
redisnew                latest    f5b47042e2a5   13 hours ago     137MB
ubuntuos                latest    ba0725824e67   3 months ago     347MB
<none>                  <none>    0dea92c08728   5 months ago     347MB
<none>                  <none>    6a51c5ed6fdf   5 months ago     347MB
<none>                  <none>    86d36e5d4bac   5 months ago     347MB
<none>                  <none>    75be11f731f5   5 months ago     347MB
linux1                  latest    0d3955e89158   5 months ago     331MB
<none>                  <none>    57ff21045edb   5 months ago     331MB
<none>                  <none>    85107a5f3881   5 months ago     341MB
ubuntu                  latest    a04dc4851cbc   6 months ago     78.1MB
```

9. docker push budarajumadhurika/redis1

What it does:

🎬 Uploads the image budarajumadhurika/redis1 to Docker Hub, so others can download it.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker push kannarohith369/redis
Using default tag: latest
The push refers to repository [docker.io/kannarohith369/redis]
803d653c7ad7: Mounted from library/redis
5f70bf18a086: Mounted from library/redis
cfb4bd9f69a7: Mounted from library/redis
7f32c228fb8a: Mounted from library/redis
0256a7df0b75: Mounted from library/redis
5b34e7b7ec41: Mounted from library/redis
eb5f13bce993: Mounted from library/redis
latest: digest: sha256:ace3d716af5c57aa3441219bcf1ed67b72337a80b1f82cd13
d3875b1059b8ea7 size: 1776

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

10. docker rm 0e993d2009a1

What it does:

🎬 Deletes the container with ID 0e993d2009a1.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm kannarohith369/redis
Error response from daemon: No such container: kannarohith369/redis

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm kannarohith369/redis
Error response from daemon: No such container: kannarohith369/redis

user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm e2e2d69dece
e2e2d69dece

user@MSI MINGW64 /c/Dockerprojects/redis
```

11. docker rmi budarajumadhurika/redis1

What it does:

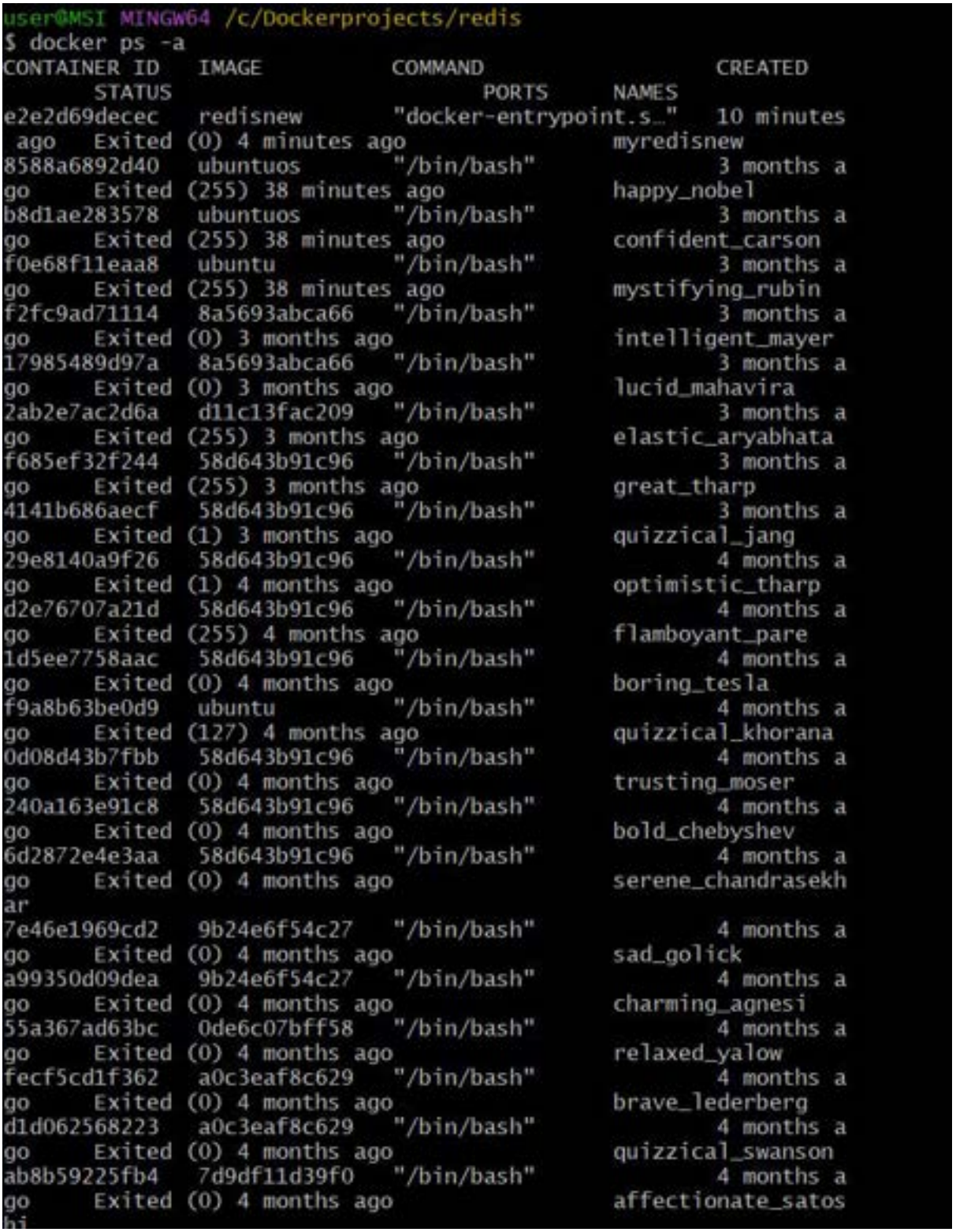
🎬 Deletes the image budarajumadhurika/redis1 from your system.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker pull kannarohith369/redis
Using default tag: latest
latest: Pulling from kannarohith369/redis
Digest: sha256:ace3d716af5c57aa3441219bcf1ed67b72337a80b1f82cd13d3875b10
59b8ea7
Status: Image is up to date for kannarohith369/redis:latest
docker.io/kannarohith369/redis:latest

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

12. docker ps -a

What it does:

Shows all containers again to confirm changes.



```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
       STATUS                          PORTS     NAMES
e2e2d69decec   redisnew       "docker-entrypoint.s…"   10 minutes
 ago   Exited (0) 4 minutes ago                       myredisnew
8588a6892d40   ubuntuos       "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                    happy_nobel
b8d1ae283578   ubuntuos       "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                    confident_carson
f0e68f11eaa8   ubuntu         "/bin/bash"              3 months a
go     Exited (255) 38 minutes ago                    mystifying_rubin
f2fc9ad71114   8a5693abca66   "/bin/bash"              3 months a
go     Exited (0) 3 months ago                        intelligent_mayer
17985489d97a   8a5693abca66   "/bin/bash"              3 months a
go     Exited (0) 3 months ago                        lucid_mahavira
2ab2e7ac2d6a   d11c13fac209   "/bin/bash"              3 months a
go     Exited (255) 3 months ago                      elastic_aryabhata
f685ef32f244   58d643b91c96   "/bin/bash"              3 months a
go     Exited (255) 3 months ago                      great_tharp
4141b686aecf   58d643b91c96   "/bin/bash"              3 months a
go     Exited (1) 3 months ago                        quizzical_jang
29e8140a9f26   58d643b91c96   "/bin/bash"              4 months a
go     Exited (1) 4 months ago                        optimistic_tharp
d2e76707a21d   58d643b91c96   "/bin/bash"              4 months a
go     Exited (255) 4 months ago                      flamboyant_pare
1d5ee7758aac   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        boring_tesla
f9a8b63be0d9   ubuntu         "/bin/bash"              4 months a
go     Exited (127) 4 months ago                      quizzical_khorana
0d08d43b7fbb   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        trusting_moser
240a163e91c8   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        bold_chebyshev
6d2872e4e3aa   58d643b91c96   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        serene_chandrasekh
ar
7e46e1969cd2   9b24e6f54c27   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        sad_golick
a99350d09dea   9b24e6f54c27   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        charming_agnesi
55a367ad63bc   0de6c07bff58   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        relaxed_yalow
fecf5cd1f362   a0c3eaf8c629   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        brave_lederberg
d1d062568223   a0c3eaf8c629   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        quizzical_swanson
ab8b59225fb4   7d9df11d39f0   "/bin/bash"              4 months a
go     Exited (0) 4 months ago                        affectionate_satos
hi
```

13. docker logout

What it does:

🎬 Logs you out of Docker Hub.

14. docker pull budarajumadhurika/redis1

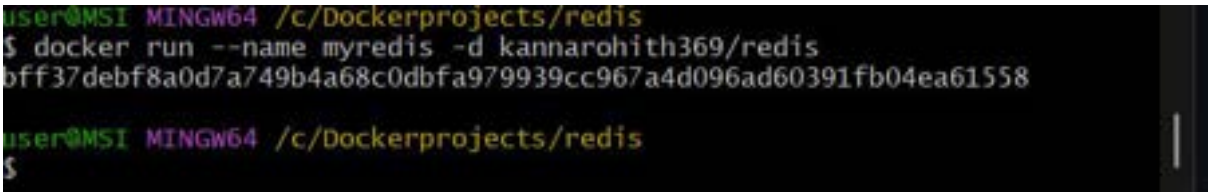
What it does:

🎬 Downloads the image budarajumadhurika/redis1 from Docker Hub.

15. docker run --name myredis -d budarajumadhurika/redis1

What it does:

🎬 Starts a new container using the image budarajumadhurika/redis1.



16. docker exec -it myredis redis-cli

What it does:

🎬 Opens the Redis command-line interface (like a terminal) inside the running container myredis.

17. SET name "Abcdef"

What it does:

🎬 Saves a key-value pair in Redis (key = name, value = Abcdef).

18. GET name

What it does:

🎬 Retrieves the value of the key name from Redis (it will return "Abcdef").

19. exit

What it does:

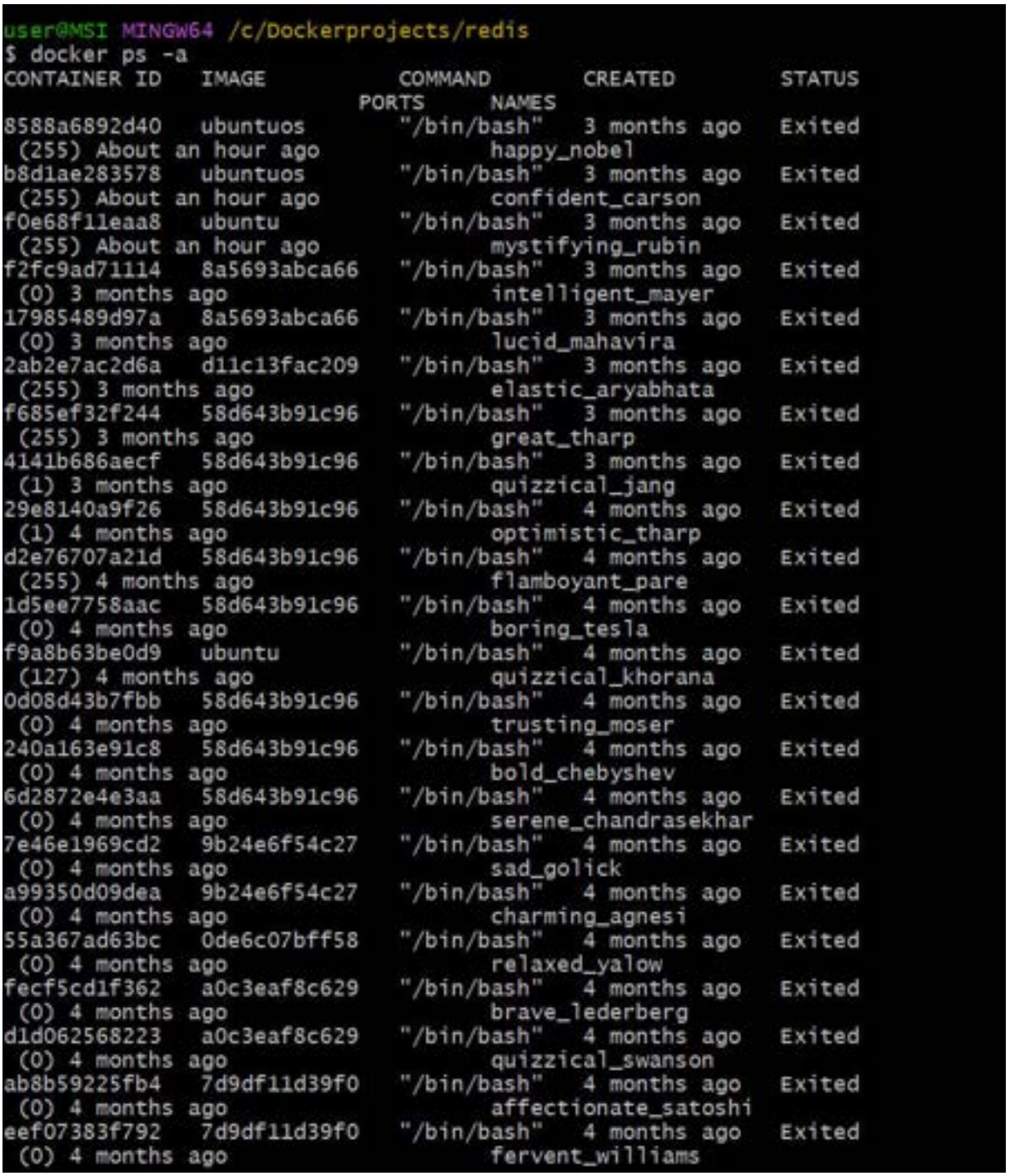
🎬 Exits the Redis CLI.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "rohith"
OK
127.0.0.1:6379> GET name
"rohith"
127.0.0.1:6379> exit

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

20. docker ps -a

What it does:

🎬 Shows all containers again to check their status.



```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND       CREATED         STATUS
                         PORTS     NAMES
8588a6892d40   ubuntuos       "/bin/bash"   3 months ago    Exited
 (255) About an hour ago                 happy_nobel
b8d1ae283578   ubuntuos       "/bin/bash"   3 months ago    Exited
 (255) About an hour ago                 confident_carson
f0e68f11eaa8   ubuntu         "/bin/bash"   3 months ago    Exited
 (255) About an hour ago                 mystifying_rubin
f2fc9ad71114   8a5693abca66   "/bin/bash"   3 months ago    Exited
 (0) 3 months ago                        intelligent_mayer
17985489d97a   8a5693abca66   "/bin/bash"   3 months ago    Exited
 (0) 3 months ago                        lucid_mahavira
2ab2e7ac2d6a   d11c13fac209   "/bin/bash"   3 months ago    Exited
 (255) 3 months ago                      elastic_aryabhata
f685ef32f244   58d643b91c96   "/bin/bash"   3 months ago    Exited
 (255) 3 months ago                      great_tharp
4141b686aecf   58d643b91c96   "/bin/bash"   3 months ago    Exited
 (1) 3 months ago                        quizzical_jang
29e8140a9f26   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (1) 4 months ago                        optimistic_tharp
d2e76707a21d   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (255) 4 months ago                      flamboyant_pare
1d5ee7758aac   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        boring_tesla
f9a8b63be0d9   ubuntu         "/bin/bash"   4 months ago    Exited
 (127) 4 months ago                      quizzical_khorana
0d08d43b7fbb   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        trusting_moser
240a163e91c8   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        bold_chebyshev
6d2872e4e3aa   58d643b91c96   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        serene_chandrasekhar
7e46e1969cd2   9b24e6f54c27   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        sad_golick
a99350d09dea   9b24e6f54c27   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        charming_agnesi
55a367ad63bc   0de6c07bff58   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        relaxed_yalow
fecf5cd1f362   a0c3eaf8c629   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        brave_lederberg
d1d062568223   a0c3eaf8c629   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        quizzical_swanson
ab8b59225fb4   7d9df11d39f0   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        affectionate_satoshi
eef07383f792   7d9df11d39f0   "/bin/bash"   4 months ago    Exited
 (0) 4 months ago                        fervent_williams
```

21. docker stop myredis

What it does:

🎬 Stops the container myredis.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker stop myredis
myredis
```

22. docker rm 50a6e4a9c326

What it does:

🎬 Deletes the container with ID 50a6e4a9c326.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rm bff37debf8a0
bff37debf8a0
```

23. docker images

What it does:

🎬 Lists all images again to confirm which ones remain.

```
$ docker images
REPOSITORY              TAG       IMAGE ID       CREATED          S
IZE
kannarohith369/redis    latest    dee473af92a5   9 minutes ago    1
37MB
redisnew                latest    f5b47042e2a5   14 hours ago     1
37MB
ubuntuos                latest    ba0725824e67   3 months ago     3
47MB
<none>                  <none>    0dea92c08728   5 months ago     3
47MB
<none>                  <none>    6a51c5ed6fdf   5 months ago     3
47MB
<none>                  <none>    86d36e5d4bac   5 months ago     3
47MB
<none>                  <none>    75be11f731f5   5 months ago     3
47MB
linux1                  latest    0d3955e89158   5 months ago     3
31MB
<none>                  <none>    57ff21045edb   5 months ago     3
31MB
<none>                  <none>    85107a5f3881   5 months ago     3
41MB
ubuntu                  latest    a04dc4851cbc   6 months ago     7
8.1MB

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

24. docker rmi budarajumadhurika/redis1

What it does:

🎬 Deletes the image budarajumadhurika/redis1 again.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker rmi kannarohith369/redis
Untagged: kannarohith369/redis:latest
Untagged: kannarohith369/redis@sha256:ace3d716af5c57aa3441219bcf1
ed67b72337a80b1f82cd13d3875b1059b8ea7
Deleted: sha256:dee473af92a5449ec8dad7a308abf96e2076fc25adf0e9961
95e976ea7a1d607

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

Step 3: Remove Login Credentials (Optional)

If you no longer need to be logged in, you can log out:

docker logout

What It Does:

🎬 Logs you out from Docker Hub and removes your stored credentials.

```
user@MSI MINGW64 /c/Dockerprojects/redis
$ docker logout
Removing login credentials for https://index.docker.io/v1/

user@MSI MINGW64 /c/Dockerprojects/redis
$
```

Conclusion:

Through this hands-on exploration of Docker commands and Dockerfile usage, students will get a clear picture about the end-to-end container workflow—from pulling images, creating and running containers, to customizing them with Dockerfiles. By practicing key commands such as docker build, docker run, docker ps, and docker exec, students will gain practical skill s in managing containerized environments. The Dockerfile exercise reinforced the importance of layered builds, efficient image creation, and environment reproducibility. Overall, these activities strengthened the understanding of containerization principles, enabling building, testing and deploying applications in isolated, consistent environments.

LAB ACTION PLAN FOR WEEK 6

Objectives:

Students will able to:

- Learn how to define and run multiple interdependent services (e.g., web server, database) in a single configuration file.
- Gain skills in writing and interpreting docker-compose.yml files for service setup.
- Deploy the same setup across different machines without manual configuration.
- Configure container networking and persistent storage within Compose.
- Reduce setup time and enable faster iteration during application development.

Students will performHands-on running multi container applicationsusing docker composeand make document that include the execution of each step along withscenario-based question solutions after completing the tasks.

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications. It allows you to define services, networks, and volumes that your application needs, all in a single file. This makes it easier to manage complex applications that require multiple containers (e.g., a web server and a database).

Structure of a Docker Compose File

A Docker Compose file is written in YAML (YAML Ain't Markup Language), a human-readable data format used for configuration. In the Docker context, it helps define the services, networks, and volumes needed for an application.

Basic Components of a Docker Compose YAML File

1. **Version**: Specifies which version of the Docker Compose file format you're using. Each version comes with its own set of features.

   ```
   version:'3.8'
   ```

2. **Services**: This section defines all the containers you want to run for your application. Each service represents a container (like a web server or database).

   Each service includes:

- **Image**: The Docker image to use.
- **Ports**: Ports to map between the container and your host machine.
- **Environment**: Environment variables required by the service.
- **Dependencies**: Which other services a container depends on.

Example:

```
services:
web:
image:nginx
ports:
-"8060:80"
db:
image:tomee
ports:
-"8050:8080"
```

3. **Networks**: Define networks to allow different services to communicate with each other. Docker Compose automatically creates a default network if not specified.

4. **Volumes**: Used for persistent storage, allowing data to persist even after the container stops or is removed.

Example Docker Compose File (Simple)

Here's a basic example of a Docker Compose file that runs WordPress and MySQL together:

```
version: '3.8'  # Docker Compose file format version

services:
wordpress:  # WordPress service
   image: wordpress:latest
   ports:
     - "8080:80"  # Map port 80 of the container to port 8080 of the host
   environment:
     WORDPRESS_DB_HOST: db:3306  # Database host
     WORDPRESS_DB_USER: wordpress
     WORDPRESS_DB_PASSWORD: wordpress
     WORDPRESS_DB_NAME: wordpress
depends_on:
```

```
      - db  # Ensures the db service starts first

  db:  # MySQL service
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
```

How to Run It

To run a multi-container setup like the one above:

1. **Save the file as docker-compose.yml.**

   Or

   docker-compose.yaml

2. **To Start the compose**

   docker-compose up –d

3. **To stop the containers**

   docker-compose down

4. **To scale the container**

   docker-compose up --scale<service name>=2 -d

1.Define and run multiple interdependent services

Task:

I. Create a new folder compose-lab

Inside it, create a file docker-compose.yml with the following content:

```
version: "3.9"
services:
  web:
```

image: nginx:latest

ports:

- "8080:80"

db:

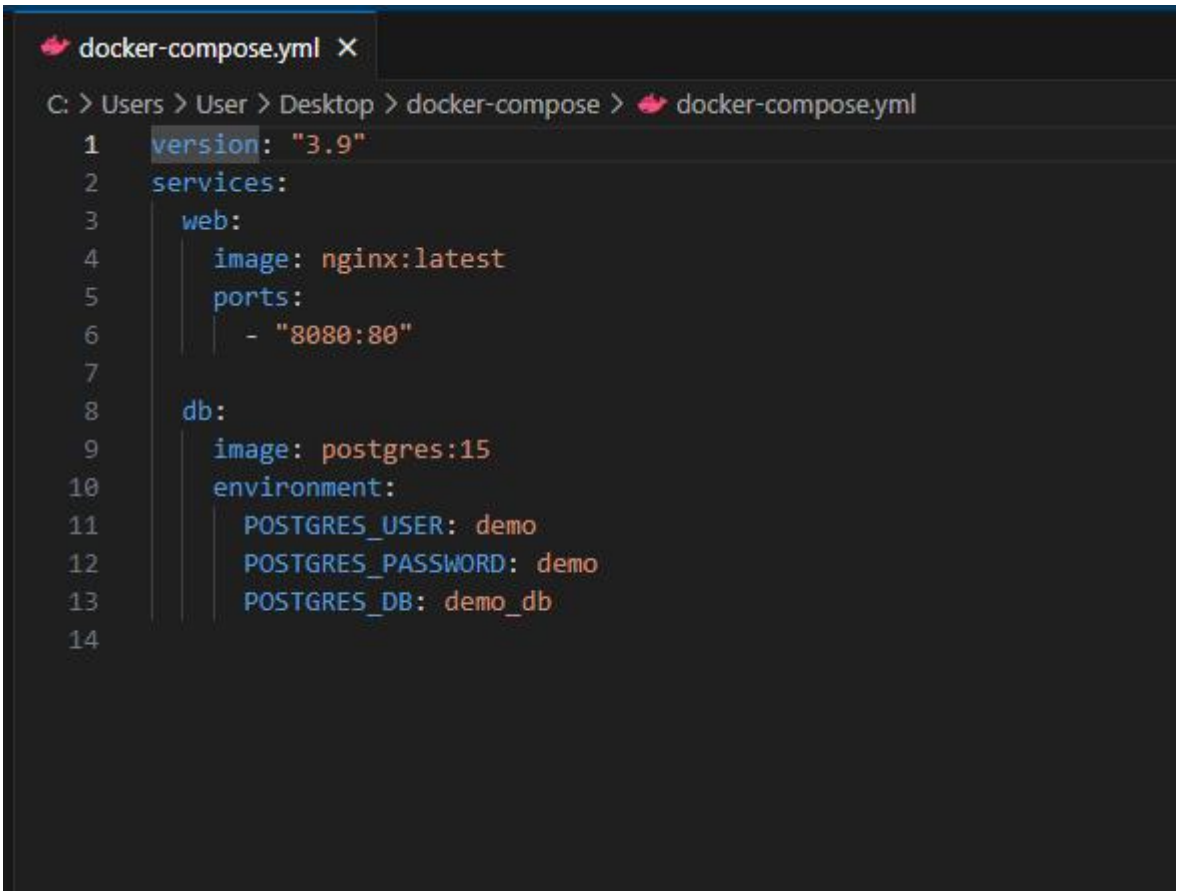
image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

POSTGRES_DB: demo_db



II.Run the setup:

docker compose up -d

III.Open your browser and visit: http://localhost:8080.

IV.Expected Output:

Nginx welcome page is displayed.

db container runs in the background.

2.Write and interpret docker-compose.yml files

Task:

I. Modify docker-compose.yml to add a Redis cache:

```
redis:
  image: redis:alpine
```

II.Add a depends_on so web waits for Redis:

```
 web:
  image: nginx:latest
  ports:
    - "8080:80"
depends_on:
    - redis
```

docker-compose.yml

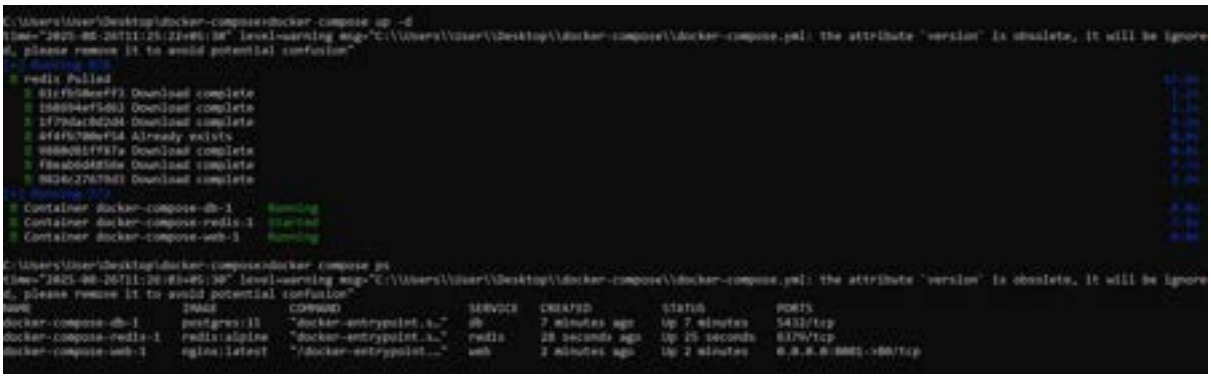
C: > Users > User > Desktop > docker-compose > docker-compose.yml

```yaml
version: "3.9"
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    depends_on:
      - redis

  redis:
    image: redis:alpine

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
```

III.Restart the setup:

docker compose up –d

docker compose ps



IV.Expected Output:

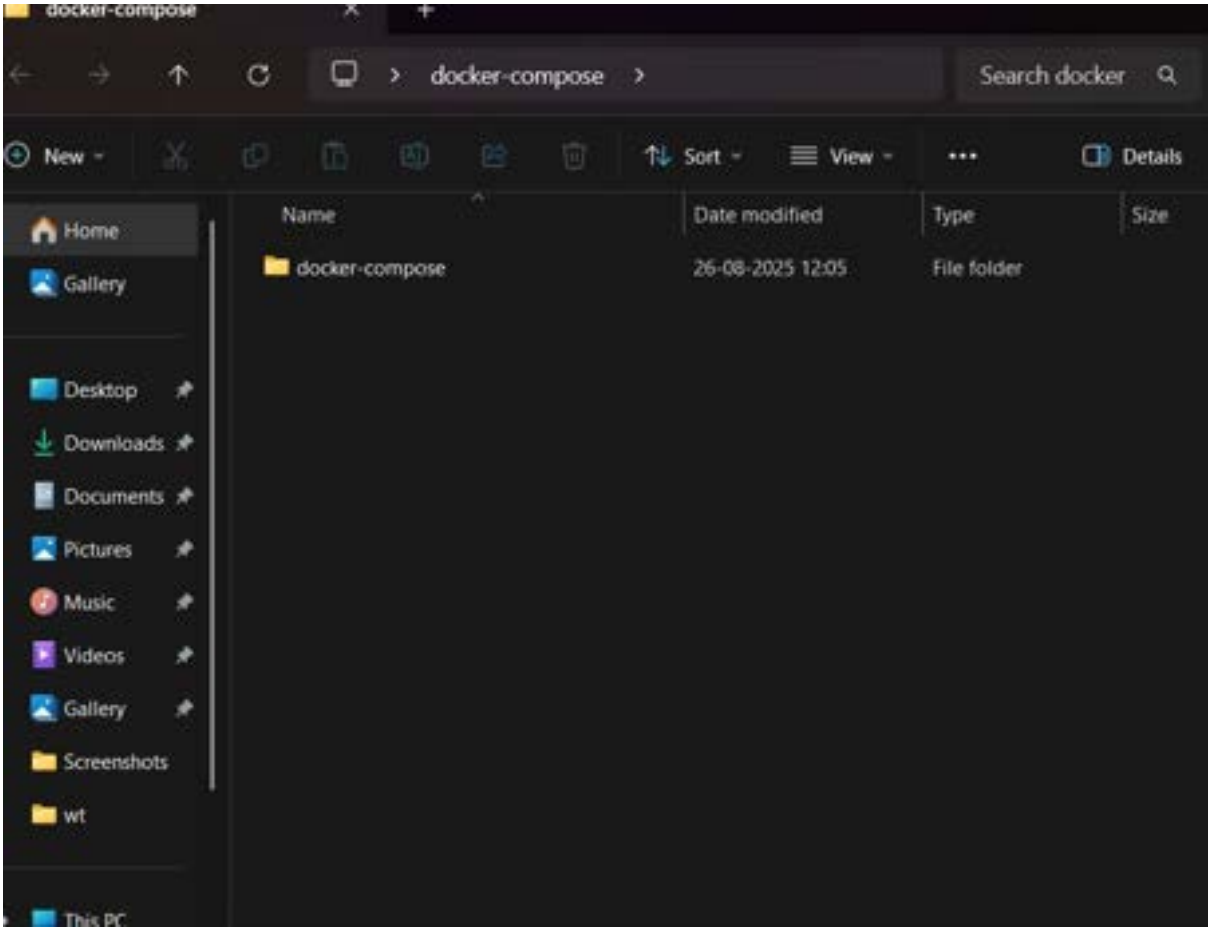
Three services (web, db, redis) are listed as running.

3.Deploy across different machines

Task:

I. Zip your compose-lab folder.

Transfer it to another machine with Docker Compose installed.



II.Run:

docker compose up -d

Check that Nginx and Postgres work there as well.

III.Expected Output:

The same services run on the new machine without changes.

Hello Docker Compose!

4.Networking and persistent storage

Task:

I. **Update your docker-compose.yml to add a custom network and volume:**

```
networks:
  app-net:

volumes:
db-data:

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    networks:
      - app-net
depends_on:
      - db

db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net
```

docker-compose.yml

C: > Users > User > Desktop > docker-compose > docker-compose.yml

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    networks:
      - app-net
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: demo
      POSTGRES_PASSWORD: demo
      POSTGRES_DB: demo_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  db-data:
```

II.Run:

docker compose up –d

III.Insert some data into Postgres (optional with psql).

```
C:\Users\User\Desktop\docker-compose>docker compose exec db psql -U demo -d demo_db
psql (15.14 (Debian 15.14-1.pgdg13+1))
Type "help" for help.

demo_db=# CREATE TABLE test (id SERIAL PRIMARY KEY, name VARCHAR(50));
CREATE TABLE
demo_db=# INSERT INTO test (name) VALUES ('Sample data');
INSERT 0 1
demo_db=# SELECT * FROM test;
 id |    name
----+-------------
  1 | Sample data
(1 row)

demo_db=# \q
```

IV.Remove containers:

docker compose down

```
[+] Running 3/3esktop\docker-compose>docker compose down
 ⠿ Container docker-compose-web-1  Removed
 ⠿ Container docker-compose-db-1   Removed
 ⠿ Network docker-compose_app-net  Removed
```

V.Start again:

docker compose up –d

VI.Expected Output:

Database data persists across restarts.

Services communicate via the app-net network using service names.

5.Faster iteration during development

Task:

I. Create a simple Flask app in app.py:

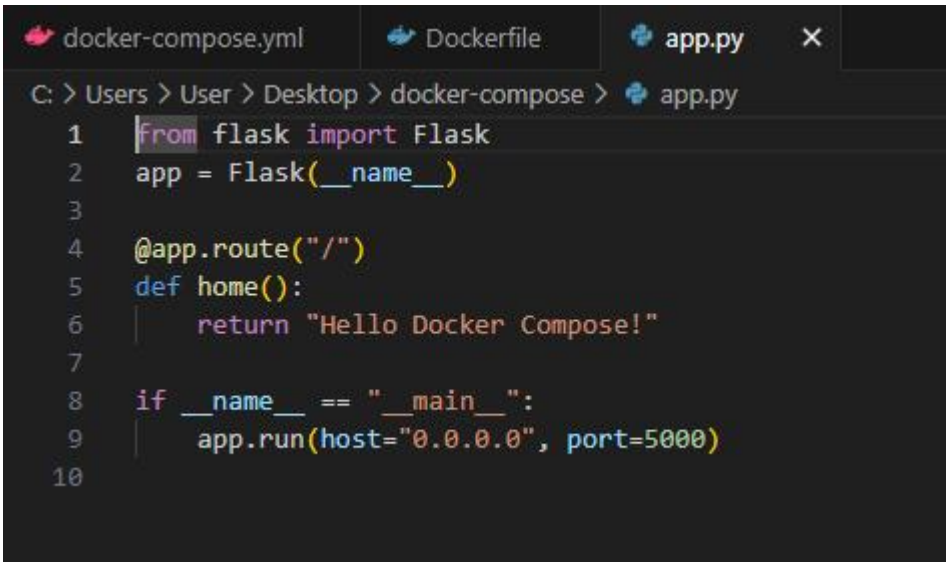
```
from flask import Flask

app = Flask(__name__)

@app.route("/")
```

def home():

    return "Hello from Flask + Docker!"

if __name__ == "__main__":

app.run(host="0.0.0.0", port=5000)



II.Add a Dockerfile in the same folder:

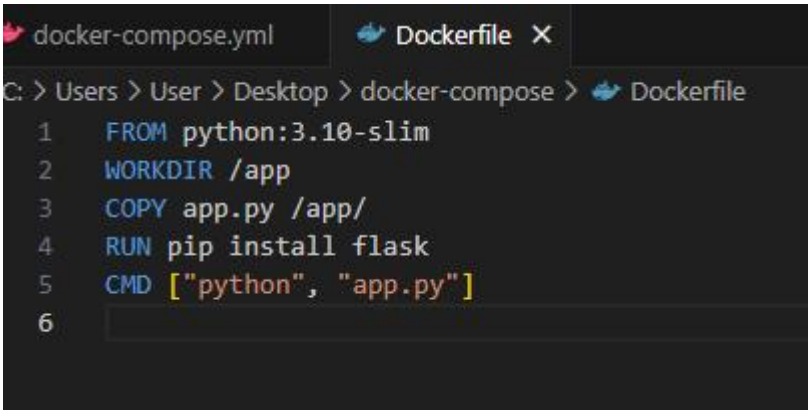
FROM python:3.10-slim

WORKDIR /app

COPY app.py /app/

RUN pip install flask

CMD ["python", "app.py"]



III.Update docker-compose.yml:

web:

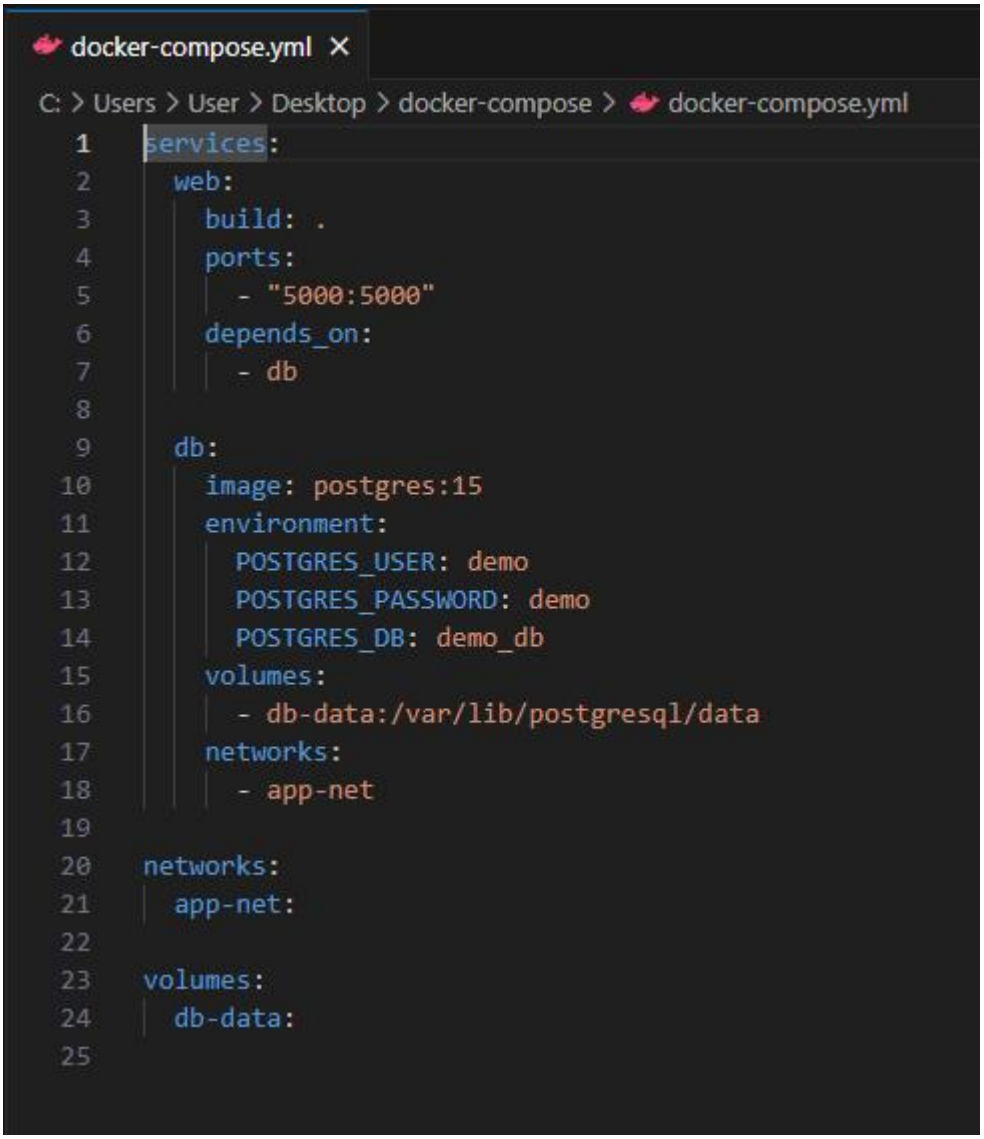
build: .

ports:

- "5000:5000"

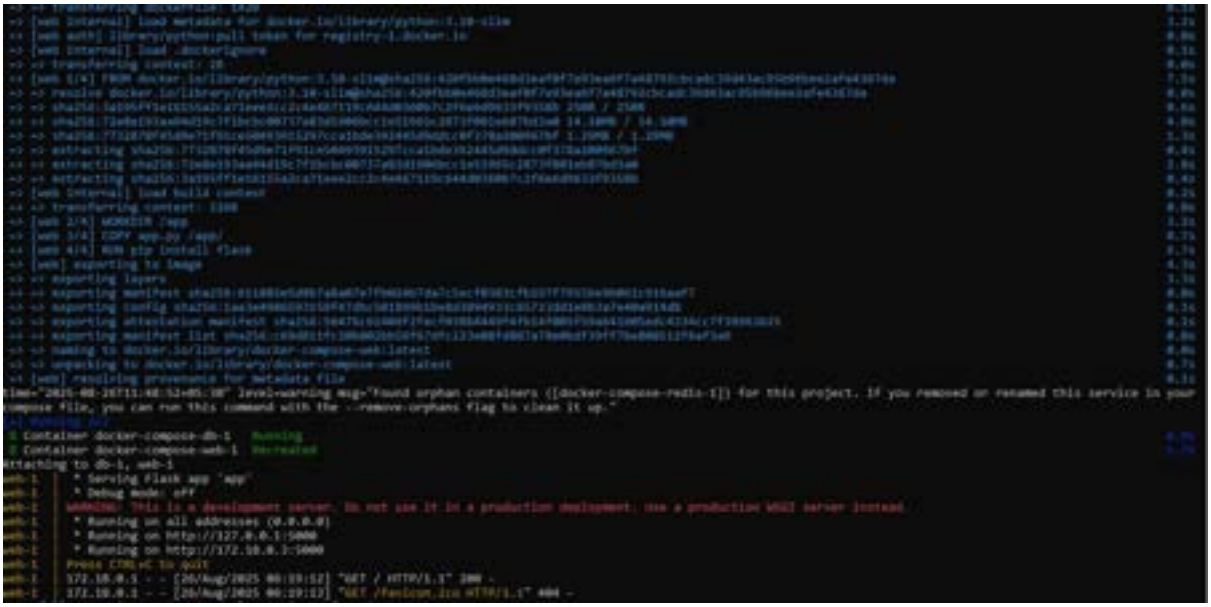
depends_on:

- db

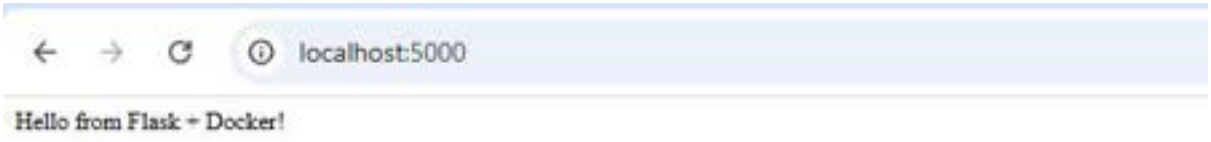


IV.Run:

docker compose up –build



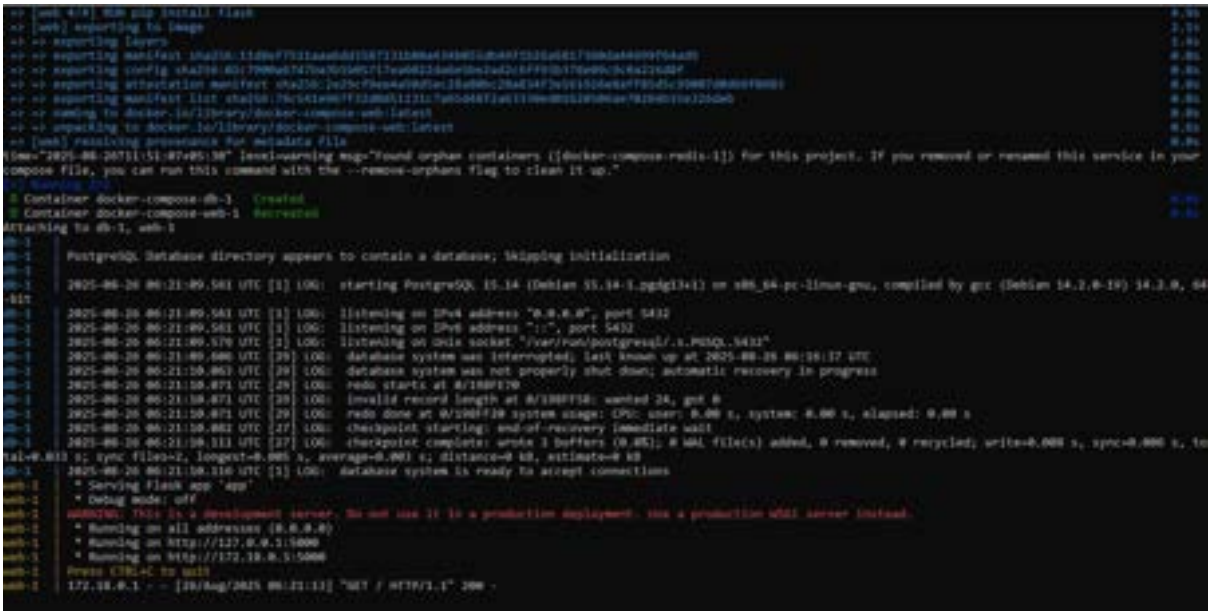
Visit http://localhost:5000.



Change the return text in app.py (e.g., "Hello Docker Compose!").

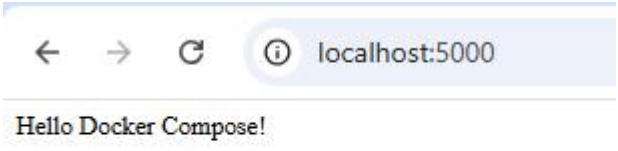
V.Rebuild:

docker compose up –build



VI.Expected Output:

New message appears instantly after rebuild.



Conclusion:

Docker Compose simplifies multi-container application management by allowing developers to define, configure, and run services in a single YAML file. It streamlines deployment, ensures consistent environments, and reduces manual setup, making it an essential tool for orchestrating complex, container-based applications efficiently.

LAB ACTION PLAN FOR WEEK 7

Objective:

To enable students to:

- Understand the structure and lifecycle of a Maven project.
- Build and package Java and Web applications using Maven.
- Add dependencies using **pom.xml**, compile and test using plugins.
- Resolve errors and conflicts arising from dependency mismatches.
- Work with parent and multi-module Maven projects.
- Generate executable JARs and deployable WARs using Maven.

Students must **document observations**, include **screenshots of executions**, and answer **scenario-based questions** after completing the tasks.

1. Understanding Maven Project Structure

Maven standardizes the project structure for both Java and web-based applications.src/

```
├── main/
│   ├── java/        → Application source code
│   └── resources/   → Configuration files like config.properties
└── test/
    ├── java/        → Unit test source code
    └── resources/   → Test resources
```

The compiled files and reports are generated in the **`target/`** directory after a successful build.

2. Steps to Perform Maven Build and Testing

----mvn clean install

- **`clean`:** Deletes the previous build (**`target/`** folder)
- **`install`:** Builds, tests, and installs the package to local **`.m2`** repository

After execution:

- **`target/`** folder contains compiled **`.class`** files and the final **`.jar`** or **`.war`**

- Artifact is stored in:
  `~/.m2/repository/groupId/artifactId/version/`

Add a Dependency (e.g., Gson):

In **`pom.xml`**:

<dependency>

<groupId>com.google.code.gson</groupId>

<artifactId>gson</artifactId>

<version>2.10</version>

</dependency>

After running:

mvn clean install

Check:

- Gson JAR is downloaded to **`.m2/repository/com/google/code/gson/gson/`**
- If version is wrong or not found → **`BUILD FAILURE`** with dependency resolution error.

Creation of Maven Java Project

Step 1. Open Eclipse IDE

└— 1.1. Launch Eclipse workspace

Step 2. Install Maven Plugin (if not installed)

└— 2.1. Go to "Help" in the top menu

└— 2.1.1. Click "Eclipse Marketplace"

└— 2.1.2. Search for "Maven Integration for Eclipse"

└— 2.1.3. Install the plugin if not already installed

Step 3. Create a New Maven Project

└— 3.1. File -> New -> Project...

└— 3.1.1. Expand "Maven"

└── 3.1.2. Select "Maven Project" and click "Next"

Step 4. Set Project Configuration

└── 4.1. Select workspace location (default or custom)

└── 4.2. Click "Next"

Step 5. Choose Maven Archetype

└── 5.1. Select an archetype(e.g "org.apache.maven.archetypes -> maven-archetype-quickstart 1.4 ")

└── 5.2. Click "Next"

Step 6. Define Project Metadata

└── 6.1. Group ID: (e.g., com.example)

└── 6.2. Artifact ID: (e.g., my-maven-project)

└── 6.3. Version: (default is usually fine)

└── 6.4. Click "Finish"

In Console, artifacts are grouped. When prompted with Y/N, type 'Y'.

Step 7. Maven Project Created

└── 7.1. Project structure is generated with a standard Maven layout

└── 7.2. Includes:

└── src/main/java (for Java source code)

└── src/test/java (for test code)

└── pom.xml (Maven configuration file)

Step 8. Update Project Settings (if needed)

└── 8.1. Right-click on the project -> Maven -> Update Project...

└── 8.2. Ensure dependencies are up to date

Step 9. Build and Run Maven Project

└── 9.1. Right-click on App.java -> Run As -> Maven Clean

└── 9.1.1. Right-click on App.java -> Run As -> Maven Install

└── 9.1.2. Right-click on App.java -> Run As -> Maven Test

└── 9.1.3. Right-click on App.java -> Run As -> Maven Build

Step 10. In the Maven Build dialog:

└── Enter Goals: clean install test

└── Click on Apply -> Click on Run

Step 11. Check console for BUILD SUCCESS message.

Step 12. Run the application:

└── Right-click on App.java -> Run As -> Java Application

└── Output: "Hello World" displayed.

Creation of Maven web Java Project

Step 1: Open Eclipse

└── 1.1 Launch Eclipse IDE.

└── 1.2 Select or create a workspace.

Step 2: Create a New Maven Project

└── 2.1. File -> New -> Project...

└── 2.1.1. Expand "Maven"

└── 2.1.2. Select "Maven Project" and click "Next"

Step 3: Choose Maven Archetype

└── 3.1. Select an archetype(e.g "'org.apache.maven.archetypes' -> 'maven-archetype-webapp' 1.4 ")

└── 3.2. Click "Next"

Step 4: Configure the Maven Project

└── 4.1 Group Id: Enter a group ID (e.g., com.example).

└── 4.2 Artifact Id: Enter an artifact ID (e.g., my-web-app).

└── 4.3 Click **Finish** to create the project.

Step 5: Add Maven Dependencies

└── 5.1 Open the **pom.xml** file in the Maven project.

└── 5.2 Add the necessary dependencies for your web project (e.g., Servlet, JSP):

Go to browser -> Open mvnrepository.com

Search for 'Java Servlet API' -> Select the latest version.

Copy the dependency code -> Paste it in MavenWeb's pom.xml under the target folder

└── Example:

```xml
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>4.0.1</version>
<scope>provided</scope>
</dependency>
```

Step 6:-. Configure server:

└── Window -> Show View -> Servers

└── Add server -> Select Tomcat v9.0 server -> Click Next

└── Configure server options (e.g., ports, server location).

Step 7:-. Modify 'tomcat-users.xml':

└── Add role and user details under <tomcat-users> tag.

Step 8:. Build the project:

└── Right-click on index.jsp -> Run As -> Maven Clean

└── Right-click on index.jsp -> Run As -> Maven Install

└── Right-click on index.jsp -> Run As -> Maven Test

└── Right-click on index.jsp -> Run As -> Maven Build

Step 9. In the Maven Build dialog:

└— Enter Goals: clean install test

└— Click on Apply -> Click on Run

Step 10. Check console for BUILD SUCCESS message.

Step 11. Run the application:

└— Right-click on index.jsp -> Run As -> Run on Server

└— Select the Tomcat server -> Click on Finish

Step 12. Output: "Hello World" webpage displayed.

Note:-Now push yours Maven java project and Maven Web Project into your github

3. Configure Java Version via Compiler Plugin

In `pom.xml`:

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>17</source>

<target>17</target>

</configuration></plugin>

Check:

- Run `mvn package`
- Inspect generated JAR: *jar tf target/myapp.jar*

4. JUnit Testing and Reports

Place test files in `src/test/java`. Example:

```
public class AppTest {
    @Test
    public void testSum() {
        assertEquals(5, 2 + 3);
    }
}
```

Run:

mvn test

Check:

- `target/test-classes/` → compiled test `.class` files
- `target/surefire-reports/` → `.txt` or `.xml` test results
  If test fails → corresponding log and failure trace shown

5. Handling Errors in pom.xml

- Typos in version numbers or missing repositories cause `BUILD FAILURE`
- Maven shows precise error in console
- Fix the dependency tag → re-run `mvn clean install`

6. Adding Resource Files

Put config.properties inside:

src/main/resources/config.properties

To read:

```
InputStream input =
getClass().getClassLoader().getResourceAsStream("config.properties");
```

After build, check target/classes/ → file should exist there.

7. Multi-Module and Parent Projects

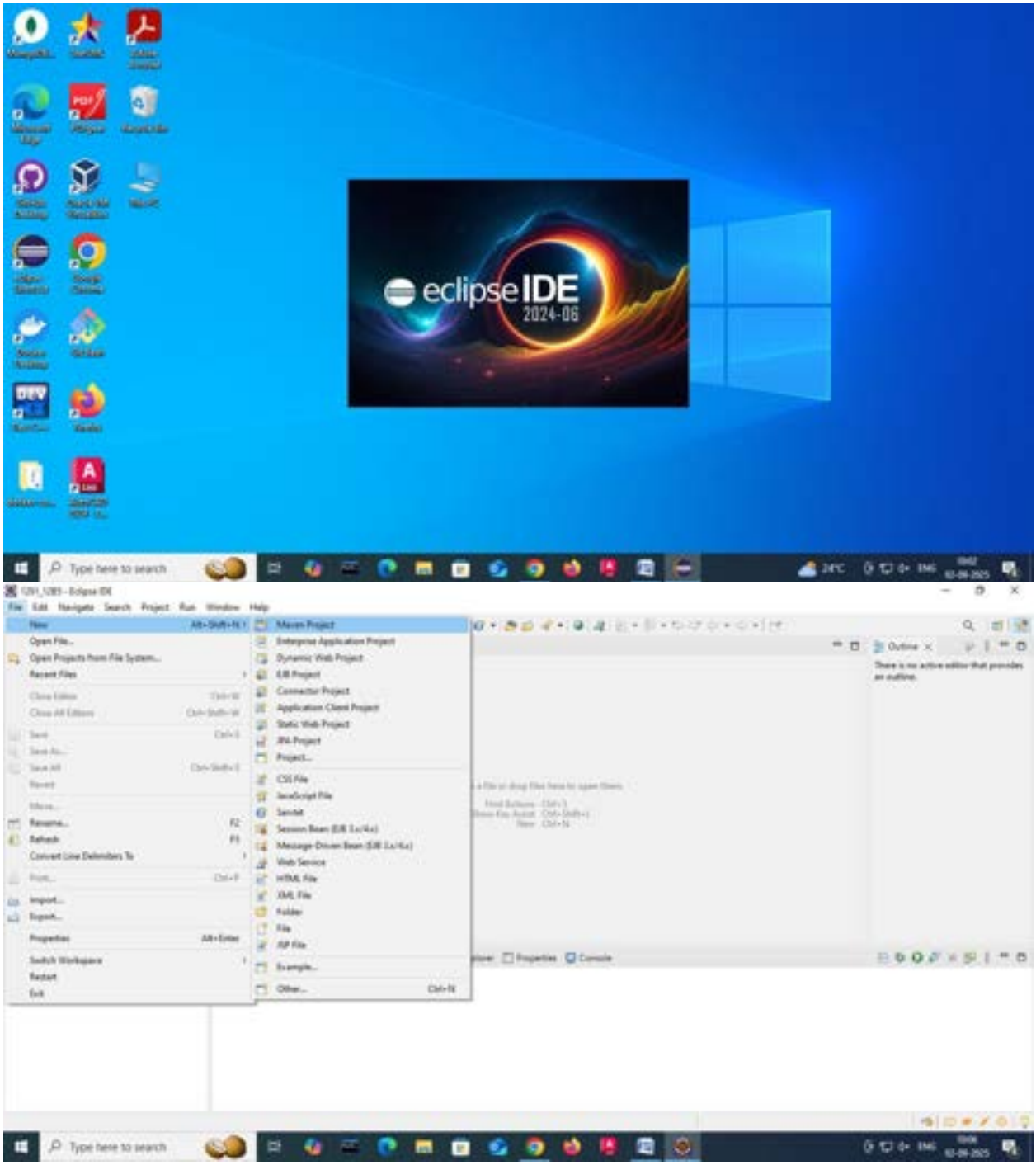
Structure:

```
parent/
├── pom.xml (packaging: pom)
├── core/
|   └── pom.xml
└── web/
    └── pom.xml
```

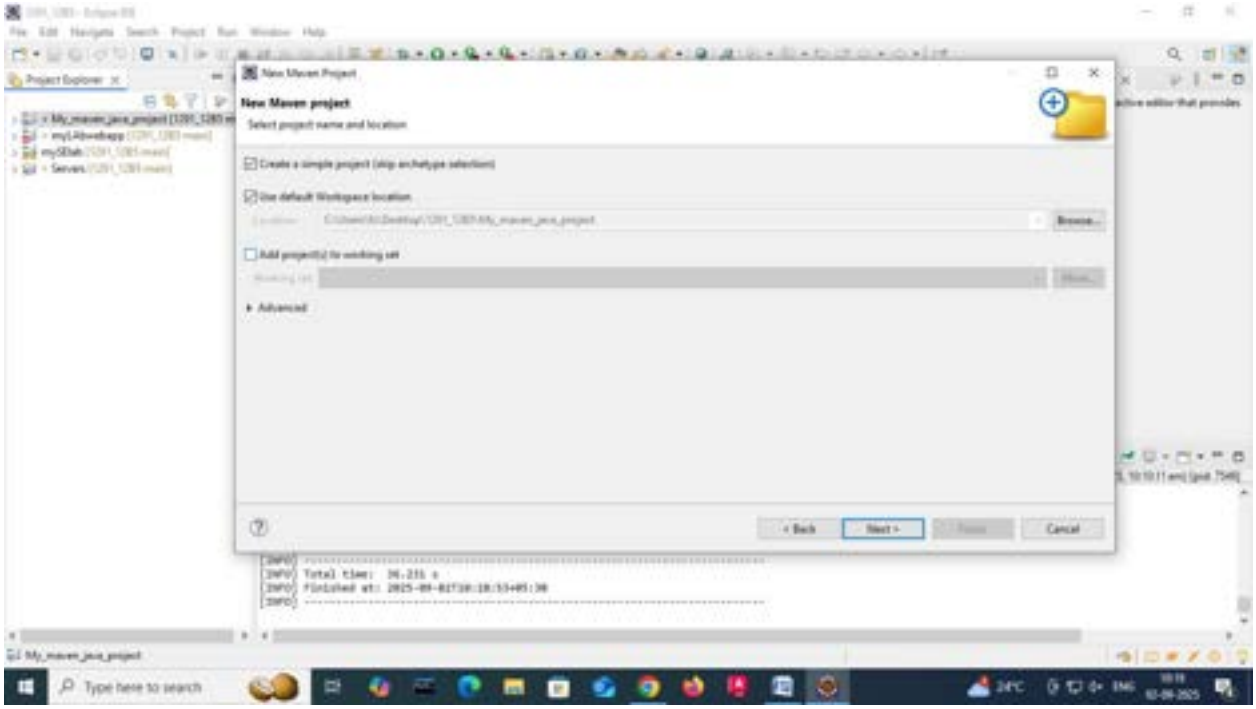
- Parent `pom.xml` defines `<modules>` and common dependencies
- Each submodule builds independently into its `target/` directory

Creating a Multi-Module Maven Project:

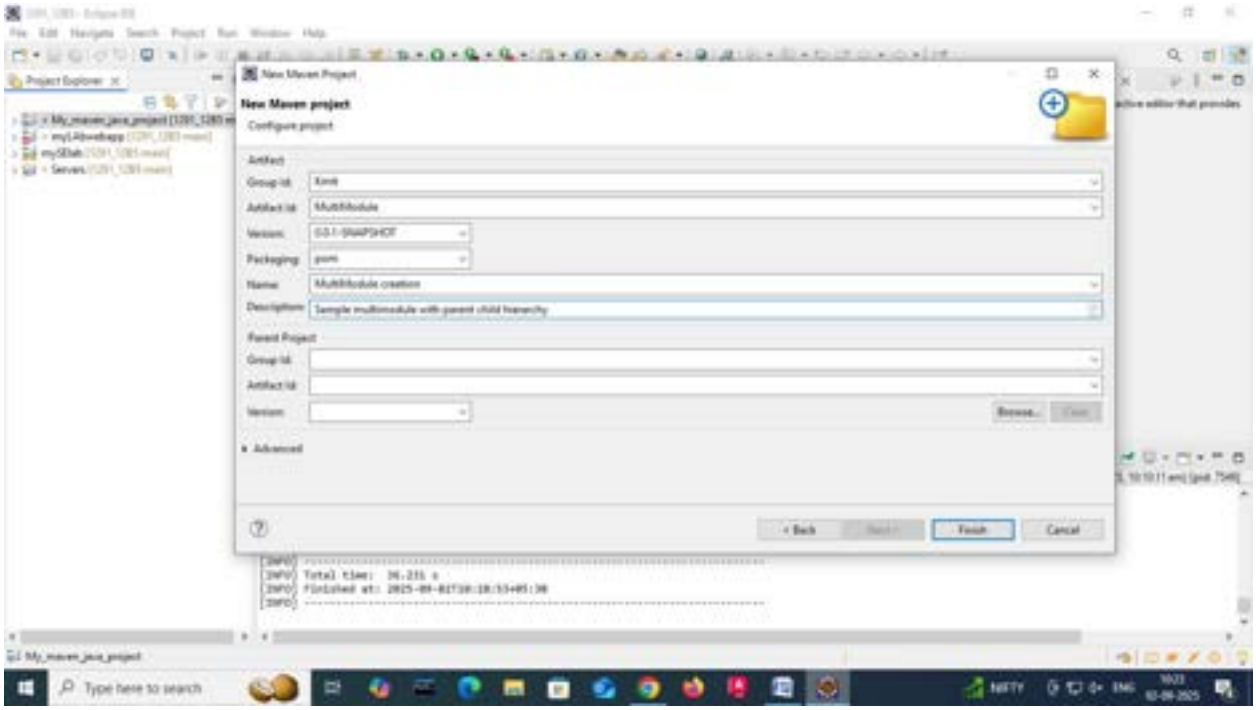
1. Open eclipse-> file-> new -> other -> Maven -> MavenProject ->click on next.



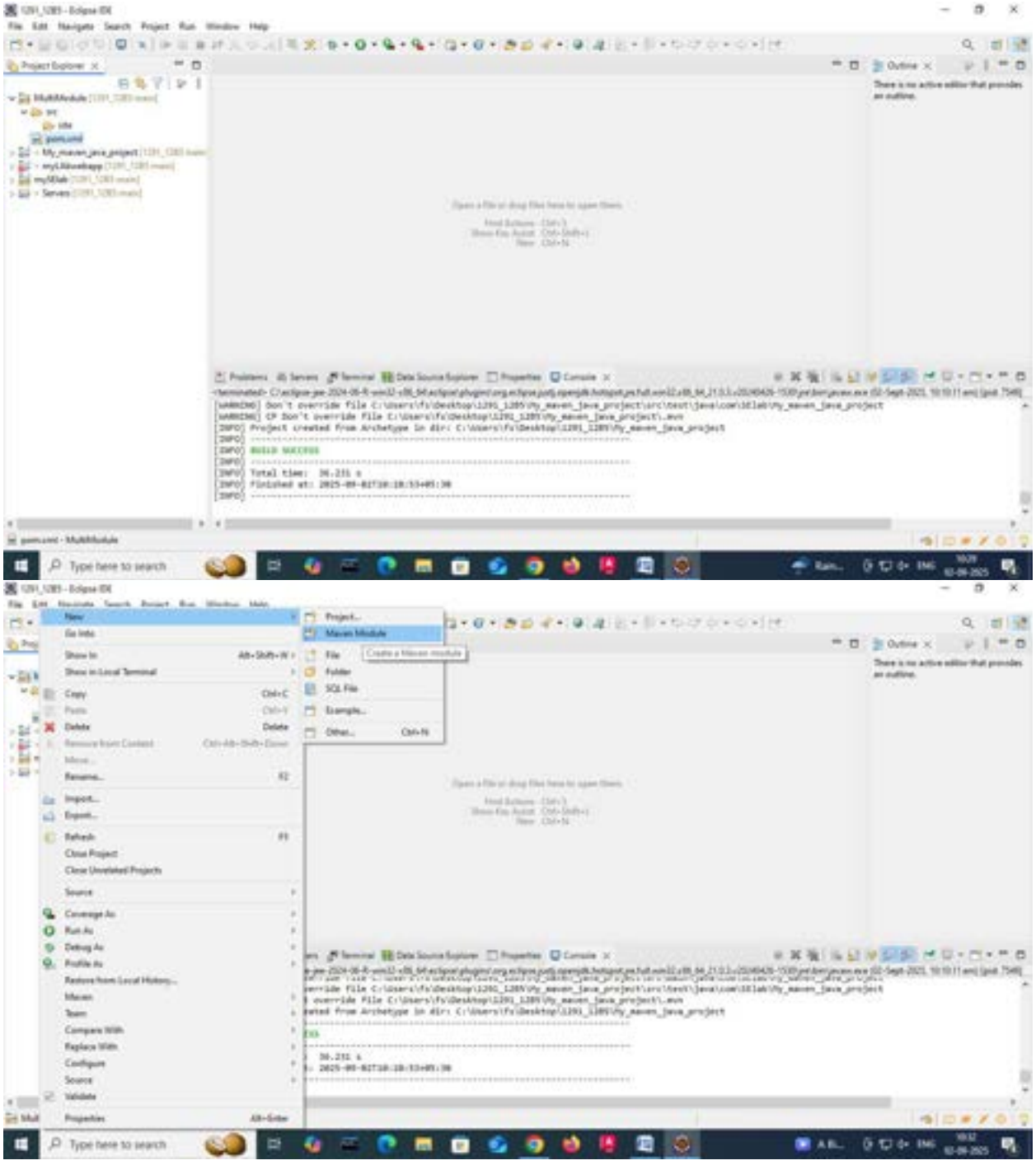
2. Check the first option: Create a Simple project(skip archetype selection) and click on next.



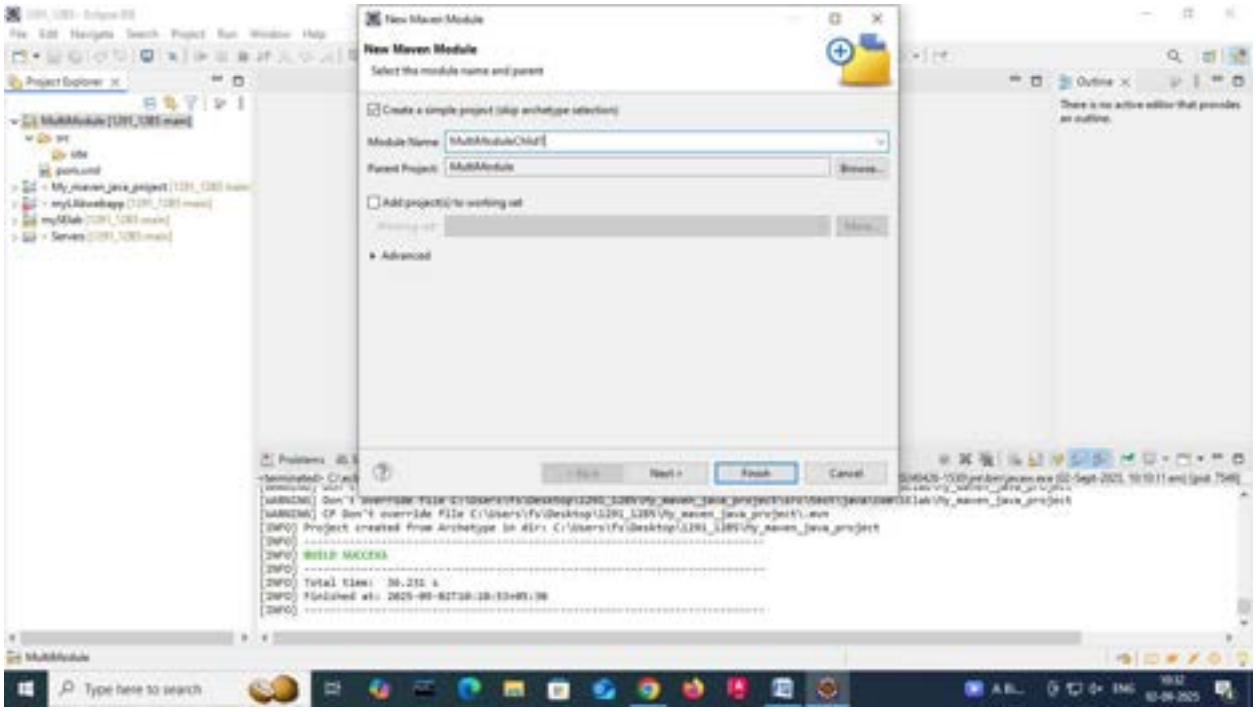
3. Give groupID: KMIT, artifact ID: MultiModule, in the packaging tab select the drop down and opt for pom.
4. Give Name: Multimodule creation and Description: Sample multimodule with parent child hierarchy and click on next.



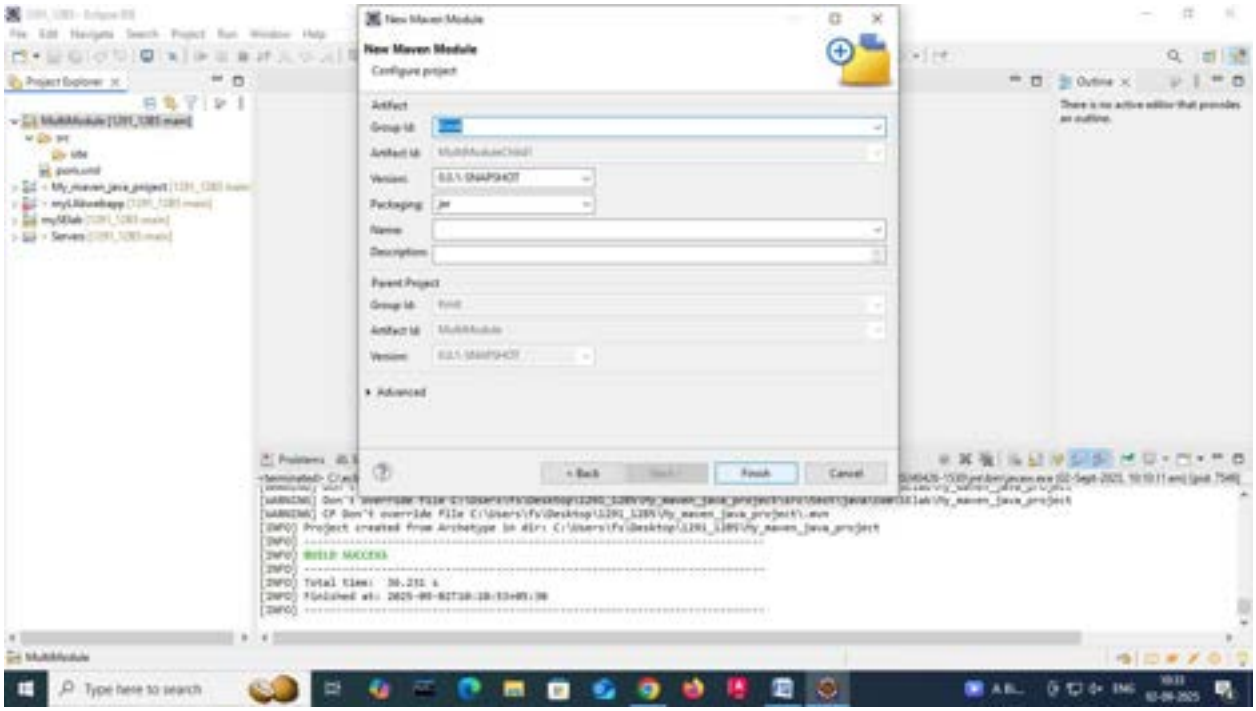
5. Now the Multimodule folder is created and displayed in the project explorer on the left. Right click on the folder and opt new -> mavenModule

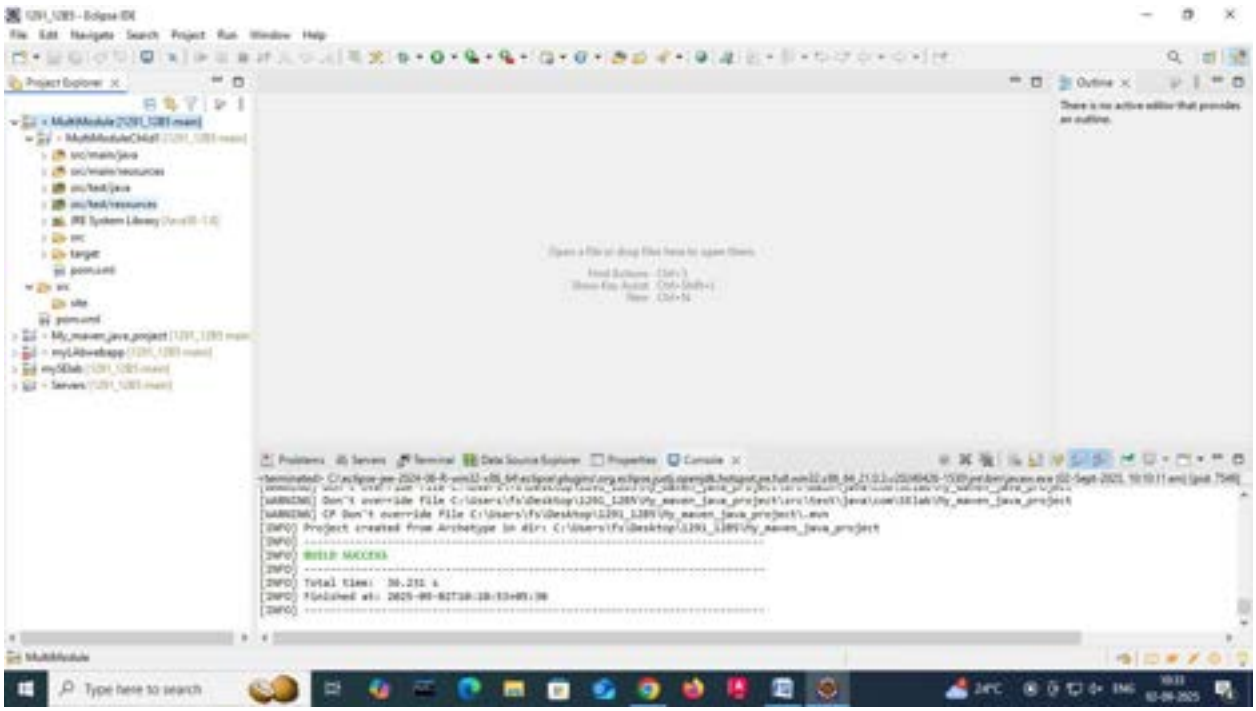


6. Check the option Create a Simple project(skip archetype selection, give the artifactID: MultiModuleChild1 and click on next and finish.

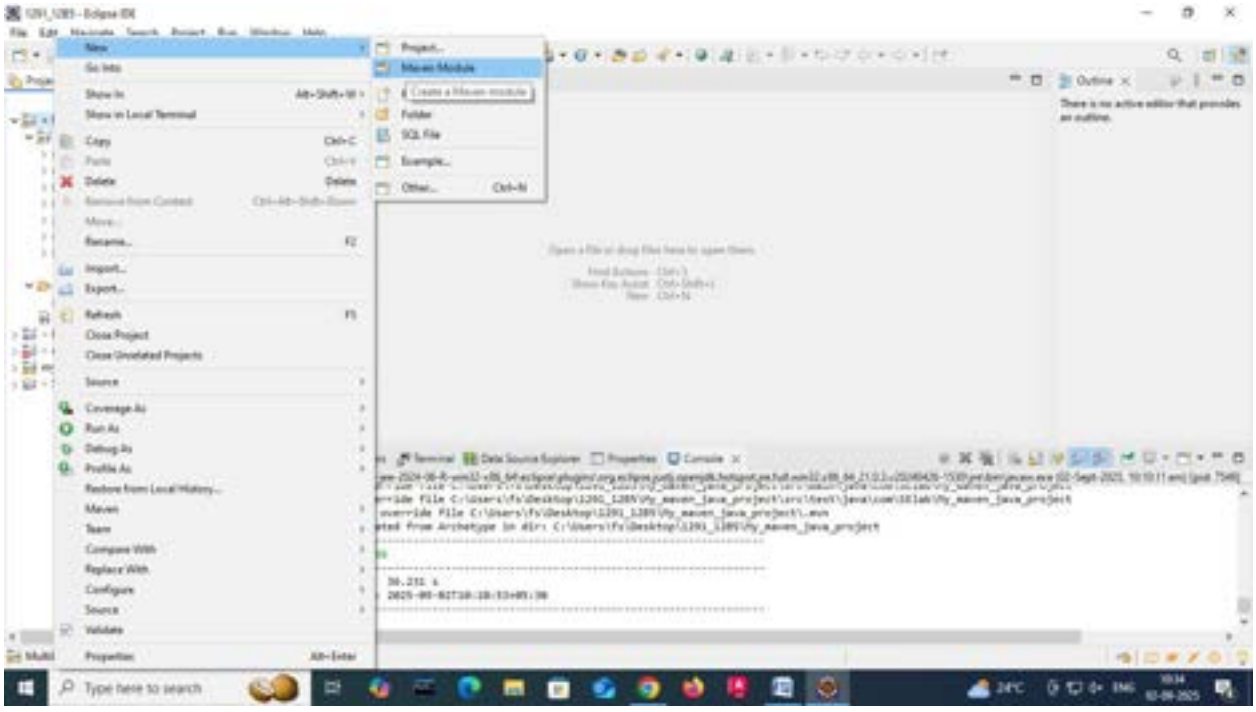


7. Now you can see, the module1 we created is displayed in the project explorer with a separate pom.xml file.

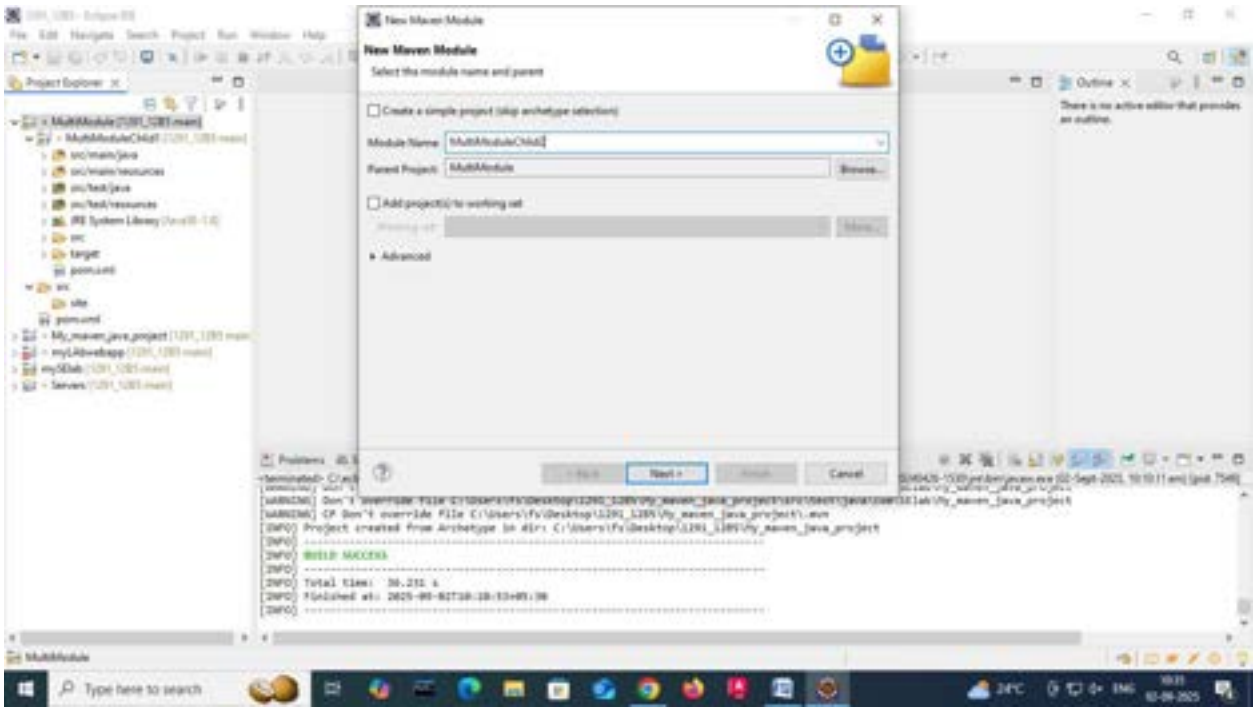




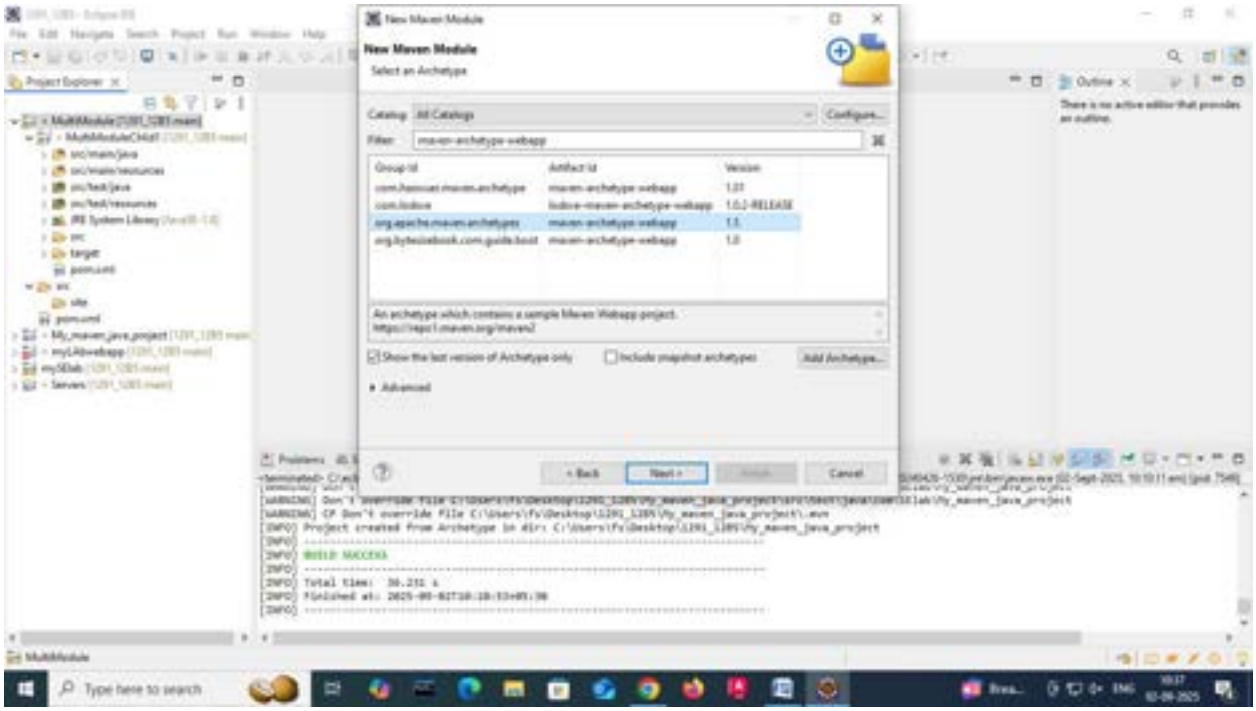
8. . Right click on the MultiModule folder in the project explorer and opt new -> mavenModule



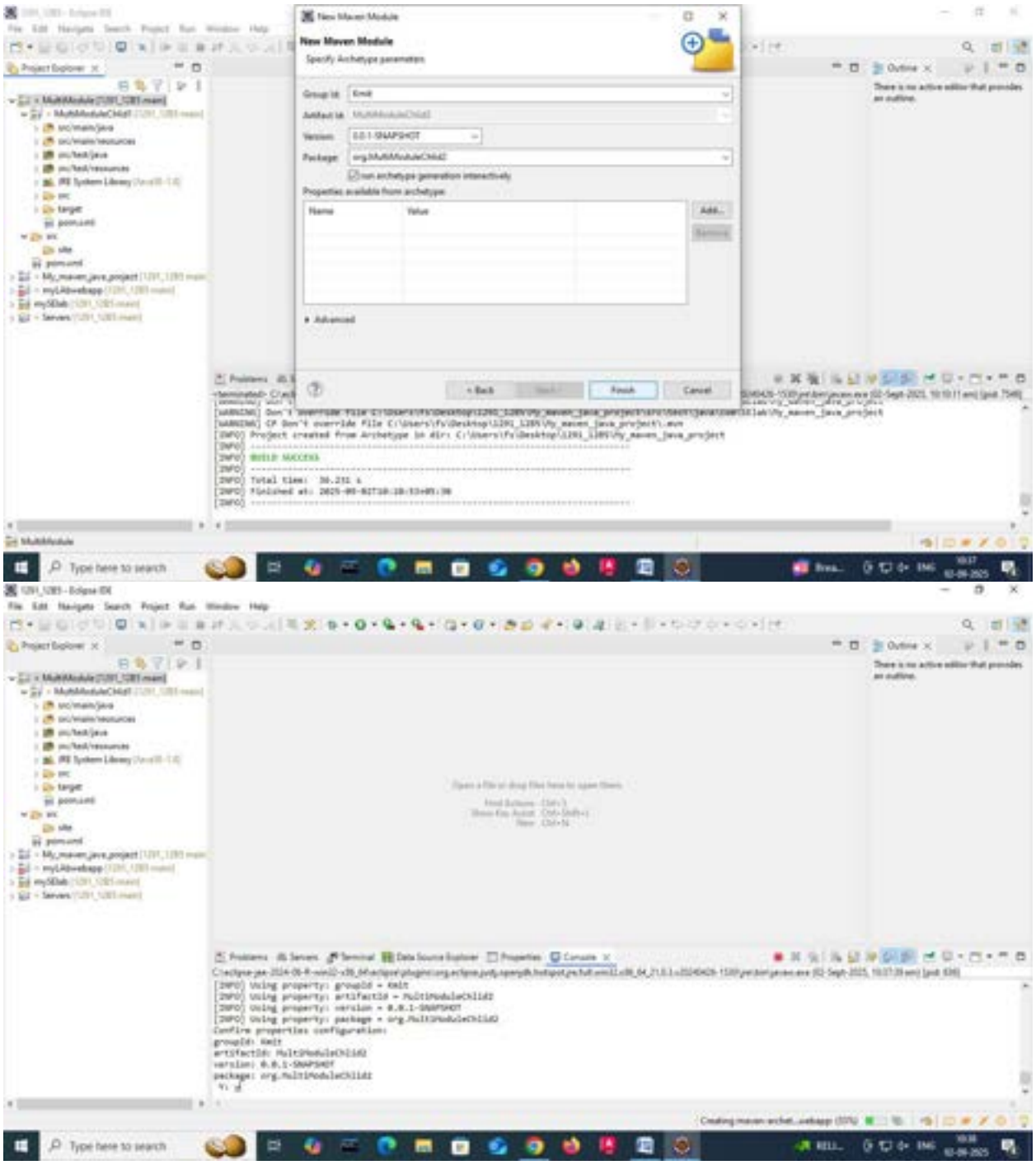
9. Give the artifactID: MultiModuleChild2 and click on next.



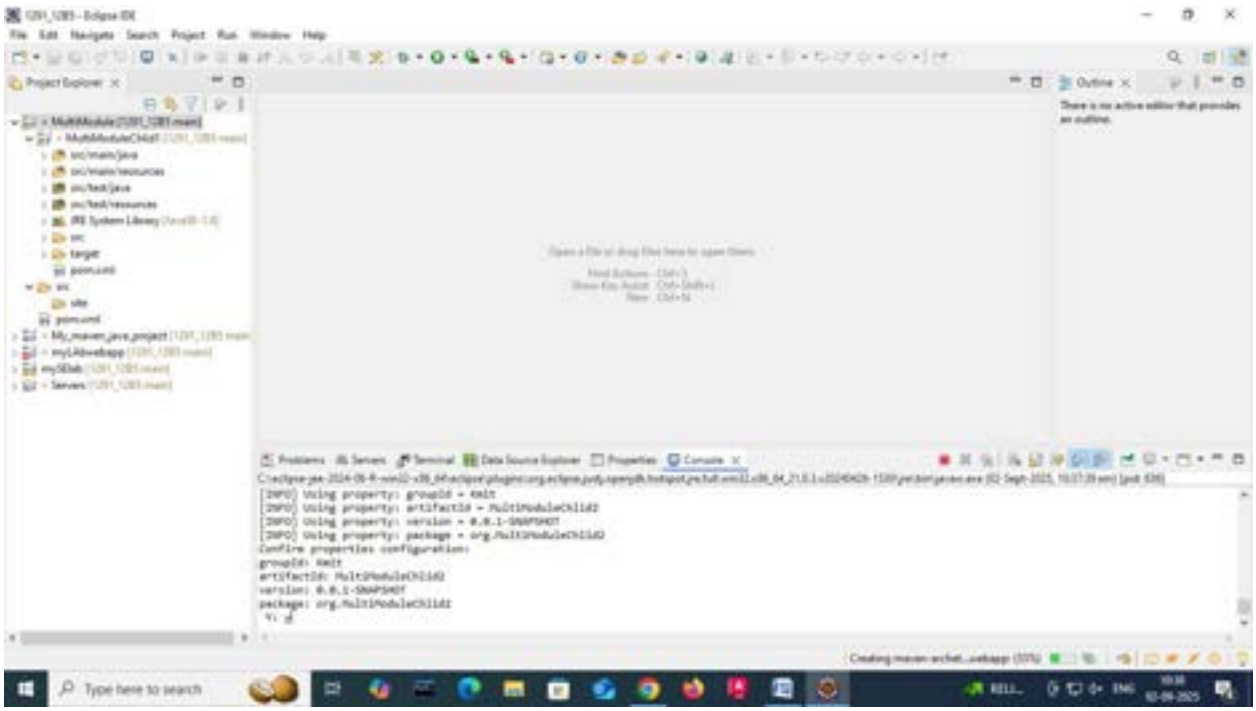
10. In filters search for maven-archetype-webapp, select the version 1.1/1.5 and click on next.



11. Now click on finish, and confirm with a Y in the console.



12. Now you can watch the new child is been added to the parent project.



Note:*Childs or Modules can be added to the projects who have the packaging type as pom. If we try to create a module within child1 it is not possible since its packaging is opted to JAR.*

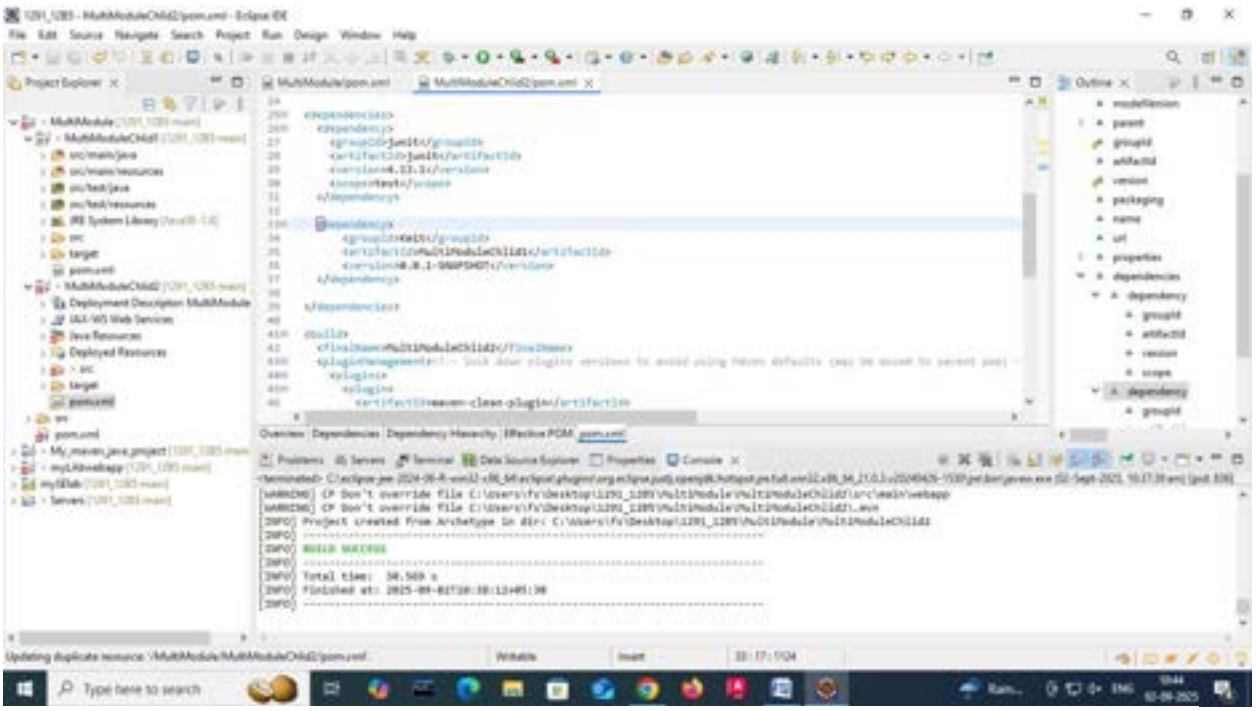
Linking two modules in a Parent Project:

Go to the module u want to connect. And add a dependency to other module on which the current module depends. For example if my child2 is dependent on child1. I have to change the dependencies in the pom.xml by adding a new dependency of child1.

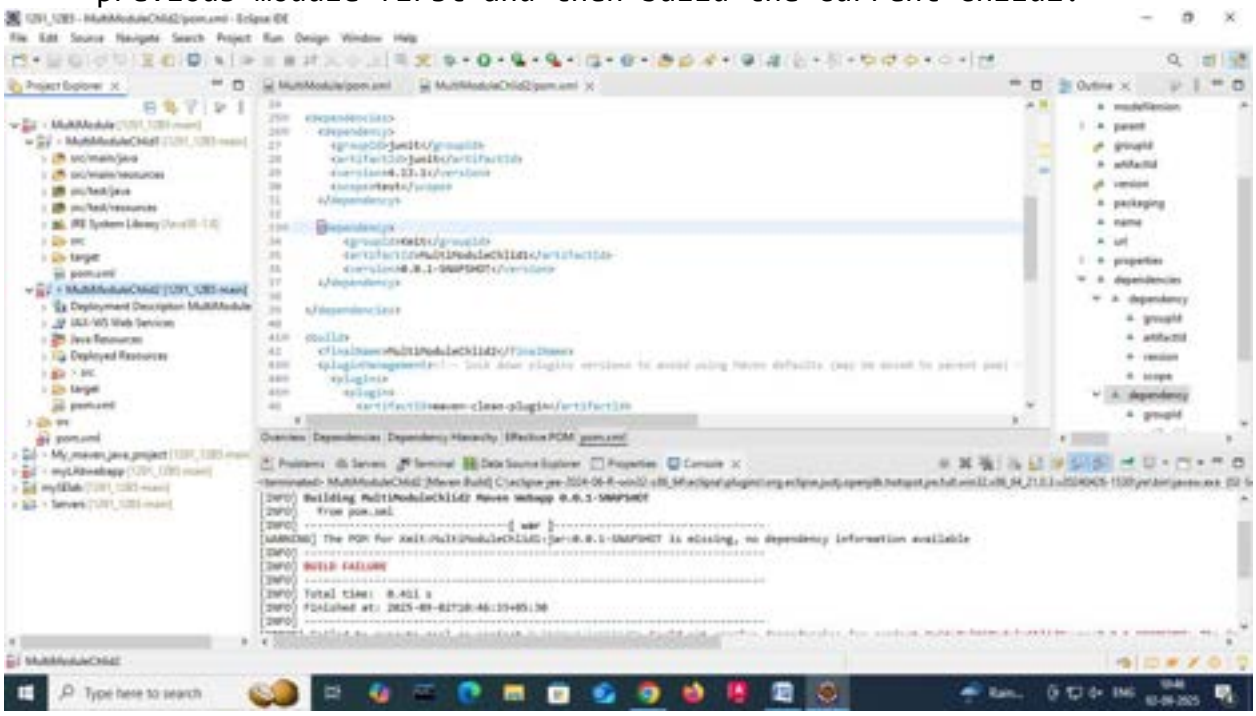
Steps to add dependency:

1. Goto pom.xml of child2.
2. Navigate to dependencies tag. Update the dependencies with child1 dependency as given below, and save pom.xml after update.

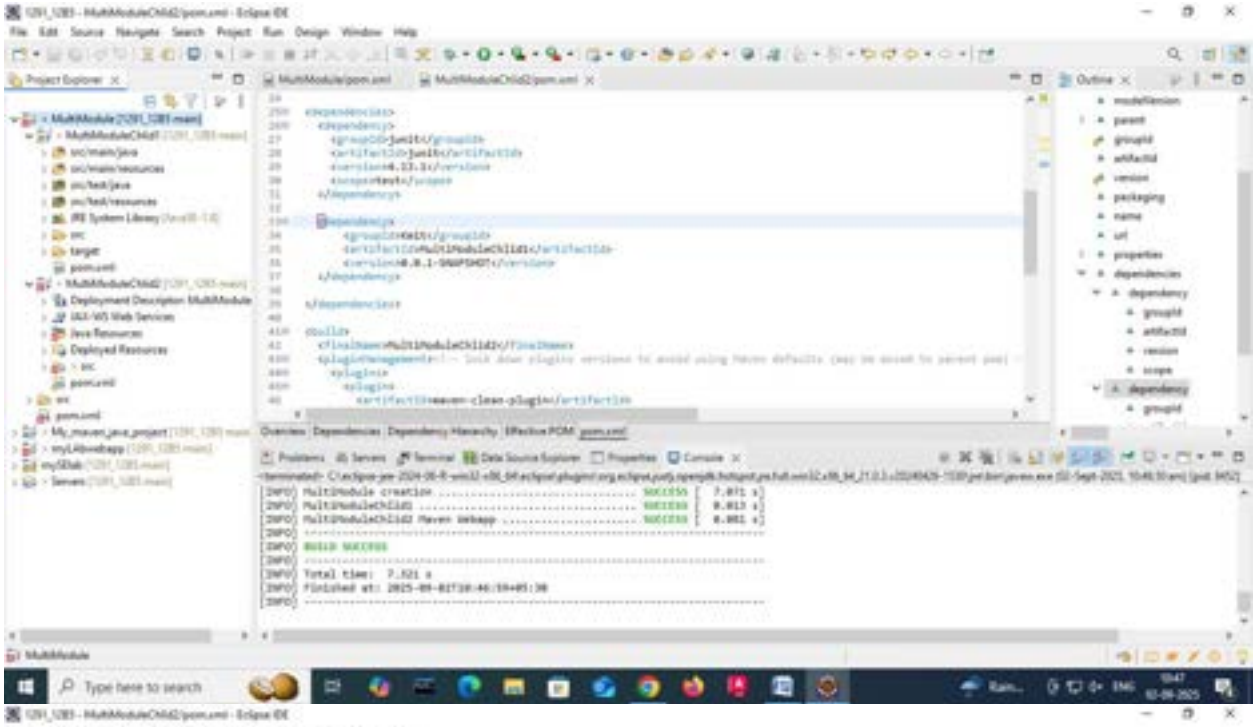
```
<dependency>
        <groupId>KMIT</groupId>
            <artifactId>MultimoduleChild1</artifactId>
            <version>0.0.1-SNAPSHOT</version>
   </dependency>
```

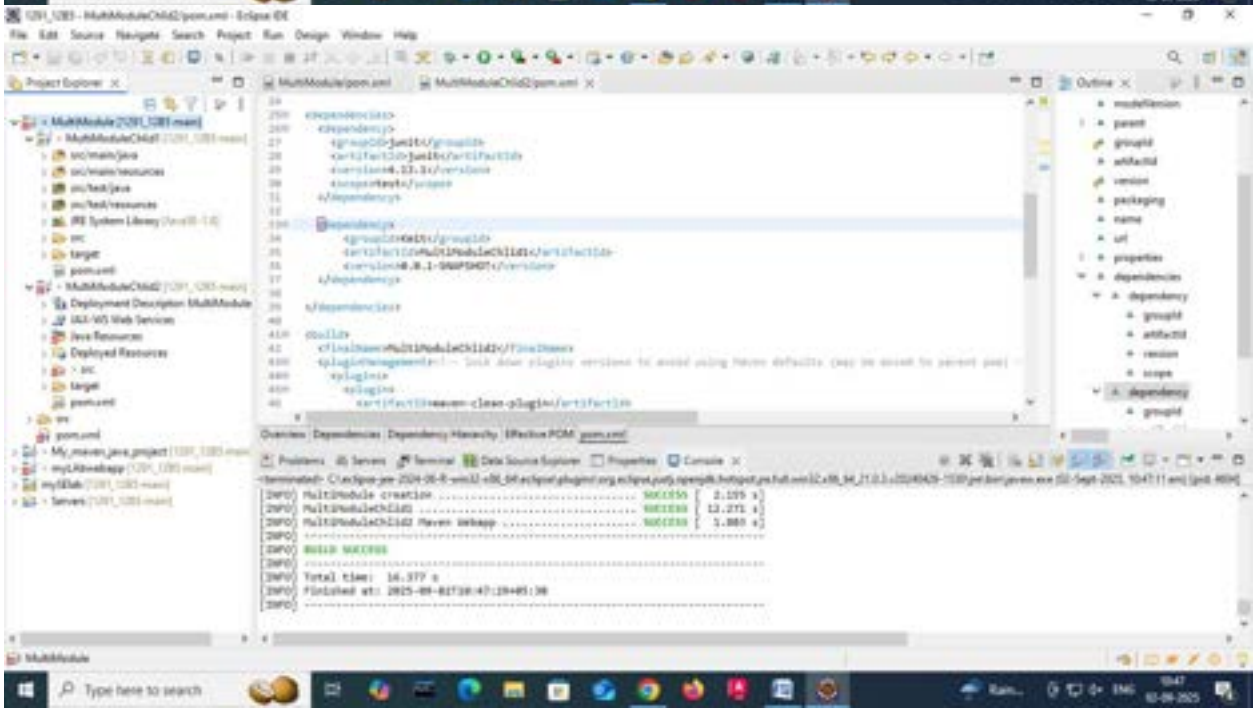


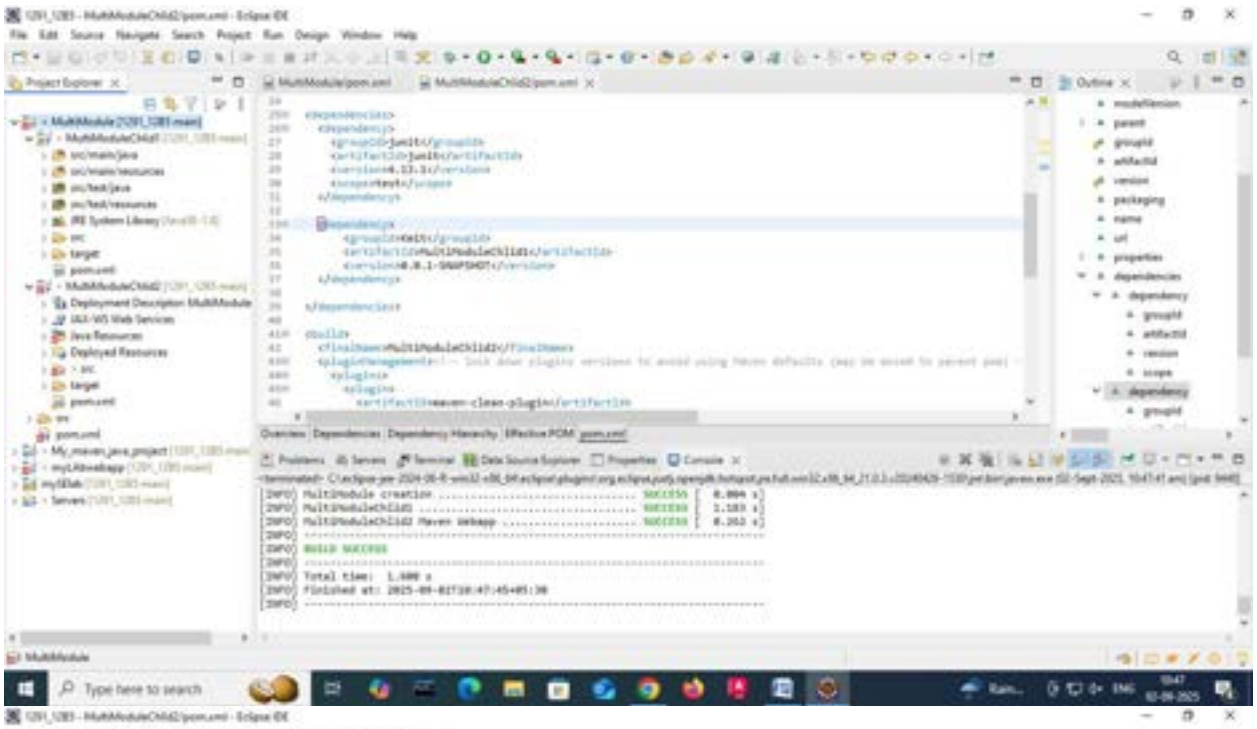
3. Now build the child2 by right clicking it and opting run as->maven build.
4. The build will be a *failure.* Because we haven't build the parent project and the child project. Child1 is dependent on parent. And Child2 is dependent on child2. SO we have to build ass the previous module first and then build the current child2.

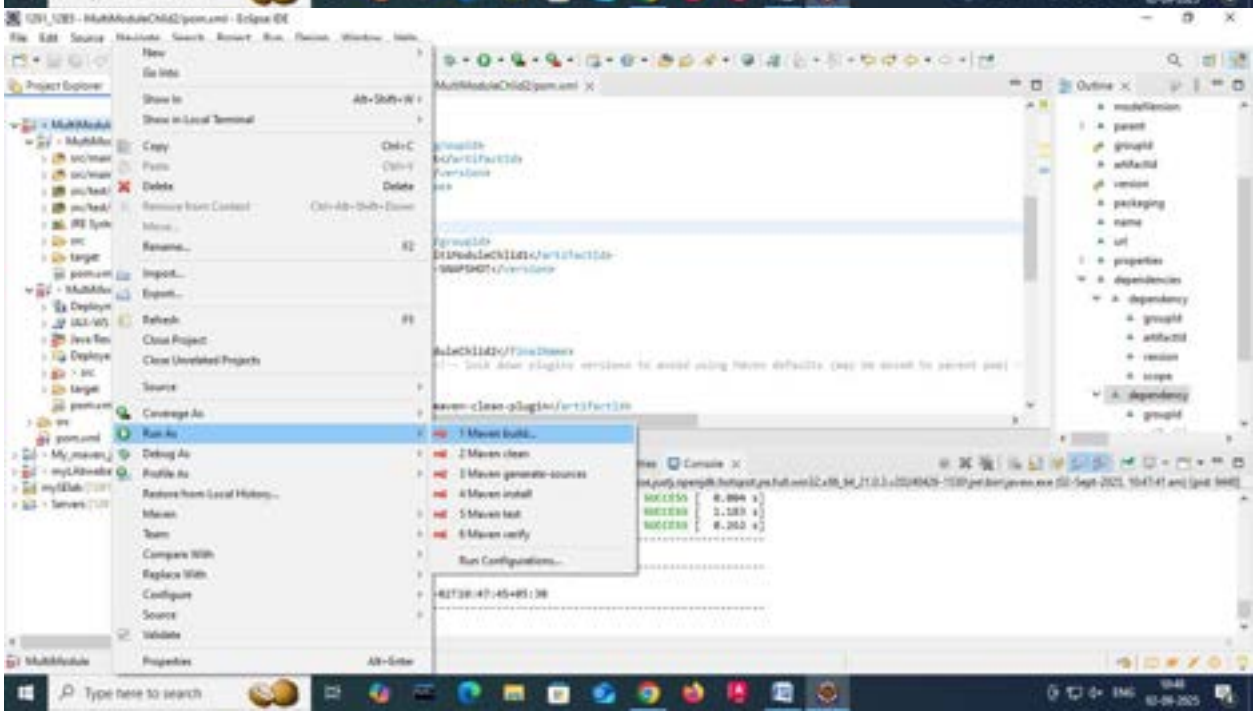


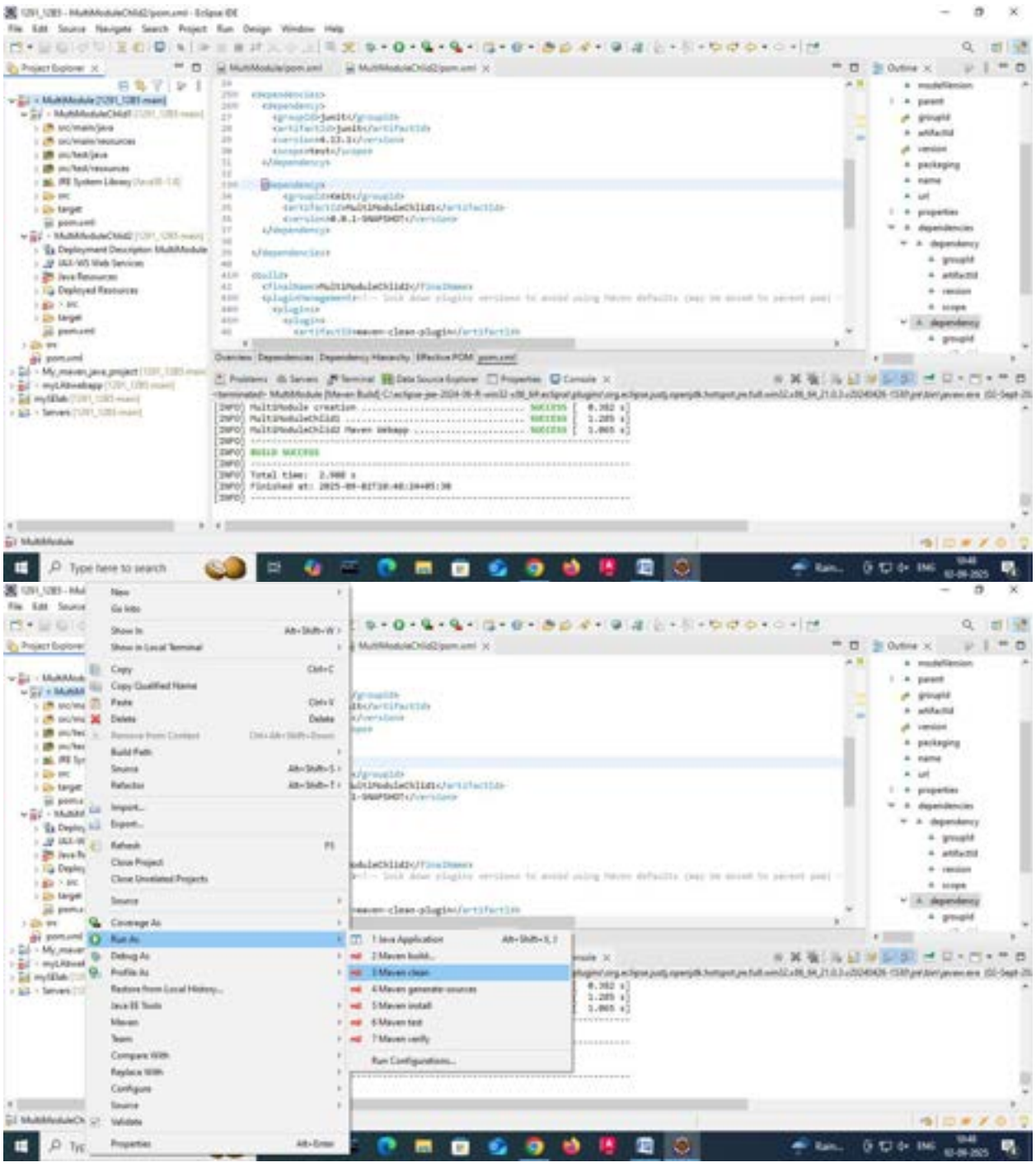
5. After building parent and child1, build child2 now the build will be a *Success*

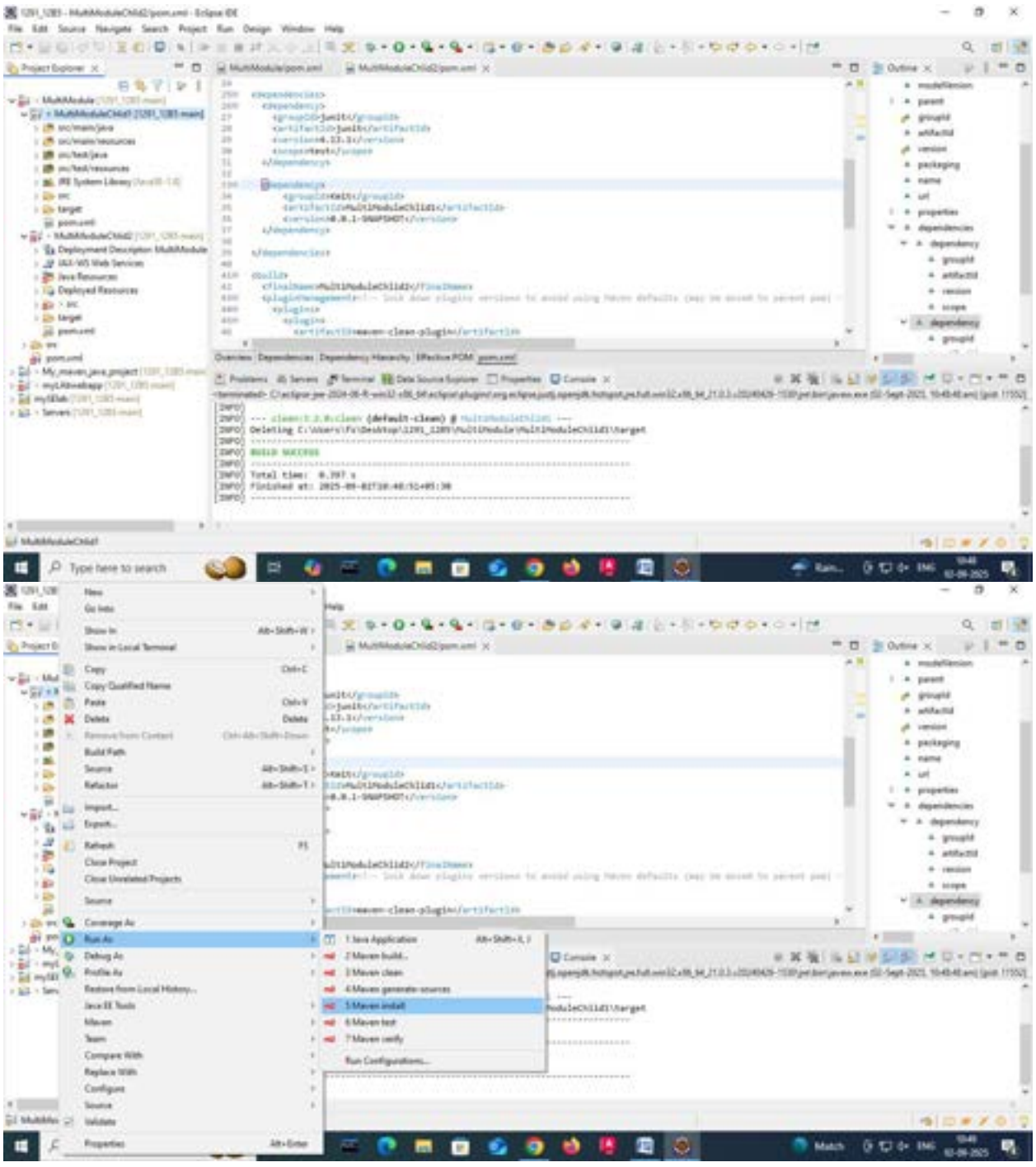


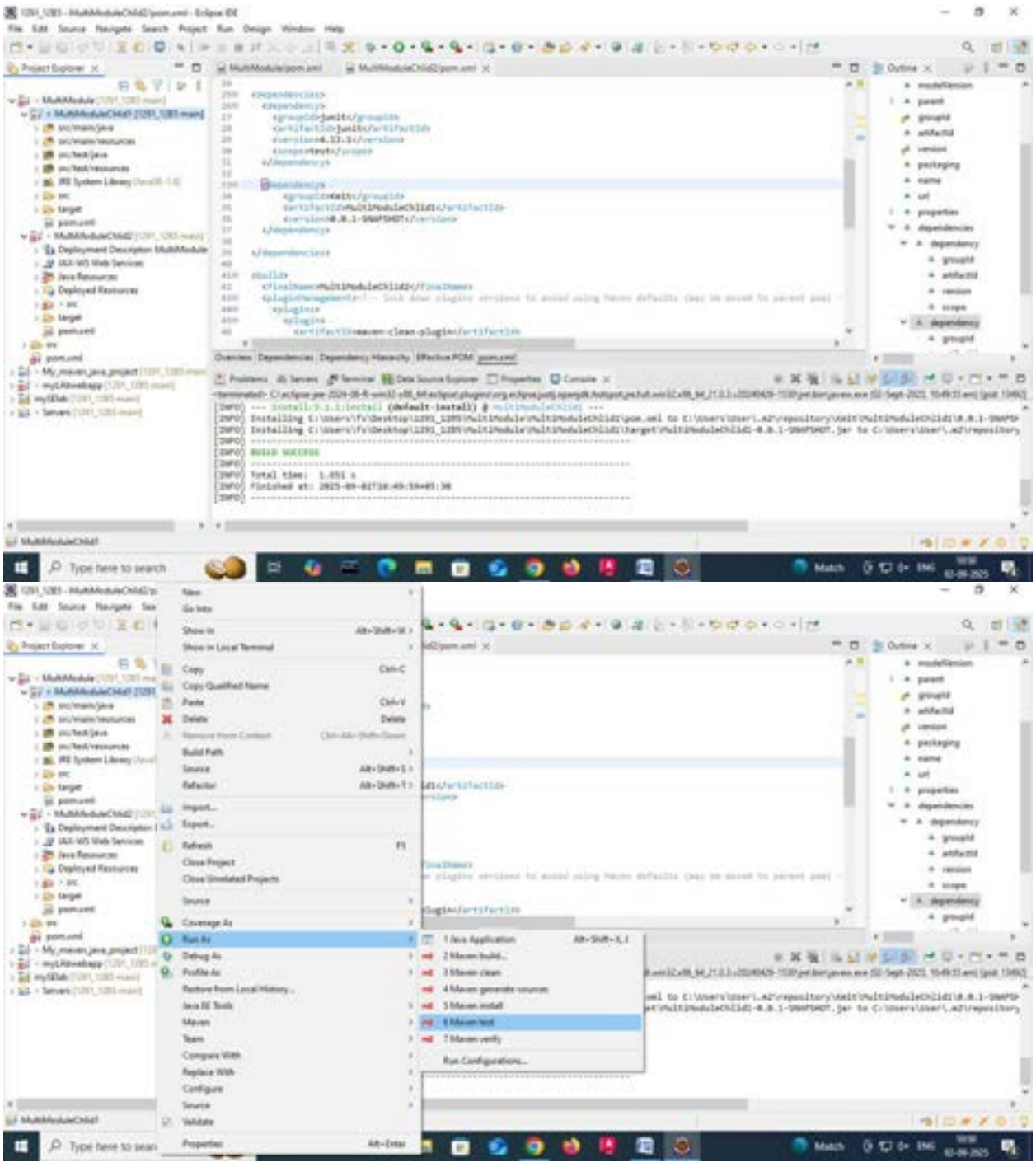


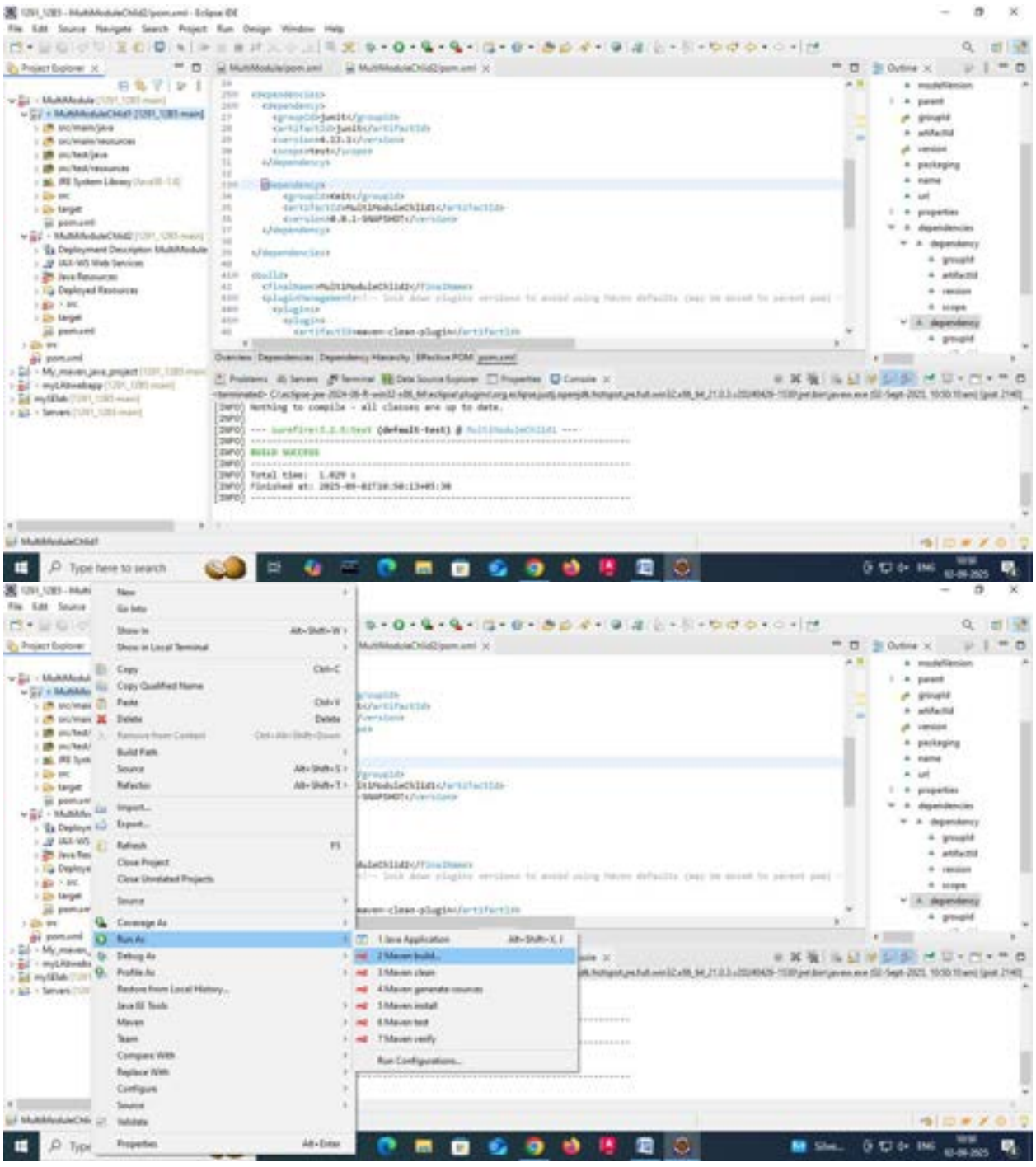


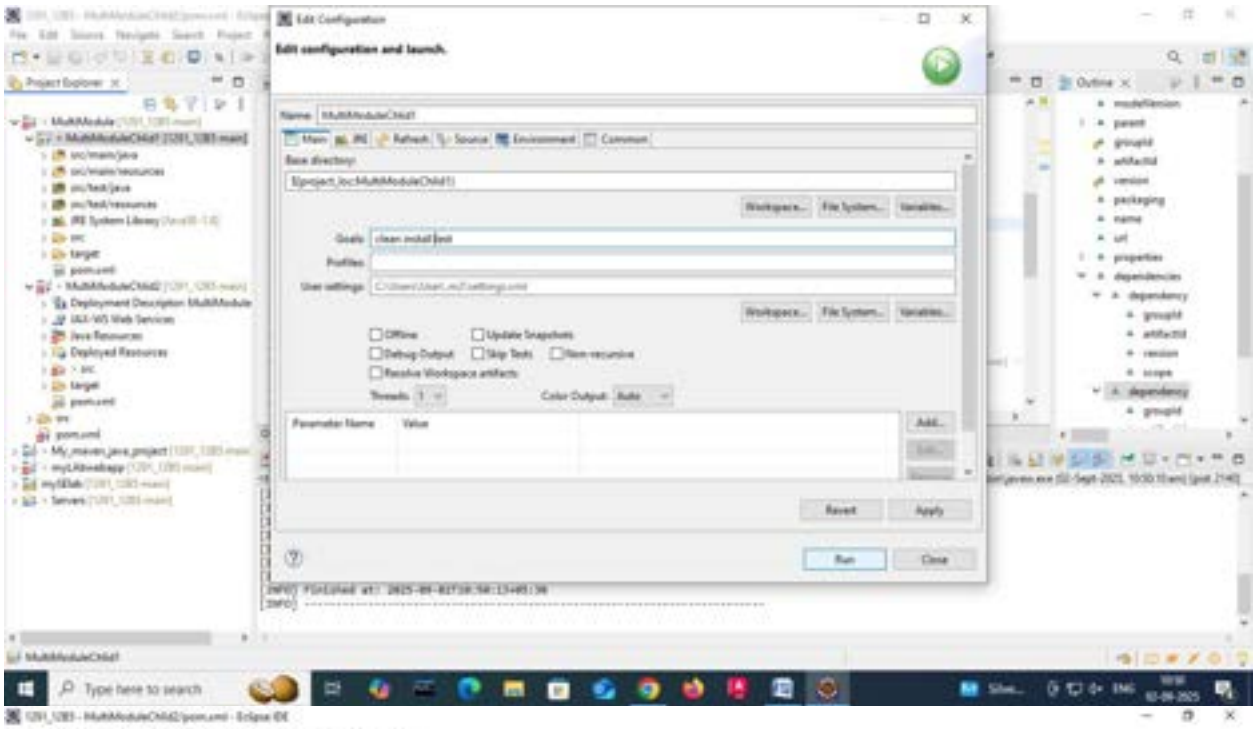


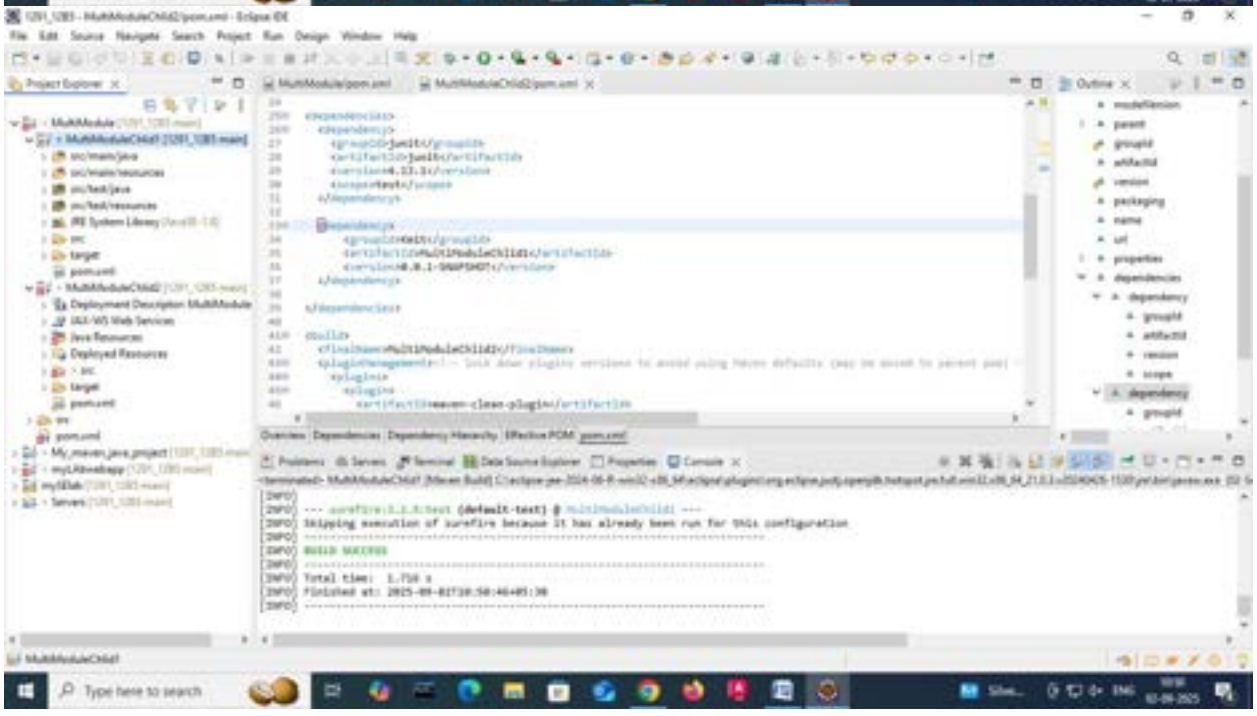


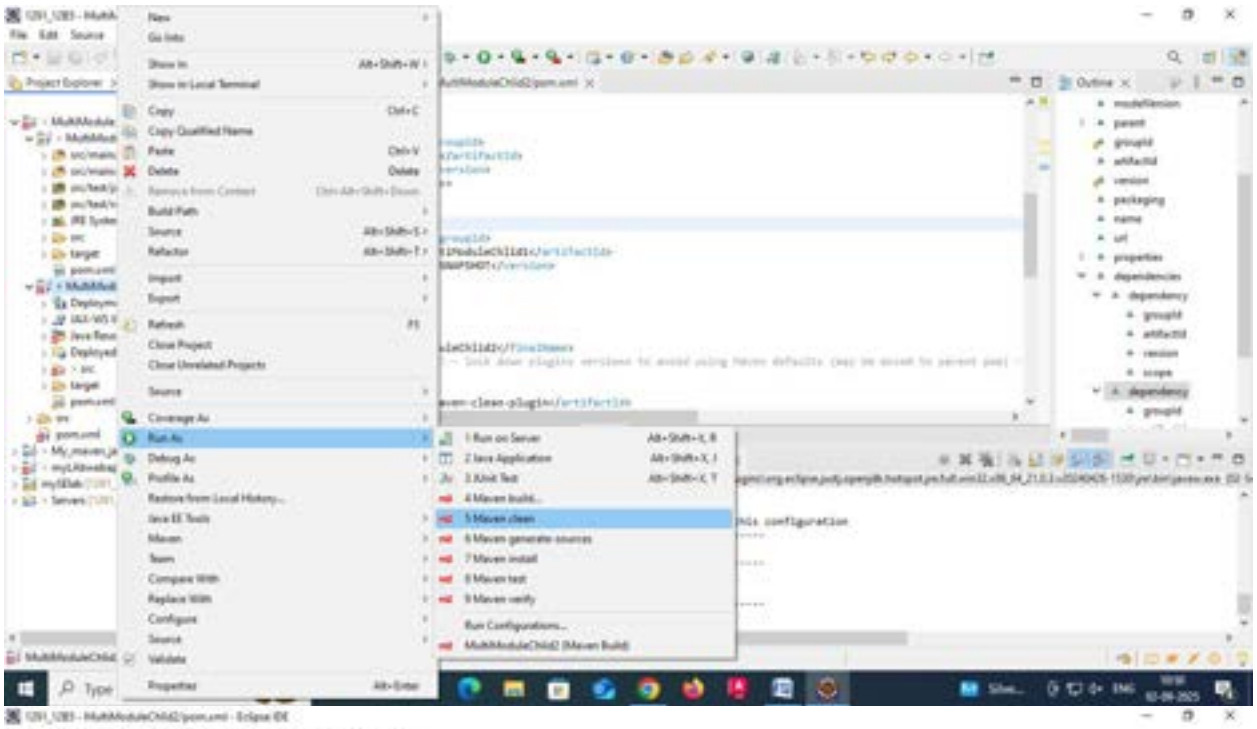


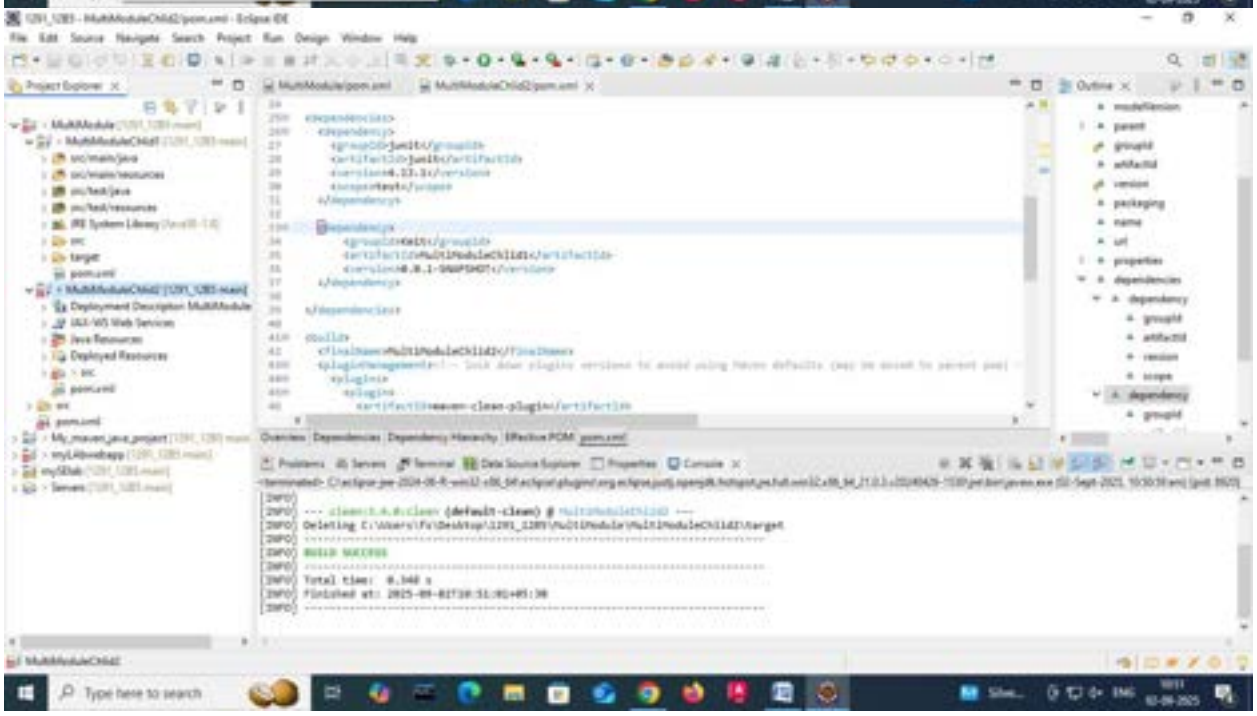


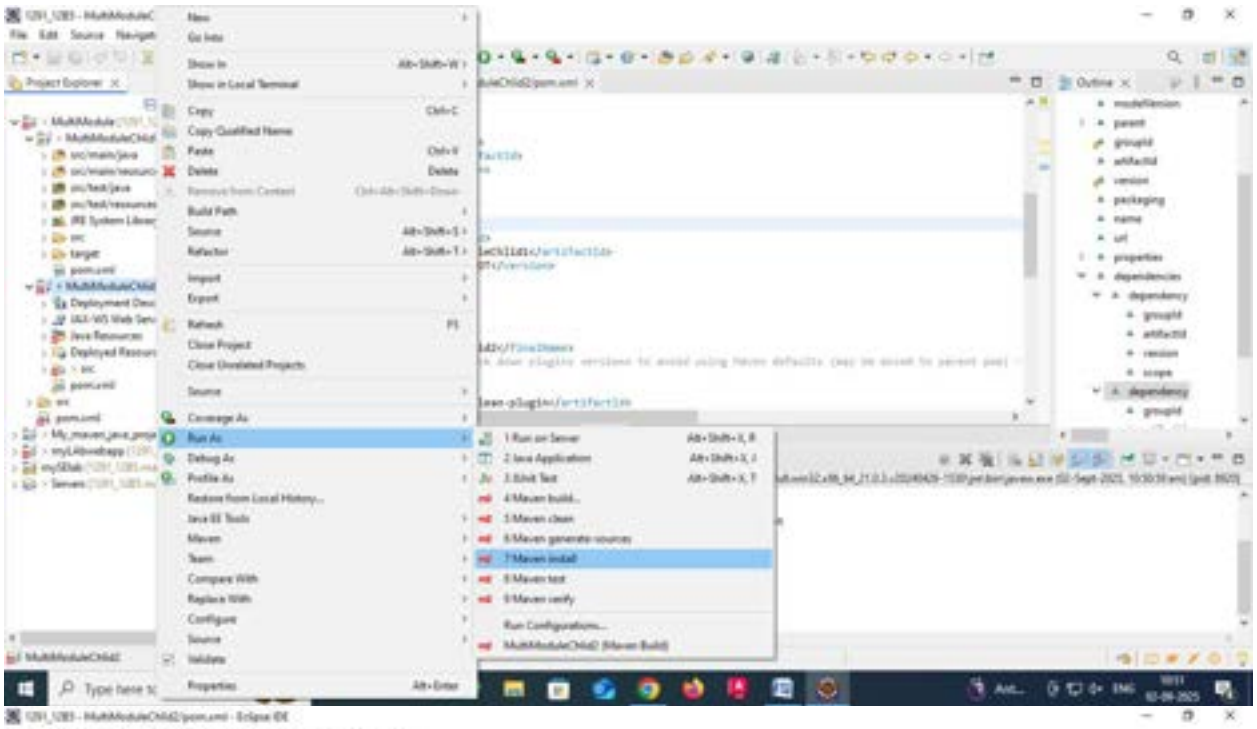


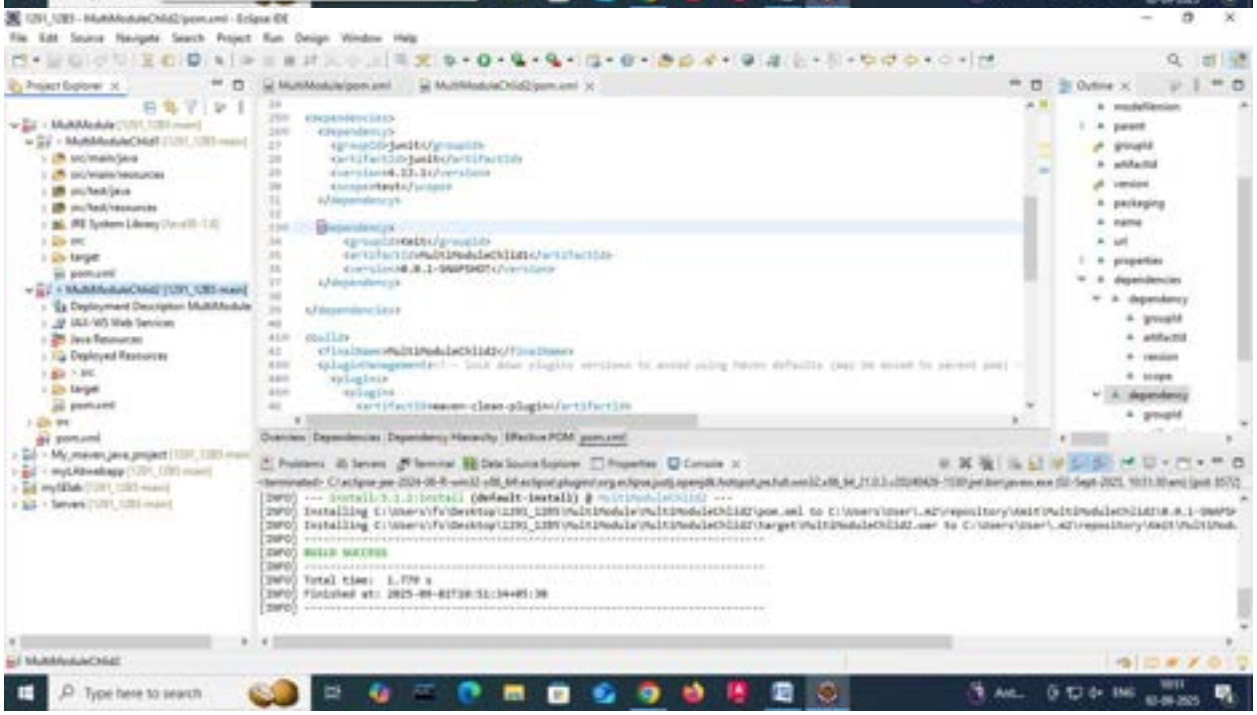

Edit Configuration
Edit configuration and launch.
Name: MultiModuleChild1
Base directory:
${project_loc:MultiModuleChild1}
Goals: clean install test
Profiles:
Offline
Update Snapshots
Debug Output
Skip Tests
Non-recursive
Resolve Workspace artifacts
Threads:
Color Output:
Parameter Name
Value
Add...
Revert
Apply
Run
Close
Type here to search

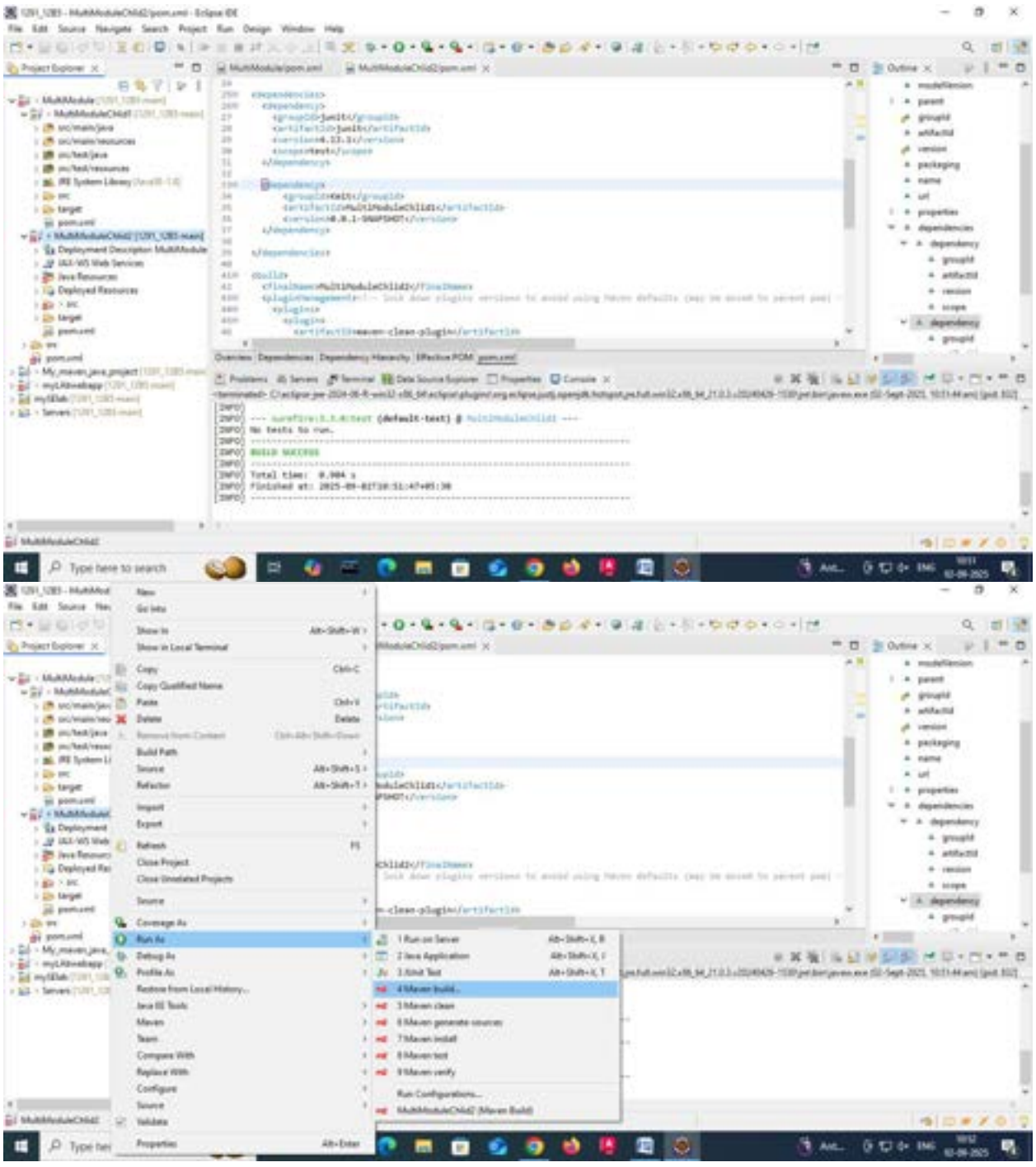

MultiModule/pom.xml
MultiModuleChild2/pom.xml
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13.1</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>test</groupId>
<artifactId>MultiModuleChild1</artifactId>
<version>0.0.1-SNAPSHOT</version>
</dependency>
</dependencies>
<build>
<finalName>MultiModuleChild2</finalName>
<pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
<plugins>
<plugin>
<artifactId>maven-clean-plugin</artifactId>
Overview
Dependencies
Dependency Hierarchy
Effective POM
pom.xml
Problems
Servers
Terminal
Data Source Explorer
Properties
Console
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ MultiModuleChild1 ---
[INFO] Skipping execution of surefire because it has already been run for this configuration
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 1.710 s
[INFO] Finished at: 2025-09-01T10:50:46+05:30
[INFO] --
Type here to search

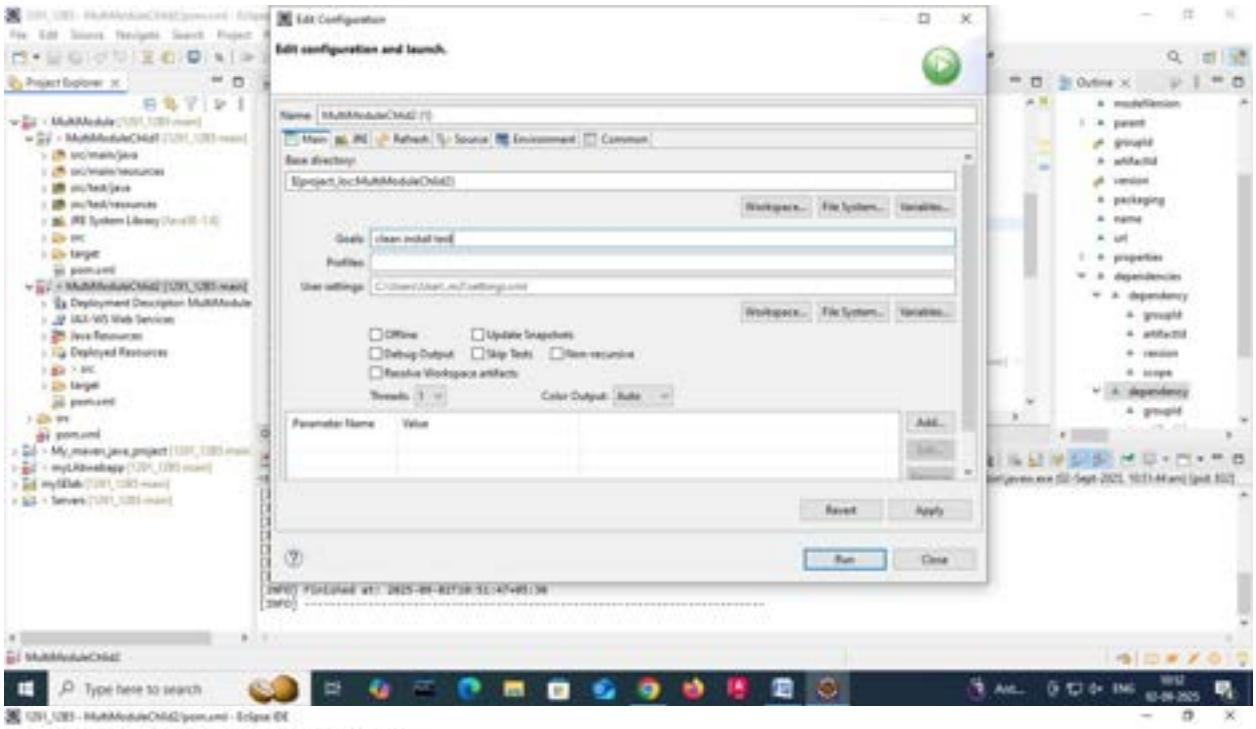

New
Go Into
Show In
Show in Local Terminal
Copy
Copy Qualified Name
Paste
Delete
Build Path
Source
Refactor
Import
Export
Refresh
Close Project
Close Unrelated Projects
Run As
Debug As
Profile As
Restore from Local History...
Java EE Tools
Maven
Team
Compare With
Replace With
Configure
Validate
Properties
1 Run on Server
2 Java Application
3 JUnit Test
4 Maven build...
5 Maven clean
6 Maven generate-sources
7 Maven install
8 Maven test
9 Maven verify
Run Configurations...
MultiModuleChild2 (Maven Build)

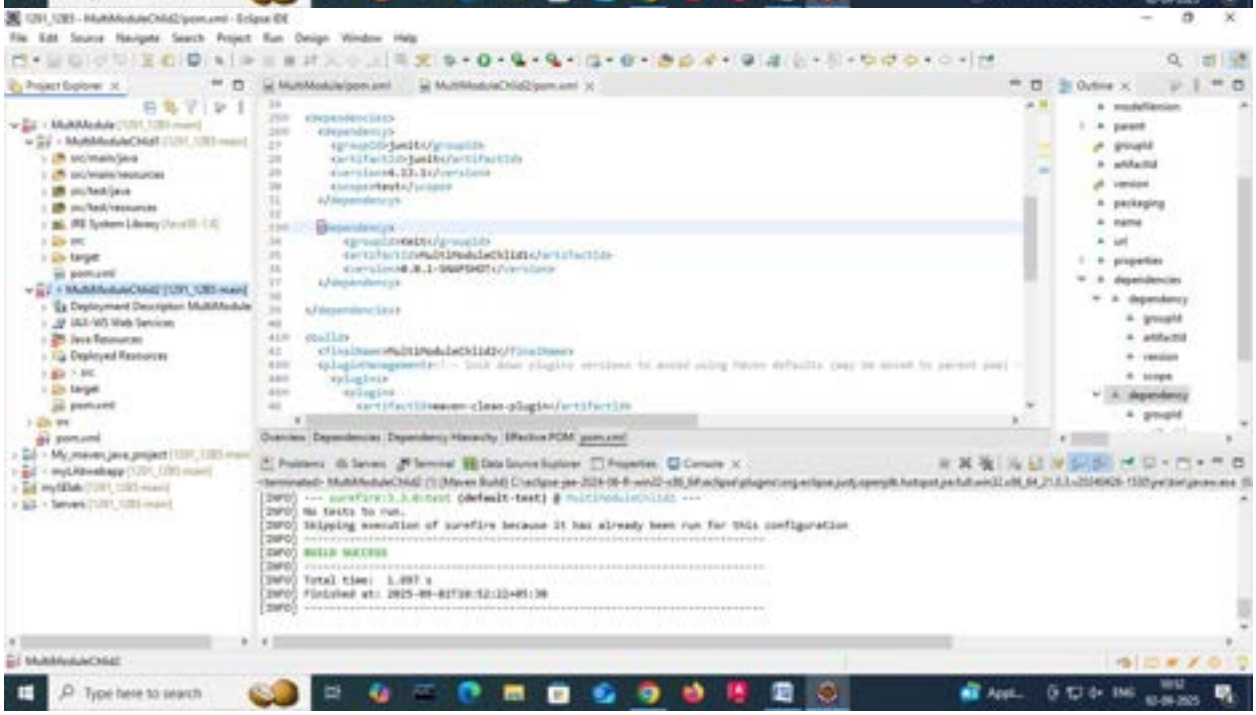

[INFO] --- clean:3.4.0:clean (default-clean) @ MultiModuleChild2 ---
[INFO] Deleting C:\Users\Fs\Desktop\1291_1285\MultiModule\MultiModuleChild2\target
[INFO] BUILD SUCCESS
[INFO] Total time: 0.340 s
[INFO] Finished at: 2025-09-01T10:51:01+05:30


Run As
1 Run on Server
2 Java Application
3 JUnit Test
4 Maven build...
5 Maven clean
6 Maven generate-sources
7 Maven install
8 Maven test
9 Maven verify
Run Configurations...
MultiModuleChild2 (Maven Build)


[INFO] --- install:3.1.1:install (default-install) @ multiModuleChild2 ---
[INFO] Installing C:\Users\fv\Desktop\1291_1285\MultiModule\MultiModuleChild2\pom.xml to C:\Users\User\.m2\repository\Amit\MultiModuleChild2\0.0.1-SNAPSH
[INFO] Installing C:\Users\fv\Desktop\1291_1285\MultiModule\MultiModuleChild2\target\MultiModuleChild2.war to C:\Users\User\.m2\repository\Amit\MultiModu
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 1.779 s
[INFO] Finished at: 2025-09-01T10:51:34+05:30
[INFO] --







8. Executable JARs

Add to `pom.xml`:

<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<configuration>
<archive>
<manifest>
<mainClass>com.example.Main</mainClass>
</manifest>
</archive>
</configuration>
</plugin>
</plugins>
</build>

Run:

```
mvn package
java -jar target/myapp.jar
```

9. Building a WAR File

Create a Maven web project with structure:

```
src/main/webapp/
  └── WEB-INF/web.xml
```

Add:

<packaging>war</packaging>

Command:

mvn package

Generates target/mywebapp.war → deploy on Tomcat server.

Conclusion

Mastering Maven empowers students to structure projects efficiently, manage complex dependencies, and automate testing and packaging. By practicing real-world scenarios—such as resolving build errors, handling resource files, applying patches, and deploying to servers—students gain valuable hands-on experience aligned with professional software development workflows. Maven not only streamlines builds but also enforces standardization and reusability across projects. Through Maven, students learn efficient project management, dependency control, multi-module integration, and reproducible builds. Understanding Maven's lifecycle, plugin system, and error handling prepares students for professional DevOps and CI/CD workflows. This lab equips students with hands-on experience in packaging, testing, resolving conflicts, and deploying real-world Java and Web applications.

Week 8: Jenkins Automation

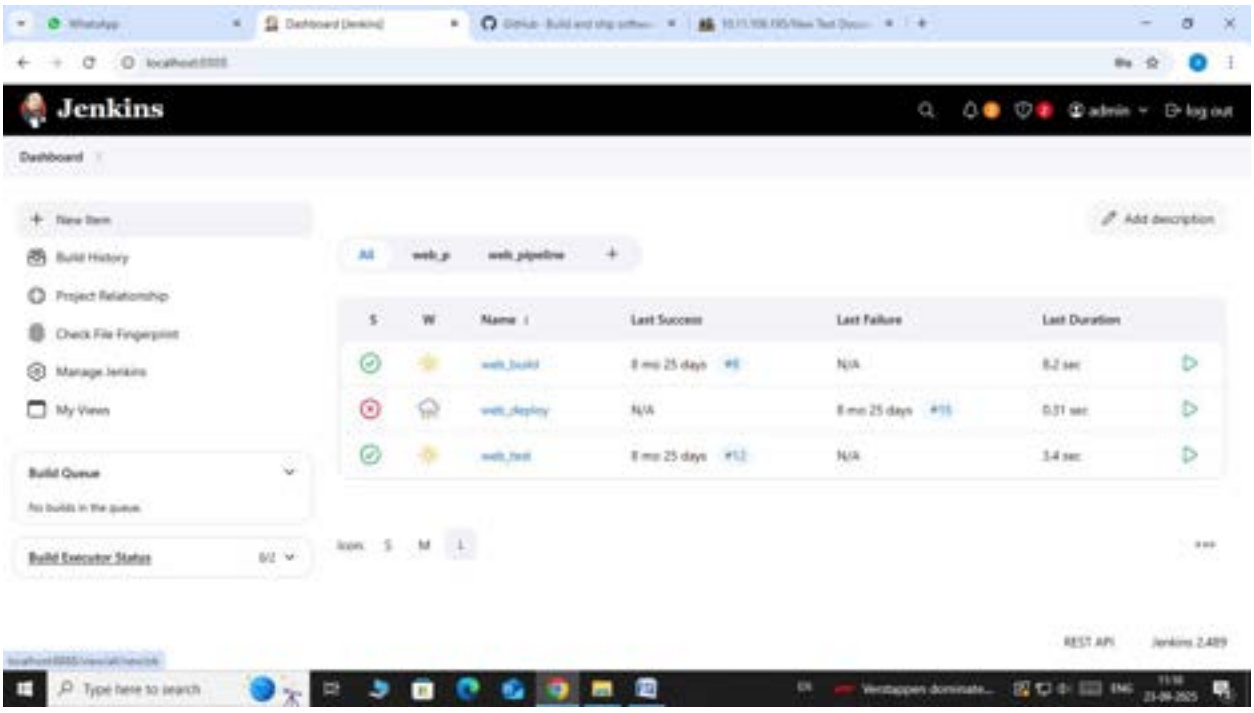
I. Hands-on practice on manual creation of Jenkins pipeline using Maven projects from Github
II. Create the job and build the pipeline for maven-java and maven-web project.
III. Questions on Jenkins
IV. Upload the Screenshots.

I. Steps for MavenJava Automation:

Maven Java Automation Steps:

Step 1: Open Jenkins (localhost:8080)

├── Click on "New Item" (left side menu



Step 2: Create Freestyle Project (e.g., MavenJava_Build)

├── Enter project name (e.g., MavenJava_Build)

├── Click "OK"

└── **Configure the project:**

├── **Description**: "Java Build demo"

├── **Source Code Management**:

└── Git repository URL: [GitMavenJava repo URL]

├── **Branches to build**: */Main or */master

└── **Build Steps**:

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: clean

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: install

└── **Post-build Actions**:

├── Add Post Build Action -> "Archive the artifacts"

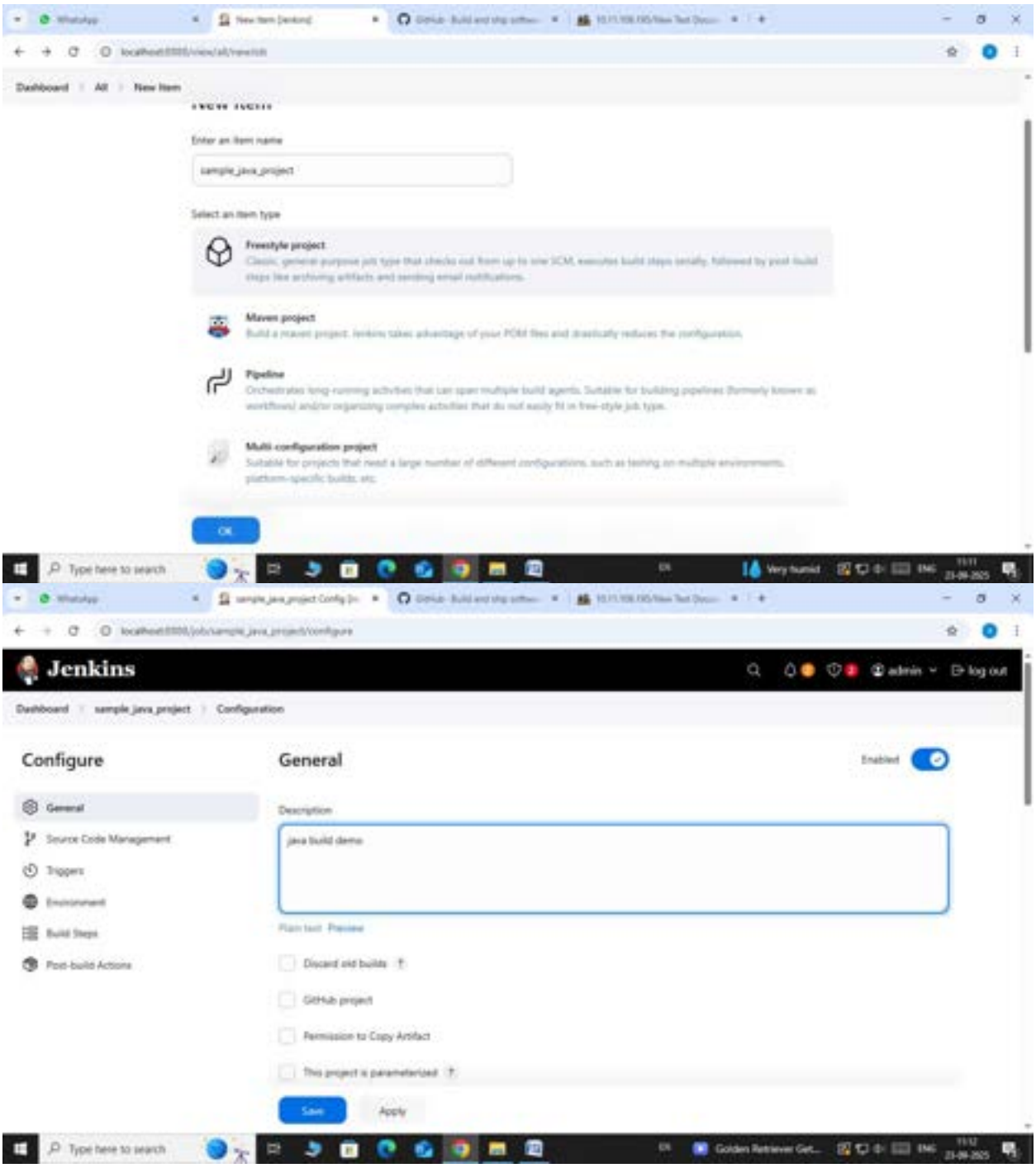
└── Files to archive: **/*

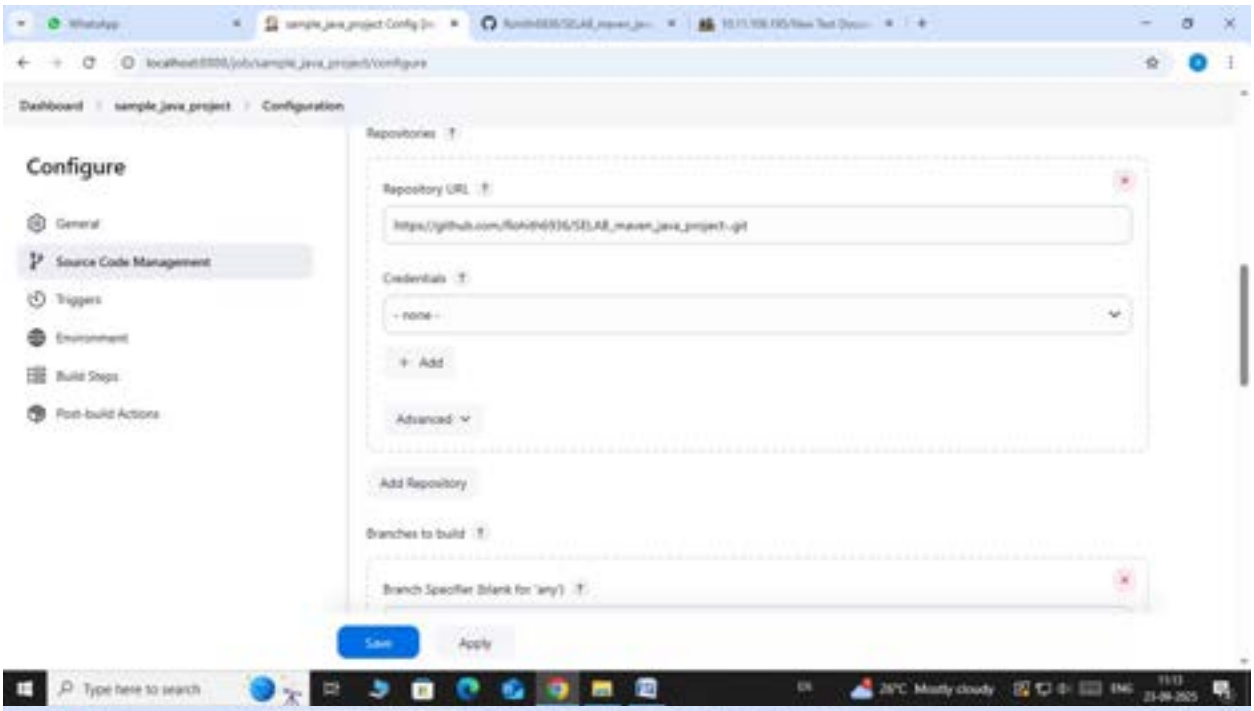
├── Add Post Build Action -> "Build other projects"

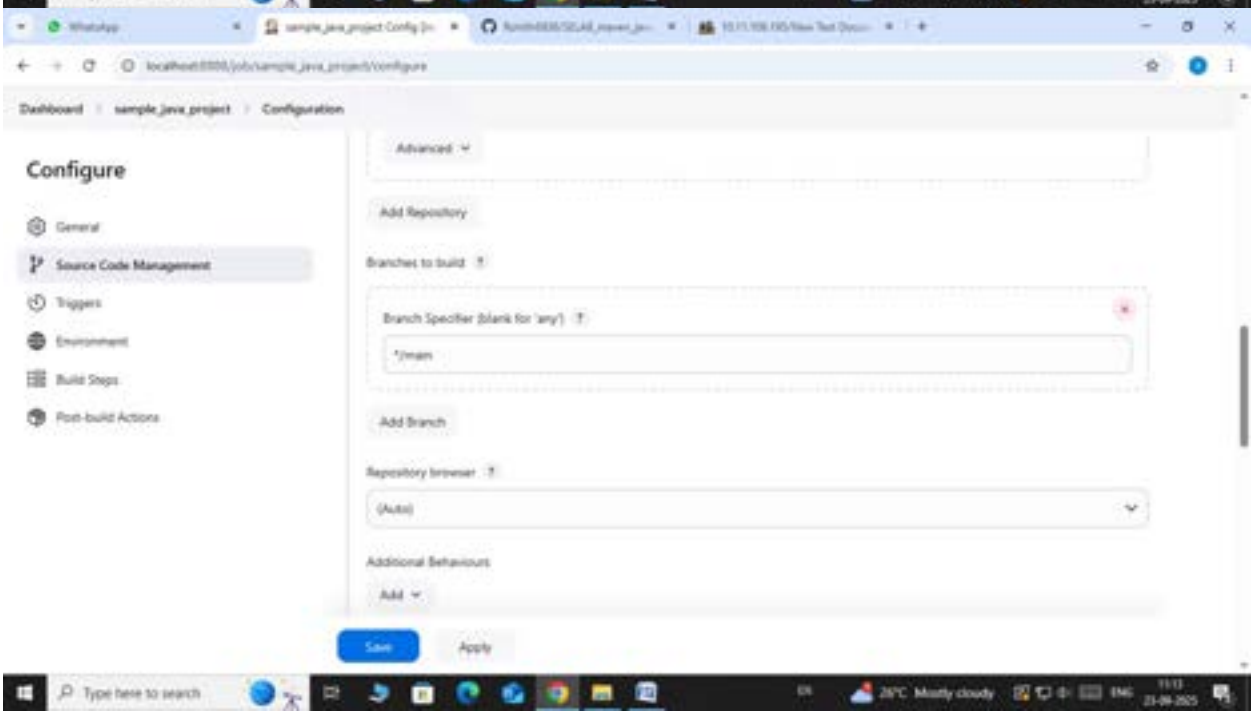
└── Projects to build: MavenJava_Test

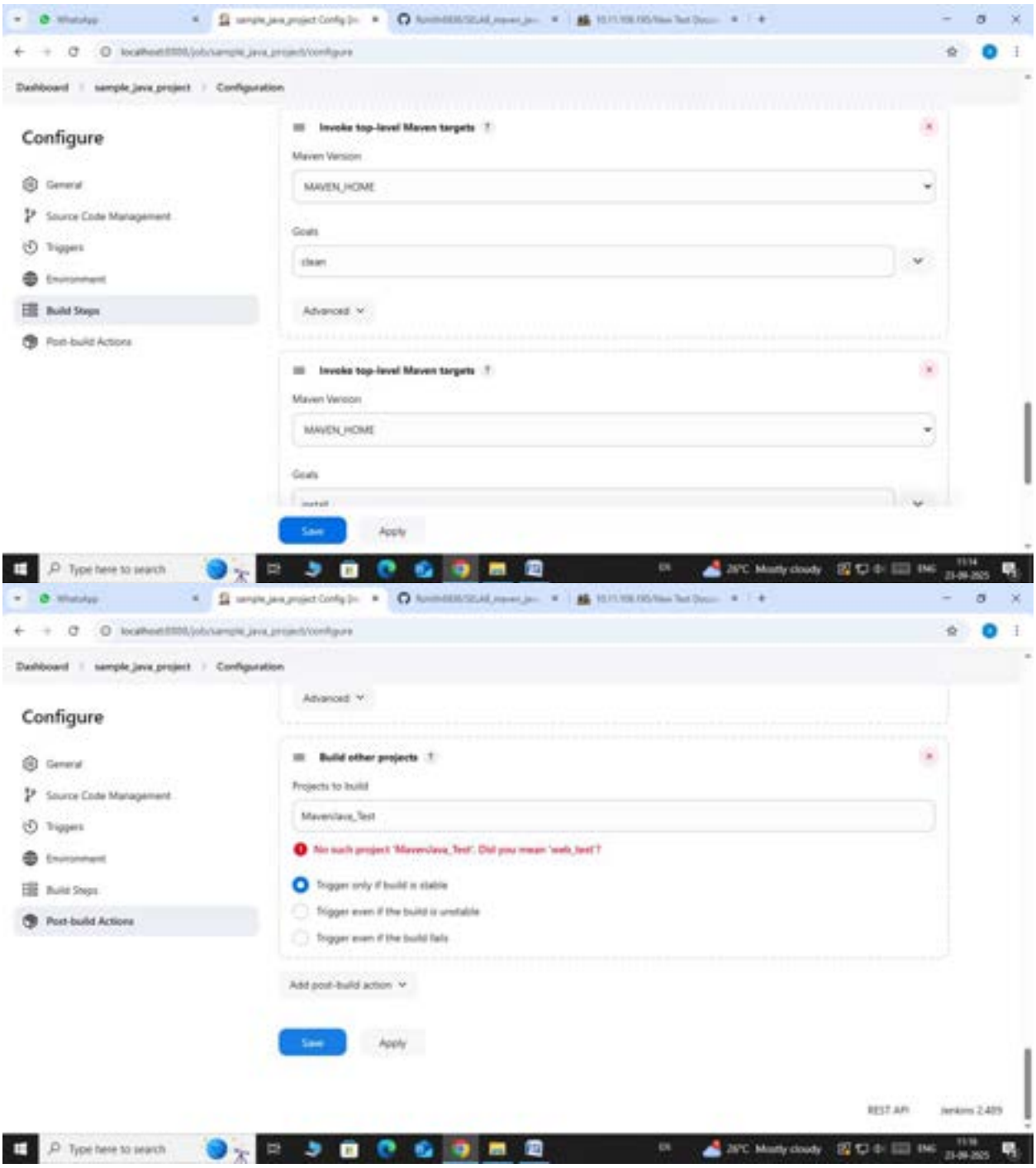
└── Trigger: Only if build is stable

└── Apply and Save


Dashboard
All
New Item
Enter an item name
sample_java_project
Select an item type
Freestyle project
Maven project
Pipeline
Multi-configuration project
OK
Jenkins
admin
log out
Dashboard
sample_java_project
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
General
Enabled
Description
java build demo
Discard old builds
GitHub project
Permission to Copy Artifact
This project is parameterized
Save
Apply


Dashboard
sample_java_project
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Repositories
Repository URL
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
Save
Apply


Dashboard
sample_java_project
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Add Branch
Repository browser
(Auto)
Additional Behaviours
Add
Save
Apply



└── **Step 3: Create Freestyle Project (e.g., MavenJava_Test)**

├── Enter project name (e.g., MavenJava_Test)

├── Click "OK"

└── **Configure the project**:

├── **Description**: "Test demo"

├── **Build Environment**:

└── Check: "Delete the workspace before build starts"

├── **Add Build Step** -> "Copy artifacts from another project"

└── Project name: MavenJava_Build

└── Build: Stable build only // tick at this

└── Artifacts to copy: **/*

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: test

└── Post-build Actions:

├── **Add Post Build Action** -> "Archive the artifacts"

└── Files to archive: **/*

└── Apply and Save

└── **Step 4**: Create Pipeline View for Maven Java project

├── Click "+" beside "All" on the dashboard

├── Enter name: MavenJava_Pipeline

├── **Select "Build pipeline view" // tick here**

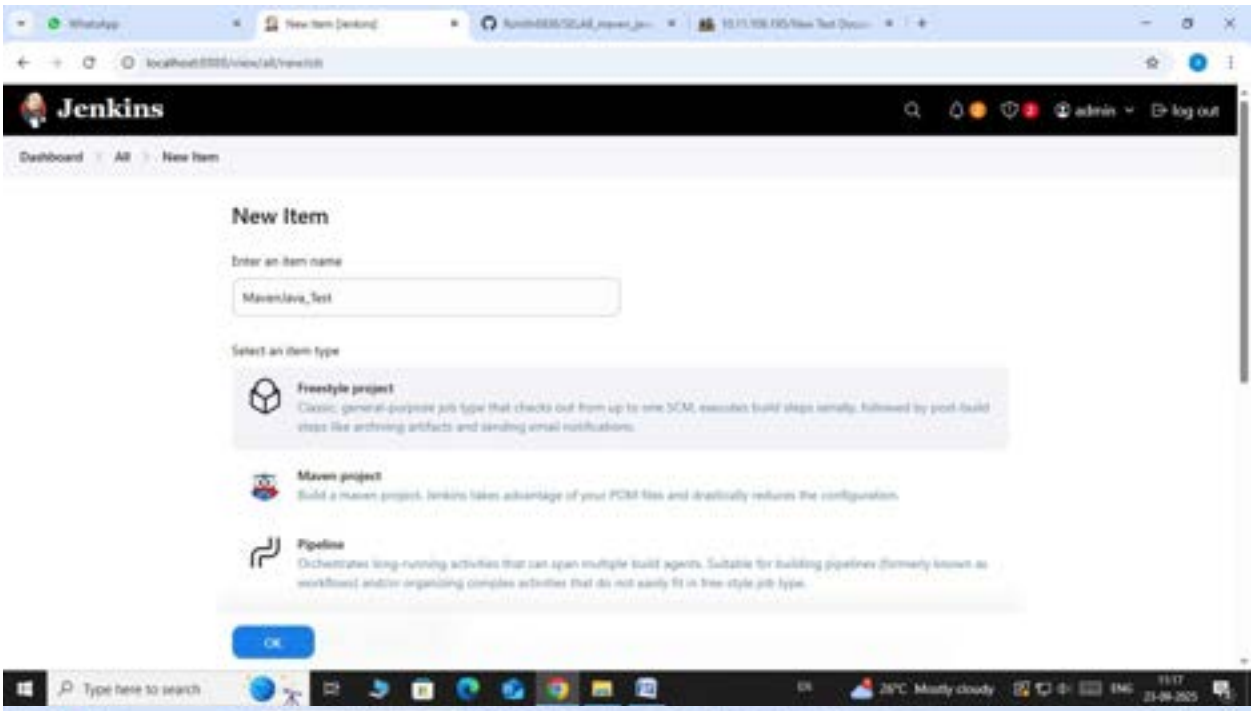
|--- **create**

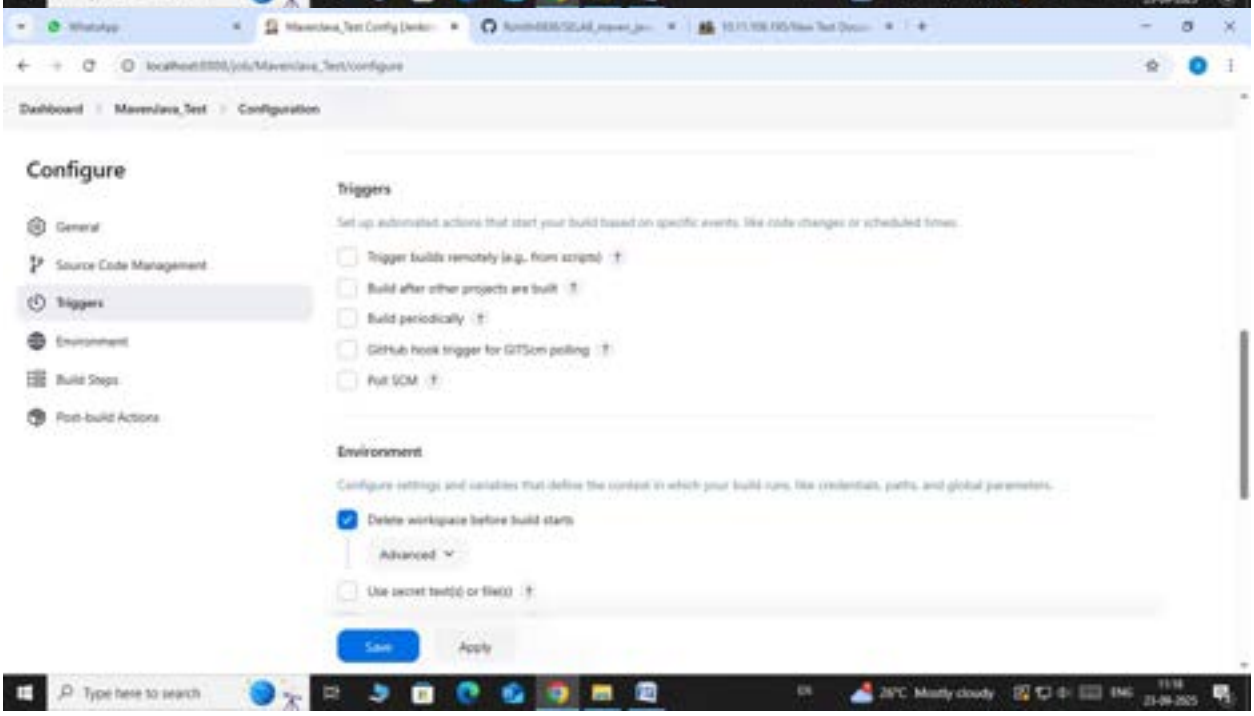
└── **Pipeline Flow**:

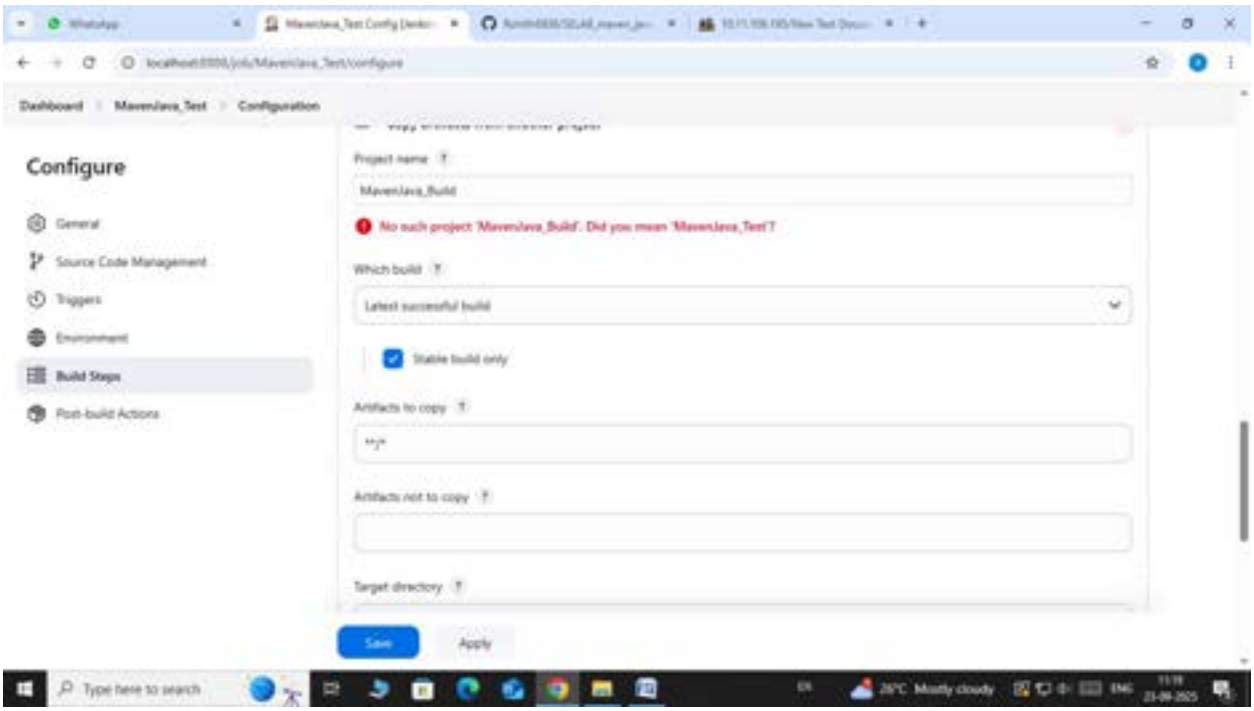
├── **Layout**: Based on upstream/downstream relationship

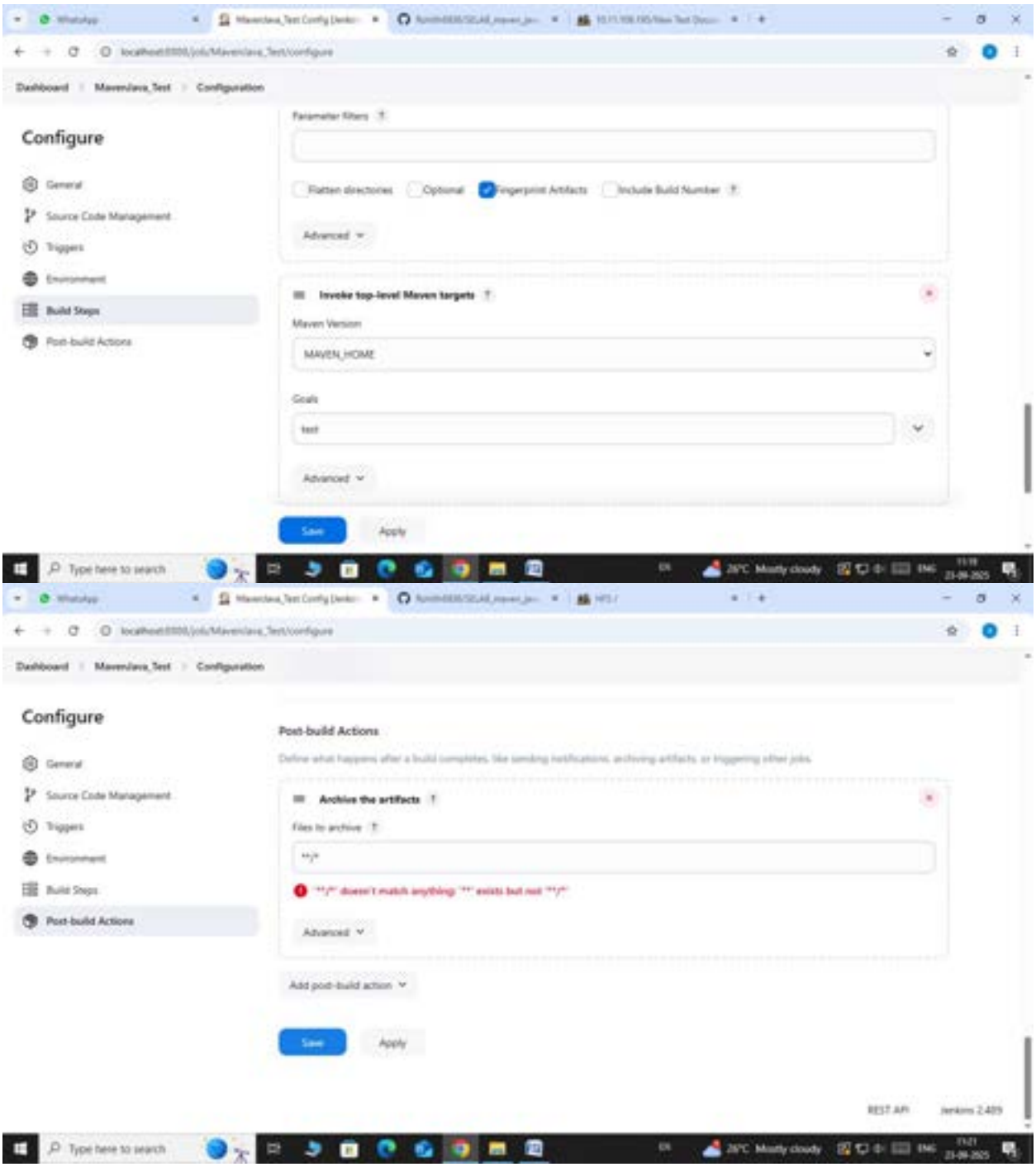
├── Initial job: MavenJava_Build

└── Apply and Save OK


Jenkins
admin
log out
Dashboard
All
New Item
New Item
Enter an item name
MavenJava_Test
Select an item type
Freestyle project
Maven project
Pipeline
OK


Dashboard
MavenJava_Test
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Triggers
Trigger builds remotely (e.g., from scripts)
Build after other projects are built
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Environment
Delete workspace before build starts
Advanced
Save
Apply

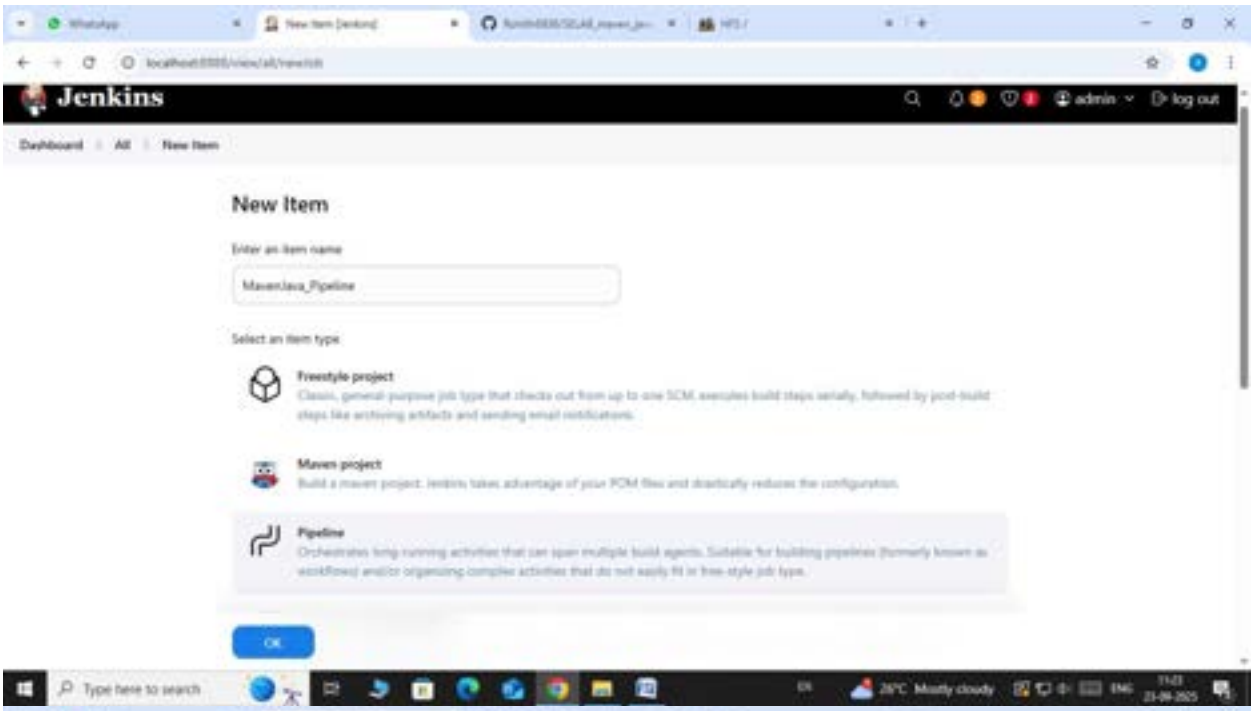

Dashboard
MavenJava_Test
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Project name
MavenJava_Build
No such project 'MavenJava_Build'. Did you mean 'MavenJava_Test'?
Which build
Latest successful build
Stable build only
Artifacts to copy
**/*
Artifacts not to copy
Target directory
Save
Apply
Type here to search
26°C Mostly cloudy

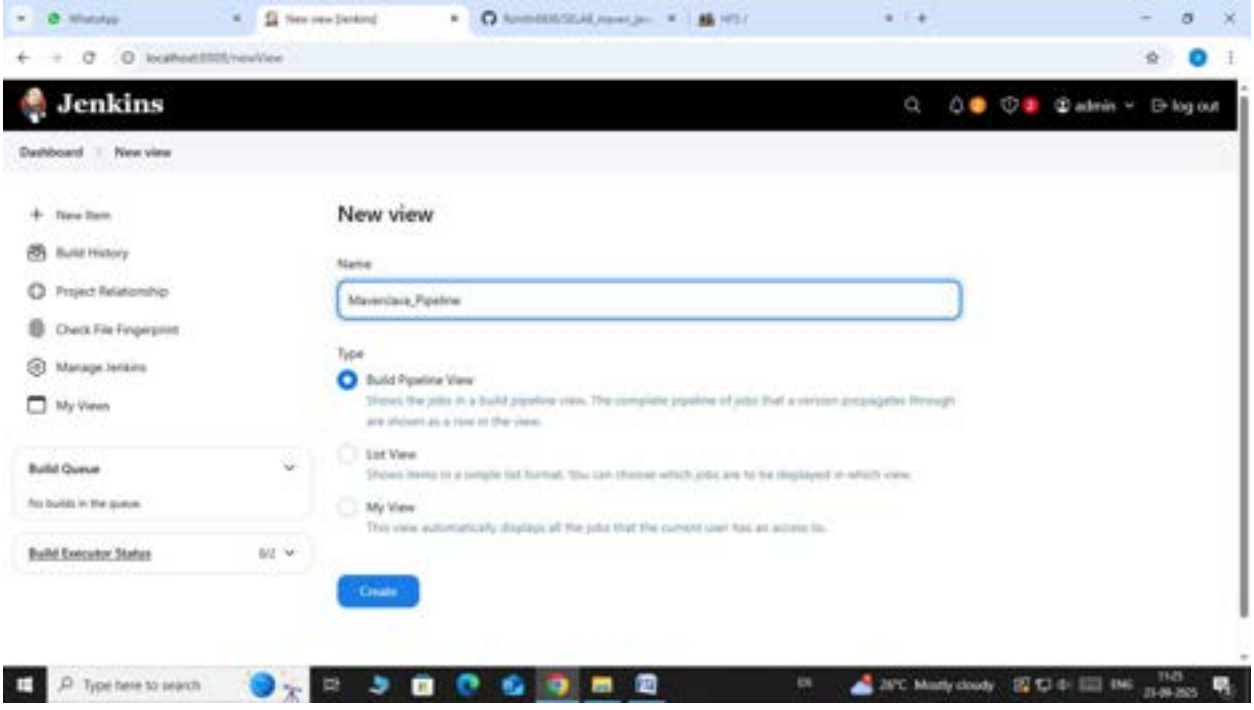


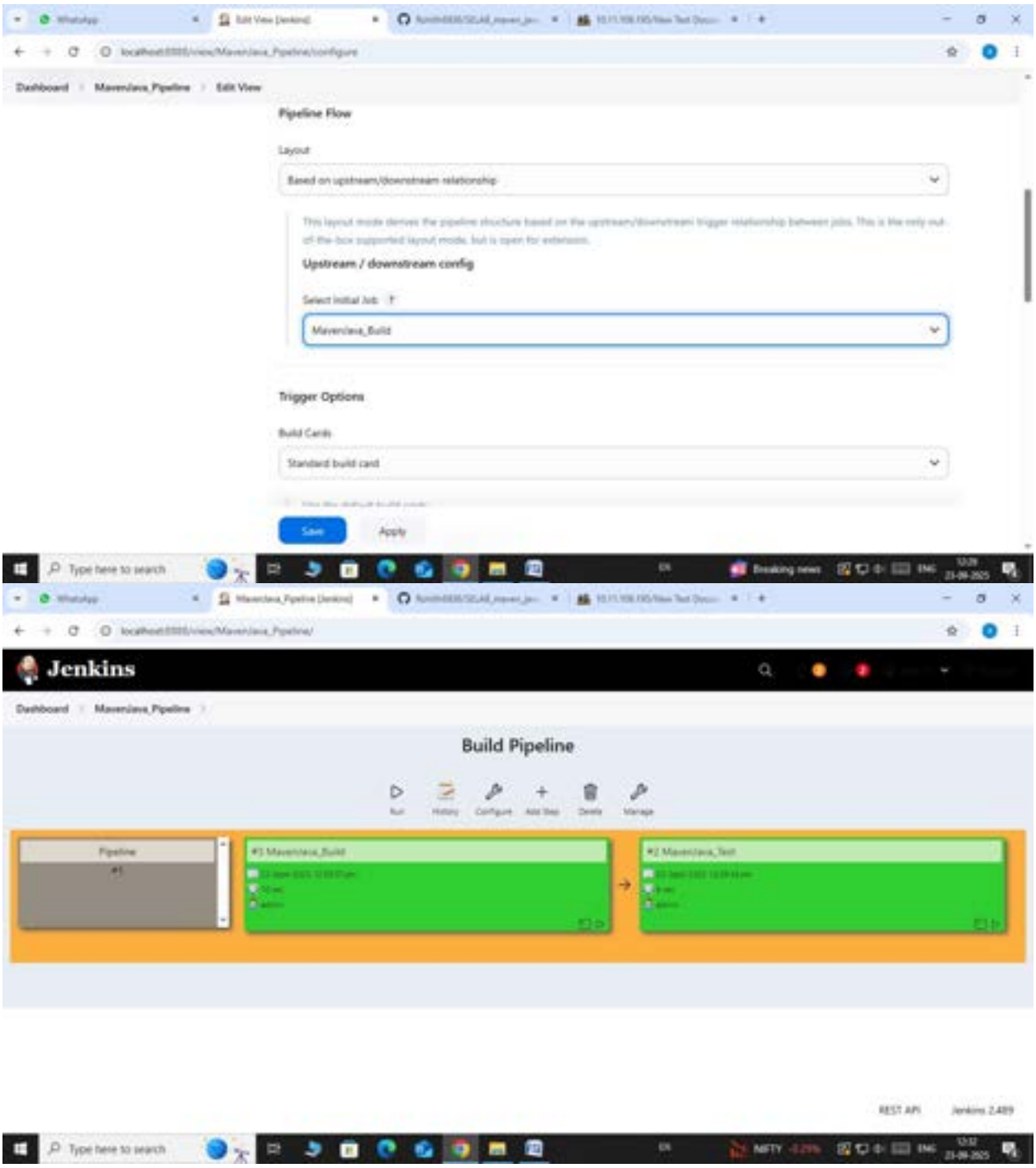
└── **Step 5**: Run the Pipeline and Check Output

├── Click on the trigger to run the pipeline

├── click on the small black box to open the console to check if the build is success


Jenkins
admin
log out
Dashboard
All
New Item
New Item
Enter an item name
MavenJava_Pipeline
Select an item type
Freestyle project
Maven project
Pipeline
OK


Jenkins
admin
log out
Dashboard
New view
New Item
Build History
Project Relationship
Check File Fingerprint
Manage Jenkins
My Views
Build Queue
No builds in the queue
Build Executor Status
New view
Name
MavenJava_Pipeline
Type
Build Pipeline View
List View
My View
Create



Note :

1. If build is success and the test project is also automatically triggered with name

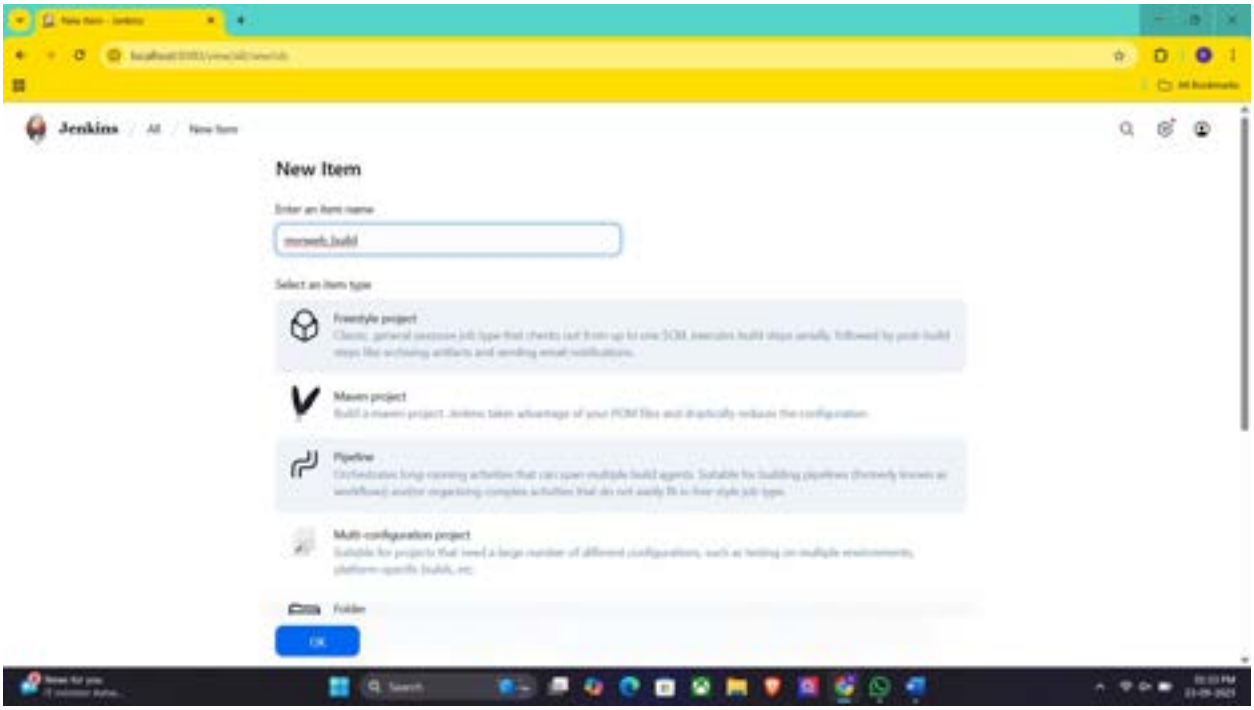
   “MavenJava_Test”

2. The pipeline is successful if it is in green color as shown ->check the console of the test project
3. The test project is successful and all the artifacts are archived successfully

II. Maven Web Automation Steps:

└── **Step 1:** Open Jenkins (localhost:8080)

├── Click on "New Item" (left side menu)



└── **Step 2**: Create Freestyle Project (e.g., MavenWeb_Build)

├── Enter project name (e.g., MavenWeb_Build)

├── Click "OK"

└── **Configure the project**:

├── **Description**: "Web Build demo"

├── **Source Code Management:**

└── Git repository URL: [GitMavenWeb repo URL]

├── *Branches to build*: */Main or master

└── **Build Steps**:

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: clean

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: install

└── **Post-build Actions**:

├── **Add Post Build Action** -> "Archive the artifacts"

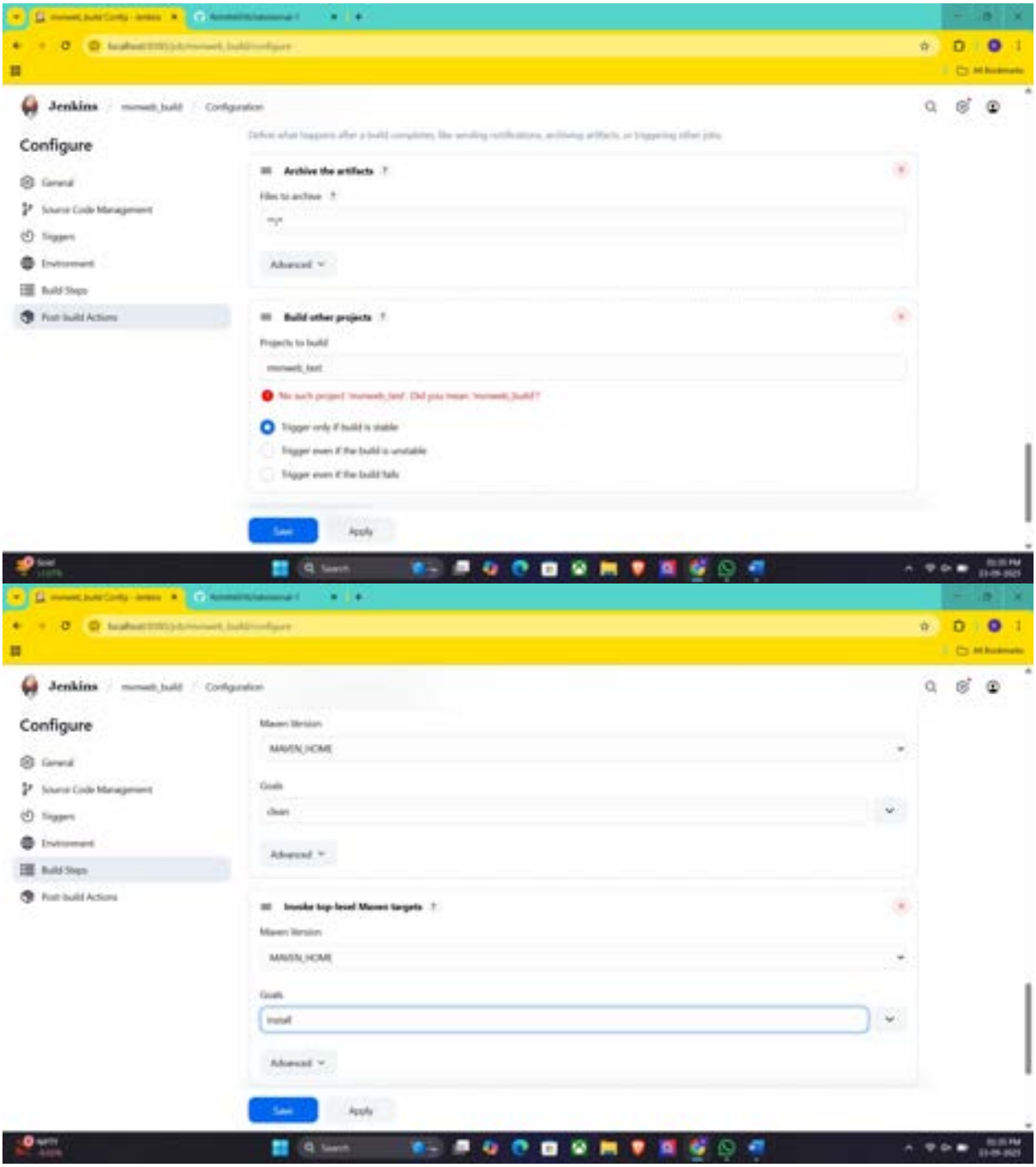
└── Files to archive: **/*

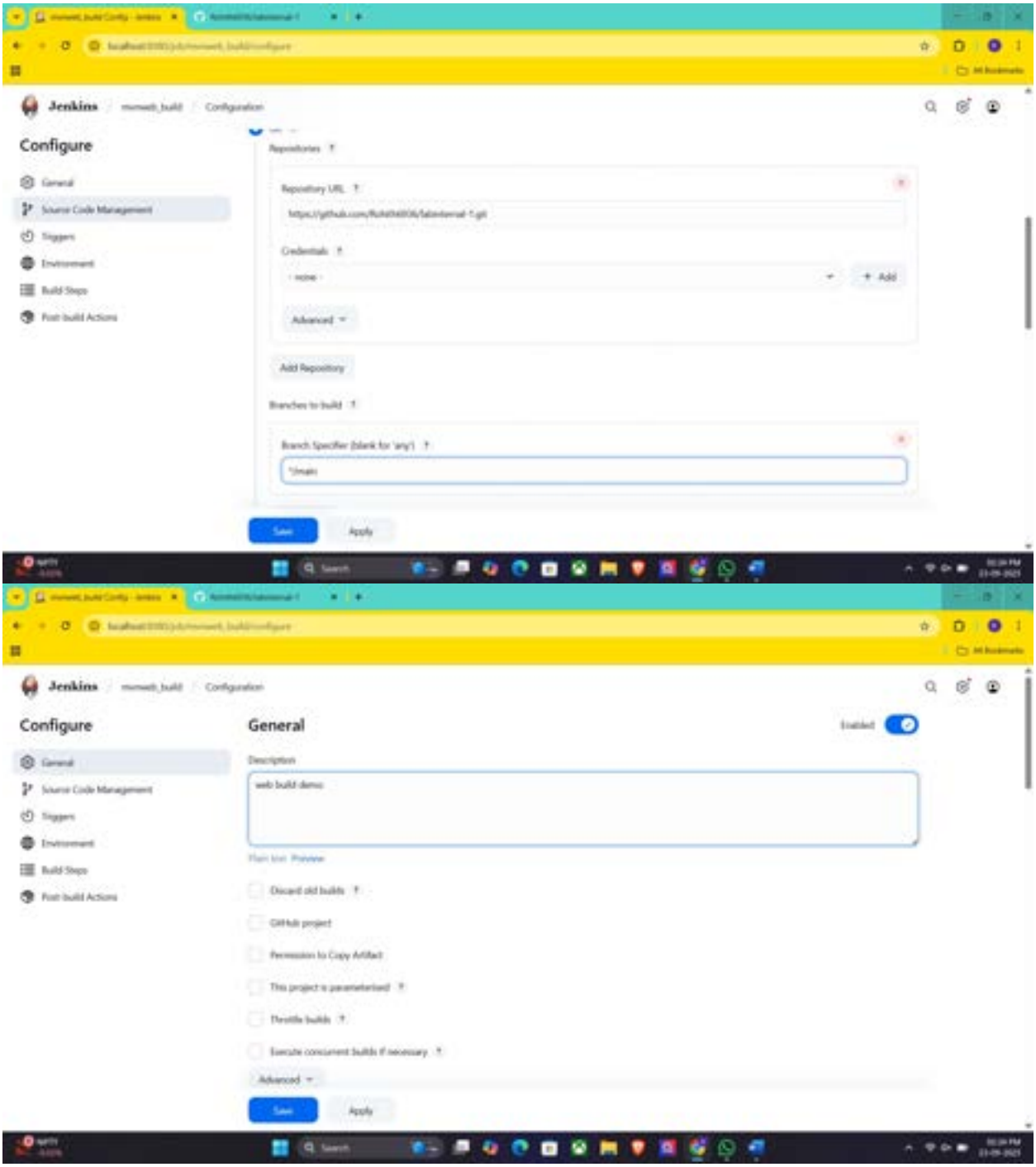
├── **Add Post Build Action** -> "Build other projects"

└── Projects to build: MavenWeb_Test

└── Trigger: Only if build is stable

└── Apply and Save


Jenkins
mvnweb_build
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Archive the artifacts
Files to archive
**/*
Advanced
Build other projects
Projects to build
mvnweb_test
Trigger only if build is stable
Trigger even if the build is unstable
Trigger even if the build fails
Save
Apply
Jenkins
mvnweb_build
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Maven Version
MAVEN_HOME
Goals
clean
Advanced
Invoke top-level Maven targets
Maven Version
MAVEN_HOME
Goals
install
Advanced
Save
Apply



└── **Step 3**: Create Freestyle Project (e.g., MavenWeb_Test)

├── Enter project name (e.g., MavenWeb_Test)

├── Click "OK"

└── **Configure the project:**

├── **Description:** "Test demo"

├── **Build Environment**:

└── Check: "Delete the workspace before build starts"

├── **Add Build Step** -> "Copy artifacts from another project"

└── Project name: MavenWeb_Build

└── Build: Stable build only

└── Artifacts to copy: **/*

├── **Add Build Step** -> "Invoke top-level Maven targets"

└── Maven version: MAVEN_HOME

└── Goals: test

└── **Post-build Actions**:

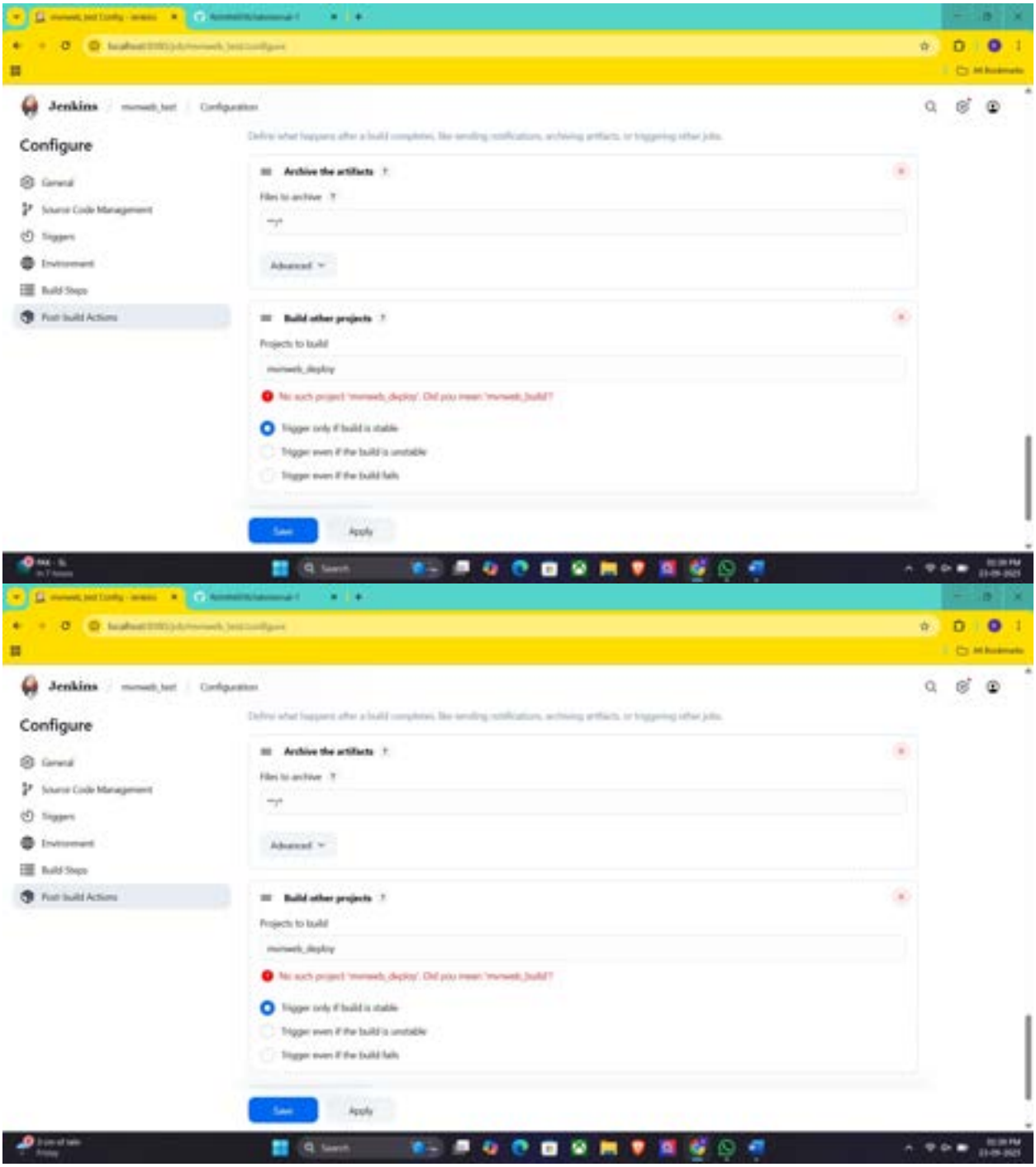
├── **Add Post Build Action** -> "Archive the artifacts"

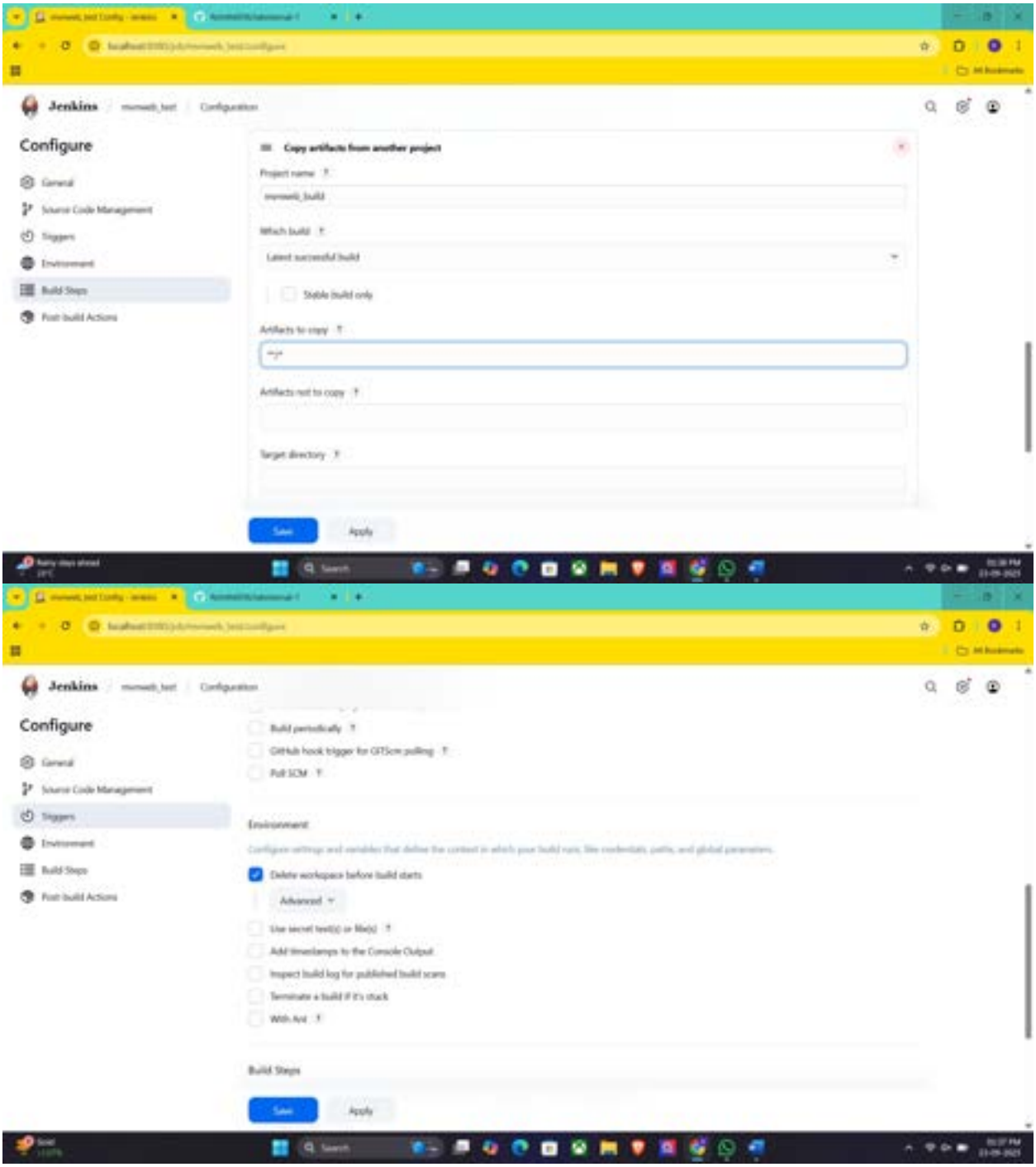
└── Files to archive: **/*

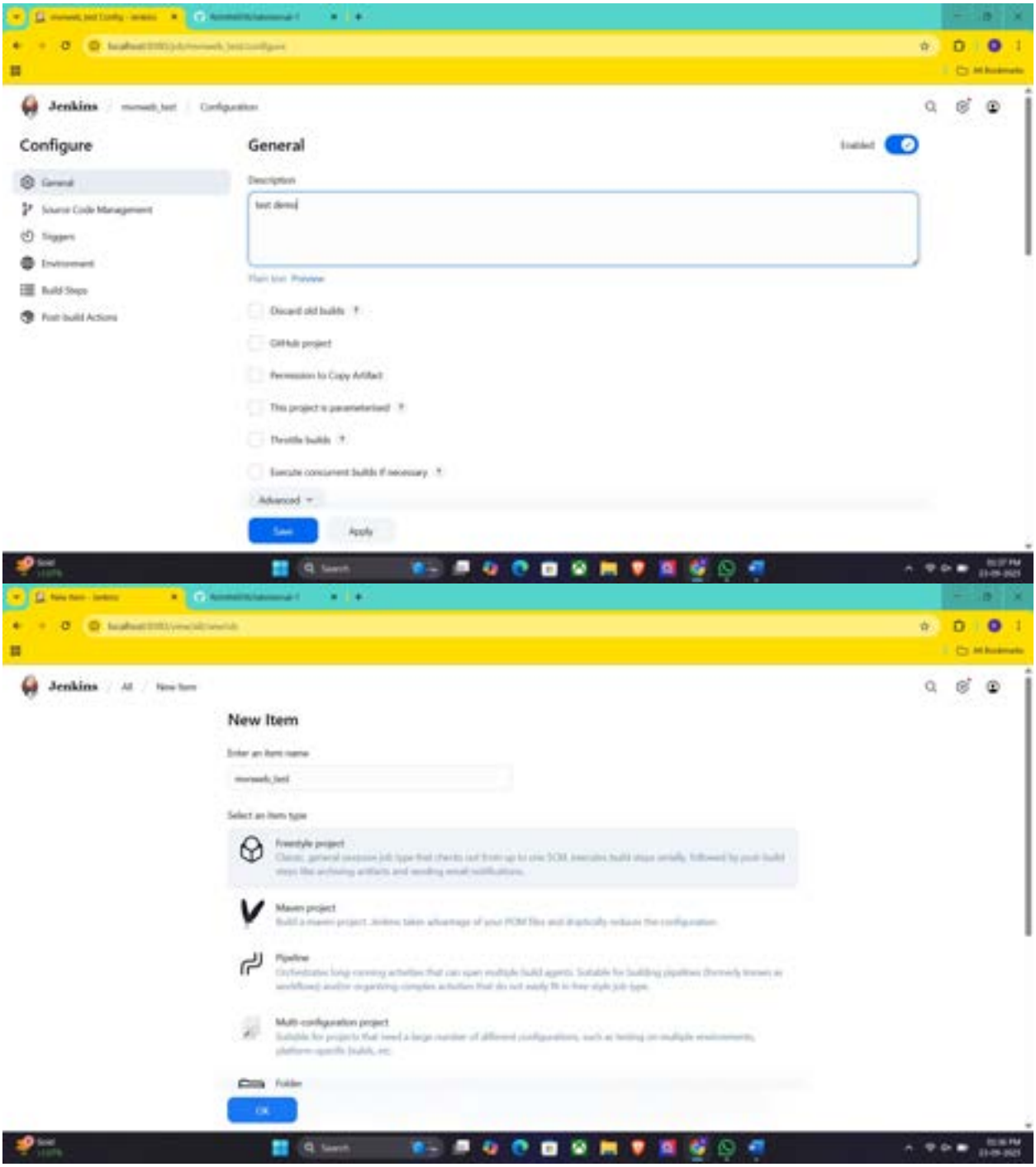
├── **Add Post Build Action** -> "Build other projects"

└── Projects to build: MavenWeb_Deploy

└── Apply and Save


Jenkins
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Archive the artifacts
Files to archive
Advanced
Build other projects
Projects to build
Trigger only if build is stable
Trigger even if the build is unstable
Trigger even if the build fails
Save
Apply


Jenkins
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Copy artifacts from another project
Project name
Which build
Latest successful build
Stable build only
Artifacts to copy
Artifacts not to copy
Target directory
Save
Apply
Jenkins
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Environment
Delete workspace before build starts
Advanced
Add timestamps to the Console Output
Terminate a build if it's stuck
With Ant
Build Steps
Save
Apply



└── **Step 4**: Create Freestyle Project (e.g., MavenWeb_Deploy)

  ├── Enter project name (e.g., MavenWeb_Deploy)

  ├── Click "OK"

  └── **Configure the projec**t:

    ├── **Description**: "Web Code Deployment"

├── **Build Environment**:

└── Check: "Delete the workspace before build starts"

├── **Add Build Step** -> "Copy artifacts from another project"

└── Project name: MavenWeb_Test

└── Build: Stable build only

└── Artifacts to copy: **/*

└── **Post-build Actions**:

├── **Add Post Build Action** -> "Deploy WAR/EAR to a container"

└── WAR/EAR File: **/*.war

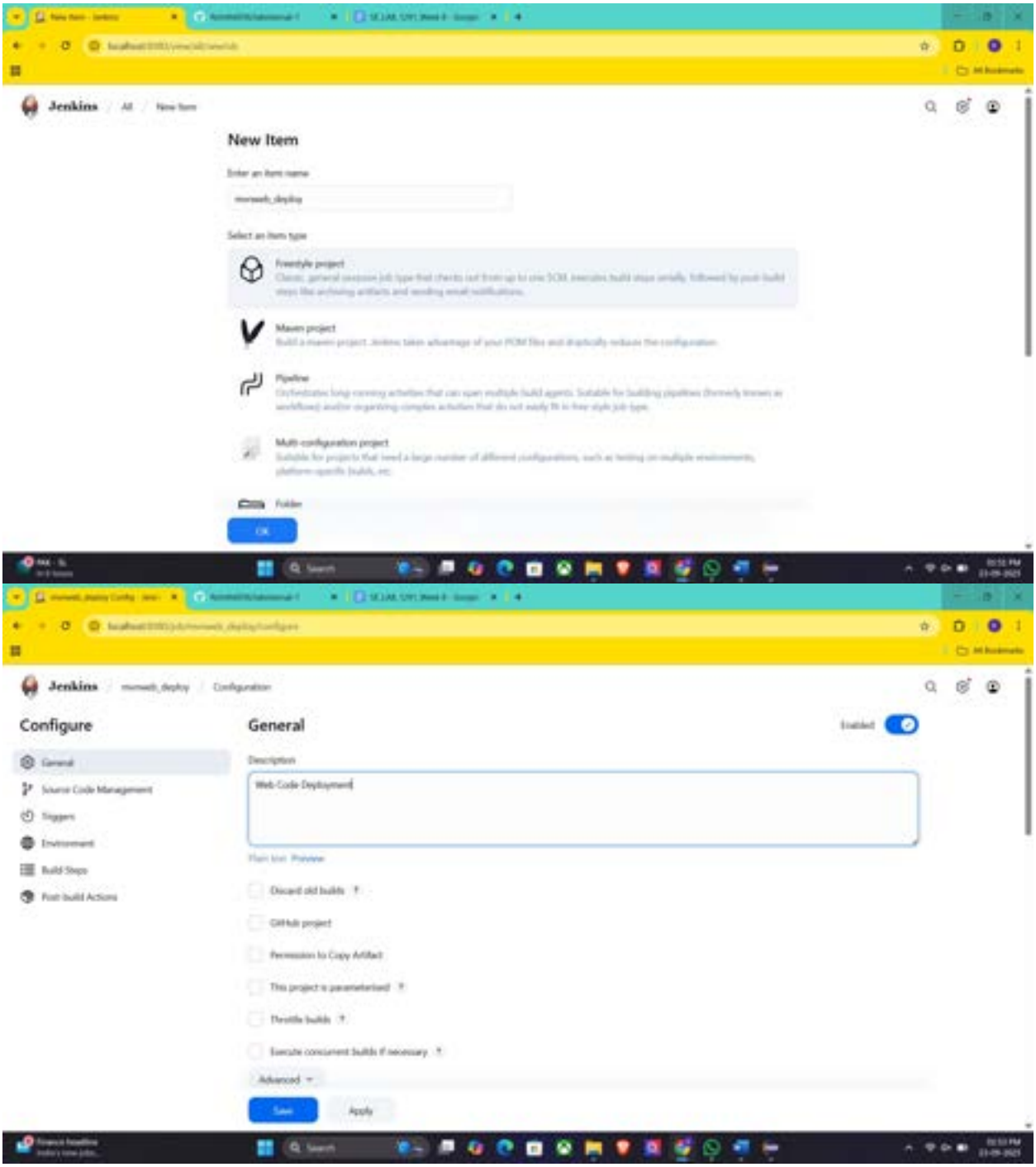
└── Context path: Webpath

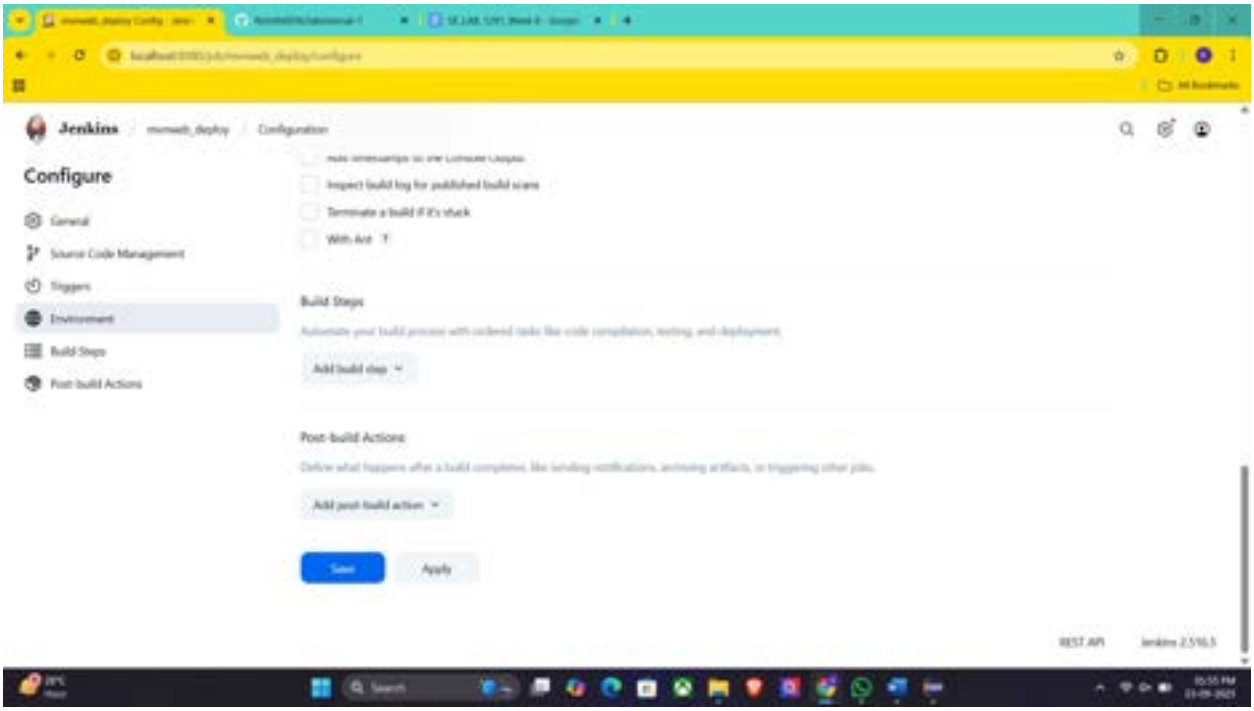
└── Add container -> Tomcat 9.x remote

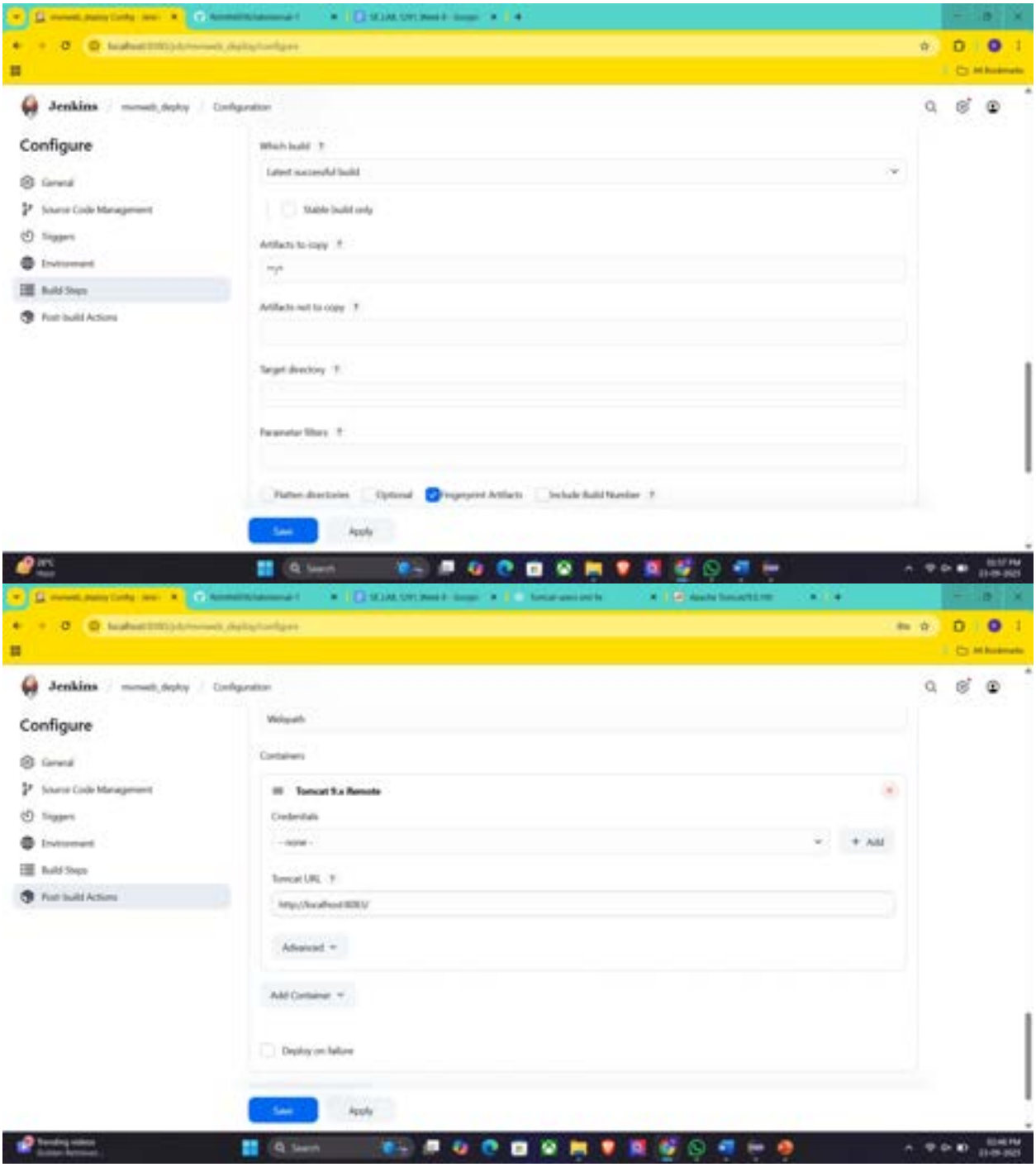
└── Credentials: Username: admin, Password: 1234

── Tomcat URL: https://localhost:8085/

└── Apply and Save


Jenkins
New Item
Freestyle project
Maven project
Pipeline
Multi-configuration project
Folder
OK
Jenkins
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
General
Enabled
Description
Web Code Deployment
Discard old builds
GitHub project
This project is parameterised
Throttle builds
Execute concurrent builds if necessary
Advanced
Save
Apply


Jenkins
mvnweb_deploy
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Inspect build log for published build scans
Terminate a build if it's stuck
With Ant
Build Steps
Add build step
Post-build Actions
Add post-build action
Save
Apply
REST API



└── **Step 5**: Create Pipeline View for MavenWeb

  ├── Click "+" beside "All" on the dashboard

  ├── Enter name: MavenWeb_Pipeline

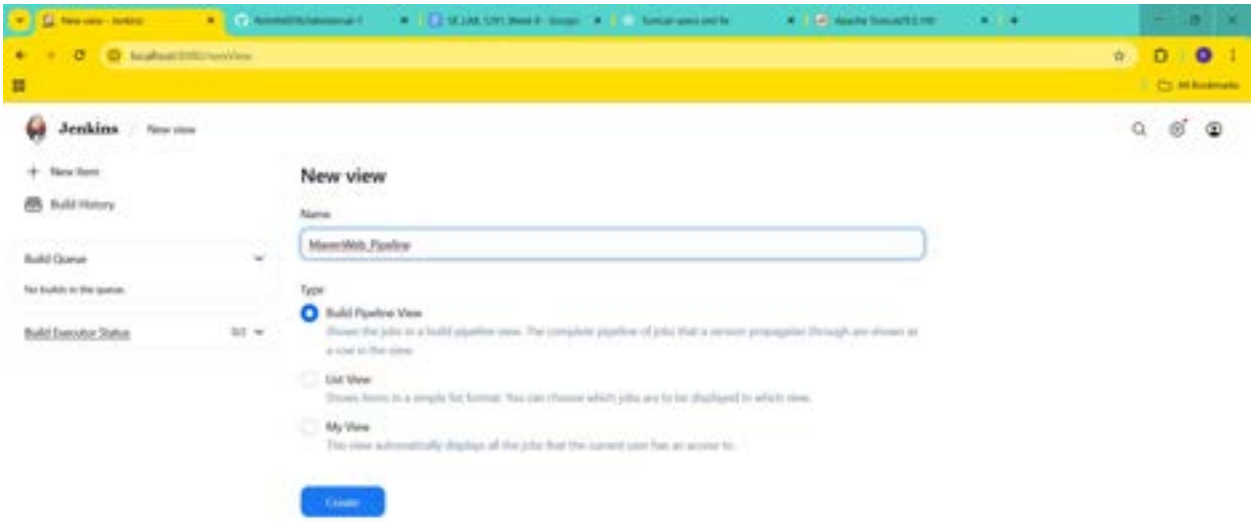
  ├── **Select "Build pipeline view"**

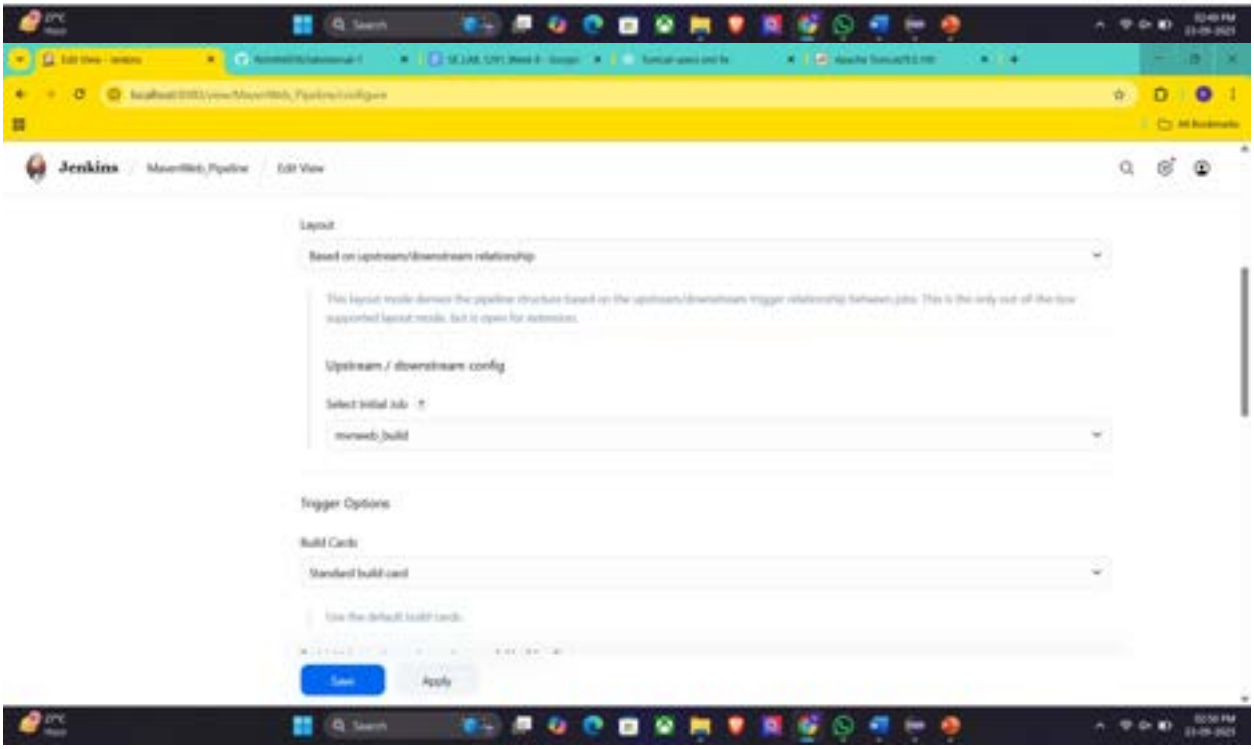
  └── **Pipeline Flow**:

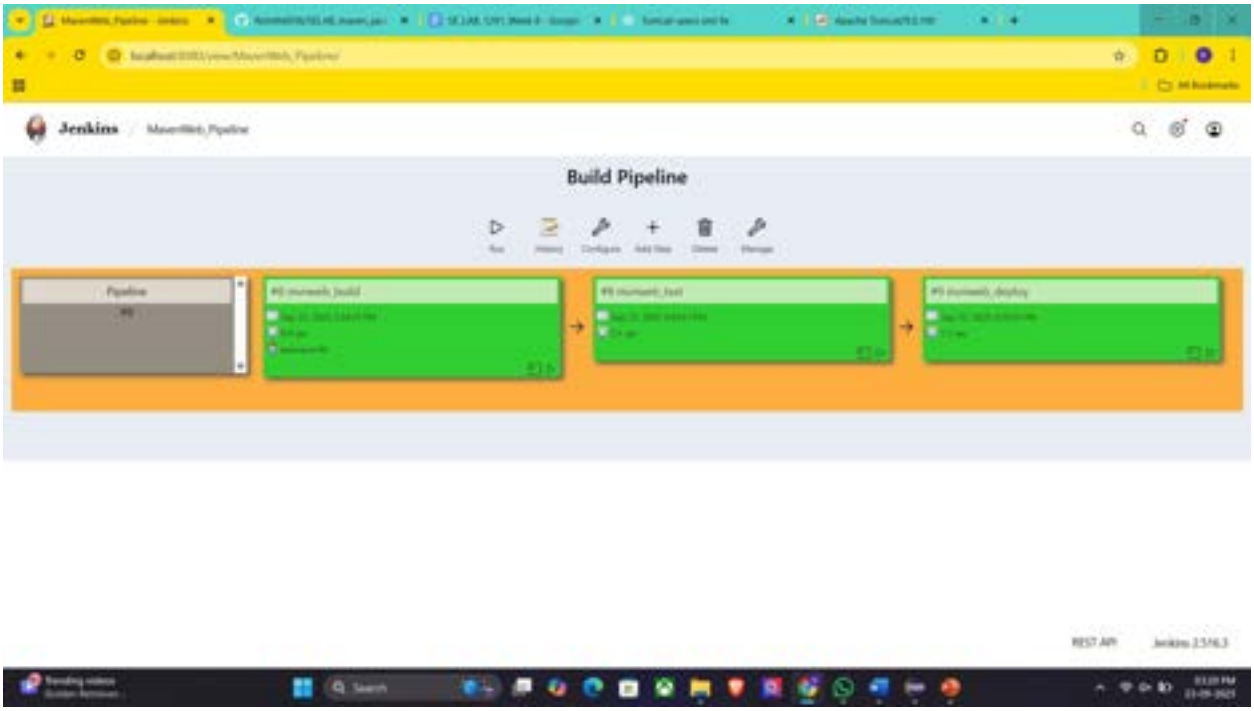
├── **Layout**: Based on upstream/downstream relationship

├── Initial job: MavenWeb_Build

└── Apply and Save


Jenkins
New view
New item
Build History
Build Queue
No builds in the queue.
Build Executor Status
New view
Name
MavenWeb_Pipeline
Type
Build Pipeline View
List View
My View
Create


Jenkins
MavenWeb_Pipeline
Edit View
Layout
Based on upstream/downstream relationship
Upstream / downstream config
Select Initial Job
mvnweb_build
Trigger Options
Build Cards
Standard build card
Save
Apply



└── **Step 6**: Run the Pipeline and Check Output

├── Click on the trigger **"RUN"** to run the pipeline

Note:

1. After Click on Run -> click on the small black box to open the console to check if the build is success
2. Now we see all the build has success if it appears in green color

├── Open Tomcat homepage in another tab

├── Click on the "/webpath" option under the manager app

Note:

1. It ask for user credentials for login ,provide the credentials of tomcat.
2. It provide the page with out project name which is highlighted.
3. After clicking on our project we can see output.

Conclusion: In this week student learnt automating Maven projects through Jenkins.

Week 9: Pipeline Creation using script

1. Evaluation of Jenkins pipeline.
2. **WORKING ON BUILD TRIGGERS FOR LAST JENKINS PIPILINE**
3. Hands-on practice on creation of scripted Jenkins pipeline.
4. Take the screenshots for above task

Procedure

General :

Description :- provide the description of the project

Build triggers: here we can provide with build triggers of you choice

Advance project option:

Definition:

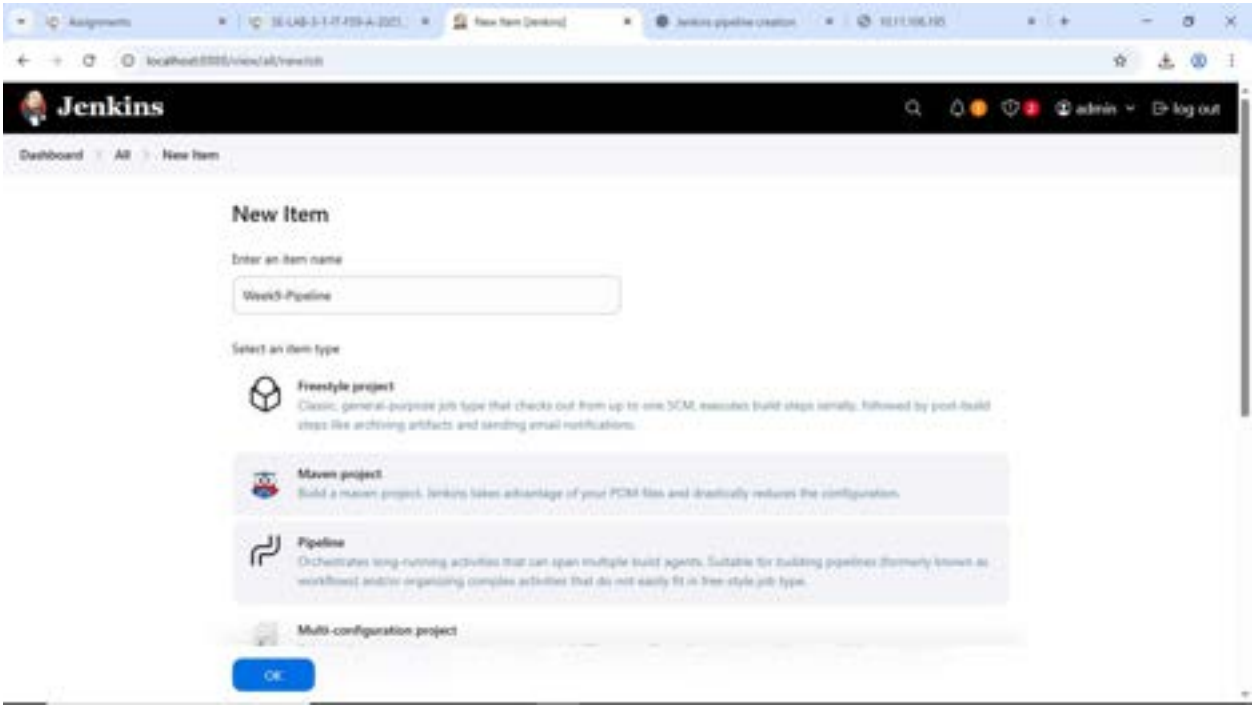
Choose pipeline script

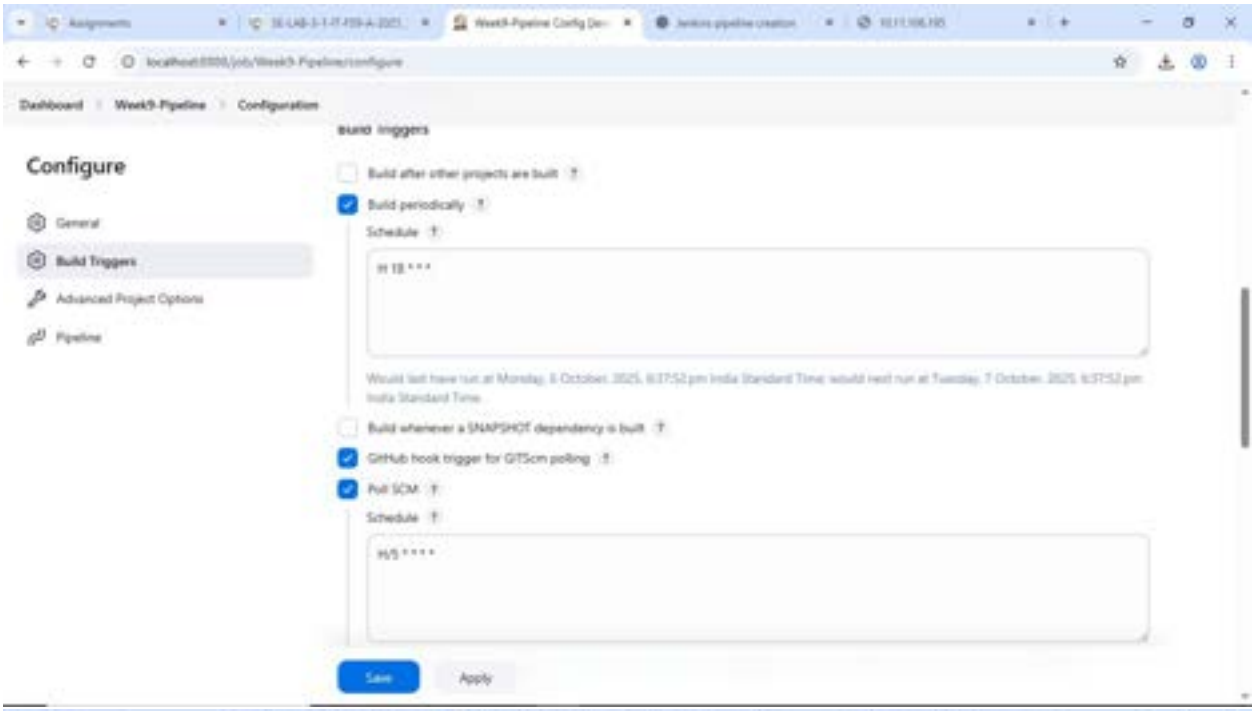
Here code for pipeline -script

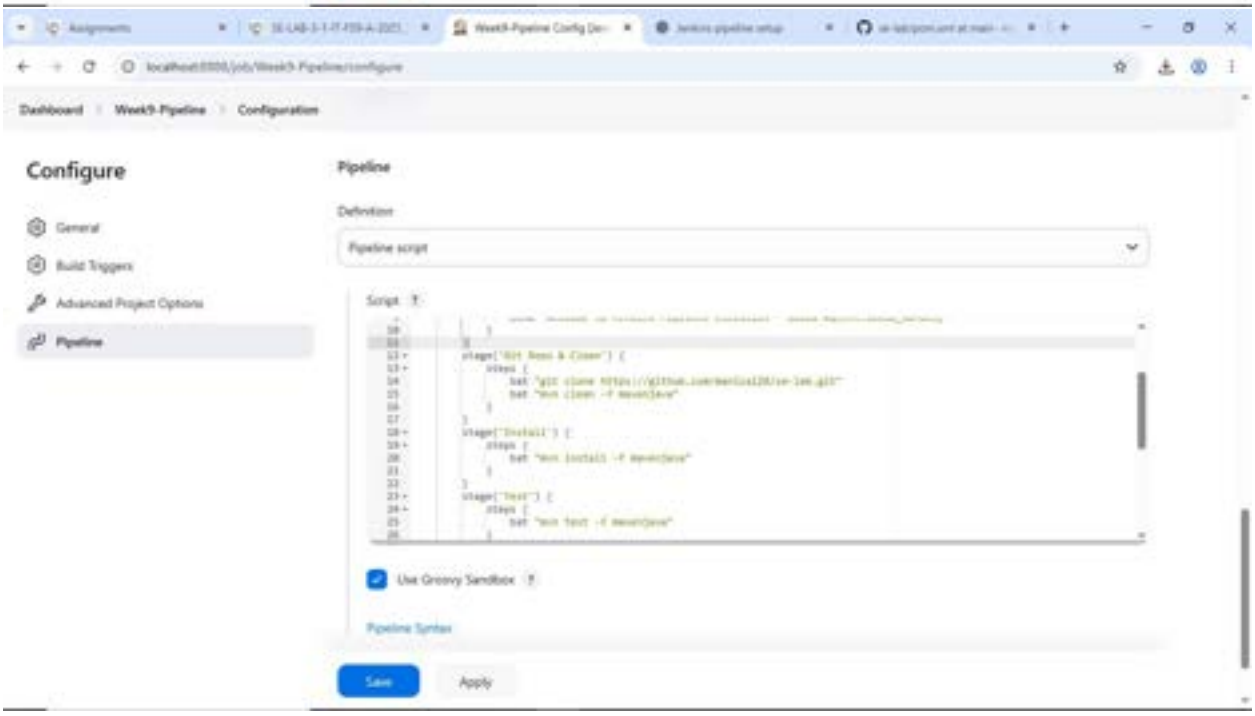
Copy the code there

```
pipeline {
   agent any
   tools{
      maven 'MAVEN-HOME'
   }
   stages {
      stage('git repo & clean') {
         steps {
            //bat "rmdir  /s /q mavenjava"
            bat "git clone provide your github link"
            bat "mvn clean -f mavenjava"
         }
      }
      stage('install') {
         steps {
            bat "mvn install -f mavenjava"#project name#
         }
      }
      stage('test') {
         steps {
```

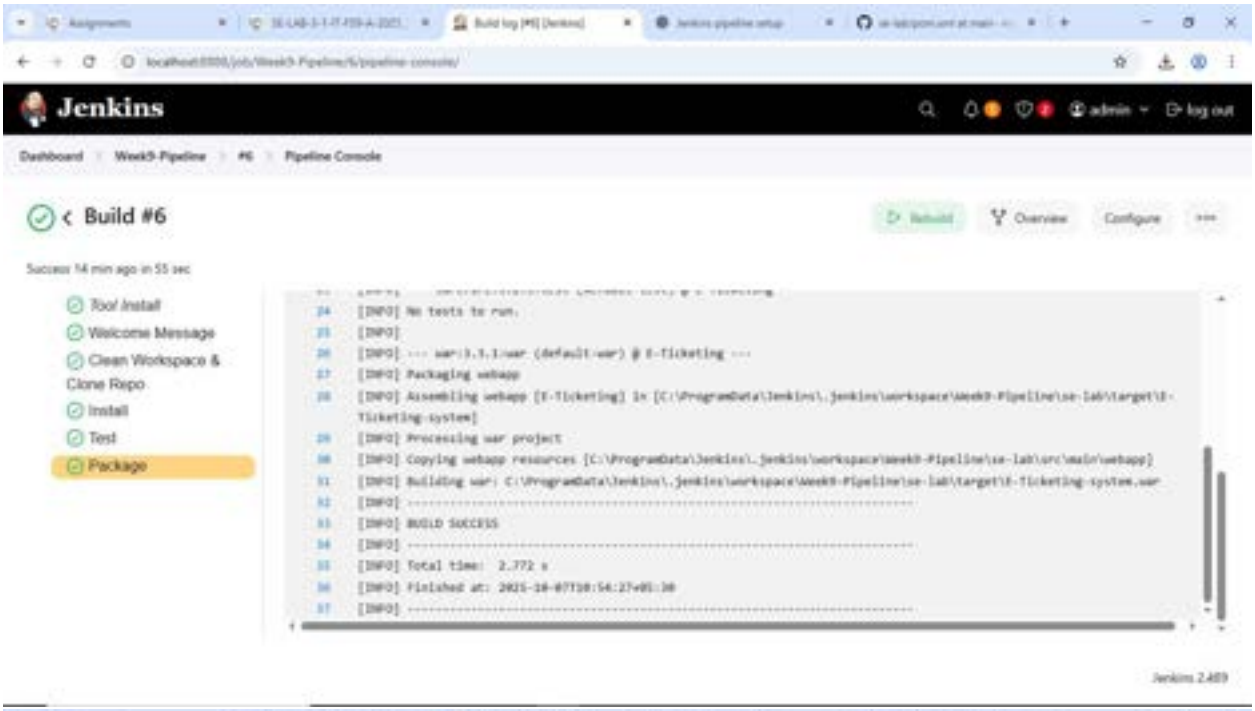
```
            bat "mvn test -f mavenjava"
        }
    }
    stage('package') {
        steps {
            bat "mvn package -f mavenjava"
        }
    }
  }
}
```

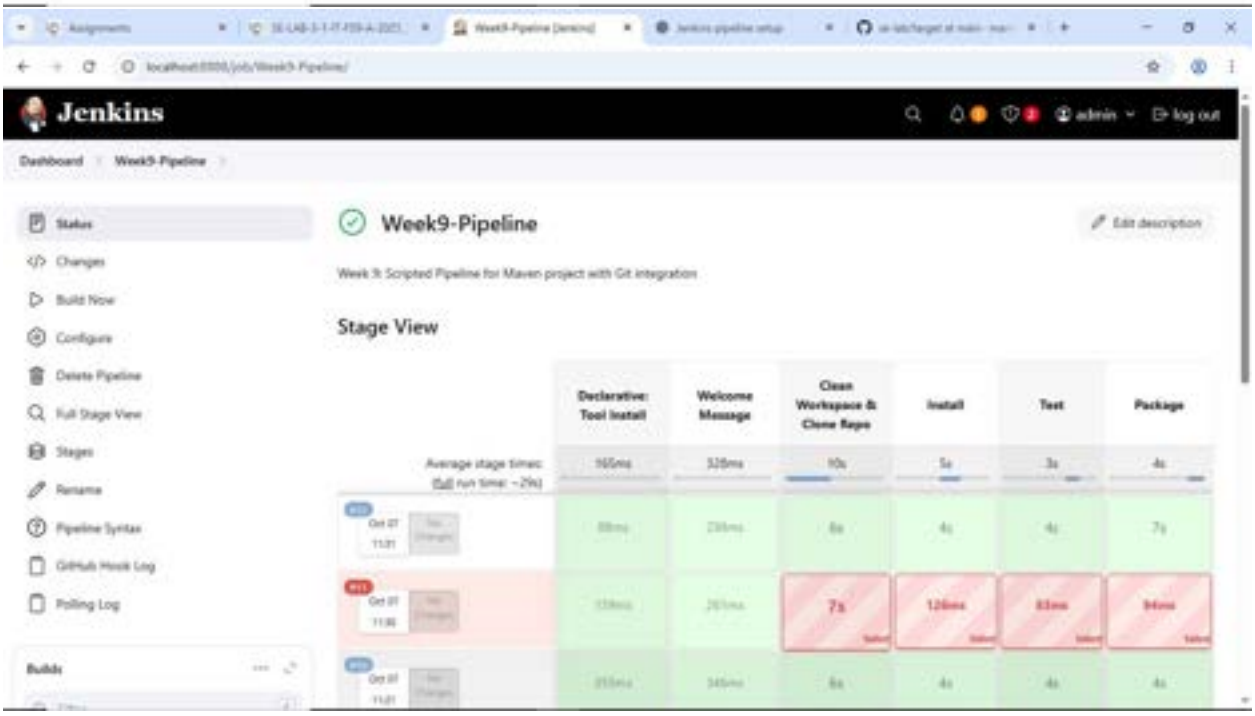


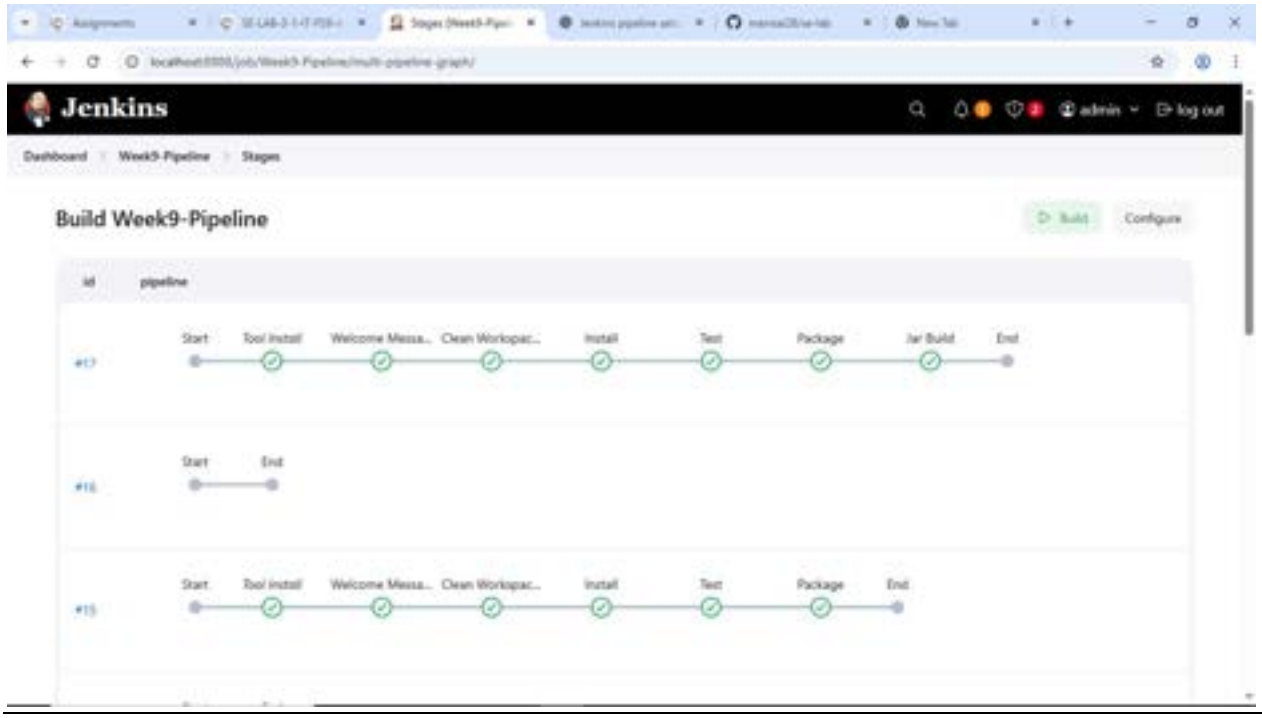

Dashboard
Week9-Pipeline
Configuration
Configure
General
Build Triggers
Advanced Project Options
Pipeline
Build Triggers
Build after other projects are built
Build periodically
Schedule
H 18 * * *
Build whenever a SNAPSHOT dependency is built
GitHub hook trigger for GITScm polling
Poll SCM
Schedule
H/5 * * * *
Save
Apply


Dashboard
Week9-Pipeline
Configuration
Configure
General
Build Triggers
Advanced Project Options
Pipeline
Pipeline
Definition
Pipeline script
Script
Use Groovy Sandbox
Pipeline Syntax
Save
Apply


Dashboard
Week9-Pipeline
#6
[INFO] --- surefire:3.2.5:test (default-test) @ E-Ticketing ---
[INFO] No tests to run.
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ E-Ticketing ---
[INFO] Packaging webapp
[INFO] Assembling webapp [E-Ticketing] in [C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\target\E-Ticketing-system]
[INFO] Processing war project
[INFO] Copying webapp resources [C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\src\main\webapp]
[INFO] Building war: C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\target\E-Ticketing-system.war
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 2.772 s
[INFO] Finished at: 2025-10-07T10:54:27+05:30
[INFO] --
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Jenkins
admin
log out
Dashboard
Week9-Pipeline
#6
Pipeline Console
Build #6
Rebuild
Overview
Configure
Success 14 min ago in 55 sec
Tool Install
Welcome Message
Clean Workspace & Clone Repo
Install
Test
Package
[INFO] No tests to run.
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ E-Ticketing ---
[INFO] Packaging webapp
[INFO] Assembling webapp [E-Ticketing] in [C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\target\E-Ticketing-system]
[INFO] Processing war project
[INFO] Copying webapp resources [C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\src\main\webapp]
[INFO] Building war: C:\ProgramData\Jenkins\.jenkins\workspace\Week9-Pipeline\se-lab\target\E-Ticketing-system.war
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 2.772 s
[INFO] Finished at: 2025-10-07T10:54:27+05:30
[INFO] --
Jenkins 2.489


Jenkins
admin
log out
Dashboard
Week9-Pipeline
Status
Changes
Build Now
Configure
Delete Pipeline
Full Stage View
Stages
Rename
Pipeline Syntax
GitHub Hook Log
Polling Log
Builds
Week9-Pipeline
Edit description
Week 9: Scripted Pipeline for Maven project with Git integration
Stage View
Declarative: Tool Install
Welcome Message
Clean Workspace & Clone Repo
Install
Test
Package
Average stage times:
(Full run time: ~29s)
165ms
328ms
10s
5s
3s
4s
7s
126ms
83ms
94ms



Apply and save

LAB ACTION PLAN FOR WEEK 10

Working with minikube and Nagios

Hands-on practice of creating, running and scaling pods in minikube.

Running Nginx server on specified port number by explaining the Nginx monitoring tool

Running Nagios server and Understanding the Monitoring tool using Docker.

AWS-free Trier account Creation steps

Upload the screenshots for the tasks

Kubernetes:

Kubernetes is a tool that automates how we run and manage applications inside the container.

Dockers will only run containers, if in any case the container fails/stopped/killed, the docker will not help us, here is where Kubernetes plays an important role, Kubernetes cluster will be responsible in creating a new container and managing various containers.

POD: In Kubernetes, a Pod is the smallest deployable unit that you can create and manage.

Minikube:

Minikube creates a VM on your local machine and deploys a simple Kubernetes cluster with one node. It's a lightweight implementation.Minikube *is* a version of Kubernetes.

Nagios:

Nagios is an **open-source IT infrastructure monitoring tool**. It monitors

Serrvers
Network devices
Applications and services

It **alerts admnistrators** when issues occur and notifies when they are resolved.

Step 1: Install Prerequisites

Before installing Minikube, ensure the following are installed:

Virtualization Support:
Verify virtualization is enabled:

Hypervisor:
Minikube supports multiple hypervisors (e.g., **Hyper-V**, **VirtualBox**, or **Docker** as a driver).
Install one of the following:
Hyper-V (pre-installed on Windows 10/11 Pro or Enterprise).
Docker Desktop (if you want to use Docker as the driver).

Step 2: Download Minikube

Open a PowerShell or Command Prompt with administrator privileges.
Download the latest Minikube executable using this command:

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-installer.exe
```

Install Minikube by running the installer:

```
.\minikube-installer.exe
```

Step 3: Add Minikube to PATH

If Minikube is not automatically added to your PATH during installation:

Open **System Properties → Environment Variables**.
Add the directory where Minikube is installed (e.g., `C:\Program Files\Minikube`) to your PATH variable.

Step 4: Start Minikube

Open a terminal (PowerShell or CMD). Do the following commands
Start Minikube with a specified driver (e.g., Hyper-V, Docker, or VirtualBox). For example:
Hyper-V:

```
minikube start --driver=hyperv
```

Docker:

```
minikube start --driver=docker
```

Verify Minikube is running:

```
minikube status
```

Step 5: Interact with Minikube

kubectl is a command-line tool used in Kubernetes to interact with and manage Kubernetes clusters.

Once Minikube is running:

Use `kubectl` to interact with the cluster.
Install `kubectl` if not already installed:
`minikube kubectl -- get pods -A`
Or download it separately from the official Kubernetes site.
Open the Minikube dashboard (optional):
`minikube dashboard`

Optional: Check Your Installation

Run the following to verify the installation:

Minikube version

```
kubectl version --client
```

Troubleshooting

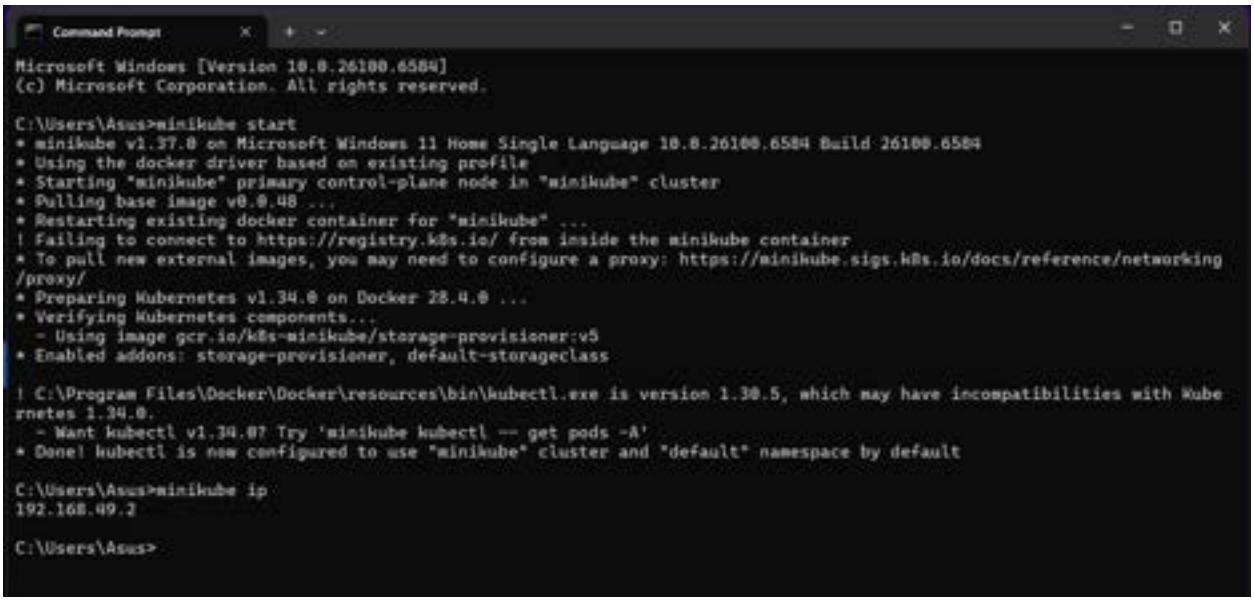
If Minikube fails to start:
Ensure your hypervisor (Hyper-V/Docker/VirtualBox) is installed and running.
Check the Minikube logs:
`minikube logs`
Updating Minikube:
`minikube update-check`
`minikube update`

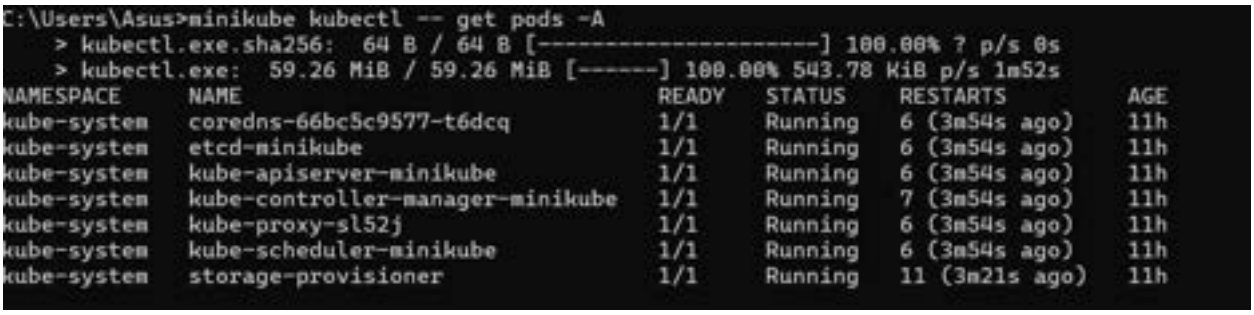
Minikube Automation Steps

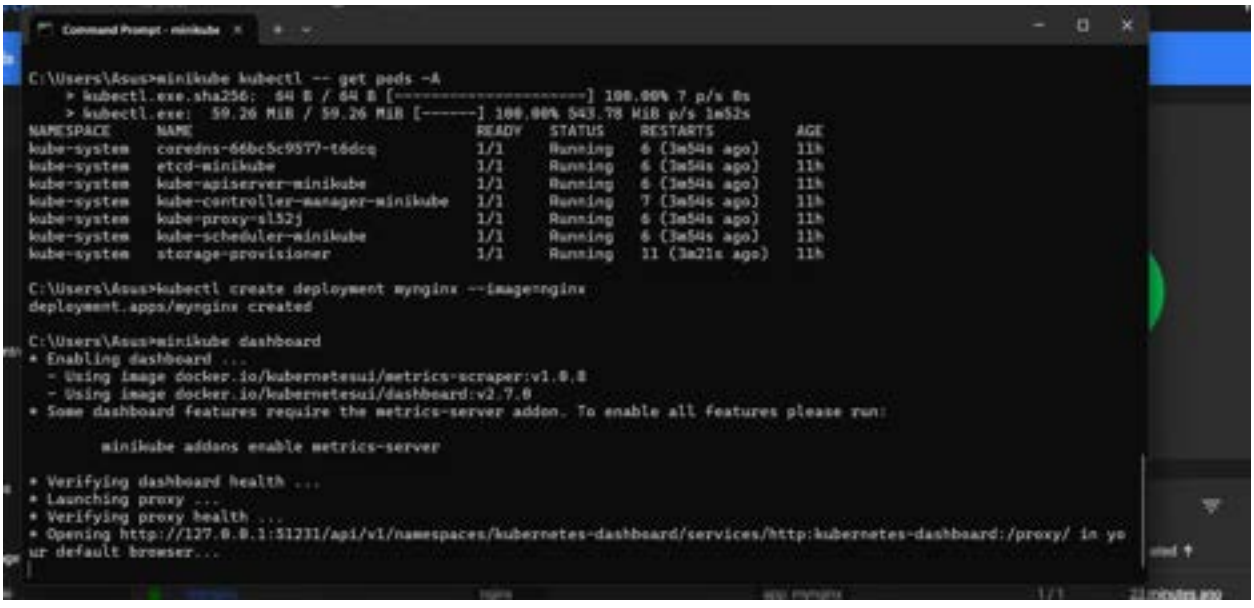
Step 1: Start Minikube Cluster

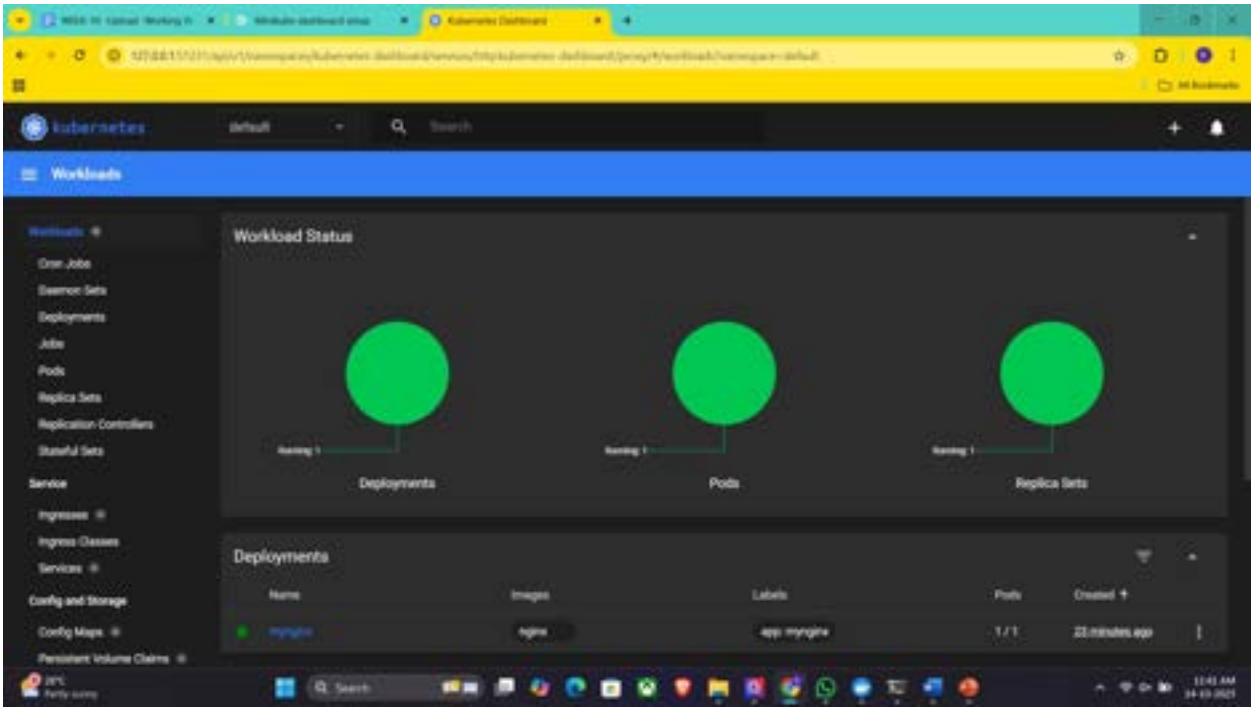
Open your terminal and run the command:

```
minikube start
```


Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Asus>minikube start
* minikube v1.37.0 on Microsoft Windows 11 Home Single Language 10.0.26100.6584 Build 26100.6584
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.48 ...
* Restarting existing docker container for "minikube" ...
! Failing to connect to https://registry.k8s.io/ from inside the minikube container
* To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
* Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.30.5, which may have incompatibilities with Kubernetes 1.34.0.
- Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
C:\Users\Asus>minikube ip
192.168.49.2
C:\Users\Asus>


C:\Users\Asus>minikube kubectl -- get pods -A
> kubectl.exe.sha256: 64 B / 64 B [----------] 100.00% ? p/s 0s
> kubectl.exe: 59.26 MiB / 59.26 MiB [------] 100.00% 543.78 KiB p/s 1m52s
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system coredns-66bc5c9577-t6dcq 1/1 Running 6 (3m54s ago) 11h
kube-system etcd-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system kube-apiserver-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system kube-controller-manager-minikube 1/1 Running 7 (3m54s ago) 11h
kube-system kube-proxy-sl52j 1/1 Running 6 (3m54s ago) 11h
kube-system kube-scheduler-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system storage-provisioner 1/1 Running 11 (3m21s ago) 11h


Command Prompt - minikube
C:\Users\Asus>minikube kubectl -- get pods -A
> kubectl.exe.sha256: 64 B / 64 B [----------] 100.00% ? p/s 0s
> kubectl.exe: 59.26 MiB / 59.26 MiB [------] 100.00% 543.78 KiB p/s 1m52s
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system coredns-66bc5c9577-t6dcq 1/1 Running 6 (3m54s ago) 11h
kube-system etcd-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system kube-apiserver-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system kube-controller-manager-minikube 1/1 Running 7 (3m54s ago) 11h
kube-system kube-proxy-sl52j 1/1 Running 6 (3m54s ago) 11h
kube-system kube-scheduler-minikube 1/1 Running 6 (3m54s ago) 11h
kube-system storage-provisioner 1/1 Running 11 (3m21s ago) 11h
C:\Users\Asus>kubectl create deployment mynginx --image=nginx
deployment.apps/mynginx created
C:\Users\Asus>minikube dashboard
* Enabling dashboard ...
- Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
- Using image docker.io/kubernetesui/dashboard:v2.7.0
* Some dashboard features require the metrics-server addon. To enable all features please run:
minikube addons enable metrics-server
* Verifying dashboard health ...
* Launching proxy ...
* Verifying proxy health ...
* Opening http://127.0.0.1:51231/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...



Step 2: Create and Manage Deployment

Create an application in Kubernetes:

Command:

```
kubectl create deployment mynginx --image=nginx
```

```
if already created then
```

```
kubectl set image deployment/myngnix nginx=nginx:latest
```

Verify the deployment using: Kubernetes responds by showing you a list that includes the names of your deployment groups

```
kubectl get deployments
```

```
C:\Users\Asus>kubectl get deployments
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
mynginx    1/1     1            1           32m

C:\Users\Asus>
```

Ensure `mynginx` appears in the output.

Check the following commands:

kubectl get pods

kubectl describe pods

Expose Deployment as a Service:

Command:

```
kubectl expose deployment mynginx --type=NodePort --port=80 --target-port=80
```

```
C:\Users\Asus>kubectl expose deployment mynginx --type=NodePort --port=80 --target-port=80
service/mynginx exposed

C:\Users\Asus>
```

Step 3: Scale the Deployment

Command:Scales the Nginx deployment to 4 replicas (pods).

```
kubectl scale deployment mynginx --replicas=4

kubectl get service myngnix
```

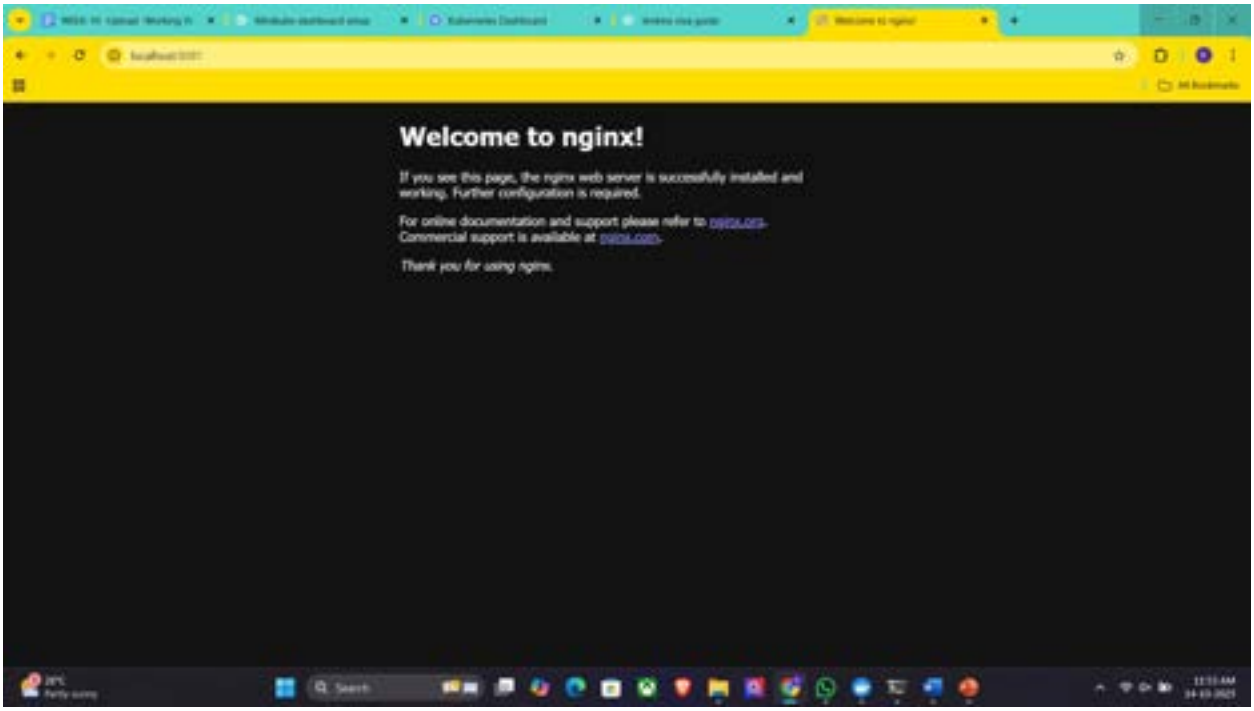
Step 4: Access the Nginx App

Using Port Forwarding:

Command:

```
kubectl port-forward svc/mynginx 8081:80
```

Access the app via http://localhost:8081.



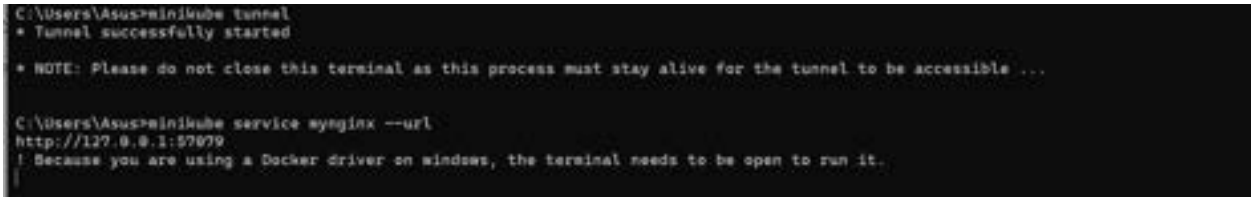
If Error, use this option, **Using Minikube Tunnel**:

Start the tunnel:

```
minikube tunnel
```

Retrieve the service URL:

```
minikube service mynginx --url
```

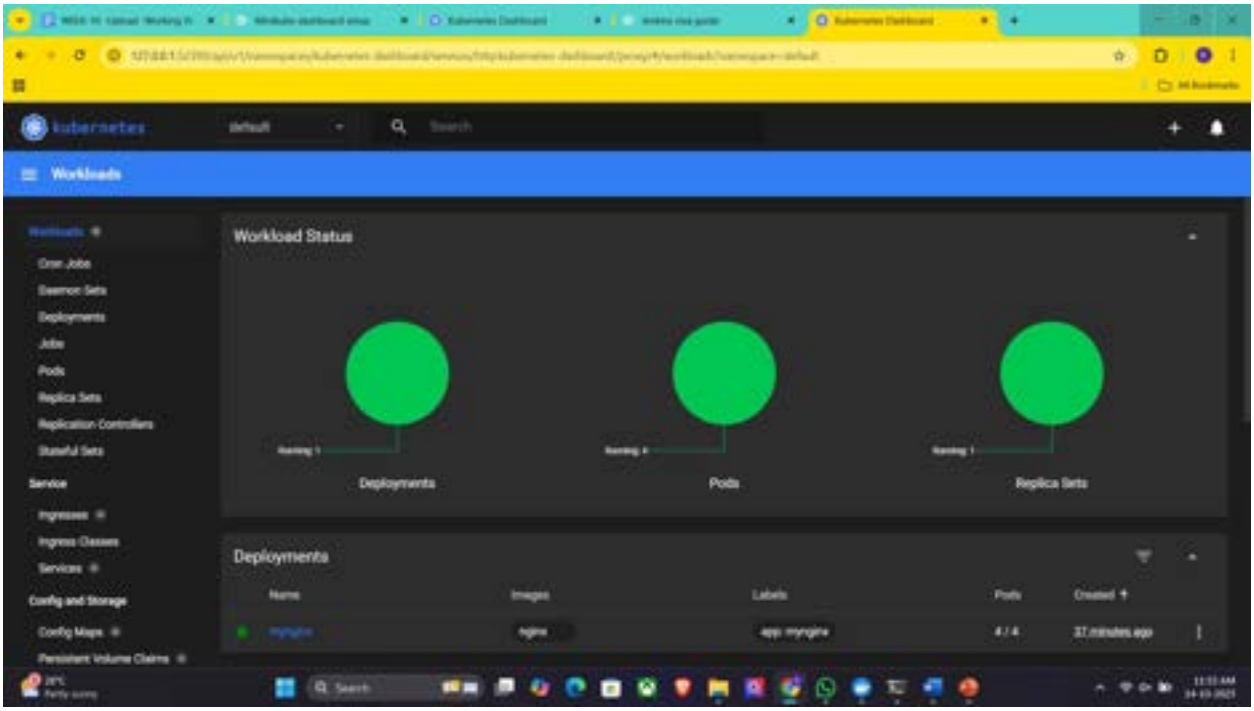


Open the provided URL in your browser.

Open the kubernets dashboard

Open the minikube dashboard

```
Minikube dashboard
```



Step 5: Stop and Clean Up

Stop Nginx Deployment:

Commands:

```
kubectl delete deployment mynginx
kubectl delete service mynginx
```

```
C:\Users\Asus>kubectl delete deployment mynginx
deployment.apps "mynginx" deleted

C:\Users\Asus>kubectl delete service mynginx
service "mynginx" deleted

C:\Users\Asus>
```

Stop Minikube (Optional):

Command:

```
minikube stop
```

Delete Minikube Cluster (Optional):

Command:`minikube delete`

Nagios Automation Steps

Step 1: Pull the Nagios Docker Image

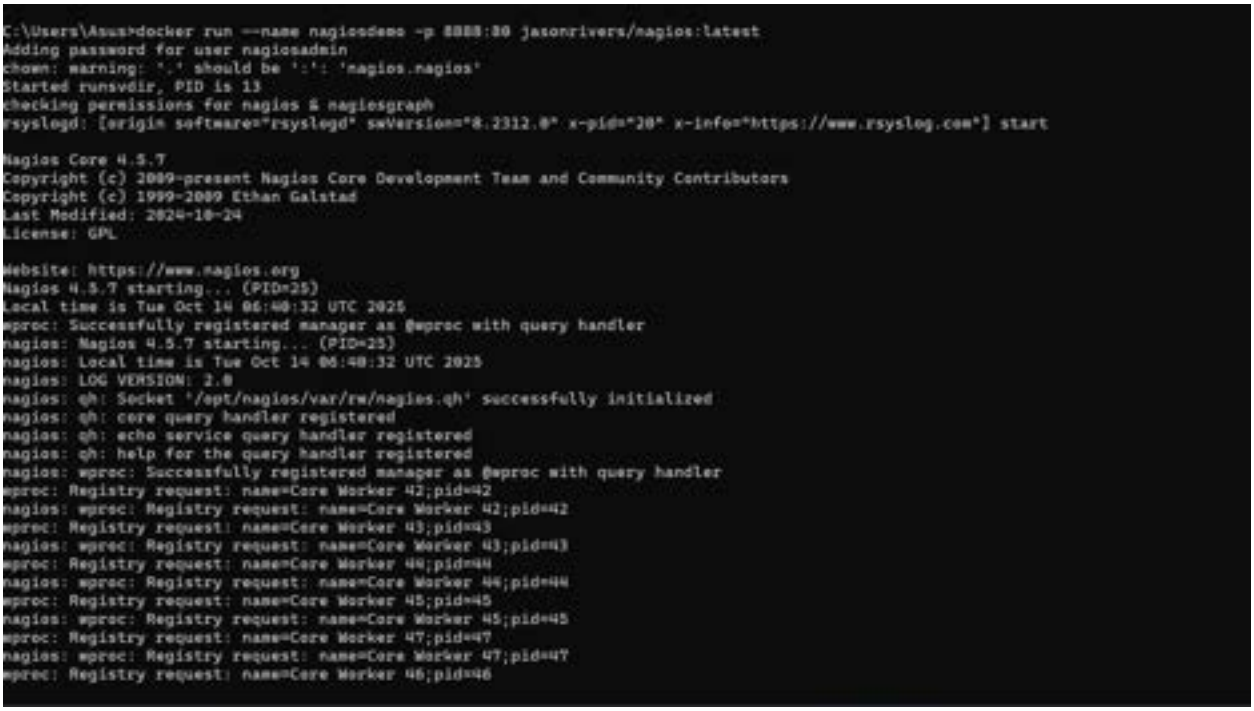
Open a terminal and run:

```
docker pull jasonrivers/nagios:latest
```

Step 2: Run Nagios

Command:

```
docker run --name nagiosdemo -p 8888:80 jasonrivers/nagios:latest
```



Step 3: Access Nagios Dashboard

Open your browser and navigate to:

```
http://localhost:8888
```

Login Credentials:

Username: `nagiosadmin`

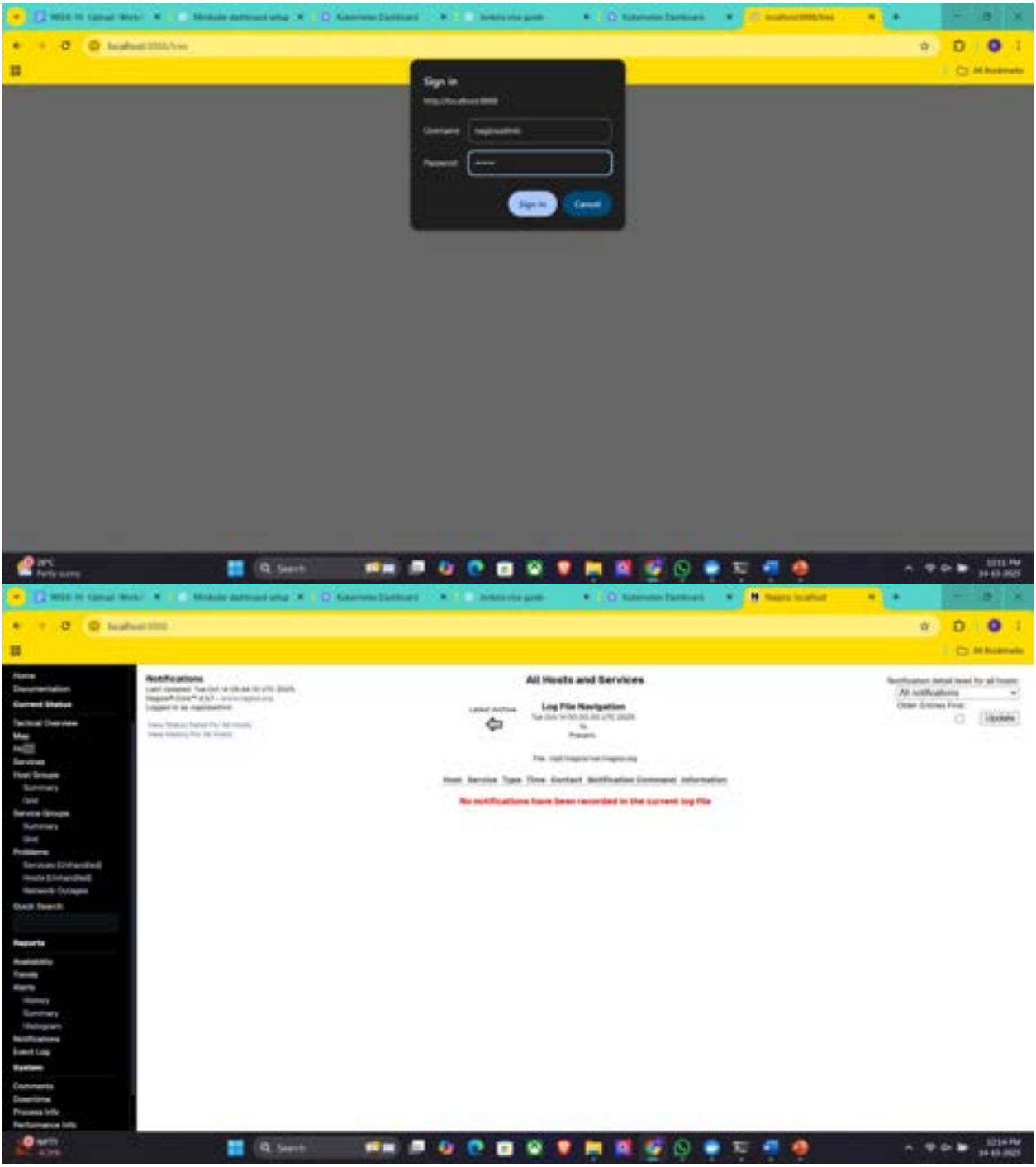
Password: `nagios`

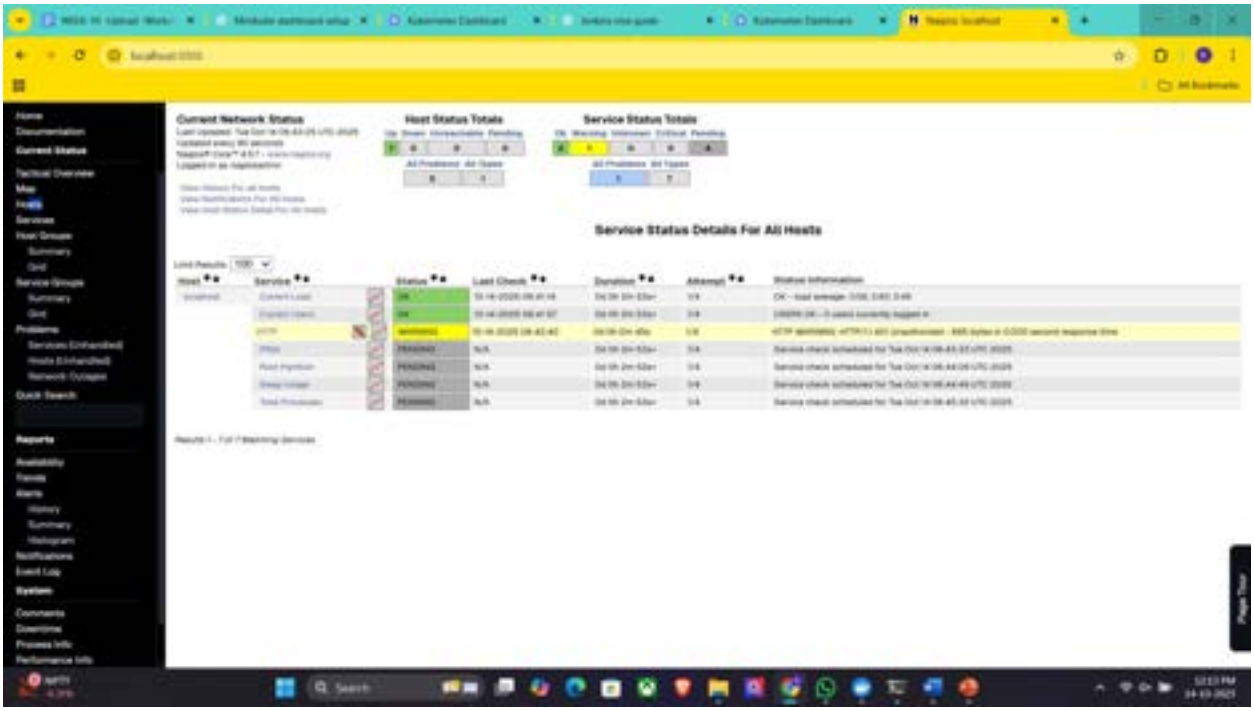
Once logged in, explore:

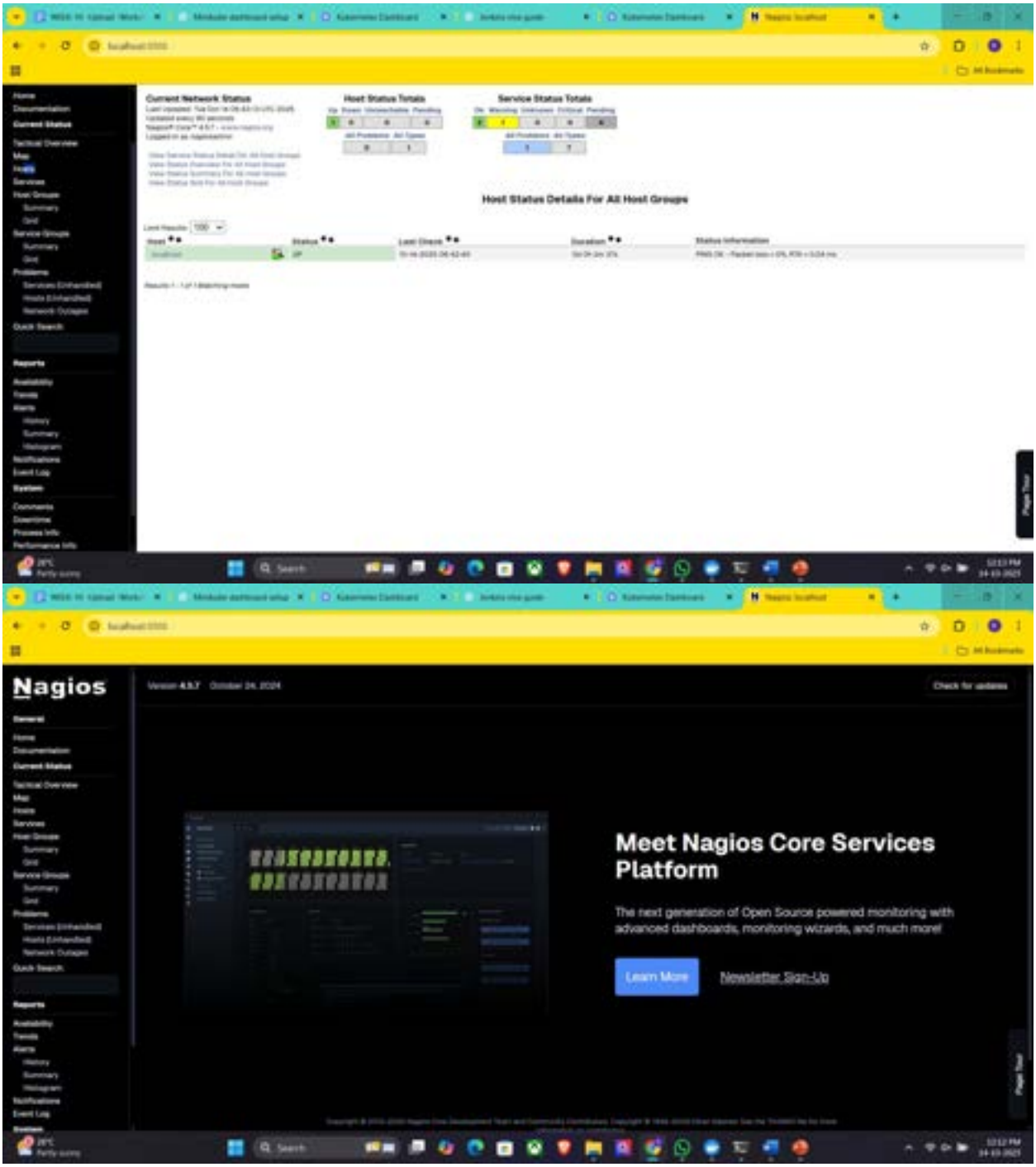
Hosts: View systems being monitored.

Services: Check tasks being monitored (e.g., CPU usage).

Alerts: Access recent notifications.


Sign in
Username
Password
Sign in
Cancel
All Hosts and Services
Log File Navigation
No notifications have been recorded in the current log file


Current Network Status
Host Status Totals
Service Status Totals
Service Status Details For All Hosts



Step 4: Monitoring Host Details

Navigate to the Host Information Page:

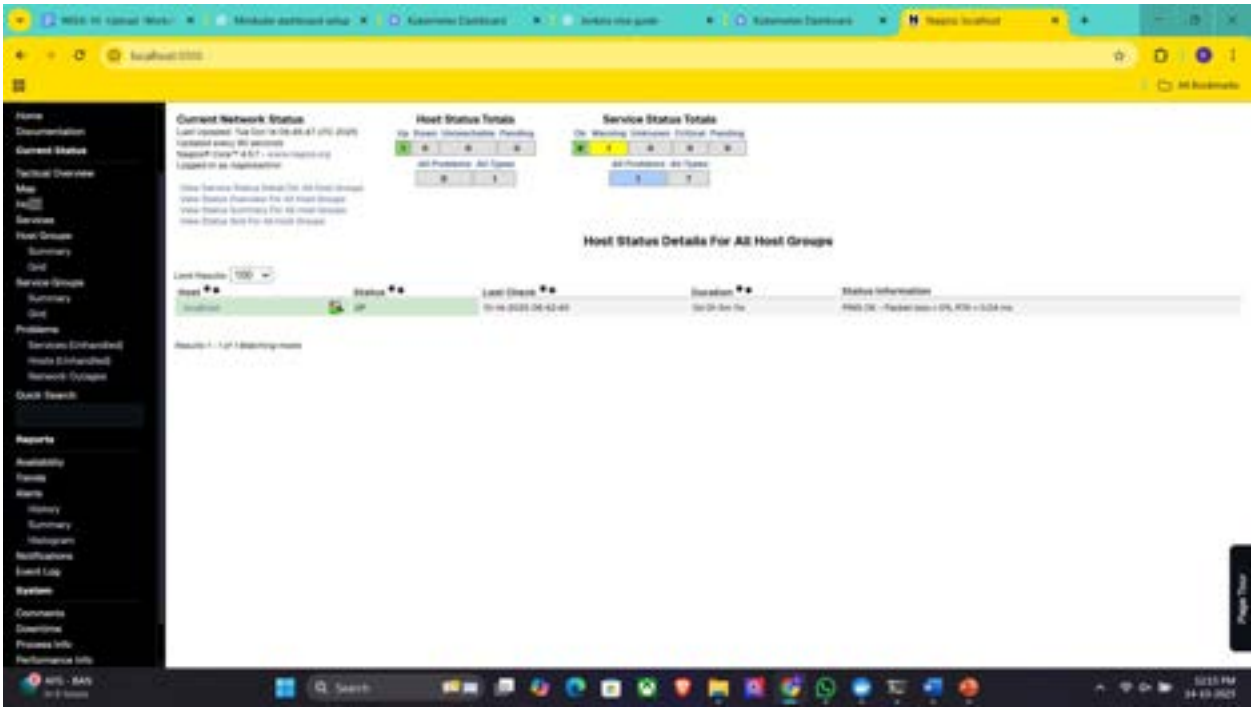
Select a host from the **Hosts** menu.

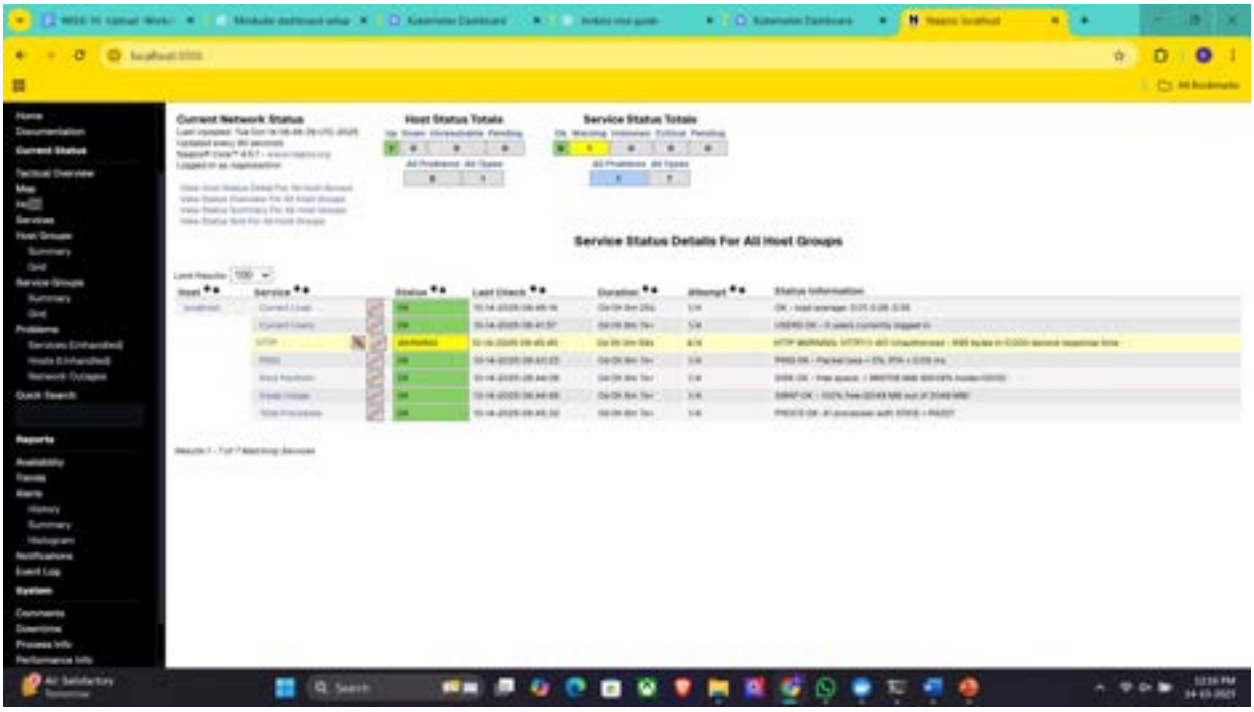
Key Details:

Host Status: Indicates if the system is UP or DOWN.

Metrics: View CPU usage, memory status, and network activity.

Actions: Reschedule checks, disable notifications, or schedule downtime.



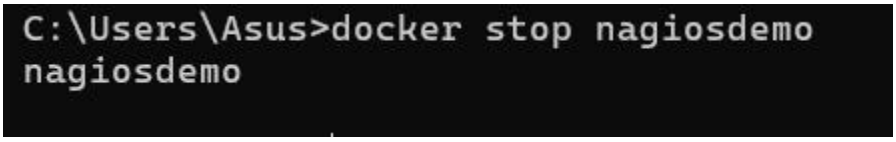


Step 5: Stop and Remove Nagios

Stop the Container:

Command:

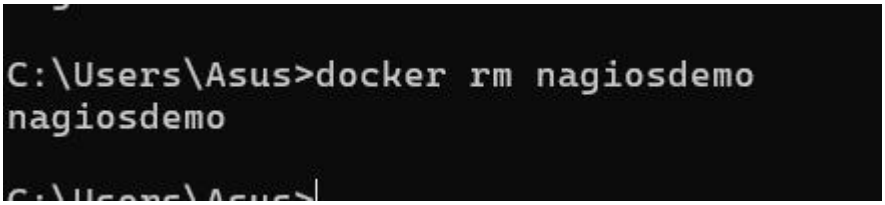
```
docker stop nagiosdemo
```



Delete the Container:

Command:

```
docker rm nagiosdemo
```



Remove the Image (Optional):

List images:

```
docker images
```

Delete the Nagios image:

```
docker rmijasonrivers/nagios:latest
```

Observe the docker containers in DockerHub, we can see the latest Nagios Installed running on port:8888

```
C:\Users\Asus>docker rmi jasonrivers/nagios:latest
Untagged: jasonrivers/nagios:latest
Untagged: jasonrivers/nagios@sha256:2a7c2b20d118baf92b47b69a3901e68dd7664617801b94e560bc4d6564d6ae54
Deleted: sha256:2ab73fb5d162ad0be4ef495fb62e17772120b32721a6c3eb9091fcc880d2989b
Deleted: sha256:631ab139f760d68457a4186da4a60f5e5f11e65aa3b26864b5df237ef75b1311
Deleted: sha256:b10e2a7f4de8bcf15a69232335422c9a736c4a4a93ca288592a2fb53a2de6f45
Deleted: sha256:3a5a34bf7e5ab76f1a3054c5854a9bc7227fd5a0dec73d4a4673456f69cec753
Deleted: sha256:0603355dfe22b531eff49627391c7a8bb944163a735d9142cfcf02529e3c8ffd
Deleted: sha256:2cdfed24d1997cb2951dc561b1c4fbc1947069e22801aab9103cf0f0c1e8c532
Deleted: sha256:5d2ae66d21d9102674574 8ae98d112256472f76f72deaa904a368b9dd41ca987
Deleted: sha256:b9759ebfa6418db67d02fd861af913fd571bd06c6d3ccddfe760d637e49f15f2
Deleted: sha256:62b02d2bded253673e8e75637dda61e85ad0fbe2e2eaa48bcfda314197add5f4
Deleted: sha256:c8bf237c41fd124329567e29514d381ae822f6d202565e013df75bb47811c94d
Deleted: sha256:5e918b338fe317b5f88044bfc83c21c5bca1f8570092859d4a8c05a8e2e036ce
Deleted: sha256:24ce75f6abff9acee33a572c2adb3cccf1a57d995ba1eaad9bf68b9a50d5bfab
Deleted: sha256:bc291db0b02d187622e2bb6f1c85dcb5baa3a37ce557e5357a628b3befbb0df3
Deleted: sha256:a0d43b5b4f90893c91713325e85539667219a1813fc95804acf0efa1925b5120
Deleted: sha256:b985a1f962d4e9e6c23c7d823a68861a4e69c7dae1a4fd01f2eb425ba130d742
Deleted: sha256:eefaf4321bcf8520d56ae8d458508da8d6c62d6b7fd484ec08c99678254e84dd
Deleted: sha256:320894a8eec97786c5249f30c572390355cd72953ee7dd70a3f6b11be9c2a047
Deleted: sha256:7c52f6b68cf3d137336b97a0fcd9beb2a8512b6c2512c8f535e2a81aa767cab1
Deleted: sha256:88100fb20cb3c5b2c708de42f7d16c2baf0992610f1fb4d435791d52ca008eb5
Deleted: sha256:067cbc3a33f52f4afbac2ea3d35fbbe2270ec8142a04d7e9837fc10f627a84d8
Deleted: sha256:4c463e1f44c076c2a117691f426399fc6236b7d8c92e375058e8045d12525e0f
Deleted: sha256:cd0357977468c3014df276354774e1d424ed389664ab653e4652940ac1d55a6d
Deleted: sha256:1277c94ea1944b9e52ab1fbe34af10214509106536e43fe3fdb802bfd1c1aa02
Deleted: sha256:57ac56afc1593da088f8f3dbfbaa11884f38aeef15b2db2e92c98380f44574cf
Deleted: sha256:202680f0cbb41cf353db89a064dd727ecc16e12fecde216b99b22ec06f3a34f9
Deleted: sha256:5c005cf4164068516d8a2e7b152775098980438e5b934794ca055a0f22c37e35c
Deleted: sha256:0d9515684b30166985cc21968825aebb9a8cd7811c00175eb395f97806881f91
Deleted: sha256:c784deeb88028b6eb1574c623e477d1075cca6b6f68db14747c5d30b30a8c077
Deleted: sha256:757fa2e4f084816630e3943408ca67e261ff5e0550da6ead9034b1d28b4d1cc5
Deleted: sha256:f77ac6741c76fbe0772ea95bf85c6e2a5e33271aa9dd226efae3b3ebc1f64117
Deleted: sha256:3f93222df80f5e6495196760d065ada12aa3c476894 7558ac1ad167d3b6683e4
Deleted: sha256:a46a5fb872b554648d9d0262f302b2c1ded46eeb1ef4dc727ecc5274605937af
```

Steps for AWS-free trier account creation

open https://aws.amazon.com

click on create AWS account

Provide email id and name details for AWS account creation then a screen will appear as below

Enter verification code received in your mail

Provide your contact details and agree Terms and conditions, then click on create

Provide billing details, click on verify and continue

Amazon chagres 2 rs and it will be credit back into your account once verification is over

Once payment success a screen appear as below, provide your working contact number after selecting country, and click on send SMS after entering the details

Provide verification code received in your mobile

Now select basic plan and click on complete sign up

Now a screen appear as follows then click on AWS management console

You will navigating to the aws page ..where you need to Select role as Academic/Researcher and interest as DevOps and click on submit

Sign into AWS console as Root user

A console screen appears aws will appear

Conclusion: In this week we learnt how to deploy pods in minikube and how to monitor our local system using Nagios tool and will know how to create the aws free trier account.

LAB ACTION PLAN FOR WEEK 11

Jenkins-CI/CD

1. CI-Continous Integration using Webhooks .
2. Sending E-mail Notification on Build Failure or success
3. Upload the screenshots for the tasks

Lab

Setting Up Jenkins CI------using GitHub Webhook with Jenkins

Step 1: Configure Webhook in GitHub

1. Go to your GitHub repository.
2. Navigate to Settings → **Webhooks.**
3. Click "**Add webhook**".
4. In the Payload URL field:
    - Enter the Jenkins webhook URL in the format:
      http://<**jenkins-server-url**>/github-webhook/

Note: If Jenkins is running on localhost, GitHub cannot access it directly.
Use ngrok to expose your local Jenkins to the internet:

- ngrok.exe http <**Jenkins local host:8080**>
    - Use the generated ngrok URL, e.g.:
    - http://abc123.ngrok.io/**github-webhook**/

5. Set Content type to:
   application/json
6. Under "Which events would you like to trigger this webhook?", select:
    - Just the push event
7. Click "Add webhook" to save.

Step 2: Configure Jenkins to Accept GitHub Webhooks

1. Open Jenkins Dashboard.
2. Select the job (freestyle or pipeline) you've already created.
3. Click Configure.
4. Scroll down to the Build Triggers section.
5. Check the box: ✓GitHub hook trigger for GITScm polling
6. Click Save**.**

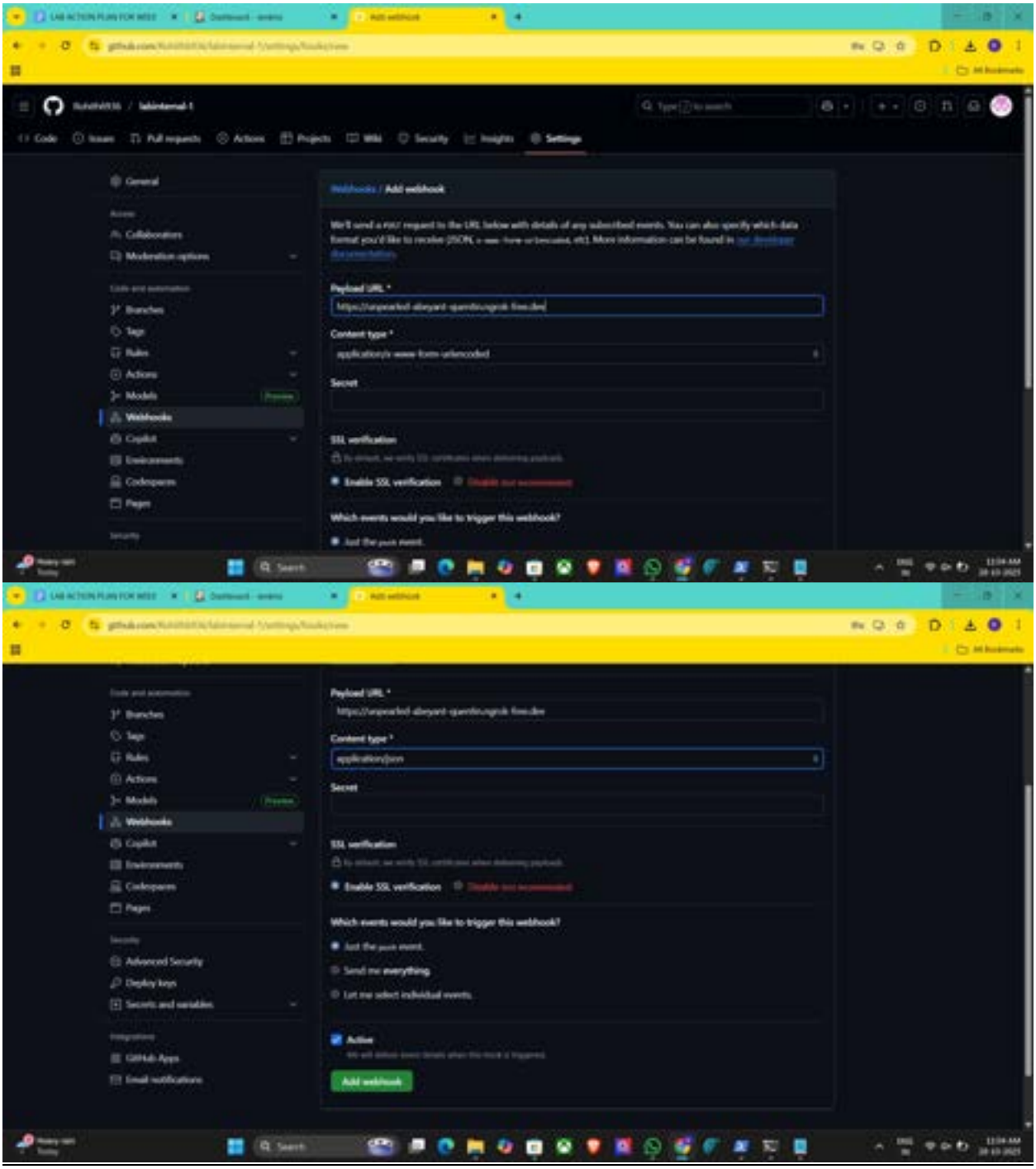
Step 3: Test the Setup

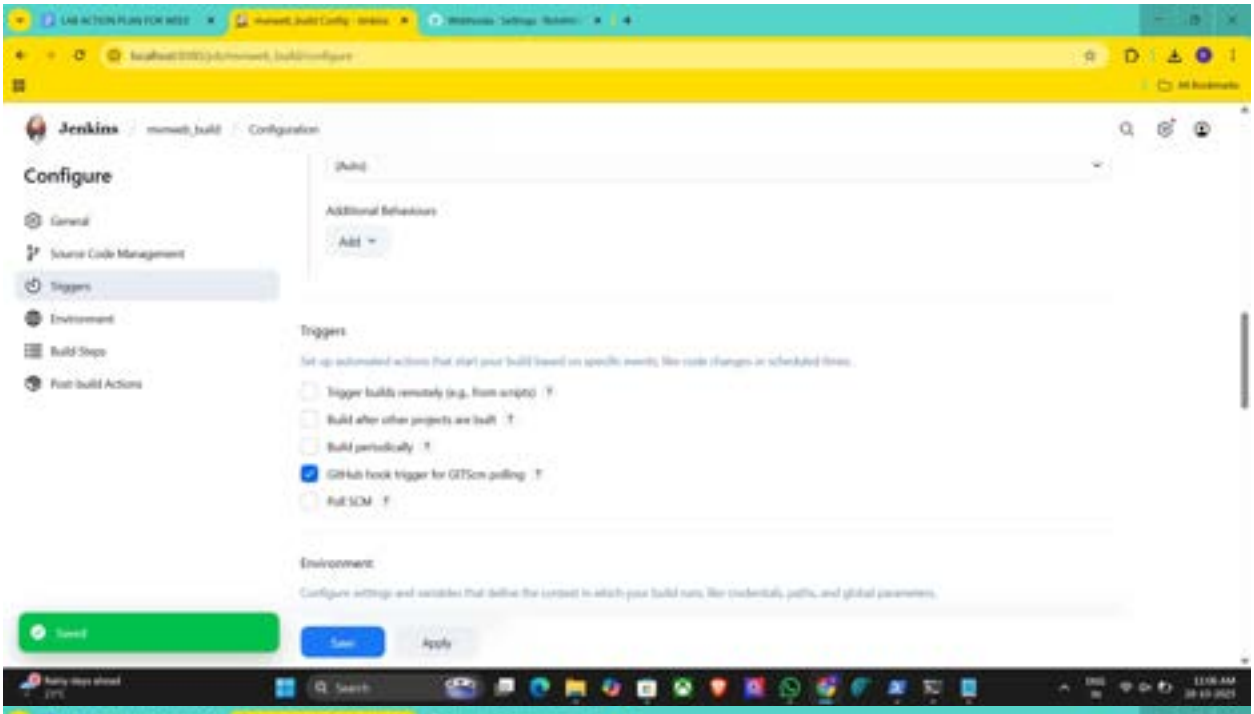
1. Make any code update in your local repo and push it to GitHub.
2. Once pushed, GitHub will trigger the webhook.
3. Jenkins will automatically detect the change and start the build pipeline**.**

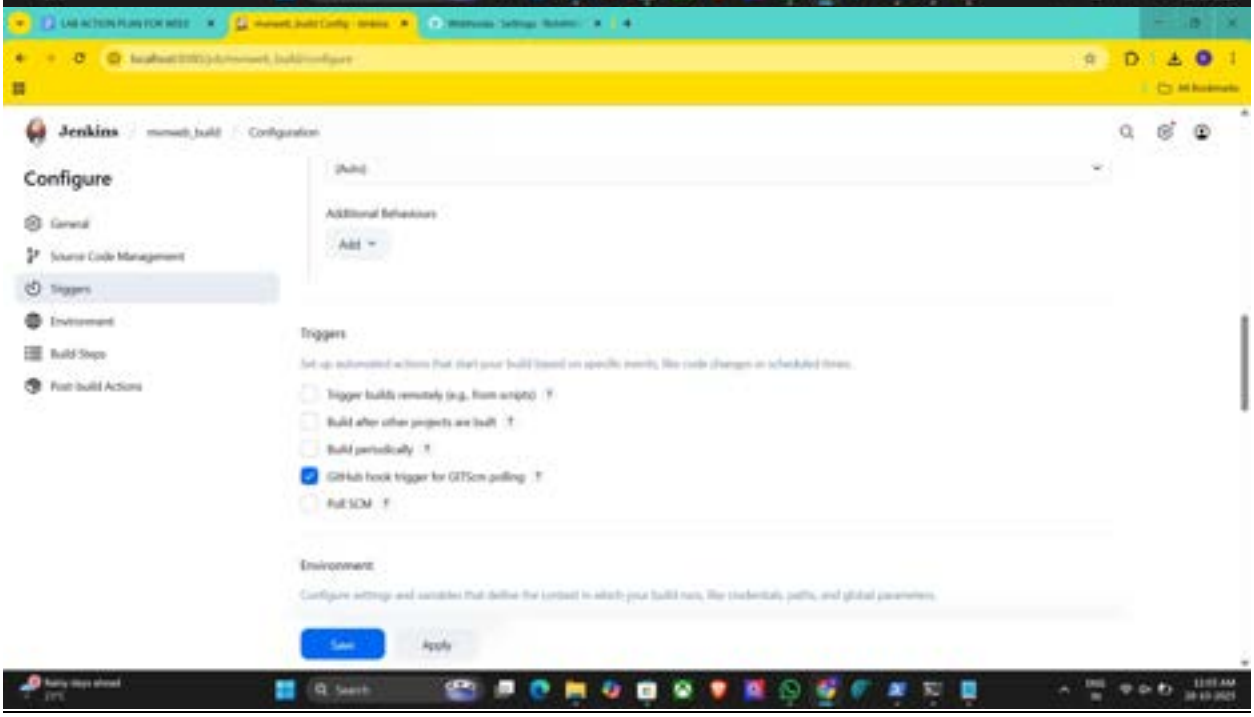
---

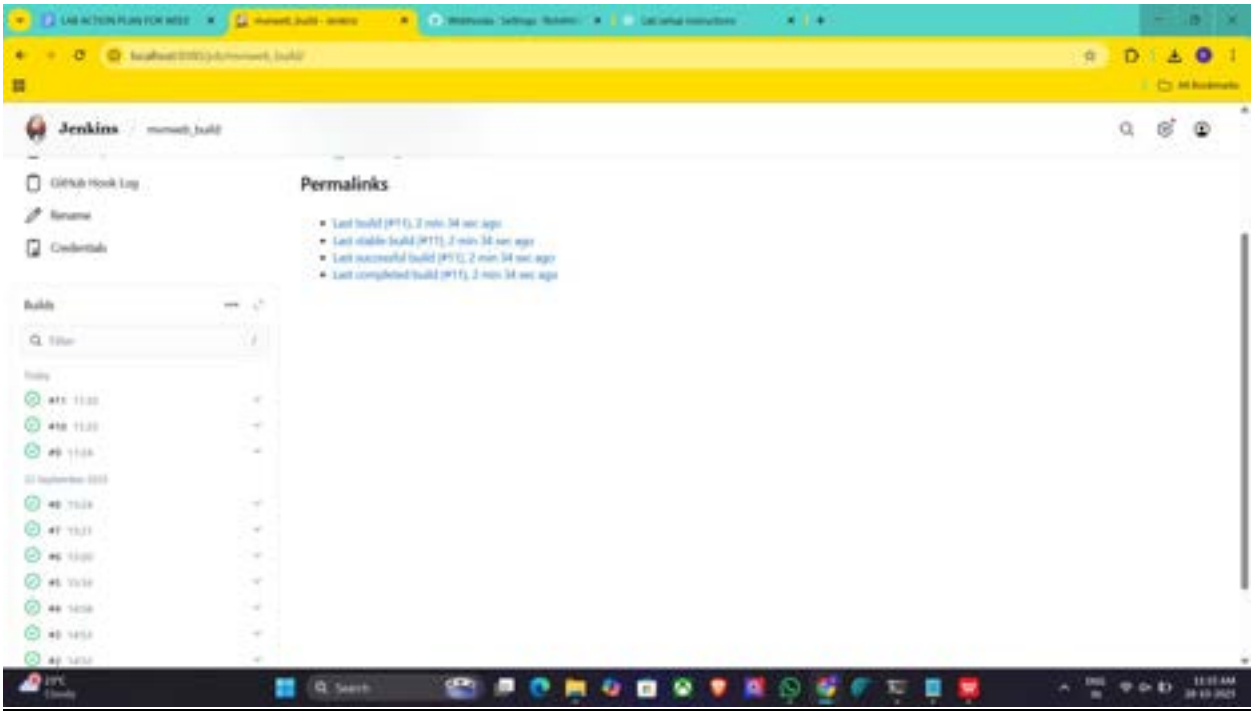
outcome

- You’ve successfully connected GitHub and Jenkins using webhooks.
- Every time you push code to GitHub, Jenkins will automatically start building your project without manual intervention.




Jenkins
mvnweb_build
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Additional Behaviours
Add
Triggers
Trigger builds remotely (e.g., from scripts)
Build after other projects are built
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Environment
Saved
Save
Apply


Jenkins
mvnweb_build
Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Additional Behaviours
Add
Triggers
Trigger builds remotely (e.g., from scripts)
Build after other projects are built
Build periodically
GitHub hook trigger for GITScm polling
Poll SCM
Environment
Save
Apply



Set-uping the ngrok

How to Install and Use ngrok

Step 1. Download ngrok
https://ngrok.com/download
Download and extract it for your OS (Windows, macOS, or Linux).

Step 2. Connect Your ngrok Account (optional but useful)
After you sign up (free), ngrok gives you an auth token.
CREATE AUTHENTICATOR [https://dashboard.ngrok.com/get-started/your-authtoken]
Run this command (replace <your_token> with yours):
ngrok config add-authtoken <your_token>
This ensures stable sessions and more control.

Step 3. Start a Tunnel for Jenkins

Assuming Jenkins runs locally on port 8085:
ngrok http 8085
You'll see output like:

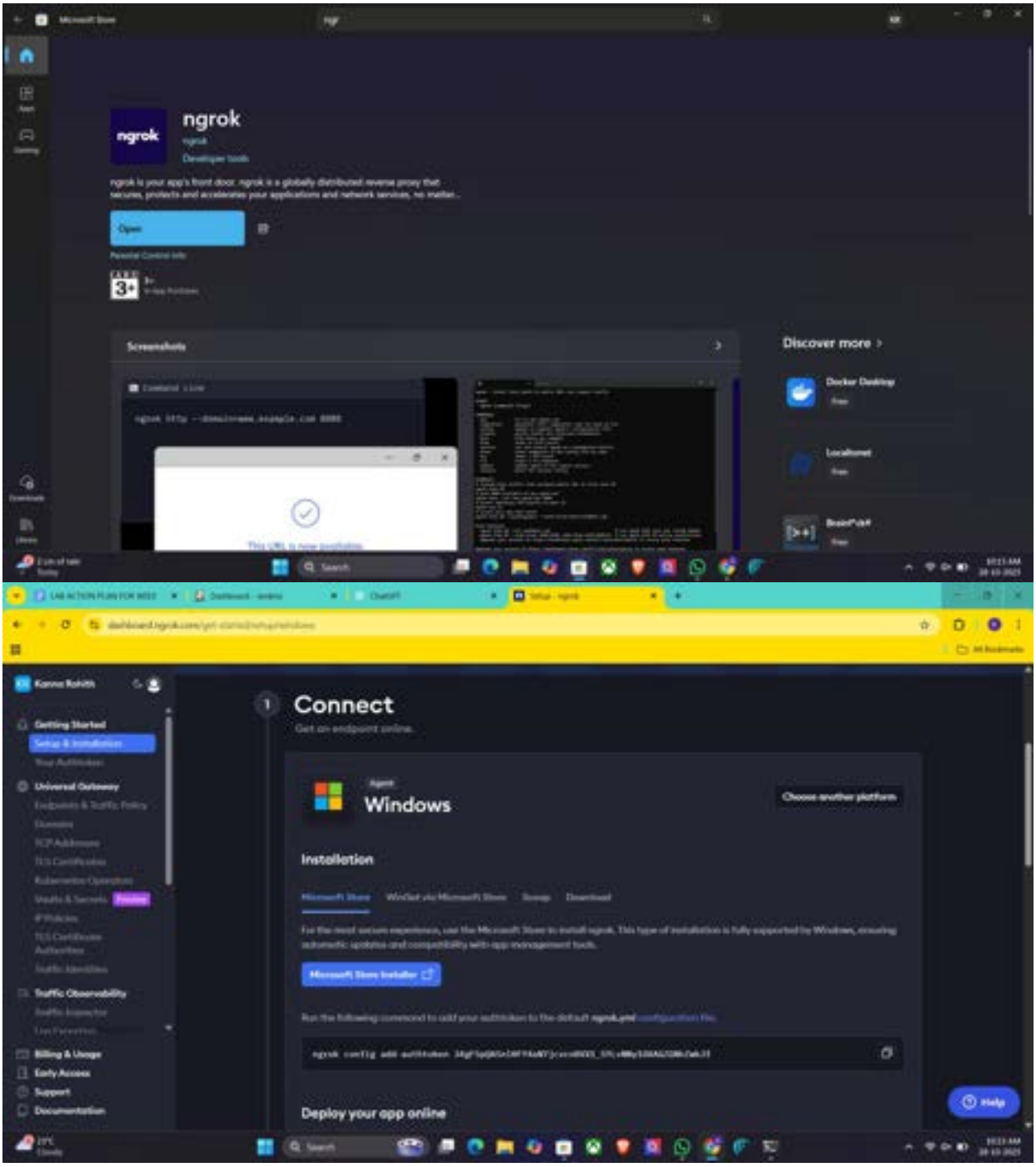
Session Status online
Forwarding https://1234abcd.ngrok.io -> http://localhost:8080
Copy the HTTPS URL (https://1234abcd.ngrok.io) — this is your public Jenkins URL for webhooks.
Step 4. Use it in GitHub Webhook

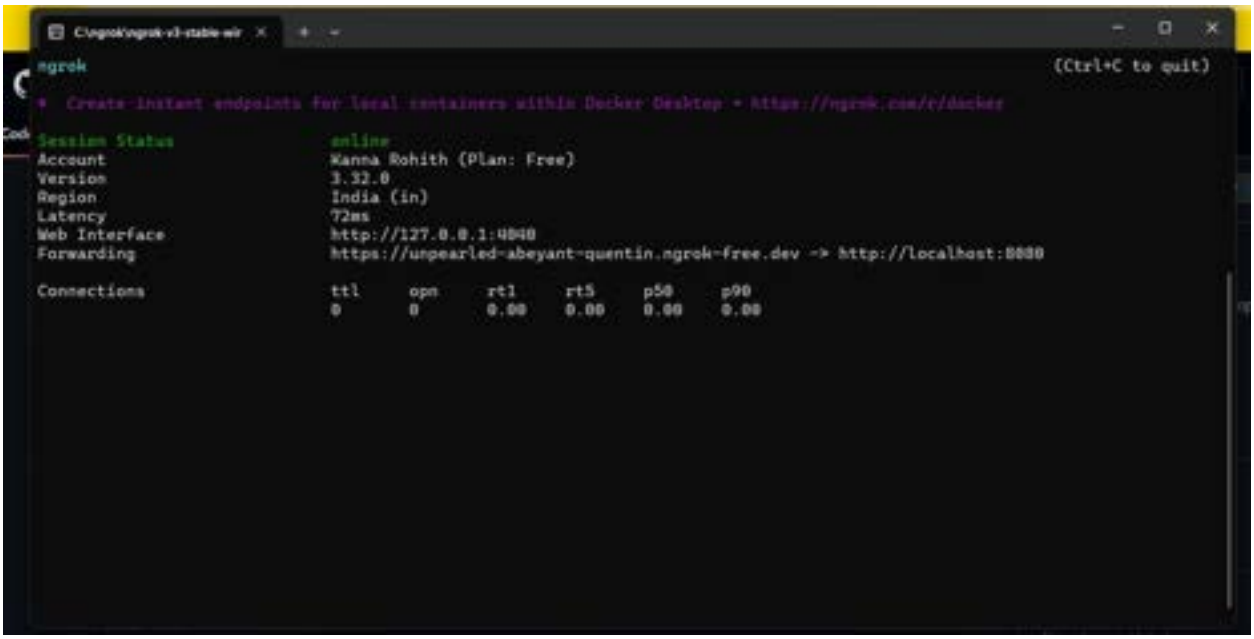
In your GitHub repo → Settings → Webhooks:

- **Payload URL:** ***[paste the url generated by ngrok]***

https://1234abcd.ngrok.io/github-webhook/ [please include this – remaining all default]

Now, whenever you push code, GitHub sends an event to that URL, which ngrok forwards to your local Jenkins.





Setting Up Jenkins Email Notification Setup (Using Gmail with App Password)

Creation of app password

1. Gmail: Enable App Password (for 2-Step Verification)

i. Go to: https://myaccount.google.com

ii. Enable 2-Step Verification

- Navigate to:
  - Security → 2-Step Verification
  - Turn it **ON**
  - Complete the OTP verification process (via phone/email)

iii. Generate App Password for Jenkins

- Go to:
  - Security → App passwords
- Select:
  - **App**: Other (Custom name)
  - **Name**: Jenkins-Demo
- Click **Generate**
- Copy the **16-digit app password**
  - Save it in a secure location (e.g., Notepad)

2. Jenkins Plugin Installation

i. Open Jenkins Dashboard

ii. Navigate to:

- Manage Jenkins → Manage Plugins

iii. Install Plugin:

- Search for and install:
  - Email Extension Plugin

---

3. Configure Jenkins Global Email Settings

i. Go to:

- Manage Jenkins → Configure System

A. E-mail Notification Section

Field	Value
SMTP Server	smtp.gmail.com
Use SMTP Auth	✓ Enabled
User Name	Your Gmail ID (e.g., archanareddykmit@gmail.com)
Password	Paste the 16-digit App Password
Use SSL	✓ Enabled
SMTP Port	465
Reply-To Address	Your Gmail ID (same as above)

➤ **Test Configuration**

- Click: Test configuration by sending test e-mail
- Provide a valid email address to receive a test mail
- ✓ Should receive email from Jenkins

B. Extended E-mail Notification Section

Field	Value
SMTP Server	smtp.gmail.com
SMTP Port	465
Use SSL	✓ Enabled
Credentials	Add Gmail ID and App Password as Jenkins credentials
Default Content Type	text/html or leave default
Default Recipients	Leave empty or provide default emails
Triggers	Select as per needs (e.g., Failure)

4. Configure Email Notifications for a Jenkins Job

i. Go to:

- Jenkins → Select a Job → Configure

---

ii. In the Post-build Actions section:

- Click: Add post-build action → **Editable Email Notification**

A. Fill in the fields:

Field	**Value**
Project Recipient List	Add recipient email addresses (comma-separated)
Content Type	Default (text/plain) or text/html
Triggers	Select events (e.g., Failure, Success, etc.)
Attachments	(Optional) Add logs, reports, etc.

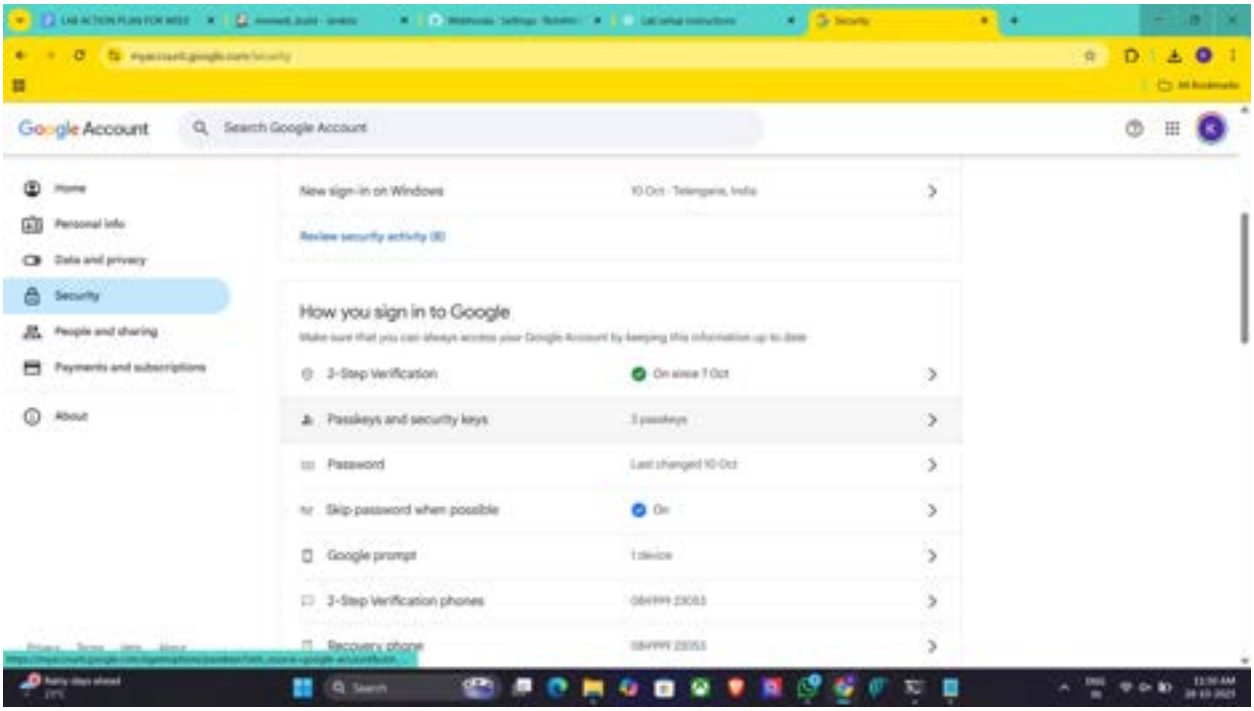
---

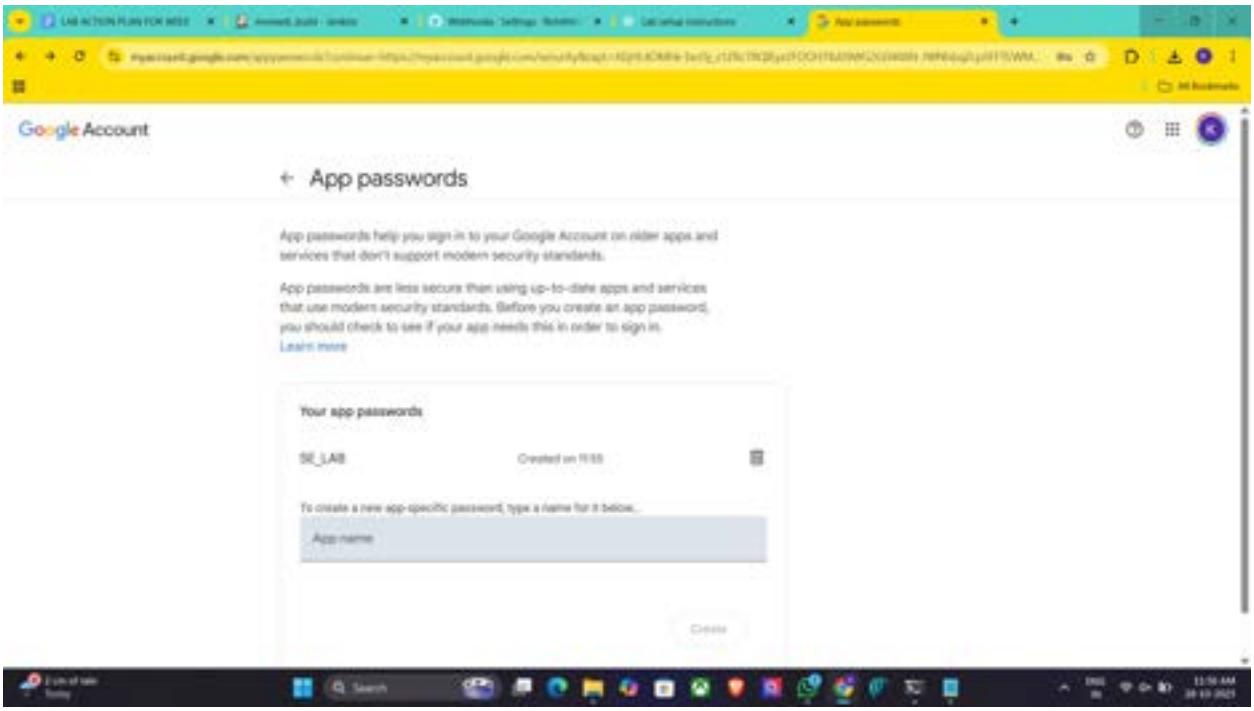
iii. Click Save

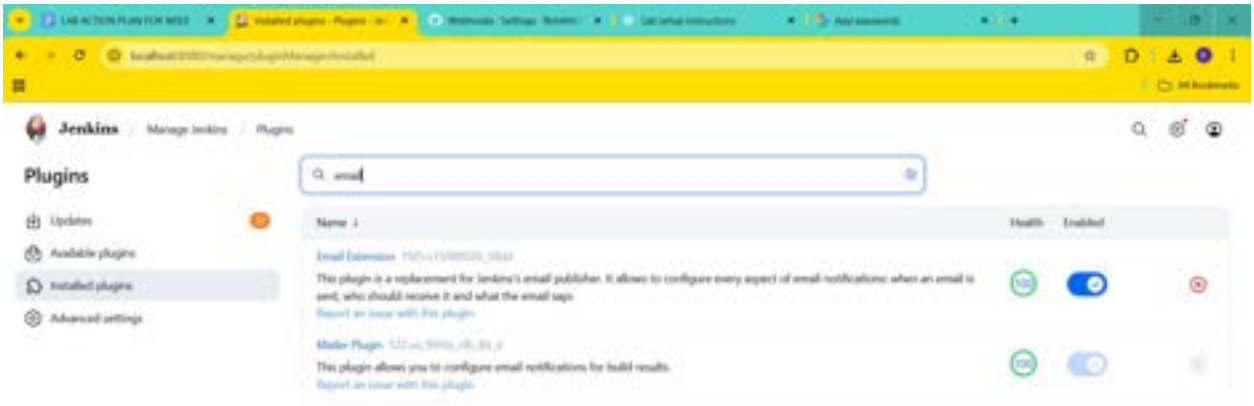
Now your Jenkins job is set up to send email notifications based on the build status!

- ---

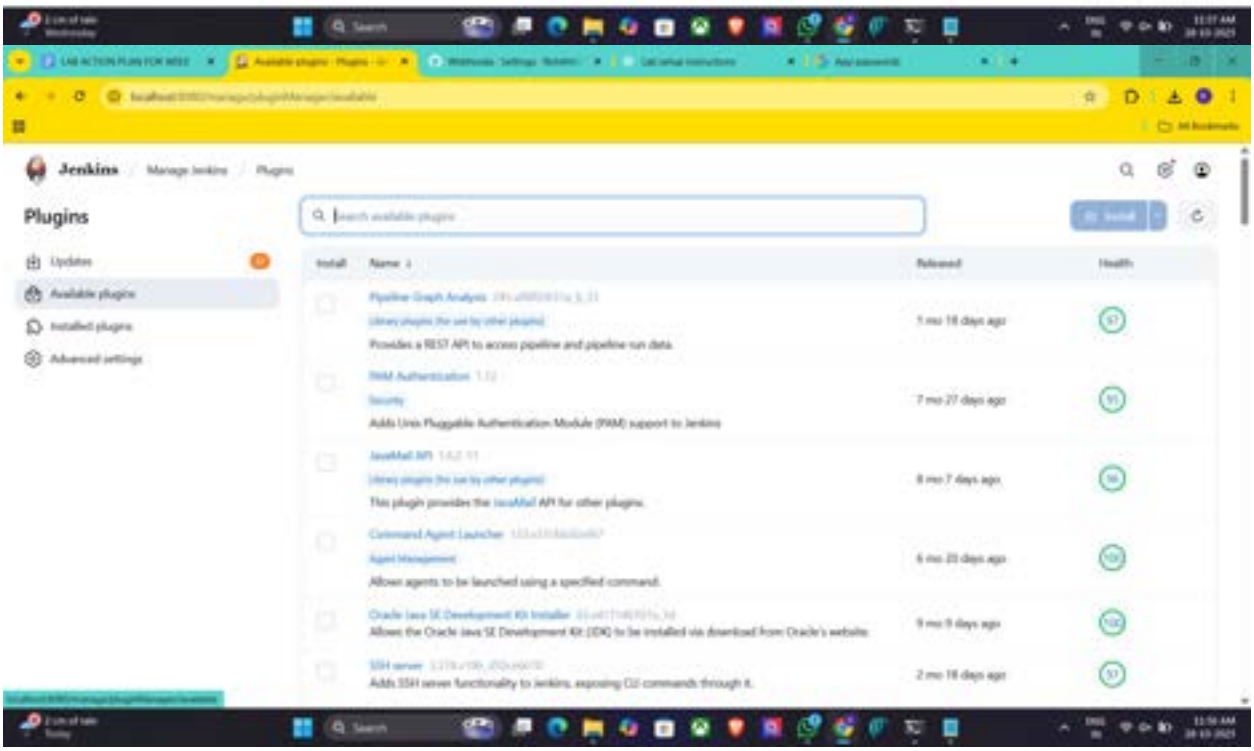
Takeaway :
Students learned how to integrate Jenkins with GitHub using webhooks to automate build triggers and configure email notifications to monitor build success or failure effectively.

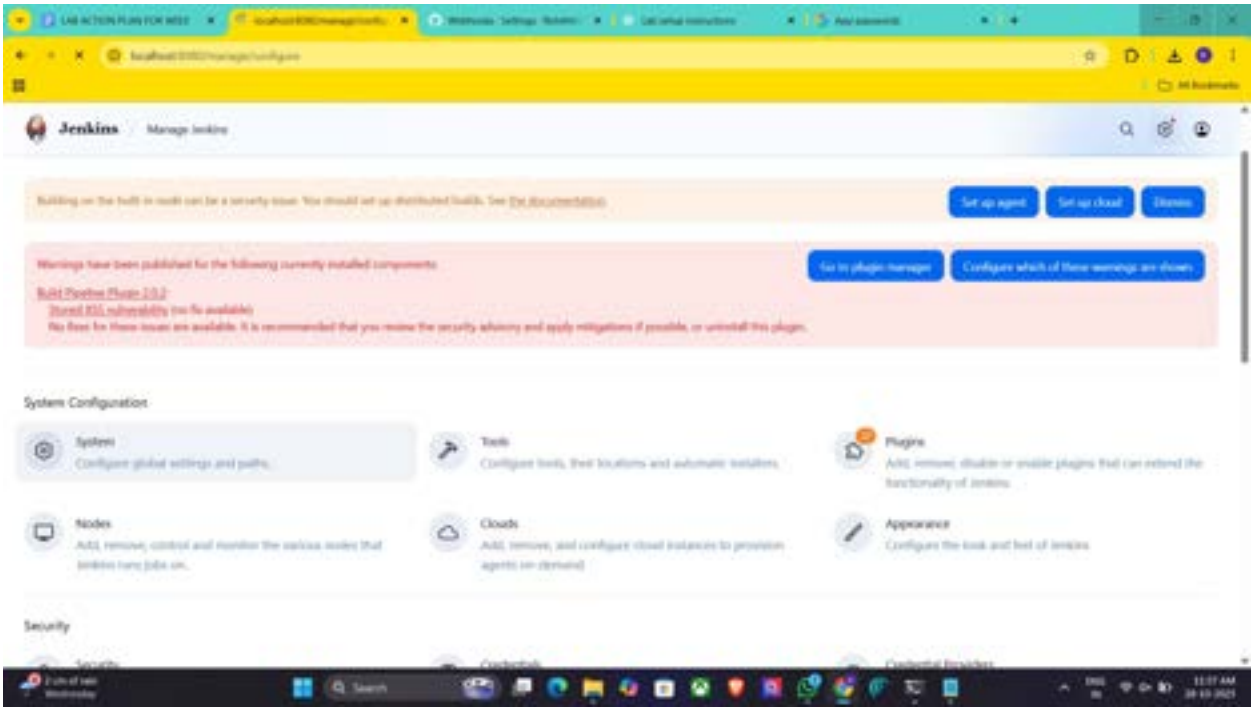

Google Account
Search Google Account
Home
Personal info
Data and privacy
Security
People and sharing
Payments and subscriptions
About
New sign-in on Windows
10 Oct · Telangana, India
Review security activity (8)
How you sign in to Google
Make sure that you can always access your Google Account by keeping this information up to date
2-Step Verification
On since 7 Oct
Passkeys and security keys
3 passkeys
Password
Last changed 10 Oct
Skip password when possible
On
Google prompt
1 device
2-Step Verification phones
Recovery phone

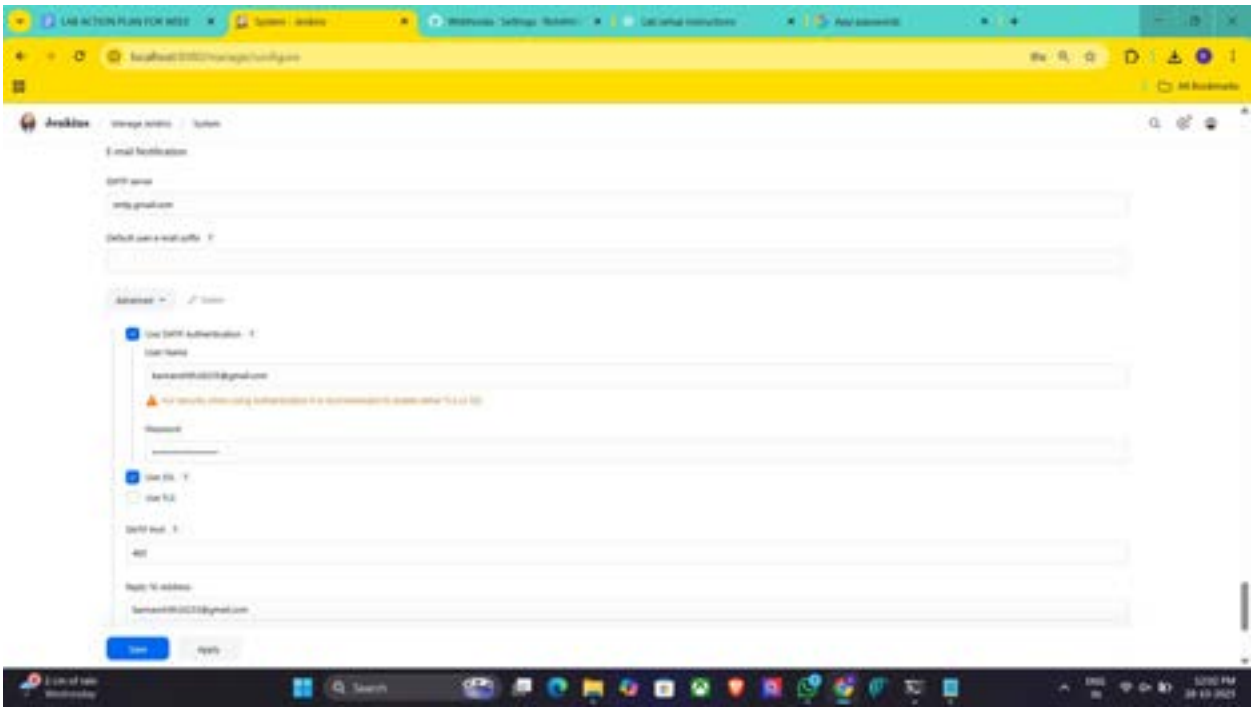

Google Account
App passwords
App passwords help you sign in to your Google Account on older apps and services that don't support modern security standards.
App passwords are less secure than using up-to-date apps and services that use modern security standards. Before you create an app password, you should check to see if your app needs this in order to sign in.
Learn more
Your app passwords
SE_LAB
To create a new app-specific password, type a name for it below...
App name
Create

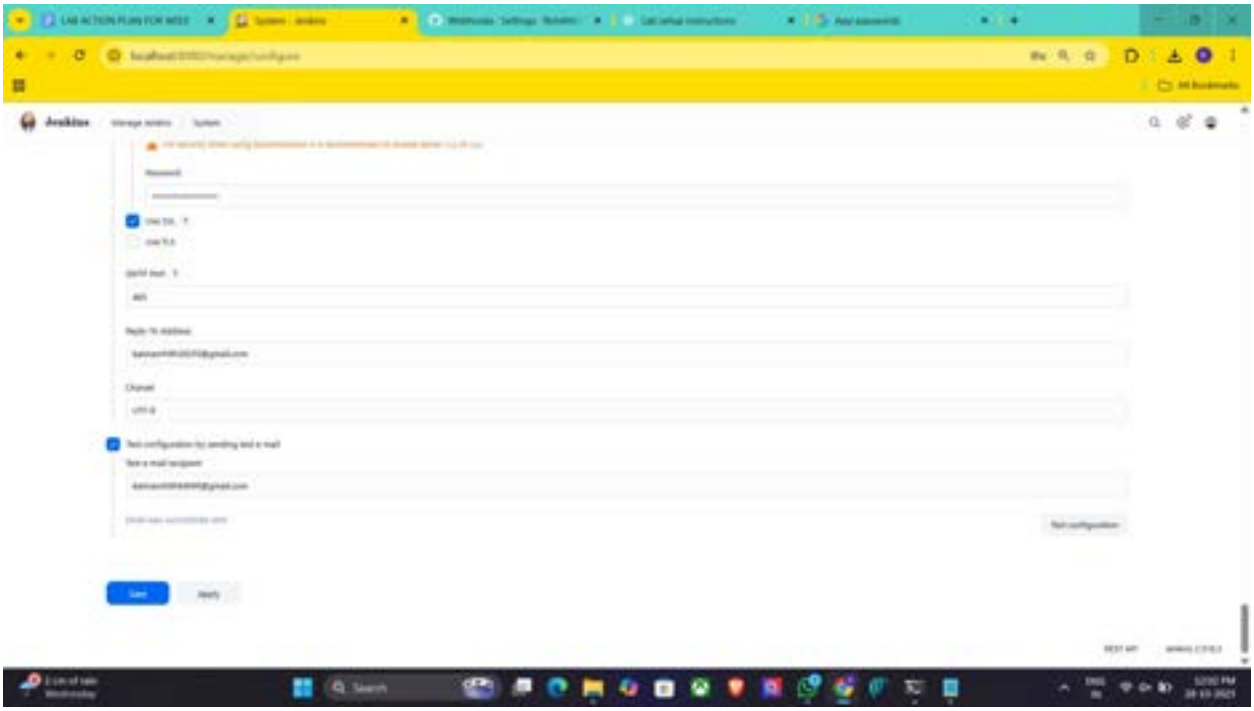


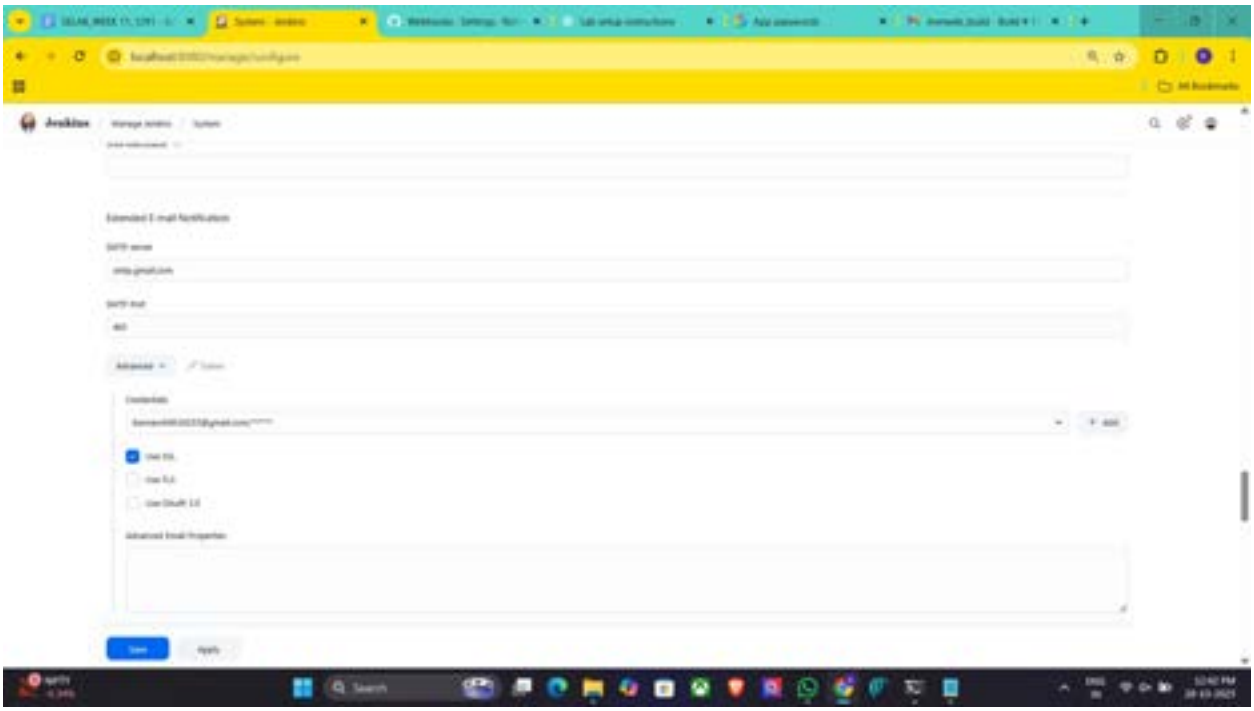
REST API Jenkins 2.516.3

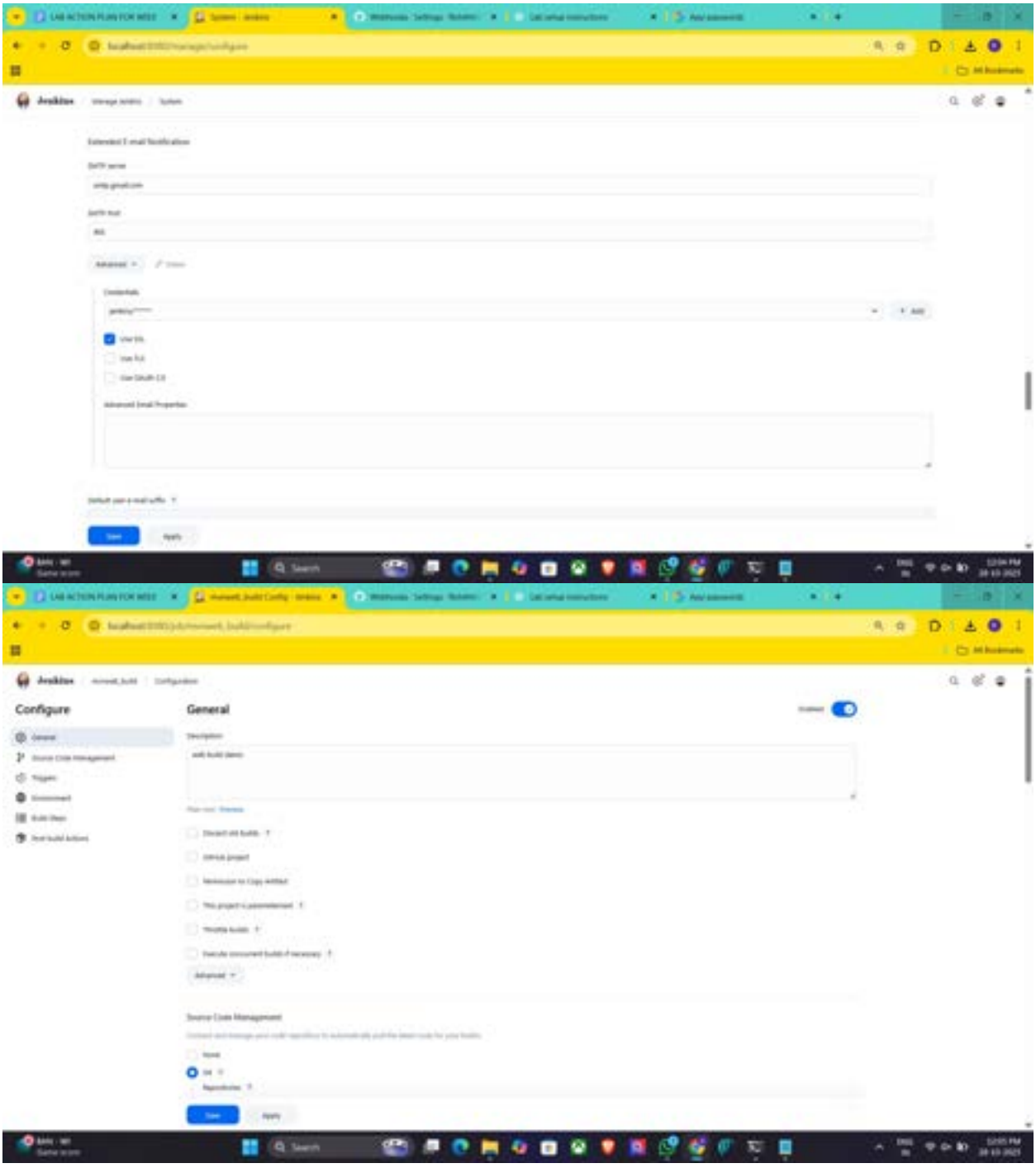



Jenkins
Manage Jenkins
Set up agent
Set up cloud
Dismiss
Go to plugin manager
System Configuration
System
Tools
Plugins
Nodes
Clouds
Appearance
Security

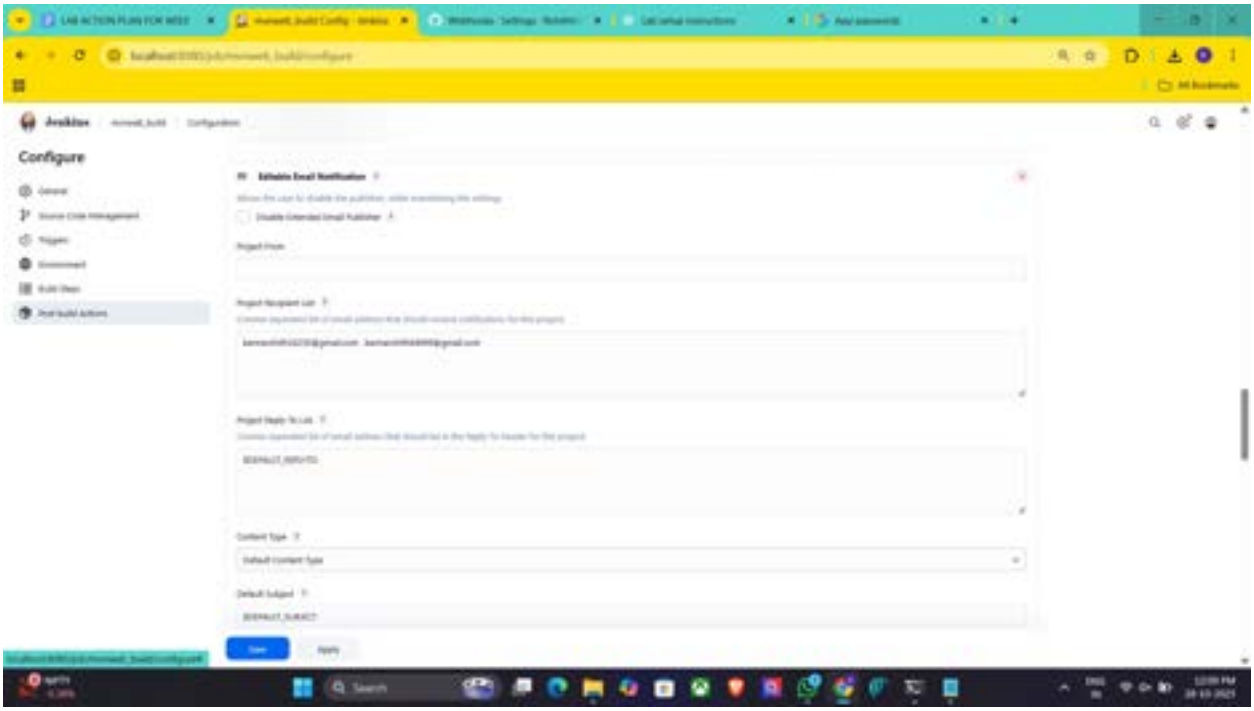

Jenkins

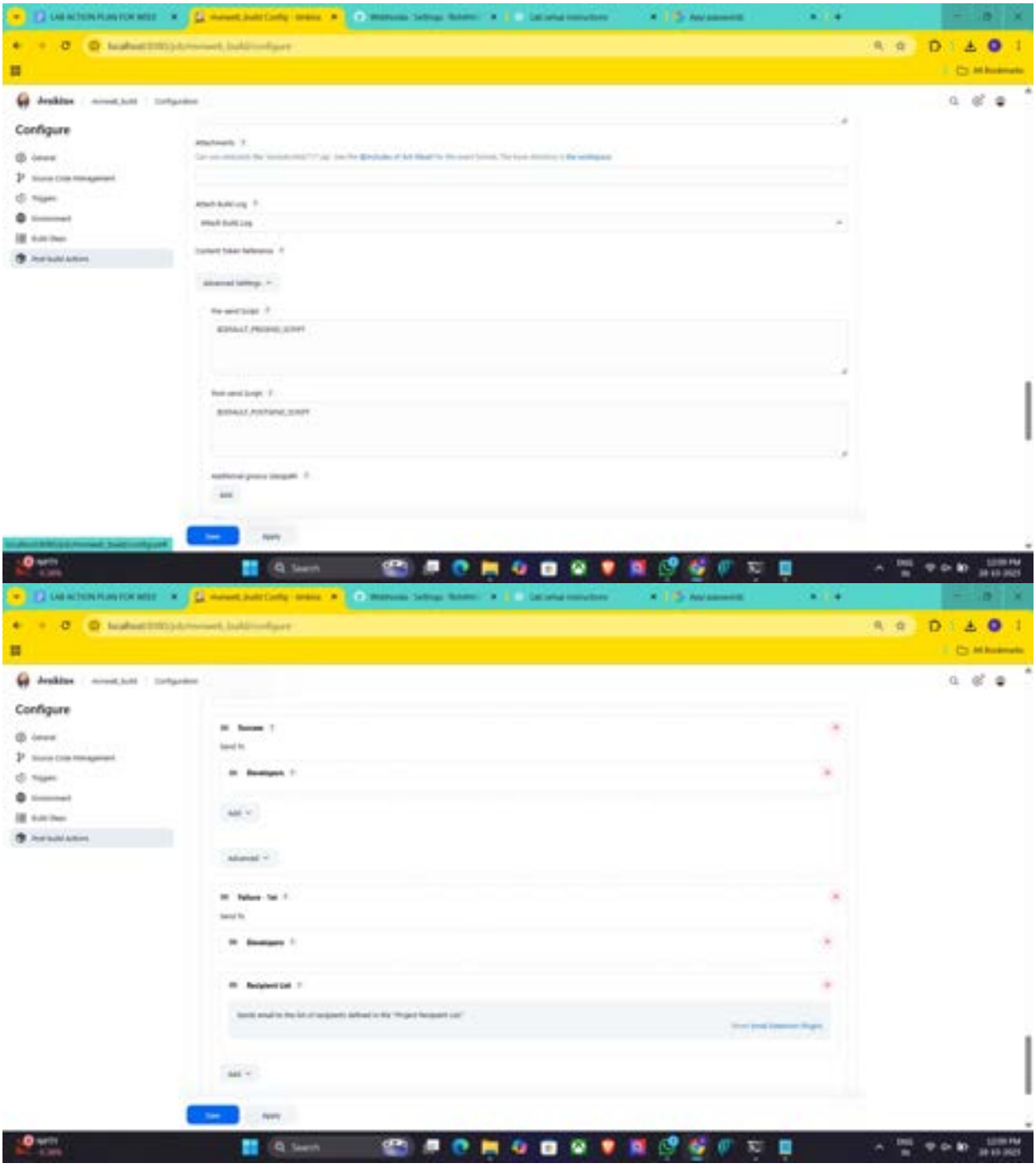




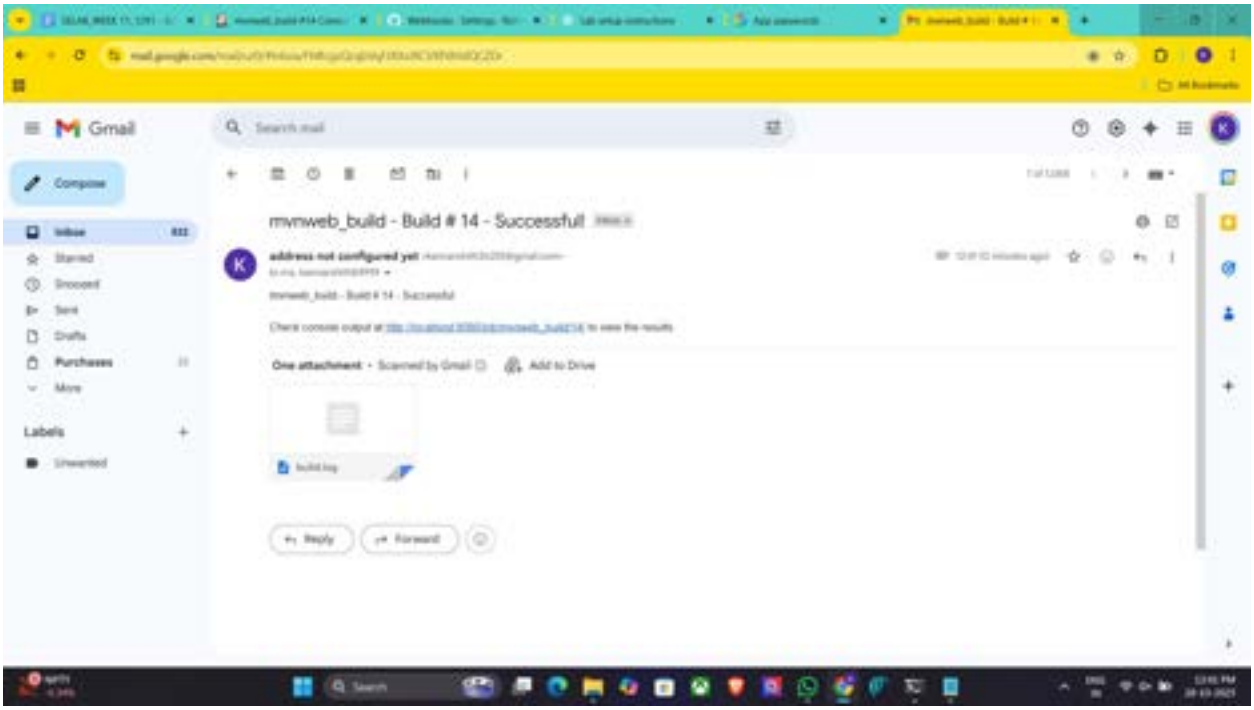


Configure
General


Configure


Configure

Configure


Gmail
Compose
Inbox
Starred
Snoozed
Sent
Drafts
Purchases
More
Labels
mvnweb_build - Build # 14 - Successful
address not configured yet
mvnweb_build - Build # 14 - Successful
Check console output at
One attachment • Scanned by Gmail
Add to Drive
build.log
Reply
Forward

Week 12: Creation of virtual machine for Ubuntu OS and Deploying the web application

1. Creation of virtual machine
2. Deploying the web application
3. Accessing it publicly

Deploying an application into cloud

Steps for Deploying application into the cloud

I. Create application and Push into github
II. Create the virtual machine and connect to it.
III. Clone the application from github, Write the Dockerfile
IV. Create the image
V. Run the image and access it public ip of virtual machine

I. Create Maven-web-java project in eclipse & push into github

II. Create the virtual machine (EC2--instance) in aws and connect to

Using Amazon EC2 eliminates your need to invest in hardware up front, so you can develop and deploy applications faster.

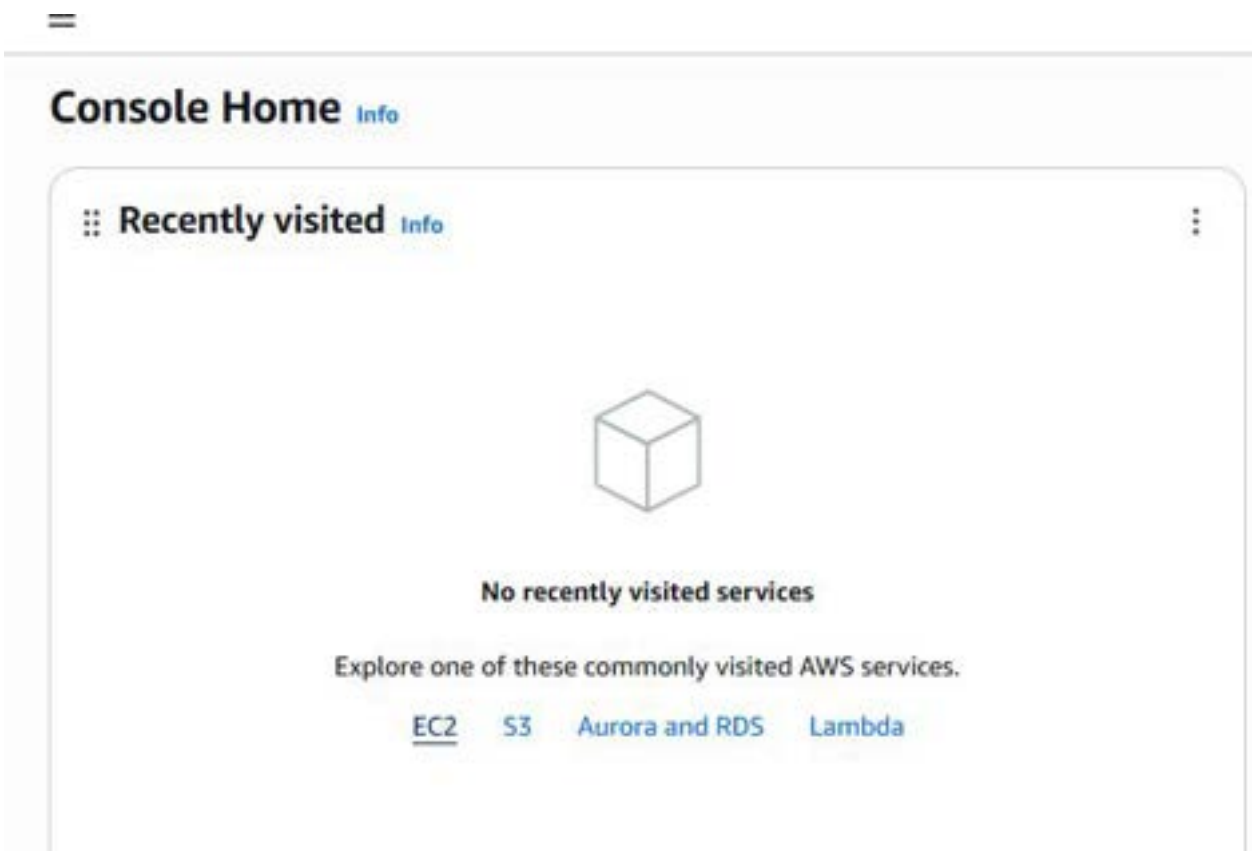
Ex : **Launch ubuntu instnace**

Step 1: Login to AWS /canvas account

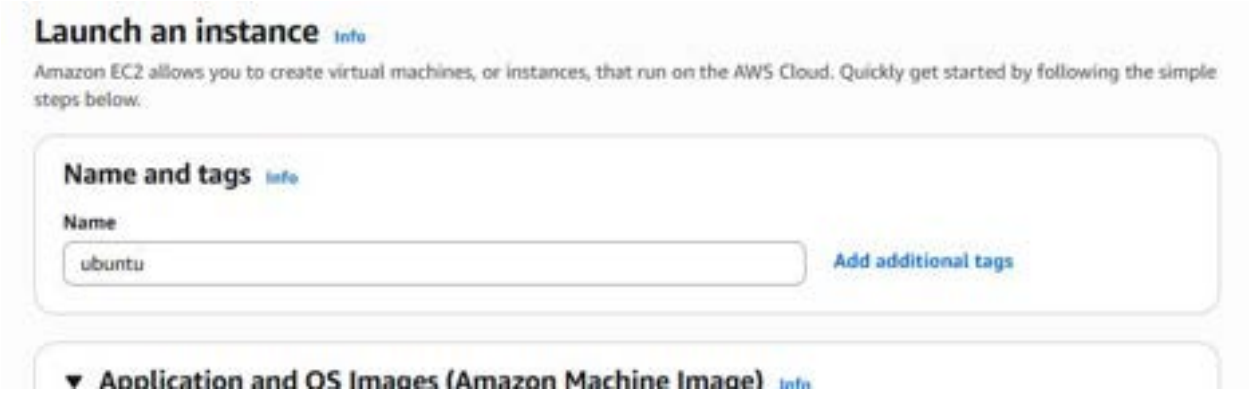
Step 2: Services -- EC2

Step 3: Choose region which is near ?

Services -- EC2 --- Launch Instance

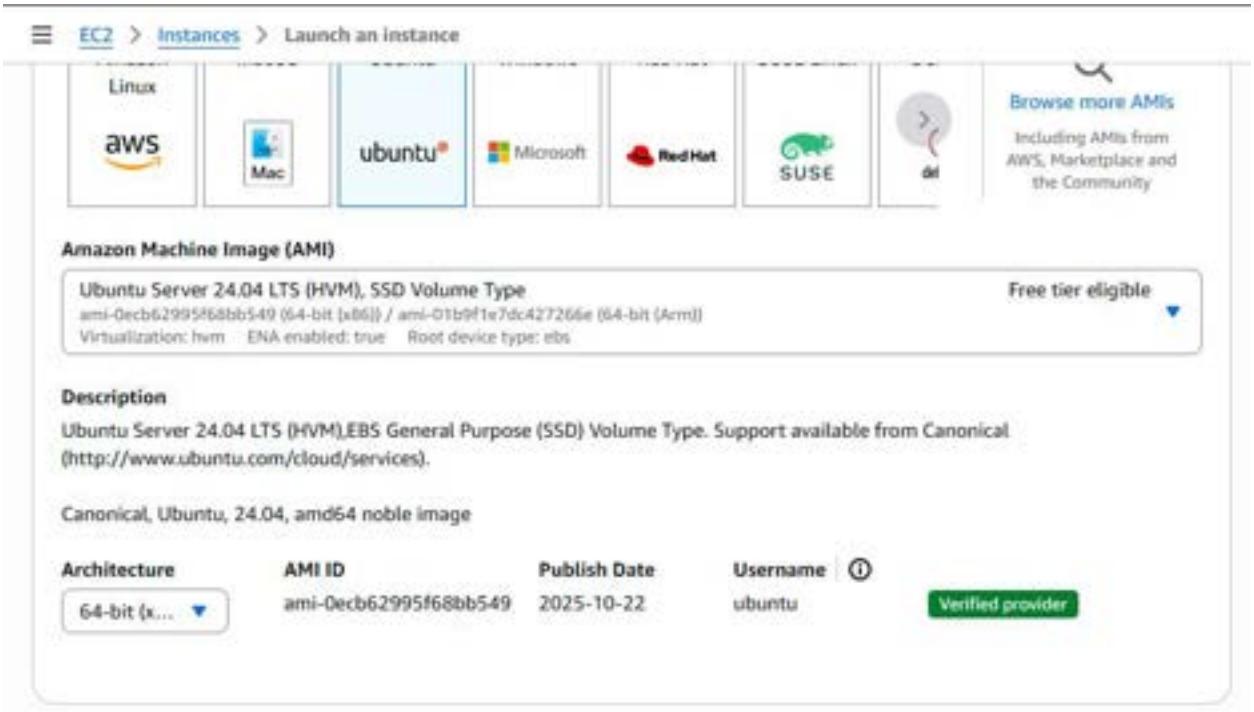


Stage 1 --Name (Giving name to the machine) ubuntu



Stage 2 -- Select AMI (Note: Select free tier eligible) ubuntu server

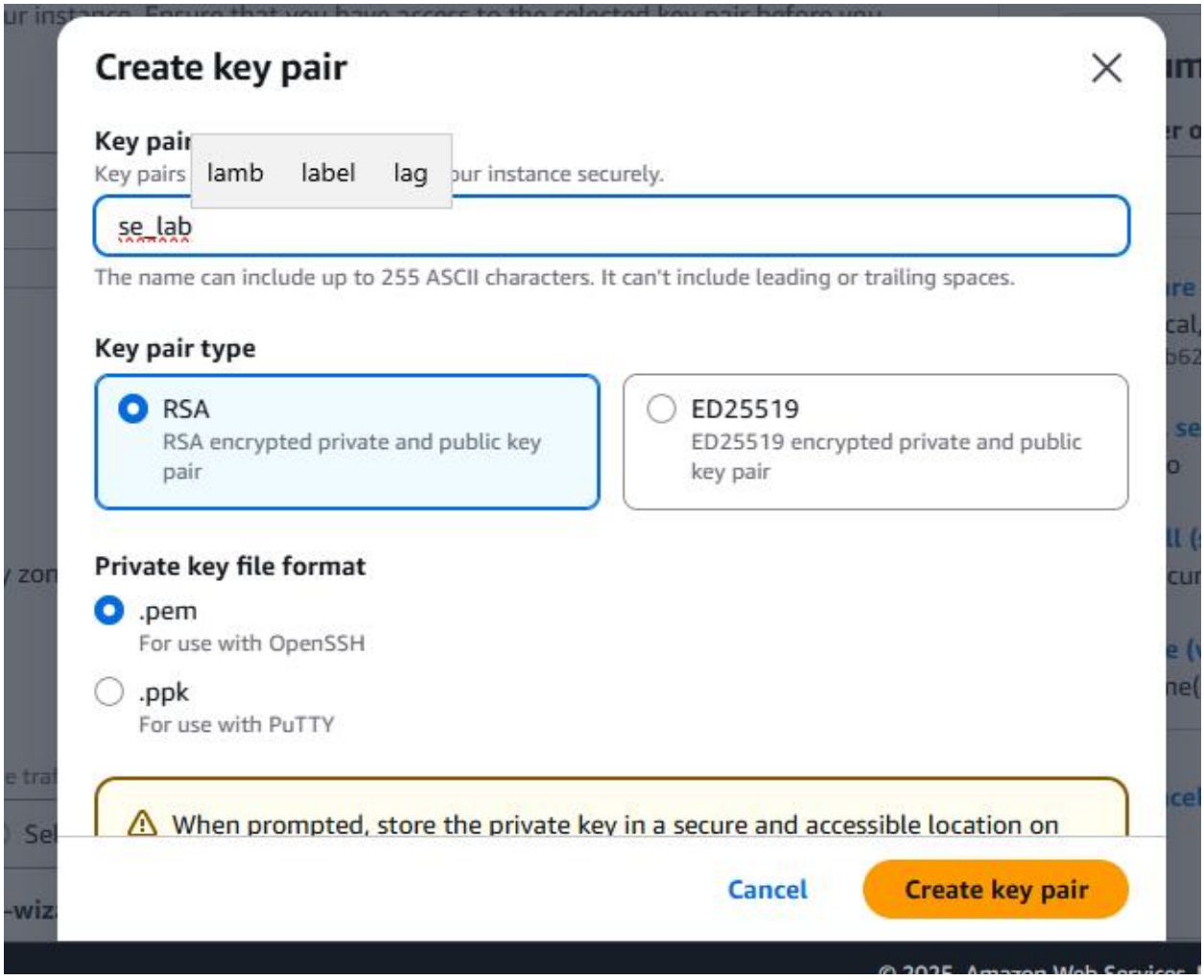
Stage 3 -- Architecture as 64-bit



Stage 4 -- **Instance type** ---- t2.micro(default 1 CPU,1 GB RAM)

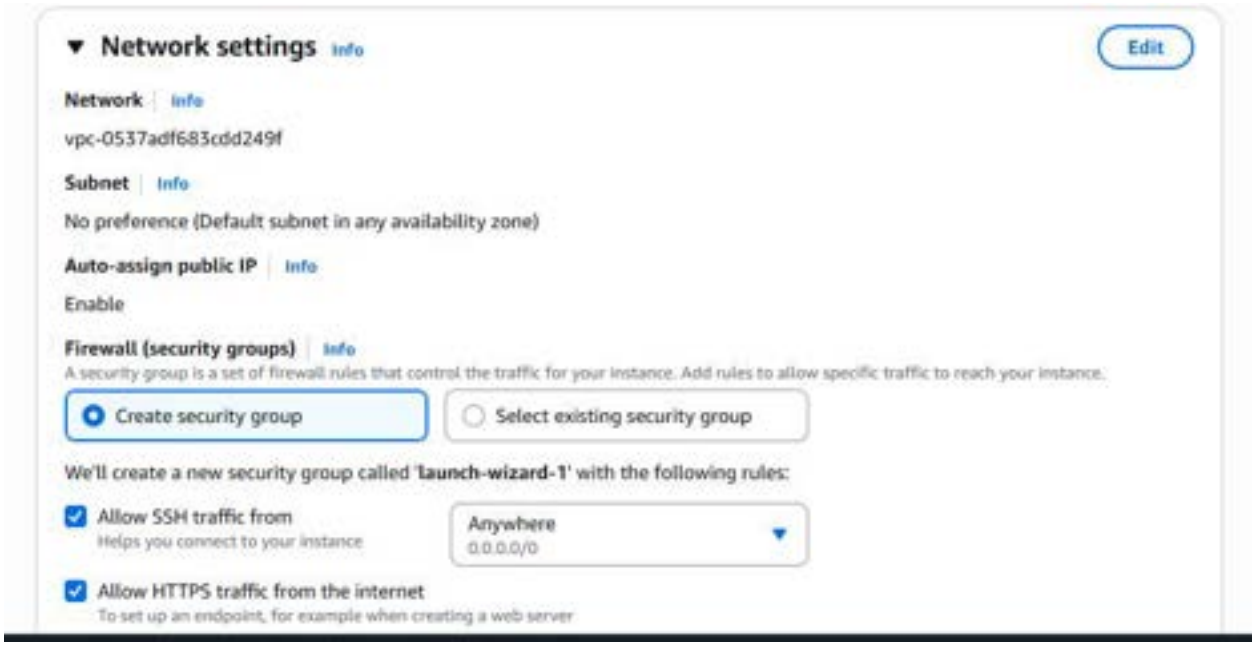
Stage 5 -- Create a new keypair---a keypair will downloaded with extension .pem

Store key in folder AWS



Stage 6 -- Network Setting ----Create Security group -- (It deals with ports)

(Note for understanding We have 0 to 65535 ports. Every port is dedicated to special purpose)



Do this step : HERE select http and https

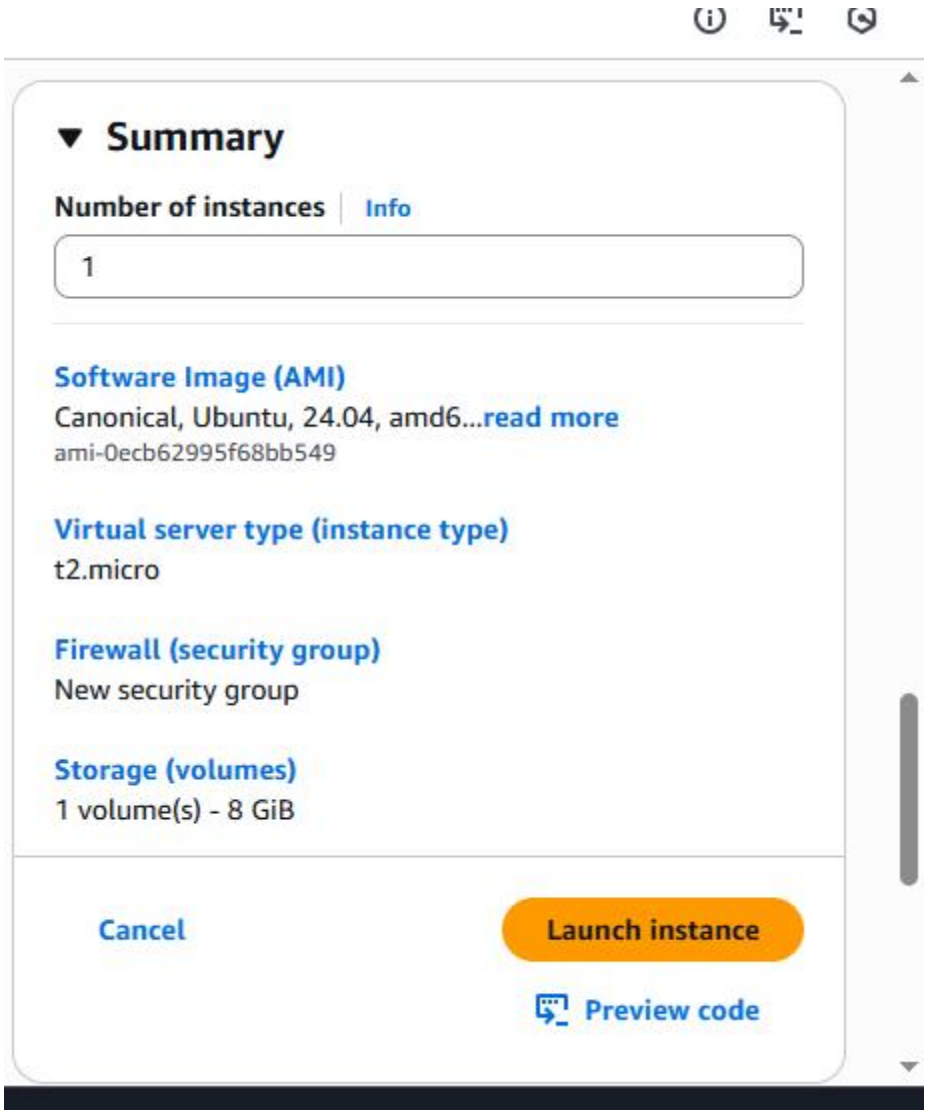
Stage 7 -- Storage - 8GB (Observation - we have root - it is same as C Drive)

Stage 8 --- click on launch instance

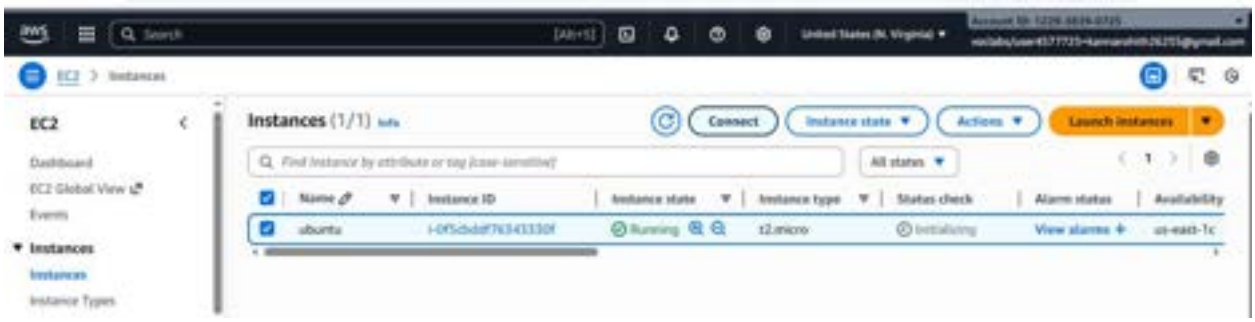
Stage 9: Number of instances ---1

++++++++++++++++++++++++++++++++

Observation - One machines created



Do this step:---once it is created select that instance and click on connect



Here copy the ssh – i command from SSH client connect tab

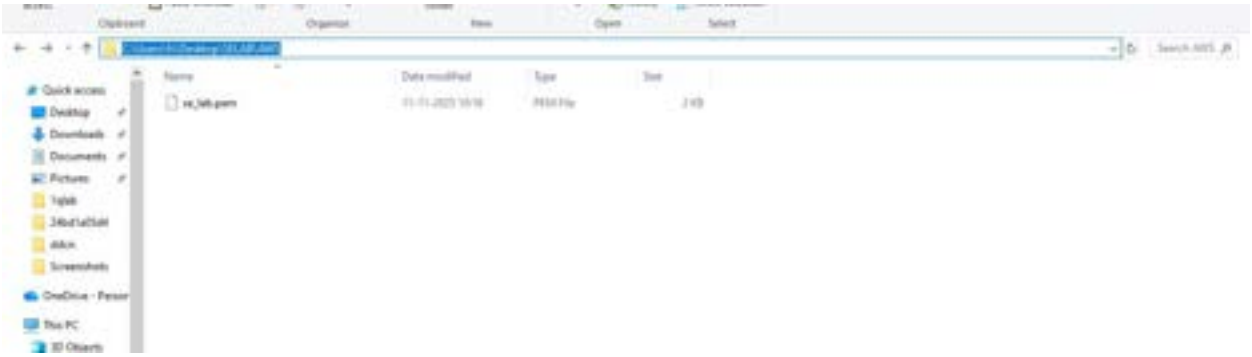
We can use powershell /gitbash /webconsole , to connect to ubuntu machine.

NOTE:- cd path of AWS folder // change path

To connect to above terminals we need to go into the path of the keypair.and

paste the

ssh -i command from the aws console



III. Clone the application from github, Write the Dockerfile

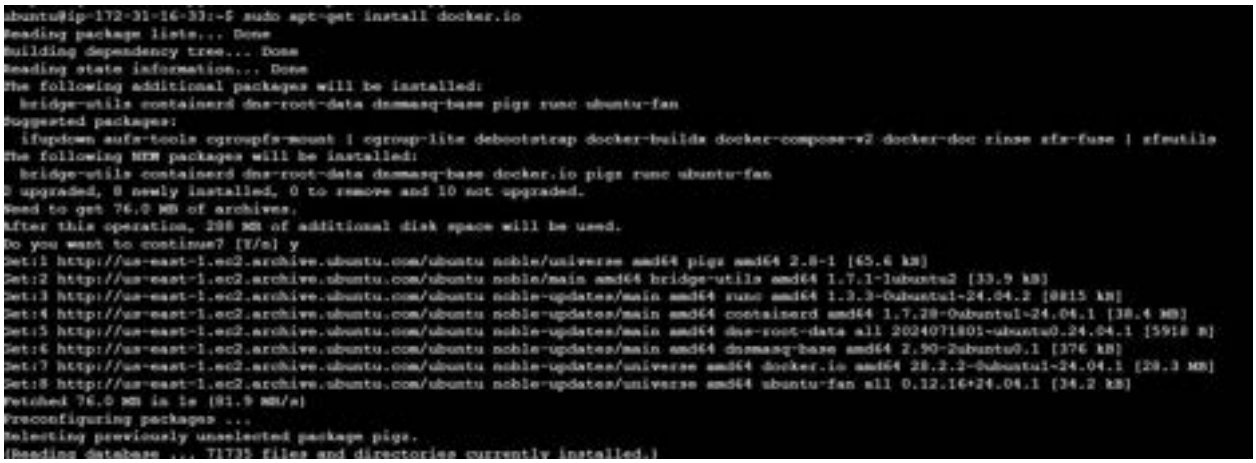
once connected to instance

Step 1:- Run the following commands to install s/w

1. Update all softwares in Ubuntu by command
   sudo apt update

```
sudo apt install udo
ubuntu@ip-172-31-16-33:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1305 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [215 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9336 B]
```

2. Install docker by command
   sudo apt-get install docker.io



3. Install git by command

   sudo apt install git

```
ubuntu@ip-172-31-16-33:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
ubuntu@ip-172-31-16-33:~$
```

4. Install nano(text editor) by command

   Sudo apt install nano

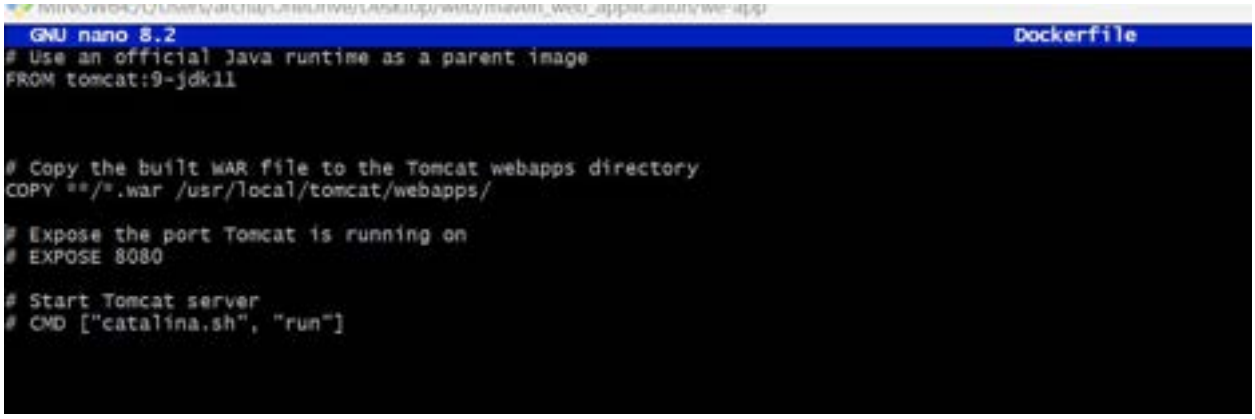
```
ubuntu@ip-172-31-16-33:~$ sudo apt install nano
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nano is already the newest version (7.2-2ubuntu0.1).
nano set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 10 not upgraded.
ubuntu@ip-172-31-16-33:~$
```

step 2:- git clone <**paste the github link of maven-web-java project**>

step 3:- navigate to the maven-web-java project

VI. Create the image

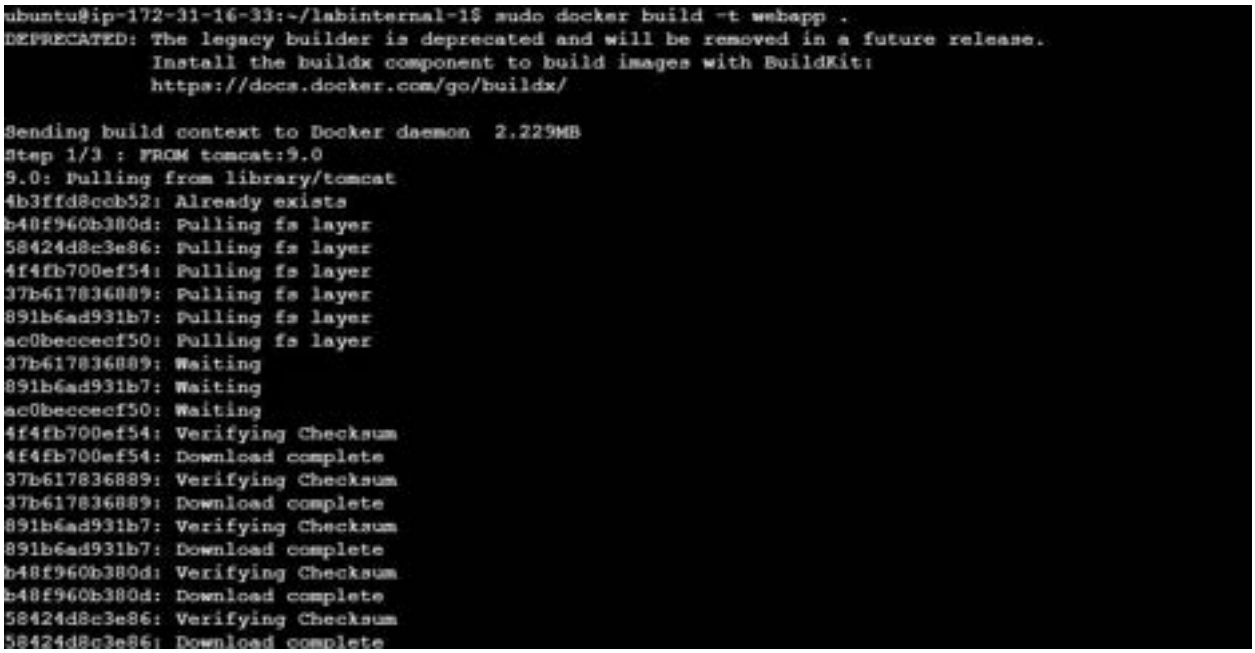
Step 4:- nano Dockerfile



V. Run the image and access it with public ip of virtual machine

Step 1:- build your image

docker build –t <imagename> .(dot)



Step 2:- check for images

```
ubuntu@ip-172-31-16-33:~/labinternal-1$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
webapp        latest    e1b44d1ddf92   8 seconds ago    414MB
tomcat        9-jdk11   f79dfb7f6fee   11 hours ago     422MB
tomcat        9.0       2e4887a16e43   11 hours ago     413MB
ubuntu@ip-172-31-16-33:~/labinternal-1$
```

Step 3:- run image

docker run –d --name app-demo –p 6060:8080 <image name>

Step:-4 **Accessing the app by public ip of virtual machine**

Note:-if your are not able to connect change the inbound rules..